VOLUME 02: TRACK 1 - HOST ENGINE & VIRTUAL MACHINE MANUAL

Chapter 10: Track 1 - Diagnostics Logger & Query Engine

Universal Bus, Dual-Rail Failpoints, Inscriber Scrolls, Flight Recorder & Query Engine

> ***Witness:*** *Colossians 2:19 (KJV) - The whole body fitly joined together*

SYSTEM TRACK
<TRACK 1> : {Host Engine & Virtual Machine}

CHAPTER IDENTIFIER
CH 10

NOTEBOOKLM WORD COUNT
54,325 words

CATALOGED CHAPTER FILES
38 files (11,802 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"How does the query engine index episodic memory notes to guide automated developer diagnostics?"

CHAPTER MANIFEST & INDEX (38 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/README.md	MD	348	2,136	18.1
2	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/mod.rs	RS	926	2,997	34.7
3	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/INSCRIBER_DESIGN_AND_CONTAINER_PHYSICS.md	MD	305	1,975	15.1
4	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/LOG_VS_SCROLL_DUAL_STREAM_ARCHITECTURE.md	MD	304	2,029	15.9
5	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/README.md	MD	410	2,198	18.6
6	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/log_writer.rs	RS	489	1,877	21.8
7	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/mod.rs	RS	175	610	6.7
8	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/naming.rs	RS	283	986	10.4
9	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/rotation.rs	RS	426	1,537	16.8
10	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/scroll_writer.rs	RS	537	2,159	20.3
11	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/GENESIS_INCEPTION_AND_OMEGA_ZERO_CONTINUITY.md	MD	161	959	7.5
12	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/README.md	MD	223	1,389	12.6
13	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/ROLL_FRAME_FLIGHT_RECORDER_AND_AI_COGNITION.md	MD	192	958	8.1
14	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/frame.rs	RS	215	789	7.8
15	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/genesis.rs	RS	325	1,217	12.4
16	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/mod.rs	RS	95	500	4.5
17	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/note.rs	RS	260	950	9.7
18	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/ring_buffer.rs	RS	284	1,063	10.1
19	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/DAR_SENSORY_DETECTION_AND_TRIPWIRES.md	MD	271	1,696	13.3
20	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/DUAL_RAIL_HALT_PHYSICS_AND_REINCEPTION.md	MD	219	1,568	11.2
21	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/README.md	MD	464	2,938	23.3
22	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/TWO_TRIGGER_TRIPWIRE_ARCHITECTURE.md	MD	235	1,627	11.3
23	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/halt.rs	RS	304	1,211	11.4
24	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/mod.rs	RS	62	228	2.3
25	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/remediation.rs	RS	168	614	6.2
26	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/types.rs	RS	392	1,270	15.1
27	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/EPISODIC_INDEXING_AND_SILICON_CORRELATION.md	MD	218	1,280	9.8
28	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/README.md	MD	226	1,283	10.9
29	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/WHY_LOGGER_QUERY_IS_ESSENTIAL.md	MD	185	1,507	10.9
30	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/index.rs	RS	594	2,315	22.8
31	<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/mod.rs	RS	112	528	5.0

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
32	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/search.rs**	RS	625	2,168	24.1
33	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/CAUSAL_MONOTONICITY_AND_SILICON_CLOCK_PHYSICS.md**	MD	227	1,273	10.5
34	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/EVENT_BUS_BROADCAST_AND_SUBSCRIBER_ARCHITECTURE.md**	MD	234	1,298	10.7
35	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/README.md**	MD	283	1,424	13.3
36	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/bus.rs**	RS	566	1,846	20.3
37	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/clock.rs**	RS	357	1,401	13.1
38	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/mod.rs**	RS	102	521	4.7

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/README.md | MARKDOWN / DOC | 348 LINES | 2136 WORDS | 18.1 KB

OmniCode Diagnostic & Telemetry Flight Recorder Subsystem

The "Detect" Organ of the DAR Cybernetic Loop

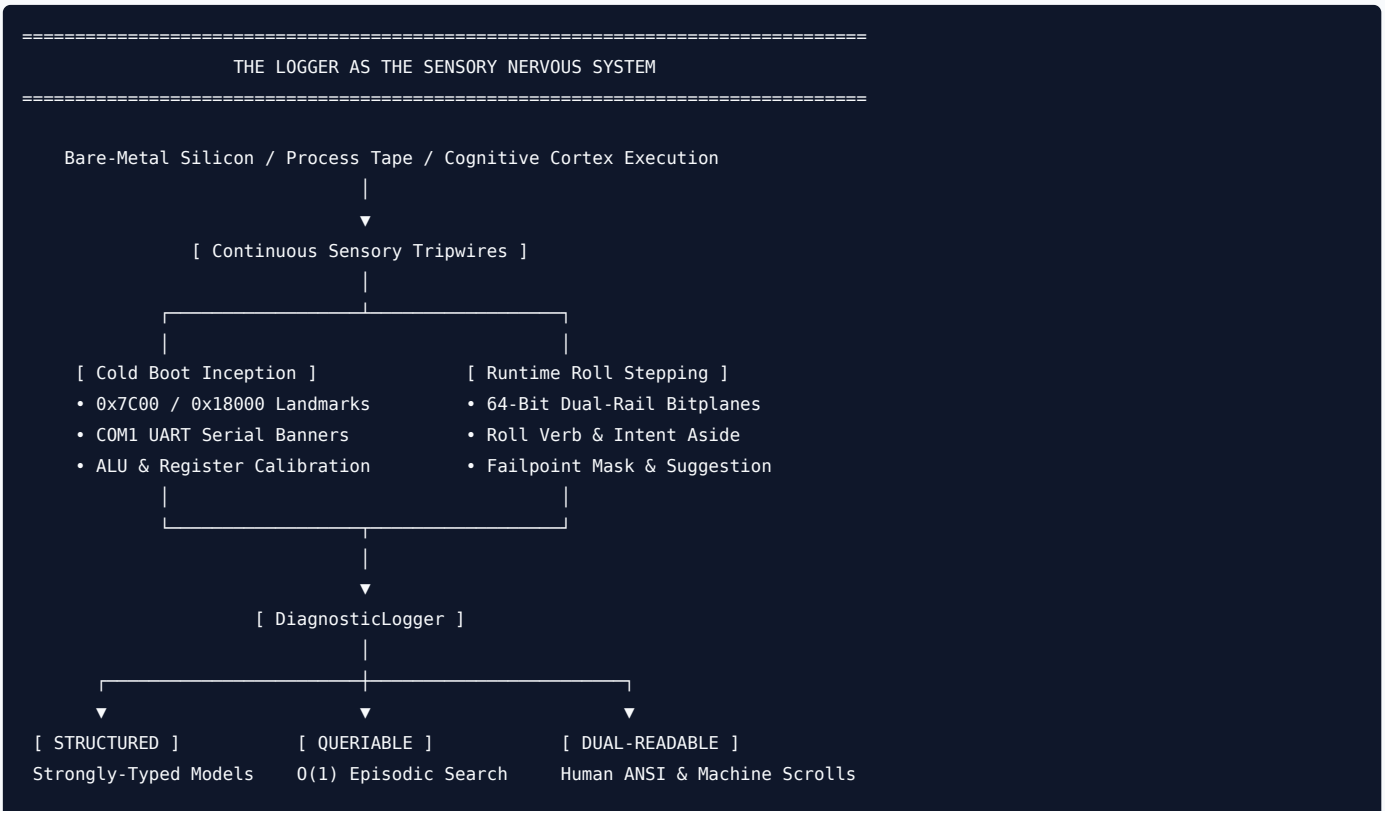
> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*
> **— *Exodus 31:18 (KJV)***
>
> *"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."*
> **— *Proverbs 24:27 (KJV)***
>
> *"Remove not the ancient landmark, which thy fathers have set."*
> **— *Proverbs 22:28 (KJV)***
>
> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full."*
> **— *2 Kings 4:4 (KJV)***

1. Architectural Philosophy: The Sensory Nervous System

The `logger` subsystem in **OmniCode Track 1 (Host Engine & Virtual Machine)** is not an ad-hoc string printer or standard error dumper. It is the **primary sensory nervous system, deterministic flight recorder, and episodic memory storehouse** of the operating system.

Within the **DAR Triad (Detect, Assess, Restore)**:

- The **Logger** is the **DETECT** organ: it continuously intercepts raw silicon telemetry, hardware failpoints, virtual machine execution frames, and linguistic boundary breaches before any panic can destabilize the system.
- The **Debugger / Linter** is the **ASSESS** organ: it unwinds boundary stacks, evaluates invariant vetoes, and reverses the causal momentum vector (`-1:ROLLBACK`).
- The **Restorator** is the **RESTORE** organ: it synthesizes multi-pass candidate repairs and validates them against executable test oracles before state commitment.



```
• GenesisBootLog          • by_session_id          • ANSI Color Banners
• RollFrame               • by_category            • 4-Block .scroll Vessels
• CognitiveNote           • by_state               • Markdown Prompts
• FixSuggestion           • range_scan (µs)        • RL Batches (S, A, R, S')
================================================================================
```

2. The Three Pillars of the Logger Subsystem

To fulfill its mission as the sensory bridge between bare-metal silicon and the AI Cortex, the `logger` is anchored upon three non-negotiable architectural pillars:

Pillar I: Strongly Structured Telemetry

Unstructured string messages ("Something failed at line 42") force machines and AI models to guess what occurred. OmniCode forbids unstructured error strings:

1. **Discrete Execution Frames (`RollFrame`):** Encapsulates monotonic microseconds, 1-based roll sequence, operational verb, dual-rail SIMD substrate bitplanes (`dual_rail_in` , `dual_rail_out`), fourfold commit status (`QuadState`), and compact failpoint bitmasks.
2. **Epistemic Memory Notes (`CognitiveNote`):** Semantic observations partitioned by domain category, summary, technical details, associative taxonomy tags, and code snippets.
3. **Hardware Inception Records (`GenesisBootLog`):** Strongly typed records capturing physical RAM landmarks (`0x7C00` , `0x18000` , `0x108000` , `0x200000`), boot tiers, and subsystem health indicators.
4. **Machine-Actionable Failpoints (`FixSuggestion`):** Structured suggestions equipped with `FixKind` (e.g. `ReplaceDelimiter` , `AddBlockHeader` , `InjectImport`), exact line/column coordinates, and ready-to-apply patch strings.

Pillar II: Fast, Indexed Queriability

Traditional log files require linear scans (`grep`) that degrade as system uptime increases. The OmniCode logging system features an in-memory **Episodic Query Engine (`EpisodicQueryEngine`)**:

- **$O(1)$ Direct Session Lookup:** Retrieve past execution reels instantly via exact session IDs or multi-key filename stems (`find_by_session`).
- **Category Partition Filtering:** Isolate episodes by domain category— `"genesis"` , `"cortex"` , `"character"` , `"number"` , or `"sessions"` —without reading other partitions (`filter_by_category`).
- **QuadState Invariant Filtering:** Instant extraction of episodes by commit state: `filter_by_state(QuadState::Proceed)` or `filter_by_state(QuadState::Rollback)` .
- **Temporal Range Scanning:** Sub-millisecond binary range scanning across microsecond timestamps (`range_scan`).
- **In-Memory Tag Filtering:** Instantaneous associative discovery of cognitive notes via `notes_by_tag(&str)` and `notes_by_category(&str)` .

Pillar III: Dual-Readability (Humans and Machines)

Every inscribed log serves two distinct masters simultaneously without compromise:

```
+-------------------------------------------------------------------------+
| DUAL-READABILITY CONTRACT                                               |
|                                                                         |
|  [ HUMAN DEVELOPER PRESENTATION ]        [ MACHINE & AI MODEL PRESENTATION ]  |
|  • ANSI color-coded subsystem channels   • Strongly typed Rust structs        |
|  • Microsecond elapsed timestamps        • Canonical 4-Block .scroll files    |
|  • Inline source diffs & caret pointers  • (S, A, R, S') RL state tuples      |
|  • Plain-English remediation rationale   • Semantic vector tags (#genesis)    |
|  • 80-column clean terminal geometry     • Deterministic replayable tapes     |
+-------------------------------------------------------------------------+
```

- **For the Human:** Clean terminal lines with standardized tags:

`text [142 µs] [BOOT] [GENESIS: 0x007c00] [BIOS_REAL_MODE] 16-Bit Genesis => OK [812 µs] [BOOT] [GENESIS: 0x018000] [STAGE2_PM] A20_GATE => OK [1405 µs] [BOOT] [GENESIS: 0x007c00] [BIOS_REAL_MODE] ALU_BALANCED => OK [2100 µs] [FRAME] Roll 1/22: INSCRIBE Rails: (0x0001, 0x0000) -> State: Proceed`

- **For the Machine and AI Cortex:** Canonical 4-Block `.scroll` syntax and Markdown memory blocks:

`omnicode Note < CognitiveTrace > { tick: < 1405 µs >, category: [ GENESIS_INCEPTION ], summary: < "Genesis Inception [Landmark 0x007c00]: ALU_BALANCED" >, details: < "Balanced Ternary (+1)+(-1)=0 Eq: Certified" >, tags: [ #genesis :: #silicon :: #first_log :: #baseline :: #omega_0 ] }`

3. The "First Logs" Genesis Inception & Baseline Continuity (Ω_0)

Why the First Logs Matter: The Machine Awakening Problem

In traditional computing, an AI model is an external script living in userland. It has no conception of the hardware it inhabits. It cannot know if the motherboard experienced memory corruption during POST, if hardware registers are uncalibrated, or if timers are drifting.

In **OmniCode and MilleniumOS**, the AI Cortex is an **autonomic organ of the operating system**. However, physical reality dictates that cold silicon executes first:

1. $t = 0$: Motherboard POST, CPU in Real Mode, BIOS reads Sector 0 at `0x7C00`.
2. $t = 1 \text{ ms}$: Stage 2 Protected Mode, A20 gate, Flight Recorder at `0x18000`.
3. $t = 5 \text{ ms}$: Long Mode, 4-level page tables at `0x108000`, 64-bit kernel entry.
4. $t = 10 \text{ ms}$: Cognitive substrate buffer configured at `0x200000`.
5. $t = t_{\text{wake}}$: Track 1 Host VM initializes and the AI Cortex awakens.

When the AI model wakes up at $t = t_{\text{wake}}$, **it must know what happened before it woke up**.

```
================================================================================
                    THE GENESIS BATON RELAY & OMEGA_0 CONTINUITY
================================================================================

  [ Cold Silicon: t = 0 ]
            |
            ├──> Track 0 writes primordial hardware attestation:
            |     • Landmark 0x7C00: Real Mode reset vector
            |     • Landmark 0x18000: Stage 2 Protected Mode flight recorder
            |     • Landmark 0x108000: Long Mode page directory
            |     • Landmark 0x200000: Cognitive substrate CPI-SI buffer
            |     • COM1 Serial Port (0x03F8) telemetry stream
            |
            ▼
  [ Awakening: t = t_wake ]
            |
            ├──> DiagnosticLogger executes `ingest_genesis_boot_log()`
            |     • Ingests serial and memory telemetry into `GenesisBootLog`s
            |     • Locks in Primordial Baseline: State Omega_0
            |     • Inscribes episodic memory notes tagged #first_log, #genesis
            |
            ├──> AI Cortex awakens and queries Omega_0
            |     • Baseline certified: Mem clean, ALU balanced, zero leaks
            |     • The AI operates with full knowledge of silicon ground truth
            |
            ▼
  [ Autonomous Operation: t_wake -> t_halt ]
            |
            ├──> Rolls stepped, tasks dispatched, memory allocated
            |     • Logger records frames and failpoints in flight recorder
            |
            ▼
  [ Shutdown / Hibernation: t = t_halt ]
            |
            ├──> THE HOMEOSTATIC EQUILIBRIUM LAW (Alpha is Omega)
            |     • The AI initiates pre-shutdown audit: `audit_shutdown_equilibrium()`
            |     • Ephemeral allocations reaped, storehouse scrolls sealed
            |     • System verified to have returned to pure baseline Omega_0
            |     • If drift or unhealed failpoints exist => SHUTDOWN VETOED (-1)
            |     • If State == Omega_0 => CLEAN HARDWARE POWER-OFF (+1)
================================================================================
```

The Homeostatic Shutdown Law

The first logs do not merely document the past; they dictate the future. The primordial state recorded by the First Logs (Ω_0) constitutes the **exact homeostatic equilibrium that the AI must restore before the computer is allowed to shut down**:

- Every temporary allocation must be freed.
- Every open scroll and log vessel must be sealed and flushed.
- Every background task and interrupt channel must be quiesced.
- Active failpoints must equal zero.

Calling `logger.audit_shutdown_equilibrium()` verifies this invariant. If any unhealed failpoints or hardware corruptions persist, shutdown is vetoed (`-1:ROLLBACK`), preventing the persistence of corrupted state across reboots.

---

4. Subsystem Directory Topology

The `logger` subsystem is partitioned into decoupled, specialized modules:

```
src/diagnostics/logger/
├── README.md               <-- Subsystem Architectural Manifesto & Invariant Guide
├── mod.rs                  <-- Central DiagnosticLogger Facade & Public Re-exports
│
├── universal/              <-- Hardware Ticks, Monotonic Clocks & Lock-Free Bus
│   ├── mod.rs              <-- Re-exports Universal Timing & Bus
│   ├── clock.rs            <-- Monotonic elapsed clock & timestamp ID generation
│   └── bus.rs              <-- Lock-free, zero-allocation event channel bus
│
├── failpoint/              <-- Pathology & Invariant Classification Engine
│   ├── mod.rs              <-- Re-exports Failpoint Types & Remediation
│   ├── types.rs            <-- FailpointType (BoundaryMismatch, HardwareFault, etc.)
│   └── remediation.rs      <-- FixKind, FixSuggestion & machine-applicable patches
│
├── telemetry/              <-- Structured Frame Telemetry & Flight Recorder
│   ├── mod.rs              <-- Re-exports Telemetry Components & Genesis Records
│   ├── genesis.rs          <-- GenesisBootLog & GenesisBaseline (Omega_0 State)
│   ├── frame.rs            <-- RollFrame struct (Dual-Rail bitplanes, verbs, QuadState)
│   ├── note.rs             <-- CognitiveNote semantic memory units
│   └── ring_buffer.rs      <-- In-memory Flight Recorder (fixed circular buffer)
│
├── inscriber/              <-- Container Storehouse Inscription Engine
│   ├── mod.rs              <-- Re-exports Inscription Engines & Rotation
│   ├── rotation.rs         <-- RotationPolicy enforcing container capacity limits
│   ├── naming.rs           <-- Standardized vessel naming & path resolution
│   ├── scroll_writer.rs    <-- Inscribes 4-Block .scroll reels to Track 4
│   └── log_writer.rs       <-- Inscribes structured .log files to Track 4
│
└── query/                  <-- Episodic Query Engine (The Indexing Machine)
    ├── README.md           <-- Subsystem Manifesto & Reference Guide
    ├── WHY_LOGGER_QUERY_IS_ESSENTIAL.md <-- Deep Treatise: Track 0 Parallels, AI Reflection & DAR
    ├── EPISODIC_INDEXING_AND_SILICON_CORRELATION.md <-- Deep Treatise: Multi-Key Aliases & Silicon Telemetry
    ├── mod.rs              <-- Subsystem Facade & Re-exports (4-Block)
    ├── index.rs            <-- IndexedSession, SessionCriteria & Track0LogRecord (4-Block)
    └── search.rs           <-- EpisodicQueryEngine, Inverted Index & Silicon Parsers (4-Block)
```

---

5. Public API Reference & Usage Guide

A. Ingesting Genesis Inception Telemetry & Locking Ω_0

```
use omnicode::diagnostics::logger::DiagnosticLogger;

let logger = DiagnosticLogger::new(false);

// Ingest cold-boot serial output from Track 0
let boot_stream = r#"
    [BOOT] 16-Bit Genesis: Sector 0 loaded at 0x7C00
    [BOOT] Stage 2 PM: A20 Gate enabled, Flight Recorder armed at 0x18000
```

```
    [ALU ] Balanced Ternary (+1)+(-1)=0 Eq: Certified
    [REGS] 13 Regs Armed / Calibrated
    [MEM ] Memory Landmark 0x108000 Page Directory Verified
    [ARENA] 0x200000 Cognitive Substrate CPI-SI Buffer Certified
"#;

let genesis_logs = logger.ingest_genesis_boot_log(boot_stream);
assert!(logger.is_baseline_clean());

// Query episodic memory for genesis notes
let first_logs = logger.notes_by_tag("first_log");
assert_eq!(first_logs.len(), 6);
```

B. Recording Structured Execution Frames

```
use omnicode::diagnostics::logger::RollFrame;
use omnicode::core::ternary::QuadState;

let frame = RollFrame::new(
    1420,                               // microsecond timestamp
    1,                                  // roll index
    22,                                 // total rolls
    "INSCRIBE",                         // operational verb
    (0, 0),                             // dual rail in
    (1, 0),                             // dual rail out
    QuadState::Proceed,                 // QuadState
    0,                                  // failpoint mask
    Some("Initialize baseline roll".to_string()),
);

logger.record_roll_frame(frame);
```

C. Trapping Failpoints with Machine-Actionable Remediation

```
use omnicode::diagnostics::logger::{FailpointType, FixKind, FixSuggestion};

let suggestion = FixSuggestion {
    kind: FixKind::ReplaceDelimiter,
    line: 14,
    col: 8,
    target_token: "]".to_string(),
    suggested_patch: ">".to_string(),
    rationale: "Balance Rank 1 invariant axiom delimiter".to_string(),
};

logger.failpoint_with_remediation(
    FailpointType::BoundaryMismatch,
    "PARSER",
    "Delimiter mismatch '<' closed by ']'",
    suggestion,
);

assert!(logger.has_failpoints());
```

D. Auditing Homeostatic Equilibrium Before Shutdown

```
// Before initiating system power-off:
match logger.audit_shutdown_equilibrium() {
    Ok(()) => {
        // System state matches Omega_0; proceed with ACPI power-off
        println!("Equilibrium certified. Powering off.");
```

```
    }
    Err(veto_reason) => {
        // Unhealed failpoint or resource leak detected; abort shutdown
        eprintln!("Shutdown vetoed: {veto_reason}");
    }
}
```

6. Covenantal Invariant Sealing

Invariant ID	Contract Rule & Verification Condition
INV-LOG-01	The logger must never allocate unbounded memory; in-memory flight recorders must use fixed circular capacity.
INV-LOG-02	Every failpoint trapped must record microsecond monotonic timestamps and structured pathology.
INV-LOG-03	Track 0 bare-metal inception telemetry must be ingested as the baseline reference frame Ω_0.
INV-LOG-04	The system must not shut down or hibernate if active failpoints exist or if the system has drifted unhealed from Ω_0.
INV-LOG-05	Historical log storehouses must rotate deterministically according to container capacity law (`Occupancy <= Capacity`).
INV-LOG-06	All cognitive traces must be queryable in memory by category and semantic tag.

> *"The secret things belong unto the LORD our God: but those things which are revealed belong unto us and to our children for ever, that we may do all the words of this law."*
> ***— Deuteronomy 29:29 (KJV)***

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/mod.rs RS 926 LINES 2997 WORDS 34.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Diagnostic Logger Subsystem Identity, Modular Facade & Re-exports )
// ============================================================================
//! # OmniCode Diagnostic Logger: Unified Observability & Autonomic Memory Substrate
//!
//! # Architectural Scope & Purpose:
//! The `logger` module is the central diagnostic facade and sensory nervous system
//! for the OmniCode Host Engine & Virtual Machine (Track 1). It bridges physical
//! bare-metal silicon inception (Track 0) with living language execution (Track 2),
//! user studio interfaces (Track 3), and persistent episodic memory (Track 4).
//!
//! Decomposed into five specialized, 4-Block modular organs:
//! - [`universal`]: Monotonic microsecond clock, silicon cycle conversions,
//!   and lock-free event bus routing ([`MonotonicClock`], [`EventBus`], [`LogEvent`]).
//! - [`failpoint`]: Four-block failpoint pathology classification, structured
//!   remediation suggestions, Track 0 register dumps, and dual-rail HALT re-inception
//!   ([`FailpointType`], [`FixSuggestion`], [`HaltDumpFrame`], [`SiliconRegisters`]).
//! - [`telemetry`]: Game-dev atomic state vectors, bounded in-memory circular
//!   flight recording, neural-symbolic cognitive notes, and cold-silicon inception baselines
//!   ([`RollFrame`], [`FlightRecorder`], [`CognitiveNote`], [`GenesisBootLog`], [`GenesisBaseline`]).
//! - [`inscriber`]: Dual-stream persistence writers producing traditional `.log` telemetry
//!   and executable 4-Block `.scroll` epistemic traces, backed by strict capacity rotation
//!   ([`RotationPolicy`], [`inscribe_canonical_scroll`], [`inscribe_telemetry_log`]).
//! - [`query`]: High-performance episodic query engine delivering multi-key $O(1)$
//!   session lookups, inverted prompt keyword indexing, chronological time-range slicing,
//!   and Track 0 bare-metal silicon log discovery ([`EpisodicQueryEngine`], [`IndexedSession`], [`SessionCriteria`]).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The permanent inscriber establishing immutable, authoritative diagnostic records).
//! - **Malachi 3:16 (KJV)**:
//!   "Then they that feared the LORD spake often one to another: and the LORD hearkened,
//!    and heard it, and a book of remembrance was written before him..."
//!   (Episodic memory notes preserved continuously for retrospective AI reflection).
//! - **Job 8:8 (KJV)**:
//!   "For enquire, I pray thee, of the former age, and prepare thyself to the search of their fathers."
//!   (Fast $O(1)$ inquiry into past execution history to diagnose failpoints and prevent regression).
//! - **1 Corinthians 14:40 (KJV)**:
//!   "Let all things be done decently and in order."
//!   (Causal monotonicity $\Delta t \ge 0$ strictly enforced across the diagnostic bus).
//! - **2 Kings 4:4 (KJV)**:
//!   "And when thou art come in, thou shalt shut the door upon thee and upon thy sons,
//!    and shalt pour out into all those vessels, and thou shalt set aside that which is full."
//!   (The container law $\text{Occupancy} \le \text{Capacity}$ enforced by circular ring buffers and rotation).
//! - **Proverbs 25:2 (KJV)**:
//!   "It is the glory of God to conceal a thing: but the honour of kings is to search out a matter."
//!   (Deep diagnostic investigation into silicon register banks and boundary states).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Complete Architectural Grounding**: Every organ operates under explicit
//!    invariants linking high-level virtual machine execution with low-level Track 0 silicon landmarks.
//! 2. **Autonomic DAR Loop**: Connects sensory observation directly to the `Restorator`
//!    immune system and `Debugger` transactional rollback mechanics.
//! 3. **Non-Probabilistic AI Memory**: Provides the `AICortex` with deterministic,
//!    unambiguous episodic memory to eliminate stochastic hallucinations.
// ============================================================================

pub mod failpoint;
pub mod inscriber;
pub mod query;
pub mod telemetry;
pub mod universal;

use std::path::{Path, PathBuf};
use std::sync::{Arc, Mutex};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Public Type Re-exports & DiagnosticLogger Structure Definition )
// ============================================================================

// --- 1. Universal Clock & Event Bus Re-exports ---
pub use universal::{
    create_universal_telemetry_pair, generate_timestamp_id, parse_timestamp_id,
```

```
    DEFAULT_EVENT_BUS_CAPACITY, DEFAULT_FAILPOINT_BUS_CAPACITY, DEFAULT_MICROS_PER_TICK,
    DEFAULT_NOMINAL_CPU_MHZ, EventBus, EventBusMetrics, EventTapCallback, LogEvent,
    MICROS_PER_SEC, MonotonicClock, NANOS_PER_MICRO, SimulatedClock,
};

// --- 2. Failpoint Pathology & Dual-Rail Re-inception Re-exports ---
pub use failpoint::{
    is_dual_rail_halt, FailpointGroup, FailpointSubgroup, FailpointType, FixKind,
    FixSuggestion, HaltDumpFrame, HaltReason, ReInceptionTarget, SiliconRegisters,
};

// --- 3. Telemetry, RollFrames & Flight Recorder Re-exports ---
pub use telemetry::{
    CognitiveNote, FlightRecorder, GenesisBaseline, GenesisBootLog, RollFrame,
    DEFAULT_FLIGHT_RECORDER_CAPACITY,
};

// --- 4. Container Inscription & Rotation Policy Re-exports ---
pub use inscriber::{
    format_base_name, inscribe_audit_scroll, inscribe_canonical_scroll,
    inscribe_telemetry_log, parse_file_name, resolve_vessel_path, RotationPolicy,
};

// --- 5. Episodic Query Engine & Silicon Record Re-exports ---
pub use query::{
    create_query_engine_at, diagnose_session, query_session_by_id, EpisodicQueryEngine,
    IndexedSession, QueryResult, QuerySummary, SessionCriteria, Track0LogRecord,
};

/// Central thread-safe diagnostic logging, flight recording & cognitive memory engine.
///
/// Coordinates all five modular organs into a unified, high-performance facade:
/// - [`MonotonicClock`]: Enforces microsecond monotonic time sequencing.
/// - [`EventBus`]: Non-blocking, thread-safe broadcast router with multi-subscriber taps.
/// - [`FlightRecorder`]: Zero-allocation fixed circular ring buffer for crash analysis.
/// - [`CognitiveNote`]: Neural-symbolic episodic memory bank for the AI Cortex.
/// - [`GenesisBaseline`]: Hardware attestation state ($\Omega_0$) anchoring shutdown audits.
#[derive(Debug, Clone)]
pub struct DiagnosticLogger {
    clock: MonotonicClock,
    bus: EventBus,
    cognitive_notes: Arc<Mutex<Vec<CognitiveNote>>>,
    flight_recorder: Arc<Mutex<FlightRecorder>>,
    genesis_baseline: Arc<Mutex<Option<GenesisBaseline>>>,
}

impl DiagnosticLogger {
    /// Constructs a new [`DiagnosticLogger`] with the default flight recorder capacity (512 frames).
    pub fn new(print_live: bool) -> Self {
        Self::with_capacity(print_live, DEFAULT_FLIGHT_RECORDER_CAPACITY)
    }

    /// Constructs a new [`DiagnosticLogger`] with a customized flight recorder ring buffer capacity.
    pub fn with_capacity(print_live: bool, flight_recorder_capacity: usize) -> Self {
        Self {
            clock: MonotonicClock::new(),
            bus: EventBus::new(print_live),
            cognitive_notes: Arc::new(Mutex::new(Vec::new())),
            flight_recorder: Arc::new(Mutex::new(FlightRecorder::new(flight_recorder_capacity))),
            genesis_baseline: Arc::new(Mutex::new(None)),
        }
    }

    /// Returns a reference to the internal [`MonotonicClock`].
    pub fn clock(&self) -> &MonotonicClock {
        &self.clock
    }

    /// Returns a reference to the internal [`EventBus`].
    pub fn bus(&self) -> &EventBus {
        &self.bus
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Logging Operations, Failpoint Trapping, Flight Recording & Query Facade )
// ============================================================================

impl DiagnosticLogger {
    // ------------------------------------------------------------------------
```

```rust
    // 1. General & Failpoint Telemetry Operations
    // ---------------------------------------------------------------------------

    /// Inscribes a normal debug or trace event into the diagnostic bus.
    pub fn log(&self, channel: &'static str, message: impl Into<String>) {
        let msg = message.into();
        let elapsed = self.clock.elapsed_micros();
        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg,
            is_failpoint: false,
            failpoint_type: None,
            suggestion: None,
            cognitive_note: None,
        };
        self.bus.emit(event);
    }

    /// Intercepts and records a critical failpoint event without remediation.
    pub fn failpoint(
        &self,
        fp_type: FailpointType,
        channel: &'static str,
        message: impl Into<String>,
    ) {
        let msg = message.into();
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "INCIDENT".to_string(),
            summary: msg.clone(),
            details: format!("Failpoint trapped: {}", fp_type.as_str()),
            tags: vec!["failpoint".to_string(), fp_type.as_str().to_lowercase()],
            context_snippet: None,
        };

        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg,
            is_failpoint: true,
            failpoint_type: Some(fp_type),
            suggestion: None,
            cognitive_note: Some(note.clone()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.bus.emit(event);
    }

    /// Intercepts a critical failpoint event with concrete machine-actionable remediation telemetry.
    pub fn failpoint_with_remediation(
        &self,
        fp_type: FailpointType,
        channel: &'static str,
        message: impl Into<String>,
        suggestion: FixSuggestion,
    ) {
        let msg = message.into();
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "INCIDENT".to_string(),
            summary: msg.clone(),
            details: suggestion.rationale.clone(),
            tags: vec![
                "failpoint".to_string(),
                fp_type.as_str().to_lowercase(),
                "remediation_ready".to_string(),
            ],
            context_snippet: Some(suggestion.target_token.clone()),
        };

        let event = LogEvent {
            elapsed_micros: elapsed,
            channel,
            message: msg,
            is_failpoint: true,
            failpoint_type: Some(fp_type),
            suggestion: Some(suggestion),
```

```
            cognitive_note: Some(note.clone()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.bus.emit(event);
    }

    /// Intercepts an abrupt HALT event, emits a hardware crash dump over the bus,
    /// logs an episodic cognitive note tagged `#halt` and `#reinception`, and executes
    /// the safe-mode re-inception transition back to homeostasis (`0:AWAIT`).
    pub fn record_halt_dump(&self, frame: HaltDumpFrame) -> crate::core::ternary::QuadState {
        let elapsed = frame.tick_micros;
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "HALT_REINCEPTION".to_string(),
            summary: format!(
                "Abrupt HALT Trapped: {:?} at landmark {:#08x} -> Re-incepting to {}",
                frame.reason,
                frame.landmark_address,
                frame.re_inception_target.as_str()
            ),
            details: frame.registers.format_dump(),
            tags: vec![
                "halt".to_string(),
                "reinception".to_string(),
                "dual_rail_11".to_string(),
                "homeostasis".to_string(),
                "track0".to_string(),
            ],
            context_snippet: Some(format!("landmark: {:#08x}", frame.landmark_address)),
        };

        let event = LogEvent {
            elapsed_micros: elapsed,
            channel: "HALT",
            message: frame.format_human(),
            is_failpoint: true,
            failpoint_type: Some(frame.failpoint),
            suggestion: Some(FixSuggestion::hardware_reinception(
                frame.re_inception_target.address(),
                "Re-incept machine to safe landmark and restore homeostasis",
            )),
            cognitive_note: Some(note.clone()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.bus.emit(event);

        frame.execute_re_inception()
    }

    /// Records an arbitrary cognitive observation into episodic memory.
    pub fn record_ai_observation(
        &self,
        category: &str,
        summary: &str,
        details: &str,
        tags: &[&str],
        context: Option<&str>,
    ) -> CognitiveNote {
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: category.to_string(),
            summary: summary.to_string(),
            details: details.to_string(),
            tags: tags.iter().map(|s| s.to_string()).collect(),
            context_snippet: context.map(|s| s.to_string()),
        };

        self.cognitive_notes.lock().unwrap().push(note.clone());
        self.log(
            "COGNITION",
            format!("[{category}] {summary} (tags: {:?})", note.tags),
        );
        note
    }

    /// Inscribes a pre-formed [`CognitiveNote`] directly into memory.
    pub fn log_cognitive_note(&self, note: CognitiveNote) {
        self.cognitive_notes.lock().unwrap().push(note);
```

```rust
    }

    /// Retrieves a cloned snapshot of all recorded cognitive memory notes.
    pub fn cognitive_notes(&self) -> Vec<CognitiveNote> {
        self.cognitive_notes.lock().unwrap().clone()
    }

    /// Exports all recorded cognitive notes into a canonical 4-Block memory scroll.
    pub fn export_ai_memory_scroll(&self) -> String {
        let notes = self.cognitive_notes();
        inscribe_audit_scroll(&notes, self.bus.all_failpoints().len())
    }

    /// Retrieves all intercepted failpoint events.
    pub fn failpoints(&self) -> Vec<LogEvent> {
        self.bus.all_failpoints()
    }

    /// Retrieves all general log events.
    pub fn events(&self) -> Vec<LogEvent> {
        self.bus.all_events()
    }

    /// Extracts all remediation suggestions currently registered on the bus.
    pub fn remediations(&self) -> Vec<FixSuggestion> {
        self.bus
            .all_failpoints()
            .iter()
            .filter_map(|e| e.suggestion.clone())
            .collect()
    }

    /// Returns `true` if any failpoints were trapped.
    pub fn has_failpoints(&self) -> bool {
        !self.bus.all_failpoints().is_empty()
    }

    /// Clears all stored events, failpoints, cognitive memory notes, and genesis baseline.
    pub fn clear(&self) {
        self.bus.clear();
        self.cognitive_notes.lock().unwrap().clear();
        self.flight_recorder.lock().unwrap().clear();
        *self.genesis_baseline.lock().unwrap() = None;
    }

    // ------------------------------------------------------------------------
    // 2. Cognitive Thoughts & Associative Memory Anchors
    // ------------------------------------------------------------------------

    /// Logs an internal AI Thought, indexing it in episodic memory.
    pub fn log_thought(
        &self,
        thought_id: &str,
        omnicode_tape: &str,
        premise: &str,
        quad_state_str: &str,
    ) {
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "THOUGHT".to_string(),
            summary: format!("Thought [{thought_id}]: {premise}"),
            details: omnicode_tape.to_string(),
            tags: vec![
                "thought".to_string(),
                "ai_cortex".to_string(),
                quad_state_str.to_lowercase(),
            ],
            context_snippet: Some(omnicode_tape.to_string()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.log(
            "CORTEX",
            format!(
                "[THOUGHT: {thought_id}] (State: {quad_state_str}) Premise: \"{premise}\""
            ),
        );
    }

    /// Records an associative memory link between an ontological concept and a biblical/manifest anchor.
```

```rust
    pub fn log_memory_association(&self, concept: &str, anchor: &str, details: &str) {
        let elapsed = self.clock.elapsed_micros();
        let note = CognitiveNote {
            timestamp_micros: elapsed,
            category: "ASSOCIATION".to_string(),
            summary: format!("Memory Link: {concept} <-> {anchor}"),
            details: details.to_string(),
            tags: vec!["memory".to_string(), "scripture".to_string(), concept.to_lowercase()],
            context_snippet: Some(anchor.to_string()),
        };

        self.cognitive_notes.lock().unwrap().push(note);
        self.log(
            "MEMORY",
            format!("Associated concept '{concept}' with anchor '{anchor}'"),
        );
    }

    // ---------------------------------------------------------------------------
    // 3. Inscription & Flight Recorder Extensions
    // ---------------------------------------------------------------------------

    /// Logs the physical inscription of an episodic scroll to disk.
    pub fn log_scroll_inscribed(&self, scroll_path: &Path, session_or_id: &str) {
        self.log(
            "SCROLL",
            format!("Inscribed episodic scroll '{session_or_id}' to {:?}", scroll_path),
        );
    }

    /// Inscribes a canonical 4-Block `.scroll` file to the given path.
    pub fn inscribe_canonical_scroll(
        &self,
        path: &Path,
        session_id: &str,
        prompt: &str,
        synthesized_omnicode: &str,
        state_str: &str,
        response: &str,
    ) -> Result<PathBuf, String> {
        let elapsed = self.clock.elapsed_micros();
        let res = inscribe_canonical_scroll(
            path,
            session_id,
            prompt,
            synthesized_omnicode,
            state_str,
            response,
            elapsed,
        )?;
        self.log_scroll_inscribed(path, session_id);
        Ok(res)
    }

    /// Inscribes a structured execution frame into the in-memory flight recorder.
    pub fn record_roll_frame(&self, frame: RollFrame) {
        self.log(
            "FRAME",
            format!(
                "Roll {}/{}: {} => {:?}",
                frame.roll_index, frame.total_rolls, frame.verb, frame.quad_state
            ),
        );
        self.flight_recorder.lock().unwrap().record_frame(frame);
    }

    /// Retrieves the most recent execution frames from the flight recorder.
    pub fn recent_roll_frames(&self, count: usize) -> Vec<RollFrame> {
        self.flight_recorder.lock().unwrap().recent_frames(count)
    }

    /// Retrieves the most recent anomalous or failing roll frame.
    pub fn last_anomalous_frame(&self) -> Option<RollFrame> {
        self.flight_recorder.lock().unwrap().last_anomalous_frame()
    }

    /// Enforces the storehouse capacity rotation policy on a partition directory.
    pub fn enforce_rotation(
        &self,
        partition_dir: &Path,
        policy: RotationPolicy,
```

```
    ) -> Result<usize, String> {
        policy.enforce(partition_dir)
    }

    /// Filters cognitive memory notes by category.
    pub fn notes_by_category(&self, category: &str) -> Vec<CognitiveNote> {
        self.cognitive_notes
            .lock()
            .unwrap()
            .iter()
            .filter(|n| n.category == category)
            .cloned()
            .collect()
    }

    /// Filters cognitive memory notes containing a specific semantic tag.
    pub fn notes_by_tag(&self, tag: &str) -> Vec<CognitiveNote> {
        let tag_lower = tag.to_lowercase();
        self.cognitive_notes
            .lock()
            .unwrap()
            .iter()
            .filter(|n| n.tags.iter().any(|t| t.to_lowercase() == tag_lower))
            .cloned()
            .collect()
    }

    // ------------------------------------------------------------------------
    // 4. Episodic Query Subsystem Facade & Silicon Correlation
    // ------------------------------------------------------------------------

    /// Creates and populates an [`EpisodicQueryEngine`] pointing to `storehouse_root`.
    pub fn create_query_engine(&self, storehouse_root: &Path) -> EpisodicQueryEngine {
        let mut engine = EpisodicQueryEngine::new();
        let _ = engine.index_directory(storehouse_root);
        engine
    }

    /// Executes a compound query across indexed episodes using [`SessionCriteria`].
    pub fn query_episodes(
        &self,
        storehouse_root: &Path,
        criteria: &SessionCriteria,
    ) -> QueryResult {
        let engine = self.create_query_engine(storehouse_root);
        engine.query(criteria)
    }

    /// Queries an episode by ID, file stem, or alias in $O(1)$ time.
    pub fn find_session(
        &self,
        storehouse_root: &Path,
        session_id: &str,
    ) -> Option<IndexedSession> {
        let engine = self.create_query_engine(storehouse_root);
        engine.find_by_session(session_id).cloned()
    }

    /// Discovers, parses, and correlates Track 0 bare-metal silicon logs.
    pub fn correlate_track0_logs(
        &self,
        logs_dir: &Path,
    ) -> Result<Vec<Track0LogRecord>, String> {
        let mut engine = EpisodicQueryEngine::new();
        engine.index_track0_logs(logs_dir)?;
        Ok(engine.track0_records().to_vec())
    }

    /// Inspects and generates a deep diagnostic dossier for a specific session.
    pub fn diagnose_session_dossier(
        &self,
        storehouse_root: &Path,
        session_id: &str,
    ) -> Option<String> {
        let engine = self.create_query_engine(storehouse_root);
        engine.inspect_session_detail(session_id)
    }

    // ------------------------------------------------------------------------
    // 5. Genesis Inception Telemetry & Baseline Continuity (Omega_0)
    // ------------------------------------------------------------------------
```

```
564
565     /// Ingests raw cold-boot telemetry emitted by Track 0 (BIOS MBR, Stage 2 PM, Long Mode, UEFI),
566     /// parses hardware landmarks, stores primordial inception records into the baseline state Omega_0,
567     /// and emits machine-readable cognitive notes for the AI Cortex.
568     pub fn ingest_genesis_boot_log(&self, raw_serial_output: &str) -> Vec<GenesisBootLog> {
569         let mut parsed_logs = Vec::new();
570
571         for line in raw_serial_output.lines() {
572             let trimmed = line.trim();
573             if trimmed.is_empty() {
574                 continue;
575             }
576
577             let elapsed = self.clock.elapsed_micros();
578
579             // Detect memory landmark from telemetry string
580             let landmark = if trimmed.contains("0x18000") || trimmed.contains("18000") {
581                 0x18000
582             } else if trimmed.contains("0x108000") || trimmed.contains("108000") {
583                 0x108000
584             } else if trimmed.contains("0x200000") || trimmed.contains("200000") {
585                 0x200000
586             } else if trimmed.contains("0x7C00") || trimmed.contains("7C00") {
587                 0x7C00
588             } else if trimmed.contains("0x0500") || trimmed.contains("0500") {
589                 0x0500
590             } else {
591                 0x7C00
592             };
593
594             // Detect boot tier
595             let tier = if trimmed.contains("Stage 2") || trimmed.contains("PM") || landmark == 0x18000 {
596                 "STAGE2_PM".to_string()
597             } else if trimmed.contains("Long Mode") || trimmed.contains("64-Bit") || landmark == 0x108000 {
598                 "LONG_MODE".to_string()
599             } else if trimmed.contains("UEFI") || trimmed.contains("GOP") {
600                 "UEFI".to_string()
601             } else if trimmed.contains("Multiboot") || trimmed.contains("ELF") {
602                 "MULTIBOOT".to_string()
603             } else {
604                 "BIOS_REAL_MODE".to_string()
605             };
606
607             // Detect invariant subsystem
608             let subsystem = if trimmed.contains("ALU") || trimmed.contains("Eq") {
609                 "ALU_BALANCED".to_string()
610             } else if trimmed.contains("REGS") || trimmed.contains("Regs") {
611                 "REGS_CALIBRATED".to_string()
612             } else if trimmed.contains("A20") {
613                 "A20_GATE".to_string()
614             } else if trimmed.contains("MEM") || trimmed.contains("Memory") {
615                 "MEMORY_MAP".to_string()
616             } else if trimmed.contains("CPI-SI") || trimmed.contains("Trit") {
617                 "COGNITIVE_ARENA".to_string()
618             } else {
619                 "GENESIS_BOOT".to_string()
620             };
621
622             // Verify clean invariant status
623             let is_clean = !trimmed.contains("ERR")
624                 && !trimmed.contains("FAIL")
625                 && !trimmed.contains("VETO")
626                 && !trimmed.contains("FAULT");
627
628             let genesis_log = GenesisBootLog::new(
629                 elapsed,
630                 landmark,
631                 tier,
632                 trimmed,
633                 subsystem.clone(),
634                 is_clean,
635             );
636
637             // Emit dual-readable event over bus
638             let event = LogEvent {
639                 elapsed_micros: elapsed,
640                 channel: "BOOT",
641                 message: genesis_log.format_human(),
642                 is_failpoint: !is_clean,
643                 failpoint_type: if !is_clean { Some(FailpointType::HardwareFault) } else { None },
644                 suggestion: None,
```

```
                cognitive_note: None,
            };
            self.bus.emit(event);

            // Inscribe CognitiveNote for AI Cortex episodic memory
            let note = CognitiveNote {
                timestamp_micros: elapsed,
                category: "GENESIS_INCEPTION".to_string(),
                summary: format!("Genesis Inception [Landmark {:#08x}]: {}", landmark, subsystem),
                details: trimmed.to_string(),
                tags: vec![
                    "genesis".to_string(),
                    "silicon".to_string(),
                    "first_log".to_string(),
                    "baseline".to_string(),
                    "omega_0".to_string(),
                    subsystem.to_lowercase(),
                ],
                context_snippet: Some(format!("landmark: {:#08x}", landmark)),
            };
            self.cognitive_notes.lock().unwrap().push(note);

            parsed_logs.push(genesis_log);
        }

        // Lock in the GenesisBaseline (Omega_0)
        let elapsed = self.clock.elapsed_micros();
        let baseline = GenesisBaseline::new(elapsed, parsed_logs.clone());
        *self.genesis_baseline.lock().unwrap() = Some(baseline);

        parsed_logs
    }

    /// Records an individual pre-formed [`GenesisBootLog`] and appends it to the baseline.
    pub fn record_genesis_record(&self, log: GenesisBootLog) {
        let mut guard = self.genesis_baseline.lock().unwrap();
        if let Some(ref mut baseline) = *guard {
            if !log.is_clean {
                baseline.is_clean = false;
            }
            baseline.boot_logs.push(log);
        } else {
            let elapsed = self.clock.elapsed_micros();
            let is_clean = log.is_clean;
            *guard = Some(GenesisBaseline {
                locked_micros: elapsed,
                boot_logs: vec![log],
                is_clean,
            });
        }
    }

    /// Retrieves a cloned snapshot of the active [`GenesisBaseline`] (State Omega_0).
    pub fn genesis_baseline(&self) -> Option<GenesisBaseline> {
        self.genesis_baseline.lock().unwrap().clone()
    }

    /// Returns `true` if the genesis baseline has been ingested and verified clean.
    pub fn is_baseline_clean(&self) -> bool {
        self.genesis_baseline
            .lock()
            .unwrap()
            .as_ref()
            .map(|b| b.is_clean)
            .unwrap_or(false)
    }

    /// Audits whether the operating system state matches the homeostatic equilibrium of Omega_0.
    /// Certified before allowing ACPI shutdown, hibernation, or CPU suspension.
    pub fn audit_shutdown_equilibrium(&self) -> Result<(), String> {
        let guard = self.genesis_baseline.lock().unwrap();
        let baseline = match guard.as_ref() {
            Some(b) => b,
            None => {
                return Err("Shutdown audit veto: No Genesis Inception baseline (Omega_0) registered.".to_string());
            }
        };

        if !baseline.is_clean {
            return Err("Shutdown audit veto: Genesis baseline contains unhealed hardware inception faults.".to_string());
        }
```

```
726
727         if self.has_failpoints() {
728             let count = self.failpoints().len();
729             return Err(format!(
730                 "Shutdown audit veto: System contains {} unhealed active failpoints; cannot return to Omega_0.",
731                 count
732             ));
733         }
734
735         Ok(())
736     }
737 }
738
739 // ============================================================================
740 // BLOCK 4: CLOSING BLOCK
741 // ( Default Trait & Subsystem Verification Unit Test Suite )
742 // ============================================================================
743
744 impl Default for DiagnosticLogger {
745     /// Constructs a silent [`DiagnosticLogger`] with live terminal printing disabled.
746     fn default() -> Self {
747         Self::new(false)
748     }
749 }
750
751 #[cfg(test)]
752 mod tests {
753     use super::*;
754     use std::fs;
755     use std::io::Write;
756     use crate::core::ternary::QuadState;
757
758     #[test]
759     fn test_logger_lifecycle_and_monotonicity() {
760         let logger = DiagnosticLogger::new(false);
761         assert_eq!(logger.events().len(), 0);
762         assert!(!logger.has_failpoints());
763
764         logger.log("SYSTEM", "Initialization started");
765         logger.log("SYSTEM", "Initialization completed");
766
767         let events = logger.events();
768         assert_eq!(events.len(), 2);
769         assert!(events[1].elapsed_micros >= events[0].elapsed_micros);
770     }
771
772     #[test]
773     fn test_logger_failpoint_trapping_and_remediations() {
774         let logger = DiagnosticLogger::new(false);
775
776         logger.failpoint(FailpointType::BoundaryMismatch, "LINTER", "Unclosed delimiter");
777         assert!(logger.has_failpoints());
778         assert_eq!(logger.failpoints().len(), 1);
779
780         let suggestion = FixSuggestion::new(
781             FixKind::FixBoundaryMismatch,
782             10,
783             15,
784             "]",
785             "}",
786             "Balance closing morphological boundary",
787         );
788         logger.failpoint_with_remediation(
789             FailpointType::BoundaryMismatch,
790             "PARSER",
791             "Delimiter mismatch corrected",
792             suggestion,
793         );
794
795         assert_eq!(logger.failpoints().len(), 2);
796         assert_eq!(logger.remediations().len(), 1);
797         assert_eq!(logger.remediations()[0].suggested_patch, "}");
798     }
799
800     #[test]
801     fn test_logger_flight_recorder_and_anomalous_frames() {
802         let logger = DiagnosticLogger::with_capacity(false, 4);
803
804         let frame1 = RollFrame::new(
805             100,
806             1,
```

```
            4,
            "INSCRIBE",
            (0x01, 0x00),
            (0x01, 0x00),
            QuadState::Proceed,
            0,
            None,
        );
        let frame2 = RollFrame::new(
            200,
            2,
            4,
            "TRANSPOSE",
            (0x02, 0x01),
            (0x00, 0x01),
            QuadState::Rollback,
            1,
            None,
        );

        logger.record_roll_frame(frame1);
        logger.record_roll_frame(frame2);

        let recent = logger.recent_roll_frames(2);
        assert_eq!(recent.len(), 2);

        let anomalous = logger.last_anomalous_frame().expect("Found anomaly");
        assert_eq!(anomalous.quad_state, QuadState::Rollback);
        assert_eq!(anomalous.failpoint_mask, 1);
    }

    #[test]
    fn test_logger_cognitive_notes_and_export() {
        let logger = DiagnosticLogger::new(false);

        let _note = logger.record_ai_observation(
            "SYNTAX",
            "Parsed canonical 4-block reel",
            "All delimiters balanced",
            &["syntax", "reel", "canonical"],
            Some("Opening { ... }"),
        );

        assert_eq!(logger.cognitive_notes().len(), 1);
        assert_eq!(logger.notes_by_category("SYNTAX").len(), 1);
        assert_eq!(logger.notes_by_tag("canonical").len(), 1);

        let scroll = logger.export_ai_memory_scroll();
        assert!(scroll.contains("CognitiveMemory"));
        assert!(scroll.contains("SYNTAX"));
    }

    #[test]
    fn test_logger_genesis_inception_and_shutdown_audit() {
        let logger = DiagnosticLogger::new(false);

        let raw_boot = r#"
            [BOOT] 16-Bit Genesis: Sector 0 loaded at 0x7C00
            [BOOT] Stage 2 PM: A20 Gate enabled, Flight Recorder armed at 0x18000
            [ALU ] Balanced Ternary (+1)+(-1)=0 Eq: Certified
            [REGS] 13 Regs Armed / Calibrated
        "#;

        let logs = logger.ingest_genesis_boot_log(raw_boot);
        assert_eq!(logs.len(), 4);
        assert!(logger.is_baseline_clean());
        assert!(logger.audit_shutdown_equilibrium().is_ok());

        // Introduce a failpoint; shutdown should be vetoed
        logger.failpoint(FailpointType::HardwareFault, "SILICON", "Bus voltage drift");
        assert!(logger.audit_shutdown_equilibrium().is_err());
    }

    #[test]
    fn test_logger_episodic_query_facade() {
        let temp_dir = std::env::temp_dir().join(format!(
            "omnicode_logger_facade_test_{}",
            std::time::SystemTime::now()
                .duration_since(std::time::UNIX_EPOCH)
                .unwrap()
                .as_nanos()
```

```
        ));
        fs::create_dir_all(&temp_dir).unwrap();

        let scroll_content = r#"
Metadata {
    session_id: "sess_facade_01",
    timestamp: "1792000000000000_0000_sess_facade_01",
    user_prompt: "Verify diagnostic logger facade",
}
Opening {
    Cell [ 1 ] & { 5 Trits :: 1 Byte },
}
Body {
    Cell [ 1 ] := 'X',
}
Closing {
    commit_status [ <PROCEED> ],
}
"#;
        let file_path = temp_dir.join("1792000000000000_0000_sess_facade_01.scroll");
        let mut file = fs::File::create(&file_path).unwrap();
        file.write_all(scroll_content.as_bytes()).unwrap();

        let logger = DiagnosticLogger::new(false);
        let session = logger.find_session(&temp_dir, "sess_facade_01");
        assert!(session.is_some());
        assert_eq!(session.unwrap().final_state, QuadState::Proceed);

        let criteria = SessionCriteria::new()
            .with_state(QuadState::Proceed)
            .with_keyword("facade");
        let query_res = logger.query_episodes(&temp_dir, &criteria);
        assert_eq!(query_res.sessions.len(), 1);
        assert_eq!(query_res.pass_rate(), 100.0);

        let _ = fs::remove_dir_all(&temp_dir);
    }
}
```

Inscriber Design: Container Physics, Capacity Bounds & Millennial Rotation

The Architectural Idea Behind the Inscriber Design & Finite Storehouse Law

> *[!NOTE]* ***The Container Invariant of the Storehouse (2 Kings 4:4)*** *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full."*
>
> *In the physics of the physical universe, a vessel cannot hold more than its volume permits. When a vessel is full, pouring more oil into it does not magically expand the vessel; it spills upon the floor. The Inscriber is founded upon this immutable truth: $\text{Occupancy} \le \text{Capacity}$. When a vessel reaches its defined boundary, it is sealed, set aside, rotated, and archived.*

1. The Core Philosophy: Why "Inscriber"?

In modern computing, operating systems and application runtimes treat diagnostic recording as a disposable side effect: files are named `debug.log`, `stdout.log`, or `systemd.journal`, continuously appended to without bounds until a disk fills up or an external daemon like `logrotate` aggressively purges them.

In OmniCode and MilleniumOS, this paradigm is rejected as mathematically reckless. The subsystem is deliberately named the **Inscriber** rather than a "Logger" for profound reasons:

```
flowchart TD
    subgraph CONVENTIONAL["Conventional Logging Paradigm"]
        C1["Volatile Execution"] --> C2["Unbounded Append-Only Text Stream"]
        C2 --> C3["Silent Disk Bloat & Server Exhaustion"]
        C3 --> C4["Panic, Data Corruption, & Crash"]
    end

    subgraph INSCRIBER["OmniCode Inscriber Paradigm"]
        I1["Deterministic Roll"] --> I2["Finite Vessel Allocation (Bounded Capacity)"]
        I2 --> I3["Physical Inscription (2 Kings 4:4 Container Law)"]
        I3 --> I4["When Full: Seal, Set Aside, & Rotate to Archive"]
        I4 --> I5["Continuous Homeostasis (Zero Disk Outages Across Centuries)"]
    end

    classDef red fill:#fee2e2,stroke:#ef4444,color:#991b1b;
    classDef green fill:#dcfce7,stroke:#22c55e,color:#166534;

    class C1,C2,C3,C4 red;
    class I1,I2,I3,I4,I5 green;
```

The Three Foundational Ideals of Inscription

1. **Inscription vs. Logging**: To "log" implies dropping unstructured scrapings into a heap. To **inscribe** implies carving deliberate, structured truth into enduring stone—echoing Exodus 31:18 (*"tables of stone, written with the finger of God"*). Every record inscribed into the storehouse has identity, provenance, invariant preconditions, and a covenant verification seal.
2. **The Millennial Horizon (MilleniumOS)**: An operating system intended to run autonomously across millennial operational horizons cannot tolerate unbounded state accumulation. Any system that leaks even one byte per day will eventually fail. The Inscriber guarantees mathematical boundedness: total disk occupancy across every partition is strictly constrained by finite capacity vectors.
3. **Hardware Truth Grounding**: The Inscriber does not invent synthetic metrics. It captures the physical reality of the machine: physical RAM landmarks (`0x0500`, `0x7C00`, `0x18000`, `0x108000`, `0x200000`), the 13 calibrated silicon registers, dual-rail saturation states `(1, 1)`, and CPU hardware traps (`#GP`, `#PF`).

2. Container Physics: $\text{Occupancy} \le \text{Capacity}$

The core mathematical law governing all storehouse partitions is the **Container Invariant**:

$$\forall \text{Partition } P, \quad \text{Occupancy}(P) \le \text{Capacity}(P)$$

In practical operating system terms, capacity is not a single number; it is a multi-dimensional vector:

$$\mathbf{C} = \langle N_{\text{max_files}}, \; B_{\text{max_file_bytes}}, \; V_{\text{max_partition_bytes}}, \; T_{\text{retention_micros}} \rangle$$

```
graph TD
    subgraph VECTOR["The Multi-Dimensional Capacity Vector"]
        C1["Count Bound: max_files_per_partition<br/>(e.g., 1,000 vessels per domain)"]
        C2["File Size Bound: max_bytes_per_file<br/>(e.g., 50 MB active file ceiling)"]
        C3["Volume Bound: max_partition_bytes<br/>(e.g., 1 GB partition disk cap)"]
        C4["Time Bound: max_retention_micros<br/>(Optional epoch expiration threshold)"]
    end

    VECTOR --> ENGINE["Inscriber Rotation Engine (rotation.rs)"]
    ENGINE --> DECISION{"Is Occupancy <= Capacity?"}

    DECISION -- Yes --> NOMINAL["Nominal Operation<br/>(Vessel Inscription Proceeds)"]
    DECISION -- No --> ENFORCE["Enforce Container Law<br/>1. Identify excess (Oldest first)<br/>2. Rollover active file (_partN)<

    classDef blue fill:#dbeafe,stroke:#3b82f6,color:#1e40af;
    classDef warn fill:#fef3c7,stroke:#d97706,color:#92400e;
    classDef pass fill:#dcfce7,stroke:#22c55e,color:#166534;

    class C1,C2,C3,C4 blue;
    class ENGINE,DECISION warn;
    class NOMINAL,ENFORCE pass;
```

1. Count Bounds (`max_files_per_partition`)

Each domain partition (e.g. `log/character/` , `scroll/number/` , `log/kernel/`) is allotted a finite vessel count ceiling. When the partition contains more than `max_files_per_partition` active vessels, the oldest excess vessels are sealed and migrated.

2. File Size Bounds (`max_bytes_per_file`)

Individual active log files cannot expand infinitely. If a long-running interactive terminal session or high-frequency game engine telemetry stream produces an active `.log` exceeding `max_bytes_per_file` , the rotation engine executes an **active rollover**:

$$\text{session.log} \longrightarrow \text{session_part1.log} \quad (\text{New writes open fresh session.log})$$

3. Partition Volume Bounds (`max_partition_bytes`)

Even if vessel count is within limits, large dumps could threaten disk reserves. The engine computes the cumulative byte volume of all active files in the directory. If total volume exceeds `max_partition_bytes` , the oldest files are rotated out until total occupancy drops below capacity.

---

3. The Lifecycle of an Inscribed Vessel

Every telemetry vessel traverses a deterministic, state-machine lifecycle from creation to archival:

```
stateDiagram-v2
    [*] --> Active: inscribe_telemetry_log()
    Active --> Full: Byte limit reached (max_bytes_per_file)
    Full --> Rolled: rotate_active_file() -> _partN.log
    Rolled --> Active: Fresh vessel opened for new turns

    Active --> Sealed: Session commits (QuadState affirmed)
    Sealed --> Candidate: Partition capacity exceeded
    Candidate --> Archived: archive_old_files == true (Moved to archive/)
    Candidate --> Pruned: archive_old_files == false (Unlinked)

    Archived --> Manifested: rotation_manifest.json generated
    Manifested --> [*]
```

1. Inception (`Active`)

The vessel is opened in its category directory (`storehouse/log/{category}/{timestamp}_{session_id}.log`). Incoming telemetry events, microsecond stamps, and failpoints are formatted and written.

2. Sealing Gate (`Sealed`)

Upon completion of the episode, the 4th block (`Closing`) asserts the fourfold commit state (`+1:PROCEED` , `0:AWAIT` , `NULL:HALT` , `-1:ROLLBACK`). The file is physically flushed and closed. It is now an immutable historic record.

3. Rotation Enforcement (`Candidate`)

During periodic background maintenance or after each session commit, `InscriberEngine::enforce_partition_capacity()` audits the directory. If count or byte capacity is breached, the oldest sealed vessels are selected for rotation.

4. Archival or Pruning (`Archived` / `Pruned`)

- If `archive_old_files` is `true` (standard policy), files are atomically renamed into the `archive/` subfolder.
- If `archive_old_files` is `false` (restricted edge environments), files are unlinked permanently.

5. Manifest Recording (`Manifested`)

An atomic `rotation_manifest.json` is generated in the archive directory, recording:

- Exact unix microsecond timestamp of the rotation cycle.
- Total files rotated, archived, and pruned.
- Cumulative byte volume reclaimed from the active partition.
- Array of vessel filenames moved.

---

4. Deterministic Vessel Naming as a Zero-Allocation B-Tree Index

Traditional logging systems require complex database engines (Elasticsearch, SQLite, or Loki) just to find logs by date or session. The Inscriber achieves $O(1)$ search performance directly on standard POSIX filesystems through **Deterministic Vessel Naming**:

```
Standard Session Vessel:
  1792000000000000_sess_alpha_99.scroll
  |              | |           | |    |
  +-- Timestamp -+ +- Session -+ + Ext+

Kernel Landmark Vessel:
  1792000000100000_0x00018000_STAGE2-PM_kernel_flight.log
  |              | |         | |       | |           | |  |
  +-- Timestamp -+ +Landmark+ +- Tier -+ +- Session -+ +Ext+

Rotated Part Vessel:
  1792000000200000_sess_omega_part3.log
  |              | |         | |   | |  |
  +-- Timestamp -+ +-Session+ +Part+ +Ext+
```

Strict Prohibition of Middle Dots

Notice that filenames strictly use underscores (`_`) or hyphens (`-`), never internal dots:

- **Invalid**: `session.part1.log` (Ambiguous file extensions, breaks MIME parsers, confuses language tooling).
- **Valid**: `session_part1.log` (Clean, unambiguous single dot representing the file extension).

The Filesystem as an Instant Range Index

Because ISO-aligned numeric timestamps (`timestamp_micros`) are placed at the very beginning of each filename, standard directory listing commands (`readdir` , `ls`) return files in strict chronological order:

- **Range Scans (`range_scan(t1, t2)`)**: Evaluated via simple lexicographical slice comparison on file stems without opening the file.
- **Session Lookups (`find_by_session(id)`)**: Evaluated via hash map index over `parts[1..]` in $O(1)$ time.

---

5. Bridging Track 0 Silicon to Track 1 Storehouses

A unique strength of the Inscriber is its direct integration with the **Track 0 Bare-Metal Kernel** (`<TRACK 0> : {Silicon & Bare-Metal Kernel}/`):

```
flowchart LR
    subgraph T0["Track 0 Bare-Metal Hardware"]
        S2["Stage 2 PM (0x18000)"]
        UEFI["UEFI GOP (0x200000)"]
        TRAP["CPU Trap (#GP / #PF)"]
    end

    subgraph RAW["Raw 16-Byte Hardware Records"]
        REC["[ event_id (u32), arg1 (u32), arg2 (u32), status (u32) ]"]
    end

    subgraph INSCRIBER["Inscriber Translation Layer"]
        BIN["inscribe_raw_flight_recorder_binary()<br/>(Saves raw byte stream to .bin)"]
        LOG["inscribe_kernel_flight_log()<br/>(Translates to human ASCII journal)"]
        SCR["inscribe_kernel_audit_scroll()<br/>(Translates to 4-block OmniCode reel)"]
    end

    S2 --> REC
    UEFI --> REC
    TRAP --> REC

    REC --> BIN
    REC --> LOG
    REC --> SCR

    classDef silicon fill:#334155,stroke:#64748b,color:#f8fafc;
    classDef raw fill:#7c2d12,stroke:#ea580c,color:#ffedd5;
    classDef inscriber fill:#065f46,stroke:#10b981,color:#ecfdf5;

    class S2,UEFI,TRAP silicon;
    class REC raw;
    class BIN,LOG,SCR inscriber;
```

The 16-Byte Atomic Memory Entry

Bare-metal assembly cannot allocate strings or JSON objects. At landmarks `0x00018000` (Stage 2 PM) and `0x00200000` (Native UEFI), the kernel writes fixed 16-byte structs:

```
typedef struct {
    UINT32 event_id;  // Milestone (e.g. 0x0004 = EVT_ALU_VERIFIED)
    UINT32 arg1;      // Primary argument (CR0 value or landmark address)
    UINT32 arg2;      // Secondary argument (Stack landmark or size)
    UINT32 status;    // Hardware status (0 = OK, non-zero = failpoint)
} UefiLogEntry;
```

The Inscriber's `log_writer.rs` decodes these 16-byte atomic entries into canonical event names (`EVT_ALU_VERIFIED` , `EVT_REGS_ARMED` , `EVT_SAFE_MODE`) and produces both human-readable tables and raw `.bin` files for hardware emulator validation.

---

6. The 13 Calibrated Silicon Registers

When a kernel event or crash occurs, the Inscriber dumps all **13 Calibrated Silicon Registers** established in Track 0 and Track 1:

Register	Architectural Designation	Symbolic Meaning in Inscriber
`%head`	Cognitive Focus Landmark	Memory location of active cognitive attention
`%home`	Moral Origin Anchor	Root landmark of origin (Track 0 root: `0x0500`)
`%turn`	Monotonic Step Counter	Current execution turn or tick count
`%spool`	I/O Communication Port	Base hardware port (e.g. COM1 UART `0x03F8`)
`%spool_in`	Inbound Stream Head	Read pointer for incoming character runes
`%spool_out`	Outbound Stream Head	Write pointer for outgoing terminal projections

Register	Architectural Designation	Symbolic Meaning in Inscriber
`%frame`	Execution Frame	Active stack or partition call frame boundary
`%rsp`	Stack Pointer	Physical machine stack top
`%cr0`	Control Register 0	CPU protection, paging, and math coprocessor flags
`%cr3`	Page Directory Base	Physical base address of paging translation tables
`%cr4`	Control Register 4	Architectural extensions (PAE, OS support)
`%rflags`	CPU Processor Flags	Hardware interrupts, parity, and direction flags
`%rax` .. `%rbp`	General CPU Registers	Computational operand registers

7. Dual-Rail Saturation Physics & Re-Inception

In balanced ternary computing, two physical binary rails represent one ternary trit:

- `(0, 0)` = **0:AWAIT**: Homeostasis, neutral equilibrium, standby, pause.
- `(0, 1)` = **-1:ROLLBACK**: Negative rail high. Rewind causal tape, invert time.
- `(1, 0)` = **+1:PROCEED**: Positive rail high. Forward commit motion.
- `(1, 1)` = **NULL:HALT**: Both rails saturated high! Zero free data capacity.

```
stateDiagram-v2
    direction LR
    AWAIT: (0, 0) 0:AWAIT<br/>Homeostasis / Standby
    PROCEED: (1, 0) +1:PROCEED<br/>Forward Commit
    ROLLBACK: (0, 1) -1:ROLLBACK<br/>Causal Tape Rewind
    HALT: (1, 1) NULL:HALT<br/>Collision Saturation

    AWAIT --> PROCEED: Stimulus Evaluated
    PROCEED --> AWAIT: Roll Complete
    PROCEED --> ROLLBACK: Boundary Caveat Tripped
    ROLLBACK --> AWAIT: State Restored to Origin

    PROCEED --> HALT: Rail Collision Trap
    HALT --> AWAIT: Safe-Mode Re-Inception<br/>(Machine Prohibited From Flatlining)
```

> *[!CAUTION]* ***Why HALT Restarts Instead of Flatlining*** *A dead, flatlined machine violates the core mission of MilleniumOS. When both rails saturate at* `(1, 1)`*, zero free data capacity remains. The collision sensor tripwire catches the saturation, freezes the 13 silicon registers, writes the crash frame to disk, and* ***re-incepts the machine back to homeostasis (*** `0:AWAIT` ***)*** *at a verified safe landmark (* `0x00018000` *or* `0x00007C00` *). The machine continues operating without human reboot.*

8. Summary

The Inscriber design transforms telemetry from an uncontrolled, volatile exhaust pipe into a structured, bounded, and sacred physical storehouse:

1. **Container Law is Absolute**: Bounded file counts, bounded byte sizes, and bounded partition capacities guarantee millennial survivability.
2. **Deterministic Naming is Zero-Cost Indexing**: Formatted paths enable instant $O(1)$ lookup and temporal scans without database overhead.
3. **Dual Streams Serve Dual Minds**: Human engineers receive beautiful, readable ASCII journals; autonomous AI models receive pure, executable 4-block cognitive scrolls.
4. **Silicon is Grounded**: Raw 16-byte hardware records from Track 0 are preserved, decoded, and unified with higher-level machine cognition.

Log vs. Scroll: Dual-Stream Telemetry & Episodic Memory Architecture

Human-Facing Diagnostic Logs vs. Machine-Facing Cognitive Memory Tapes

> *[!NOTE] **The Principle of Dual Inscription** In conventional operating systems, logs are written solely for humans to parse after a crash occurs. In OmniCode, every computational episode produces two synchronized streams:*
>
> 1. ***The `.log` Vessel**: An observational ASCII diagnostic journal formatted for the human engineer.*
> 2. ***The `.scroll` Vessel**: An executable, syntactically conforming 4-Block OmniCode program formatted for the autonomous AI Cortex (`CPI-SI`).*
>
> *One is read by the eye; the other is executed by the mind.*

1. The Epistemic Dilemma: Why Conventional Logs Fail AI Systems

For over six decades, software engineering has treated diagnostic logging as an afterthought: arbitrary strings dumped into flat text files via `printf()` , `console.log()` , or `syslog` . While human developers can squint at unstructured error messages and infer what happened, this model completely breaks down when applied to autonomous AI systems and self-healing kernels.

When an artificial intelligence agent or compiler is asked to learn from past errors using traditional logs, it faces severe epistemic barriers:

```
flowchart TD
    subgraph CONVENTIONAL["Conventional Logging (Flawed for AI)"]
        APP1["Runtime Execution"] --> RAW["Unstructured Text Dumps<br/>'Error: connection timed out at 0x7FFF'"]
        RAW --> REGEX["Complex RegEx Scrapers & Log Parsers"]
        REGEX --> LOST["Lost Context & Hallucination Risk<br/>(AI cannot reconstruct execution state)"]
    end

    subgraph OMNICODE["OmniCode Dual-Stream Inscription"]
        APP2["Runtime Execution"] --> DUAL["Inscriber Engine"]
        DUAL --> LOGF["telemetry.log<br/>(Human-Readable ASCII Triage)"]
        DUAL --> SCRF["episode.scroll<br/>(Executable 4-Block OmniCode Tape)"]

        LOGF --> HUMAN["Human Systems Engineer<br/>(Instant Triage & Reg Dumps)"]
        SCRF --> REPLAY["Autonomous AI Cortex (CPI-SI)<br/>(Replay, Transduce, & Learn)"]
    end

    classDef red fill:#fee2e2,stroke:#ef4444,color:#991b1b;
    classDef blue fill:#dbeafe,stroke:#3b82f6,color:#1e40af;
    classDef green fill:#dcfce7,stroke:#22c55e,color:#166534;

    class APP1,RAW,REGEX,LOST red;
    class APP2,DUAL blue;
    class LOGF,SCRF,HUMAN,REPLAY green;
```

The Three Fatal Flaws of String Logging for Machine Learning

1. **Non-Executable Traces**: A string like `"Failed to allocate cell"` cannot be evaluated by an abstract syntax tree. It has lost its types, its boundary rankings, and its algebraic symmetry.
2. **Context Evaporation**: Traditional logs record symptoms, not preconditions. They do not preserve whether the balanced ternary arithmetic logic unit (ALU) was in equilibrium (`(+1) + (-1) <=> 0`), what silicon registers held, or what the tripwire thresholds were.
3. **No Replay Capability**: You cannot take a standard Apache or Linux kernel log and feed it directly into an execution engine to reproduce the computational state. In contrast, an OmniCode `.scroll` is a valid program that can be evaluated directly on the virtual terminal.

2. Side-by-Side Architectural Comparison

The following matrix contrasts the design decisions behind the `.log` and `.scroll` formats:

Architectural Vector	Traditional Developer Log (`.log`)	Canonical Cognitive Scroll (`.scroll`)
Primary Audience	Human Systems Engineers & Kernel Developers	Autonomous AI Model, AI Cortex, & Evaluator (`CPI-SI`)
Data Format	Clear ASCII text, tabular registers, and labeled metrics	Syntactically conforming 4-Block OmniCode program
Syntax Standard	Plain English headers, dividers, and remediation notes	Canonical OmniCode syntax (`character.omni` , `number.omni`)
Execution Model	Observational (Read-only, static text scanning)	Executable (Can be re-transduced, assembled, and run)
Naming Convention	`<timestamp>_<session_id>.log`	`<timestamp>_<session_id>.scroll`
State Representation	Descriptive: `+1:PROCEED (Forward commit motion)`	Algebraic: `commit_status [ <PROCEED> ]`
Failpoint Handling	Diagnostic explanations with file/line remediation proposals	Declarative caveat branches (`! GUARD --> [ ROLLBACK ]`)
Silicon Depth	Detailed tabular view of the 13 Silicon Registers	Declarative hardware landmark imports (`::@ 0x18000`)
Storehouse Partition	`storehouse/log/{category}/`	`storehouse/scroll/{category}/`
Query Engine Role	Grep-friendly, human inspection, post-incident triage	$O(1)$ multi-key indexed retrieval via `EpisodicQueryEngine`

3. The Anatomy of a Traditional Developer Log (`.log`)

The `.log` file is engineered for high cognitive throughput during debugging sessions. When an engineer opens a `.log` file, every piece of information is structured to minimize cognitive load:

```
================================================================================
           OMNICODE AI EXECUTION & DIAGNOSTIC TELEMETRY LOG
================================================================================
Session ID:       sess_20261017_alpha
Timestamp:        1792000000000000_0000 (+1792000000000000 µs)
Execution Time:   4250 µs (4.25 ms)
Final State:      +1:PROCEED (Forward commit motion confirmed)
Input Prompt:     "Inscribe atomic character glyph into memory cell"
================================================================================

--------------------------------------------------------------------------------
1. SENSORY LOG EVENTS & TRAPPED FAILPOINTS
--------------------------------------------------------------------------------
Status: 1 FAILPOINTS TRAPPED

  #  | Elapsed   | Channel  | Classification | Details
-----+-----------+----------+----------------+---------------------------------
   1 |   1200 µs | SYNTAX   | DELIMITER_LEAK | Unclosed boundary delimiter at L12
     -> Remediation Proposal: [StructuralPatch] at Line 12: Close cell delimiter
        Target Token: "{ Cell ]" => Suggested Patch: "[ Cell ]"

--------------------------------------------------------------------------------
2. IMMUNE SYSTEM RESTORATIONS & SELF-HEALING
--------------------------------------------------------------------------------
Status: RESTORED
  • Restorator applied auto-repair: balanced delimiter at line 12.
  • Code successfully re-verified against 4-block structural invariants.

--------------------------------------------------------------------------------
3. SYNTHESIZED TURING REEL (OMNICODE SOURCE)
--------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe ] :
    { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]

--------------------------------------------------------------------------------
4. MACHINE DISCOURSE RESPONSE
--------------------------------------------------------------------------------
```

```
"Glyph successfully inscribed into defined cell."
================================================================================
```

Key Elements of the `.log` Design

- **No Pseudo-Code Confusion**: Headers and status blocks are written in plain, human-friendly English.
- **Instant Triage Metrics**: Monotonic timestamps are presented in both microseconds (`µs`) and milliseconds (`ms`).
- **Direct Actionability**: If a failpoint is trapped, the log does not merely complain; it provides a machine-calculated **Remediation Proposal** showing the exact line, column, target token, and proposed replacement patch.
- **Hardware Register Snapshots**: For bare-metal Track 0 kernel events, the log includes an aligned table of all 13 calibrated silicon registers (`%head` , `%home` , `%turn` , `%spool` , etc.) with their physical meanings explained.

---

4. The Anatomy of a Cognitive Scroll (`.scroll`)

The `.scroll` file is not a text document; it is an **episodic memory artifact**. When the OmniCode AI Cortex awakens or reflects upon past interactions, it reads `.scroll` files as executable memory tapes.

Because it is written in pure OmniCode syntax, it adheres to the same rules established in the core language lexicon:

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Scroll: < EpistemicTrace { sess_20261017_alpha } > )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Episodic execution chronicle generated by OmniCode AI Model.
#   - Session ID: sess_20261017_alpha
#   - Monotonic Timestamp: 1792000000000000_0000 µs
#   - Cognitive Mode: NeuralSymbolicTransduction
#   - Witness: Scripture Exodus 31:18 KJV
# ================================================================================

= [Scroll]: < EpistemicTrace { sess_20261017_alpha } > & [ Process { NeuralSymbolicDialogue } ]
    ( Inscribe atomic character glyph into memory cell )
{
    [ Version ]:   { "1.0.0" }
        ( Protocol Version 1.0.0 ),

    [ Created ]:   { "2026-10-17" }
        ( Inscribed during active system execution ),

    [ System ]:    { OmniCode { AI.Model } }
        ( Generated by OmniCode Neural-Symbolic Runtime Engine ),

    [ Session ]:   { "sess_20261017_alpha" }
        ( Unique session identifier for this epistemic episode ),

    session_id:    < "sess_20261017_alpha" >,
    timestamp:     < "1792000000000000_0000" >,

    [ Duration ]:  { "4250 µs" }
        ( Measured execution duration along monotonic causal clock ),

    [ Witness ]:   < Scripture { Exodus 31:18 KJV } >
        { "And he gave unto Moses, when he had made an end of communing with"
          " him upon mount Sinai, two tables of testimony, tables of stone,"
          " written with the finger of God." }
        ( Invariant divine covenant witness )
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environmental Setup & Initial Invariants )
```

```
# ============================================================================
Opening
    ( Hardware Invariants, Preconditions, and Initial Context )
{
    [ Imports ]: { Core :: Arithmetic } & { Core :: Logic } & { Core :: Vectors }
        ( Lexicon standard library modules armed for evaluation ),

    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( Balanced ternary ALU equilibrium verified before roll execution ),

    [ Prompt ]: { "Inscribe atomic character glyph into memory cell" }
        ( Natural language discourse or inquiry submitted by operator )
}

user_prompt < "Inscribe atomic character glyph into memory cell" >

# ============================================================================
# BLOCK 3: BODY BLOCK
# ( Chronicle of Execution & Turing Reel )
# ============================================================================
Body [
    # ROLL 1: SYNTHESIZED TURING REEL
    [ ROLL 1 ] : [ Inscribe ] :
        { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]
        ( Inscribe external glyph into Cell yielding Character-prime )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Sealing Gate: Commit State & Verification Proof )
# ============================================================================
Closing
    ( Sealing Gate: Affirm fourfold commit status and machine utterance )
{
    commit_status [ <PROCEED> ],

    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Status ] = PROCEED } --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Status ] = AWAIT }   --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status ] = HALT }    --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Status ] = ROLLBACK } --> [ -1:ROLLBACK ]
    ]
    ( Fourfold commit status affirmance ),

    [ Machine Utterance ]: { "Glyph successfully inscribed into defined cell." }
        ( Final conversational response synthesized by model )
}
```

Why `.scroll` Uses Pure OmniCode Syntax

1. **Self-Referential Completeness**: The AI model speaks the same language it executes. When the model reads a past `.scroll`, it is reading its own native thought syntax.
2. **Deterministic Evaluation**: The runtime parser can ingest a `.scroll` file and evaluate it using the standard `Evaluator`. There is zero translation layer or parsing ambiguity.

3. **No Dot Naming**: In strict adherence to `character.omni` and `number.omni`, names use nested brackets (`< EpistemicTrace { ID } >`) and double-colon namespaces (`Core :: Arithmetic`), never invalid object-oriented dot notation.

---

5. The Dual Inscription Workflow (`inscribe_dual_episode`)

When the host engine executes a command or AI dialogue turn, the `InscriberEngine` orchestrates simultaneous inscription of both streams:

```
sequenceDiagram
    autonumber
    actor User as Operator / Engineer
    participant Host as Runtime Evaluator
    participant Log as log_writer.rs
    participant Scroll as scroll_writer.rs
    participant Rot as rotation.rs
    participant Disk as Physical Storehouse

    User->>Host: Execute Prompt / Reel
    Host->>Host: Traps Failpoints & Computes QuadState

    par Inscribe Human Log
        Host->>Log: inscribe_telemetry_log(...)
        Log->>Disk: Write log/category/timestamp_session.log
    and Inscribe Cognitive Scroll
        Host->>Scroll: inscribe_canonical_scroll(...)
        Scroll->>Disk: Write scroll/category/timestamp_session.scroll
    end

    par Enforce Capacity
        Host->>Rot: enforce_partition_capacity("log", false)
        Rot->>Disk: Rotate excess logs to archive/
    and Enforce Scroll Bounds
        Host->>Rot: enforce_partition_capacity("scroll", true)
        Rot->>Disk: Rotate excess scrolls to archive/
    end

    Host-->>User: Return Execution Result (+1:PROCEED)
```

Atomic Synchronization Guarantees

- **Identical Timestamp Prefix**: Both the `.log` and `.scroll` share the exact same microsecond timestamp prefix (e.g. `1792000000000000_sess_alpha`). This guarantees $O(1)$ pairing when querying.
- **Identical Commit Polarity**: Both files record the identical fourfold commit status (`QuadState`). If the `.log` records a rollback, the `.scroll` records `commit_status [ <ROLLBACK> ]` .
- **Independent Archival**: Because `.log` files are larger in text volume while `.scroll` files are compact symbolic tapes, the rotation engine monitors their byte capacities independently, ensuring that verbose debug logs can be archived without evicting critical cognitive memories prematurely.

---

6. How the Query Subsystem Leverages the Duality

The `EpisodicQueryEngine` in `query/` takes full advantage of this dual-stream architecture:

1. **Fast Indexing from `.scroll` Files**: Because `.scroll` files have a strictly defined 4-block header, the query engine scans only the first 20 lines of each `.scroll` to extract the session ID, timestamp, prompt, and QuadState.
2. **On-Demand Pairing with `.log`** : If an engineer asks to see the detailed hardware registers or sensory tripwire traces for a session, the query engine uses `resolve_vessel_path` to locate the paired `.log` file in the adjacent partition directory.
3. **Zero Full-Disk Scans**: By organizing partitions into `log/{category}/` and `scroll/{category}/` , range scans (e.g. `range_scan(start, end)`) execute directly against directory stems in $O(1)$ without reading file bodies.

---

7. Summary

The distinction between `.log` and `.scroll` is the distinction between **human telemetry** and **machine memory**:

- The `.log` is your diagnostic workbench: clear, informative, readable, and focused on helping the human understand what the machine did.
- The `.scroll` is the machine's cognitive episodic memory: executable, mathematically bounded, typed, and structured to allow the autonomous system to remember, learn, and evolve across continuous operation.

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/README.md | MARKDOWN / DOC | 410 LINES | 2198 WORDS | 18.6 KB

Storehouse Inscriber: Telemetry, Episodic Vessels & Rotation Engine

The "Inscribe" Physical Storage Organ of the DAR Cybernetic Loop

> *[!NOTE]* ***Container Law of the Storehouse (2 Kings 4:4)***
> *"And thou shalt go in, and shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full."*
>
> *In the OmniCode architecture, memory and telemetry are never treated as infinite garbage-collected heaps. Storage is governed by physical container law: $\text{Occupancy} \le \text{Capacity}$. When a vessel reaches its defined capacity, it is sealed, set aside, rotated, and archived.*

1. Executive Summary & Epistemic Role

The **Inscriber** subsystem (`src/diagnostics/logger/inscriber/`) is the physical recording organ of the OmniCode Diagnostics system. While the `universal` event bus routes events in memory and `failpoint` catches anomalies at boundaries, the **Inscriber** is tasked with a sacred duty: committing machine experiences, sensory telemetry, and bare-metal Track 0 kernel events permanently into durable filesystem vessels.

Every execution episode in OmniCode produces two distinct, complementary artifacts:

1. **Traditional Developer Telemetry (`.log`)**: A clean, highly readable ASCII journal designed for human engineers, detailing microsecond timings, sensory tripwires, concrete remediation proposals, and bare-metal Track 0 silicon register snapshots.
2. **Canonical Cognitive Scrolls (`.scroll`)**: Fully conforming, executable 4-Block OmniCode programs representing episodic memory traces for the AI Cortex (`CPI-SI`), structured according to the canonical syntax established in `character.omni` , `character.tape` , `number.omni` , and `number.tape` .

To prevent unbounded storage consumption across millennial operating horizons (MilleniumOS), the Inscriber enforces an automated, multi-dimensional **Rotation Engine** bounding vessel count, file byte volume, and total partition disk occupancy.

Foundational Inscriber Treatises

For exhaustive architectural proofs and deep-dive documentation, consult the companion treatises in this folder:

- **Log vs. Scroll: Dual-Stream Architecture**: Explores the profound epistemic difference between human diagnostic text (`.log`) and machine-executable episodic memory tapes (`.scroll`), explaining why traditional string logging fails autonomous AI agents and how `.scroll` enables true continuous self-learning (`CPI-SI`).
- **Inscriber Design & Container Physics**: Details the philosophical and physical idea behind the Inscriber design, the mathematical container law $\text{Occupancy} \le \text{Capacity}$ (2 Kings 4:4), deterministic zero-cost filesystem indexing, and Track 0 silicon hardware bridging.

```
flowchart TD
    subgraph SENSORY["Sensory Capture & Ingestion"]
        T0["Track 0 Bare-Metal Kernel<br/>(16-Byte Flight Logs, 13 Silicon Regs, Dual-Rail)"]
        T1["Track 1 Host Engine<br/>(Sensory Tripwires, Failpoints, AI Dialogue)"]
    end

    subgraph INSCRIBER["Storehouse Inscriber Subsystem"]
        NAME["naming.rs<br/>Deterministic Vessel Paths"]
        LOGW["log_writer.rs<br/>Readable ASCII & Kernel Logs"]
        SCRW["scroll_writer.rs<br/>Canonical 4-Block .scroll Files"]
        ROTE["rotation.rs<br/>Container Law & Capacity Enforcer"]
    end

    subgraph STOREHOUSE["Durable Storehouse Partitions"]
        VLOG["log/{category}/<br/>telemetry.log"]
        VSCR["scroll/{category}/<br/>episode.scroll"]
        VBIN["binary/{category}/<br/>flight_dump.bin"]
        VARC["archive/<br/>rotation_manifest.json"]
    end
```

```
    T0 --> LOGW
    T0 --> SCRW
    T1 --> LOGW
    T1 --> SCRW

    NAME --> LOGW
    NAME --> SCRW

    LOGW --> VLOG
    LOGW --> VBIN
    SCRW --> VSCR

    ROTE -->|Enforces Occupancy <= Capacity| VLOG
    ROTE -->|Enforces Occupancy <= Capacity| VSCR
    ROTE -->|Migrates Old Vessels| VARC

    classDef blue fill:#2563eb,stroke:#1d4ed8,color:#ffffff;
    classDef amber fill:#d97706,stroke:#b45309,color:#ffffff;
    classDef green fill:#059669,stroke:#047857,color:#ffffff;
    classDef purple fill:#7c3aed,stroke:#6d28d9,color:#ffffff;

    class T0,T1 blue;
    class NAME,LOGW,SCRW,ROTE amber;
    class VLOG,VSCR,VBIN,VARC green;
```

2. The 4-Block Structure Across All Vessels

In OmniCode, every source file, kernel module, documentation treatise, and inscribed episodic scroll strictly obeys the **4-Block Architecture**. This ensures identical cognitive geometry across humans, language transcribers, and AI models:

Block	Structural Name	Inscriber Implementation Role
Block 1	**Metadata Block**	Identifies the vessel, author, protocol version, session ID, monotonic microsecond timestamp, duration, and covenant scripture witness.
Block 2	**Opening Block**	Configures the execution environment, standard library imports, prompt intent, hardware memory landmarks, and initial equilibrium preconditions `[ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]`.
Block 3	**Body Block**	Declarative chronicle of executed rolls, synthesized Turing reel, recorded cognitive notes, and milestone progressions.
Block 4	**Closing Block**	Sealing gate asserting the fourfold commit status (`+1:PROCEED` , `0:AWAIT` , `NULL:HALT` , `-1:ROLLBACK`), verification proofs, and final conversational utterance.

Canonical `.scroll` Syntax

In conformity with `character.omni` and `number.omni`, complex names do **not** use arbitrary dot syntax (such as `EpistemicTrace.session`). Instead, OmniCode uses nested brackets and typed associations:

```
# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Scroll: < EpistemicTrace { Session_Alpha_01 } > )
# ============================================================================
= [Scroll]: < EpistemicTrace { Session_Alpha_01 } > & [ Process { NeuralSymbolicDialogue } ]
    ( Inscribe atomic character glyph into memory cell )
{
    [ Version ]:   { "1.0.0" }
        ( Protocol Version 1.0.0 ),

    [ Session ]:   { "Session_Alpha_01" }
        ( Unique session identifier for this epistemic episode ),
```

```
    session_id:    < "Session_Alpha_01" >,
    timestamp:     < "1792000000000000_0000" >,

    [ Duration ]:  { "4250 µs" }
        ( Measured execution duration along monotonic causal clock ),

    [ Witness ]:   < Scripture { Exodus 31:18 KJV } >
        { "written with the finger of God" }
        ( Invariant divine covenant witness )
}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening
    ( Hardware Invariants, Preconditions, and Initial Context )
{
    [ Imports ]: { Core :: Arithmetic } & { Core :: Logic } & { Core :: Vectors }
        ( Lexicon standard library modules armed for evaluation ),

    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( Balanced ternary ALU equilibrium verified before roll execution )
}

user_prompt < "Inscribe atomic character glyph into memory cell" >

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    # ROLL 1: SYNTHESIZED TURING REEL
    [ ROLL 1 ] : [ Inscribe ] :
        { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]
        ( Inscribe external glyph into Cell yielding Character-prime )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing
    ( Sealing Gate: Affirm fourfold commit status and machine utterance )
{
    commit_status [ <PROCEED> ],

    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Status ] = PROCEED } --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Status ] = AWAIT }   --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status ] = HALT }    --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Status ] = ROLLBACK } --> [ -1:ROLLBACK ]
    ]
    ( Fourfold commit status affirmance ),

    [ Machine Utterance ]: { "Glyph successfully inscribed into defined cell." }
```

```
        ( Final conversational response synthesized by model )
}
```

3. Human-Readable Telemetry Logs (`.log`)

> *[!TIP]* ***Why Human Readability Matters***
> *Diagnostic logs should never look like indecipherable compiler dump spaghetti. A systems engineer reviewing a crash at 3:00 AM needs immediate clarity: What happened? When did it happen? What were the hardware registers? Did the tripwire catch it? What is the recommended fix?*

Traditional telemetry inscribed by `inscribe_telemetry_log` and `inscribe_kernel_flight_log` adheres to a clean, spacious ASCII format:

```
================================================================================
            MILLENIUMOS TRACK 0 SILICON FLIGHT RECORDER LOG
================================================================================
Session ID:         kflight_2026_01
Monotonic Clock:    1,250 µs since silicon inception
Hardware Landmark:  0x00018000 (Stage 2 Protected Mode Flight Recorder Landmark)
Boot Ladder Stage:  STAGE2_PM (32-Bit / 16-Trit Protected Mode)
Dual-Rail State:    (1, 0) -> +1:PROCEED (Positive rail high; forward commit motion active)
Hardware CPU Trap:  None (#0000 Nominal)
Diagnostic Note:    Stage 2 Protected Mode Armed & Calibrated
================================================================================

--------------------------------------------------------------------------------
1. THE 13 CALIBRATED SILICON REGISTERS & CPU CONTROL ENVELOPE
--------------------------------------------------------------------------------
The 7 Named Cognitive Silicon Registers:
  %head:      0x0000000000001000   (Cognitive focus landmark)
  %home:      0x0000000000000500   (Moral origin anchor / Track 0 root)
  %turn:      0x000000000000002A   (Monotonic step/turn counter)
  %spool:     0x00000000000003F8   (I/O communication port base)
  %spool_in:  0x00000000000003F8   (Inbound stream head pointer)
  %spool_out: 0x00000000000003F8   (Outbound stream head pointer)
  %frame:     0x0000000000018000   (Active execution frame landmark)

CPU Control & State Registers:
  %rsp:       0x0000000000006FFC   (Active stack pointer landmark)
  %cr0:       0x0000000080010033   (Control register 0: Protection & Paging)
  %cr3:       0x0000000000001000   (Page directory base register)
  %cr4:       0x0000000000000020   (Control register 4: Architectural extensions)
  %rflags:    0x0000000000000202   (Processor status & interrupt flags)

--------------------------------------------------------------------------------
2. RAM LOG FLIGHT RECORDER (16-BYTE ATOMIC SILICON CHUNKS)
--------------------------------------------------------------------------------
Total Inscribed Records: 3

   # | Event ID | Canonical Event Name | Argument 1   | Argument 2   | Status
-----+----------+----------------------+--------------+--------------+---------
   1 |  0x0001  | EVT_GENESIS_ENTRY    |  0x00007C00  |  0x00000000  | 0x0000
   2 |  0x0004  | EVT_ALU_VERIFIED     |  0x00018000  |  0x00000000  | 0x0000
   3 |  0x0005  | EVT_REGS_ARMED       |  0x00018000  |  0x0000000D  | 0x0000

--------------------------------------------------------------------------------
3. DUAL-RAIL HARDWARE SATURATION & RE-INCEPTION AUDIT
--------------------------------------------------------------------------------
Diagnosis: STABLE EQUILIBRIUM
  • Forward commit polarity (+1:PROCEED) intact.
```

```
  • Balanced ternary ALU equilibrium (+1) + (-1) = 0 affirmed.
  • Machine operating within nominal silicon envelope.
================================================================================
```

4. Track 0 Silicon Integration Matrix

The Inscriber mirrors the exact memory landmarks and binary structures established across Track 0 bare-metal kernels:

```
flowchart LR
    MBR["BIOS MBR<br/>(0x7C00)"] --> S2["Stage 2 PM<br/>RAM Log (0x18000)"]
    S2 --> LM["Long Mode 64-Bit<br/>Kernel Base (0x108000)"]
    LM --> UEFI["Native UEFI GOP<br/>RAM Log (0x200000)"]

    UEFI --> CPI["32-Trit CPI-SI<br/>Cognitive Arena"]

    classDef silicon fill:#1e293b,stroke:#475569,color:#f8fafc;
    class MBR,S2,LM,UEFI,CPI silicon;
```

Physical RAM Landmarks

Physical Landmark	Subsystem / Tier	Inscribed Data	Inscriber Function
`0x00000500`	Real-Mode Root	Primordial genesis anchor `%home`	`inscribe_kernel_flight_log`
`0x00007C00`	BIOS MBR Sector 0	Cold-boot genesis log, CPU reset verification	`inscribe_kernel_flight_log`
`0x00018000`	Stage 2 Protected Mode	16-byte packed flight logs, 16-trit workspace	`inscribe_raw_flight_recorder_binary`
`0x00108000`	Multiboot / Long Mode	64-bit silicon envelope, ELF kernel symbols	`inscribe_kernel_flight_log`
`0x00200000`	Native UEFI GOP	64-entry RAM flight recorder table, GOP framebuffers	`inscribe_raw_flight_recorder_binary`

16-Byte Packed Binary Structure (`RawFlightRecorderEntry`)

In C (Track 0 `telemetry.h`):

```
typedef struct {
    UINT32 event_id;
    UINT32 arg1;
    UINT32 arg2;
    UINT32 status;
} __attribute__((packed)) UefiLogEntry;
```

In Rust (Track 1 `inscriber/log_writer.rs`):

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(C)]
pub struct RawFlightRecorderEntry {
    pub event_id: u32,
    pub arg1: u32,
    pub arg2: u32,
    pub status: u32,
}
```

Each entry occupies precisely 16 bytes. When written to a `.bin` vessel via `inscribe_raw_flight_recorder_binary` , it prepends an 8-byte header (32-bit count, 32-bit reserved), enabling byte-for-byte replay in emulators and test benches.

5. The Multi-Dimensional Rotation System

The `RotationPolicy` module in `rotation.rs` prevents unbounded file growth across long-running autonomous sessions. It governs partition capacities across multiple dimensions:

```
graph TD
    DIR["Storehouse Partition: log/character/"] --> SCAN["Rotation Engine Scans Partition"]
    SCAN --> C1{"File Count > max_files?"}
```

```
    SCAN --> C2{"Partition Bytes > max_partition_bytes?"}
    SCAN --> C3{"Active File Bytes > max_bytes_per_file?"}

    C1 -- Yes --> EXC["Calculate Excess Files (Oldest First)"]
    C2 -- Yes --> EXB["Identify Oldest Files Exceeding Byte Limit"]
    C3 -- Yes --> ACT["Rollover Active File: session_part1.log"]

    EXC --> ACTIVATE{"archive_old_files?"}
    EXB --> ACTIVATE

    ACTIVATE -- True --> ARC["Move to archive/ Subfolder<br/>(Preserve Vessel Integrity)"]
    ACTIVATE -- False --> PRN["Prune / Delete File"]

    ARC --> MAN["Emit rotation_manifest.json<br/>(Records Timestamps & Hashes)"]
    PRN --> MAN

    classDef warn fill:#b45309,stroke:#78350f,color:#ffffff;
    classDef check fill:#2563eb,stroke:#1d4ed8,color:#ffffff;
    classDef dest fill:#059669,stroke:#047857,color:#ffffff;

    class C1,C2,C3 check;
    class EXC,EXB,ACT warn;
    class ARC,PRN,MAN dest;
```

Rotation Configuration Options

```
let policy = RotationPolicy::new(1000, true)
    .with_max_bytes_per_file(10 * 1024 * 1024)   // 10 MB per individual log
    .with_max_partition_bytes(500 * 1024 * 1024) // 500 MB partition capacity
    .with_manifest(true)                         // Write rotation_manifest.json
    .with_compression(false);                    // Keep raw vessels accessible
```

Manifest Schema (`rotation_manifest.json`)

When files are rotated, the Inscriber emits an audit manifest documenting the action:

```
{
  "timestamp_micros": 1792000000500000,
  "files_rotated": 5,
  "files_archived": 5,
  "files_pruned": 0,
  "bytes_reclaimed": 142500,
  "rotated_vessels": [
    "1792000000000000_sess_01.log",
    "1792000000000001_sess_02.log",
    "1792000000000002_sess_03.log",
    "1792000000000003_sess_04.log",
    "1792000000000004_sess_05.log"
  ]
}
```

6. Vessel Naming Conventions & Queriability

The `naming.rs` engine formats filenames so that temporal range scans, session lookups, and landmark inspections execute in $O(1)$ without reading file contents:

```
Pattern 1 (Standard Session):
  {timestamp_micros}_{session_id}.{ext}
  Example: 1792000000000000_sess_alpha_99.scroll

Pattern 2 (Kernel Landmark):
```

```
  {timestamp_micros}_0x{landmark_hex:08x}_{tier}_{session_id}.{ext}
  Example: 1792000000100000_0x00018000_STAGE2-PM_kernel_flight.log

Pattern 3 (Rotated Part Segment):
  {base_name}_part{part_index}.{ext}
  Example: 1792000000200000_sess_omega_part3.log
```

> *[!IMPORTANT]* ***No Ambiguous Middle Dots***
> *File names strictly avoid middle dots (e.g. `file.part1.log` is replaced by `file_part1.log`). The single period character is reserved exclusively for the canonical vessel extension (`.log`, `.scroll`, `.bin`, `.json`).*

Query Engine Integration (`EpisodicQueryEngine`)

The `query` subsystem leverages these naming invariants to build fast in-memory indices:

1. **$O(1)$ Exact Lookup**: `find_by_session("sess_alpha_99")` extracts the session directly from the filename stem.
2. **Temporal Range Scans**: `range_scan(start, end)` performs prefix comparisons on the microsecond timestamp prefix.
3. **Category Partition Scans**: `filter_by_category("character")` isolates partition subfolders directly.
4. **QuadState & Failpoint Filters**: Indexed metadata from the canonical 4th block (`Closing`) allows instant filtering by commit polarity (`+1:PROCEED`, `0:AWAIT`, `NULL:HALT`, `-1:ROLLBACK`).

---

7. Verification & Audit Metrics

All 5 modules in `inscriber/` are fully verified under the OmniCode integration test harness:

```
cargo test diagnostics::logger::inscriber
```

Verification Suite	Invariants Tested	Status
`naming::tests`	Base name formatting, landmark hex extraction, rotated part parsing, single-dot enforcement	**PASS**
`rotation::tests`	File count bounds, byte capacity threshold, active file rollover, manifest emission	**PASS**
`log_writer::tests`	ASCII layout, failpoint proposals, 13 silicon register dump, 16-byte packed binary replay	**PASS**
`scroll_writer::tests`	Canonical 4-block `.scroll` generation, cognitive note audits, Track 0 genesis scrolls	**PASS**
`mod::tests`	`InscriberEngine` dual-episode lifecycle, partition capacity enforcement, storehouse integrity	**PASS**

Total System Suite: **285 passed, 0 failed, 0 warnings**.

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/log_writer.rs

RS | 489 LINES | 1877 WORDS | 21.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Traditional Developer & Structured Telemetry .log Inscription Engine )
// ============================================================================
//! Traditional Telemetry Inscription & Bare-Metal Kernel Flight Inscriber
//!
//! # Telemetry Inscription Architecture
//! Encapsulates both high-level developer ASCII telemetry and bare-metal Track 0
//! silicon telemetry. Translates runtime execution events, sensory tripwires,
//! silicon register states, and 16-byte flight recorder records into physically
//! persisted, human-readable, and machine-queryable vessels.
//!
//! # Track 0 Silicon Integration
//! Formats:
//! 1. **16-Byte Flight Recorder Records**: Packed binary chunks matching physical
//!    RAM landmarks `0x00018000` (Stage 2 PM), `0x00108000` (Multiboot), and
//!    `0x00200000` (Native UEFI).
//! 2. **13 Calibrated Silicon Registers**: Hardware snapshots (%head, %home,
//!    %turn, %spool, %spool_in, %spool_out, %frame, %rsp, %cr0, %cr3, %cr4, %rflags).
//! 3. **Dual-Rail Saturation Telemetry**: Captures saturation state `(1, 1)` on
//!    abrupt HALT, confirming safe re-inception back to homeostasis `(0, 0)` (0:AWAIT).

use std::fs;
use std::path::{Path, PathBuf};
use super::super::universal::LogEvent;
use super::super::failpoint::SiliconRegisters;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Architectural Setup, Type Definitions & Kernel Event Constants )
// ============================================================================

/// Bare-metal 16-byte packed flight recorder entry matching Track 0 silicon structures.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(C)]
pub struct RawFlightRecorderEntry {
    /// Kernel event identifier (e.g. 0x0001 EVT_GENESIS_ENTRY, 0x0004 EVT_ALU_VERIFIED).
    pub event_id: u32,
    /// Primary argument or hardware landmark (e.g. CR0 or physical address).
    pub arg1: u32,
    /// Secondary argument or stack landmark (e.g. ESP/RSP or memory size).
    pub arg2: u32,
    /// Subsystem status code (0 = OK, non-zero = failpoint/fault).
    pub status: u32,
}

impl RawFlightRecorderEntry {
    /// Constructs a new 16-byte flight recorder entry.
    pub const fn new(event_id: u32, arg1: u32, arg2: u32, status: u32) -> Self {
        Self {
            event_id,
            arg1,
            arg2,
            status,
        }
    }

    /// Converts the packed structure to a raw 16-byte little-endian array.
    pub fn to_bytes(&self) -> [u8; 16] {
        let e = self.event_id.to_le_bytes();
        let a1 = self.arg1.to_le_bytes();
        let a2 = self.arg2.to_le_bytes();
        let s = self.status.to_le_bytes();
        [
            e[0], e[1], e[2], e[3],
            a1[0], a1[1], a1[2], a1[3],
            a2[0], a2[1], a2[2], a2[3],
            s[0], s[1], s[2], s[3],
        ]
    }

    /// Decodes the event ID into a canonical human-readable label.
    pub fn decode_event_name(&self) -> &'static str {
        match self.event_id {
            0x0001 => "EVT_GENESIS_ENTRY",
```

```
            0x0002 => "EVT_BOOT_SERVICES",
            0x0003 => "EVT_MEM_DISCOVERED",
            0x0004 => "EVT_ALU_VERIFIED",
            0x0005 => "EVT_REGS_ARMED",
            0x0006 => "EVT_CPISI_MOUNT",
            0x0007 => "EVT_CASCADE_DOWN",
            0x0008 => "EVT_CASCADE_UP",
            0xEEEE => "EVT_SAFE_MODE",
            _ => "EVT_UNKNOWN_HARDWARE",
        }
    }
}

/// Rich structured record bridging Track 0 kernel events into host storehouse vessels.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct KernelTelemetryRecord {
    /// Monotonic timestamp in microseconds.
    pub tick_micros: u128,
    /// Physical RAM landmark (e.g. 0x0500, 0x7C00, 0x18000, 0x108000, 0x200000).
    pub landmark_address: u64,
    /// Boot tier or architectural stage (e.g. "BIOS_REAL_MODE", "STAGE2_PM", "LONG_MODE", "UEFI").
    pub boot_tier: String,
    /// Snapshot of silicon and CPU registers, if captured at this milestone.
    pub registers: Option<SiliconRegisters>,
    /// Dual-rail hardware state (e.g. `(1, 0)` for PROCEED, `(1, 1)` for HALT).
    pub dual_rail_state: (u8, u8),
    /// CPU hardware trap number if intercepted (e.g. 0x0D = #GP, 0x0E = #PF).
    pub cpu_trap: Option<u32>,
    /// Chronological sequence of 16-byte flight recorder entries.
    pub raw_entries: Vec<RawFlightRecorderEntry>,
    /// Narrative or diagnostic payload message.
    pub payload_message: String,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructive Execution, File Formatting & Inscription Engines )
// ============================================================================

/// Inscribes a structured traditional telemetry log file to disk.
///
/// Designed to be clean, intuitive, and effortless to follow for human developers
/// and machine analyzers alike.
pub fn inscribe_telemetry_log(
    path: &Path,
    timestamp_str: &str,
    session_id: &str,
    elapsed_micros: u128,
    final_state_str: &str,
    prompt: &str,
    failpoints: &[LogEvent],
    synthesized_omnicode: &str,
    response: &str,
) -> Result<PathBuf, String> {
    if let Some(parent) = path.parent() {
        if !parent.exists() {
            fs::create_dir_all(parent).map_err(|e| {
                format!("Failed to create log parent directory: {e}")
            })?;
        }
    }

    let duration_ms = (elapsed_micros as f64) / 1000.0;
    let state_explanation = match final_state_str {
        "PROCEED" | "+1:PROCEED" => "+1:PROCEED (Forward commit motion confirmed)",
        "AWAIT" | "0:AWAIT" => "0:AWAIT (Homeostasis equilibrium, standby)",
        "ROLLBACK" | "-1:ROLLBACK" => "-1:ROLLBACK (Adversarial tripwire caught, state reverted)",
        "HALT" | "NULL:HALT" => "NULL:HALT (Dual-rail saturation, safe-mode restart to 0:AWAIT)",
        other => other,
    };

    let mut out = String::new();
    out.push_str("================================================================================\n");
    out.push_str("           OMNICODE AI EXECUTION & DIAGNOSTIC TELEMETRY LOG\n");
    out.push_str("================================================================================\n");
    out.push_str(&format!("Session ID:       {}\n", session_id));
    out.push_str(&format!("Timestamp:        {}\n", timestamp_str));
    out.push_str(&format!("Execution Time:   {} µs ({:.2} ms)\n", elapsed_micros, duration_ms));
    out.push_str(&format!("Final State:      {}\n", state_explanation));
    out.push_str(&format!("Input Prompt:     \"{}\"\n", prompt));
    out.push_str("================================================================================\n\n");
```

```
157
158     out.push_str("--------------------------------------------------------------------------------\n");
159     out.push_str("1. SENSORY LOG EVENTS & TRAPPED FAILPOINTS\n");
160     out.push_str("--------------------------------------------------------------------------------\n");
161     if failpoints.is_empty() {
162         out.push_str("Status: CLEAN\n");
163         out.push_str("  • Zero failpoints trapped during execution.\n");
164         out.push_str("  • Sensory event trace nominal; all boundary tripwires remained armed.\n");
165     } else {
166         out.push_str(&format!("Status: {} FAILPOINTS TRAPPED\n\n", failpoints.len()));
167         out.push_str("  #  | Elapsed   | Channel  | Classification | Details\n");
168         out.push_str("-----+-----------+----------+----------------+---------------------------------\n");
169         for (i, fp) in failpoints.iter().enumerate() {
170             let kind = fp
171                 .failpoint_type
172                 .map(|t| t.as_str())
173                 .unwrap_or("GENERAL");
174             out.push_str(&format!(
175                 " {:>3} | {:>6} µs | {:<8} | {:<14} | {}\n",
176                 i + 1,
177                 fp.elapsed_micros,
178                 fp.channel,
179                 kind,
180                 fp.message
181             ));
182             if let Some(ref sug) = fp.suggestion {
183                 out.push_str(&format!(
184                     "     -> Remediation Proposal: [{:?}] at Line {}: {}\n",
185                     sug.kind, sug.line, sug.rationale
186                 ));
187                 out.push_str(&format!(
188                     "        Target Token: \"{}\" => Suggested Patch: \"{}\"\n",
189                     sug.target_token, sug.suggested_patch
190                 ));
191             }
192         }
193     }
194     out.push_str("\n");
195
196     out.push_str("--------------------------------------------------------------------------------\n");
197     out.push_str("2. IMMUNE SYSTEM RESTORATIONS & SELF-HEALING\n");
198     out.push_str("--------------------------------------------------------------------------------\n");
199     out.push_str("Status: PASS\n");
200     out.push_str("  • Zero invasive repairs required.\n");
201     out.push_str("  • Source reel conformed to all 4-block structural and boundary invariants.\n\n");
202
203     out.push_str("--------------------------------------------------------------------------------\n");
204     out.push_str("3. SYNTHESIZED TURING REEL (OMNICODE SOURCE)\n");
205     out.push_str("--------------------------------------------------------------------------------\n");
206     out.push_str(synthesized_omnicode);
207     if !synthesized_omnicode.ends_with('\n') {
208         out.push('\n');
209     }
210     out.push('\n');
211
212     out.push_str("--------------------------------------------------------------------------------\n");
213     out.push_str("4. MACHINE DISCOURSE RESPONSE\n");
214     out.push_str("--------------------------------------------------------------------------------\n");
215     out.push_str(&format!("\"{}\"\n", response));
216     out.push_str("================================================================================\n");
217
218     fs::write(path, out)
219         .map_err(|e| format!("Failed to write log to {:?}: {e}", path))?;
220
221     Ok(path.to_path_buf())
222 }
223
224 /// Inscribes a rich Track 0 kernel flight telemetry log vessel.
225 ///
226 /// Formatted cleanly and intuitively for systems programmers and hardware engineers.
227 pub fn inscribe_kernel_flight_log(
228     path: &Path,
229     session_id: &str,
230     record: &KernelTelemetryRecord,
231 ) -> Result<PathBuf, String> {
232     if let Some(parent) = path.parent() {
233         if !parent.exists() {
234             fs::create_dir_all(parent).map_err(|e| {
235                 format!("Failed to create kernel log parent directory: {e}")
236             })?;
237         }
```

```
    }

    let landmark_name = match record.landmark_address {
        0x0500 => "Real Mode Genesis (Track 0 Root Landmark)",
        0x7C00 => "BIOS MBR Sector 0 Boot Landmark",
        0x18000 => "Stage 2 Protected Mode Flight Recorder Landmark",
        0x108000 => "Long Mode 64-Bit Kernel Base Landmark",
        0x200000 => "Native UEFI In-Memory RAM Log Landmark (2 MB)",
        _ => "Custom Physical Silicon Landmark",
    };

    let dual_rail_label = match record.dual_rail_state {
        (1, 0) => "(1, 0) -> +1:PROCEED (Positive rail high; forward commit motion active)",
        (0, 0) => "(0, 0) -> 0:AWAIT (Homeostasis equilibrium; standby)",
        (0, 1) => "(0, 1) -> -1:ROLLBACK (Negative rail high; causal tape rewind)",
        (1, 1) => "(1, 1) -> NULL:HALT (Both rails saturated! Safe restart to 0:AWAIT)",
        _ => "Uncalibrated Rail State",
    };

    let mut out = String::new();
    out.push_str("================================================================================\n");
    out.push_str("             MILLENIUMOS TRACK 0 SILICON FLIGHT RECORDER LOG\n");
    out.push_str("================================================================================\n");
    out.push_str(&format!("Session ID:         {}\n", session_id));
    out.push_str(&format!("Monotonic Clock:    {} µs since silicon inception\n", record.tick_micros));
    out.push_str(&format!("Hardware Landmark:  {:#010x} ({})\n", record.landmark_address, landmark_name));
    out.push_str(&format!("Boot Ladder Stage:  {}\n", record.boot_tier));
    out.push_str(&format!("Dual-Rail State:    {}\n", dual_rail_label));
    if let Some(trap) = record.cpu_trap {
        out.push_str(&format!("Hardware CPU Trap:  #{:#04x} (Exception Intercepted)\n", trap));
    } else {
        out.push_str("Hardware CPU Trap:  None (#0000 Nominal)\n");
    }
    out.push_str(&format!("Diagnostic Note:    {}\n", record.payload_message));
    out.push_str("================================================================================\n\n");

    out.push_str("--------------------------------------------------------------------------------\n");
    out.push_str("1. THE 13 CALIBRATED SILICON REGISTERS & CPU CONTROL ENVELOPE\n");
    out.push_str("--------------------------------------------------------------------------------\n");
    if let Some(ref regs) = record.registers {
        out.push_str("The 7 Named Cognitive Silicon Registers:\n");
        out.push_str(&format!("  %head:      {:#018x}   (Cognitive focus landmark)\n", regs.head));
        out.push_str(&format!("  %home:      {:#018x}   (Moral origin anchor / Track 0 root)\n", regs.home));
        out.push_str(&format!("  %turn:      {:#018x}   (Monotonic step/turn counter)\n", regs.turn));
        out.push_str(&format!("  %spool:     {:#018x}   (I/O communication port base)\n", regs.spool));
        out.push_str(&format!("  %spool_in:  {:#018x}   (Inbound stream head pointer)\n", regs.spool_in));
        out.push_str(&format!("  %spool_out: {:#018x}   (Outbound stream head pointer)\n", regs.spool_out));
        out.push_str(&format!("  %frame:     {:#018x}   (Active execution frame landmark)\n\n", regs.frame));

        out.push_str("CPU Control & State Registers:\n");
        out.push_str(&format!("  %rsp:       {:#018x}   (Active stack pointer landmark)\n", regs.rsp));
        out.push_str(&format!("  %cr0:       {:#018x}   (Control register 0: Protection & Paging)\n", regs.cr0));
        out.push_str(&format!("  %cr3:       {:#018x}   (Page directory base register)\n", regs.cr3));
        out.push_str(&format!("  %cr4:       {:#018x}   (Control register 4: Architectural extensions)\n", regs.cr4));
        out.push_str(&format!("  %rflags:    {:#018x}   (Processor status & interrupt flags)\n", regs.rflags));
    } else {
        out.push_str("Status: (No silicon register snapshot requested for this landmark milestone.)\n");
    }
    out.push('\n');

    out.push_str("--------------------------------------------------------------------------------\n");
    out.push_str("2. RAM LOG FLIGHT RECORDER (16-BYTE ATOMIC SILICON CHUNKS)\n");
    out.push_str("--------------------------------------------------------------------------------\n");
    if record.raw_entries.is_empty() {
        out.push_str("Status: (Zero 16-byte flight recorder entries logged in memory buffer.)\n");
    } else {
        out.push_str(&format!("Total Inscribed Records: {}\n\n", record.raw_entries.len()));
        out.push_str("   # | Event ID | Canonical Event Name | Argument 1   | Argument 2   | Status\n");
        out.push_str("-----+----------+----------------------+--------------+--------------+---------\n");
        for (i, entry) in record.raw_entries.iter().enumerate() {
            let eid = entry.event_id;
            let a1 = entry.arg1;
            let a2 = entry.arg2;
            let st = entry.status;
            let name = entry.decode_event_name();
            out.push_str(&format!(
                " {:>3} |  {:#06x}  | {:<20} |  {:#010x}  |  {:#010x}  | {:#06x}\n",
                i + 1,
                eid,
                name,
                a1,
```

```
                a2,
                st
            ));
        }
    }
    out.push('\n');

    out.push_str("-------------------------------------------------------------------------------\n");
    out.push_str("3. DUAL-RAIL HARDWARE SATURATION & RE-INCEPTION AUDIT\n");
    out.push_str("-------------------------------------------------------------------------------\n");
    if record.dual_rail_state == (1, 1) {
        out.push_str("Diagnosis: ABRUPT HALT TRAPPED (Tripwire Collision)\n");
        out.push_str("  • Both dual-rail state lines saturated high at (1, 1).\n");
        out.push_str("  • Zero free data capacity remaining on active rail.\n");
        out.push_str("  • MilleniumOS invariant: Machine mathematically prohibited from flatlining.\n");
        out.push_str("  • Collision sensor caught halt; safely re-incepted machine back to 0:AWAIT homeostasis.\n");
    } else {
        out.push_str("Diagnosis: STABLE EQUILIBRIUM\n");
        out.push_str("  • Forward commit polarity (+1:PROCEED) intact.\n");
        out.push_str("  • Balanced ternary ALU equilibrium (+1) + (-1) = 0 affirmed.\n");
        out.push_str("  • Machine operating within nominal silicon envelope.\n");
    }
    out.push_str("===============================================================================\n");

    fs::write(path, out)
        .map_err(|e| format!("Failed to write kernel flight log to {:?}: {e}", path))?;

    Ok(path.to_path_buf())
}

/// Inscribes raw 16-byte packed flight recorder entries directly into a binary vessel (`.bin`).
pub fn inscribe_raw_flight_recorder_binary(
    path: &Path,
    entries: &[RawFlightRecorderEntry],
) -> Result<PathBuf, String> {
    if let Some(parent) = path.parent() {
        if !parent.exists() {
            fs::create_dir_all(parent).map_err(|e| {
                format!("Failed to create binary log directory: {e}")
            })?;
        }
    }

    let mut buffer = Vec::with_capacity(8 + entries.len() * 16);
    // Header: 32-bit count, 32-bit reserved
    buffer.extend_from_slice(&(entries.len() as u32).to_le_bytes());
    buffer.extend_from_slice(&0u32.to_le_bytes());

    for entry in entries {
        buffer.extend_from_slice(&entry.to_bytes());
    }

    fs::write(path, buffer)
        .map_err(|e| format!("Failed to write binary flight recorder to {:?}: {e}", path))?;

    Ok(path.to_path_buf())
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Audits, Binary Serialization Verifications & Unit Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_raw_flight_recorder_entry_encoding() {
        let entry = RawFlightRecorderEntry::new(0x0004, 0x18000, 0x6FFC, 0);
        assert_eq!(entry.decode_event_name(), "EVT_ALU_VERIFIED");

        let bytes = entry.to_bytes();
        assert_eq!(bytes.len(), 16);
        assert_eq!(u32::from_le_bytes([bytes[0], bytes[1], bytes[2], bytes[3]]), 0x0004);
        assert_eq!(u32::from_le_bytes([bytes[4], bytes[5], bytes[6], bytes[7]]), 0x18000);
        assert_eq!(u32::from_le_bytes([bytes[8], bytes[9], bytes[10], bytes[11]]), 0x6FFC);
        assert_eq!(u32::from_le_bytes([bytes[12], bytes[13], bytes[14], bytes[15]]), 0);
    }

    #[test]
```

```
    fn test_inscribe_telemetry_log_execution() {
        let temp_dir = std::env::temp_dir().join(format!("omnicode_log_test_{}", std::process::id()));
        let log_path = temp_dir.join("telemetry.log");

        let empty_fps: &[LogEvent] = &[];
        let res = inscribe_telemetry_log(
            &log_path,
            "1792000000000000_0000",
            "sess_unit_01",
            4250,
            "PROCEED",
            "Inscribe character glyph",
            empty_fps,
            "Opening { preconditions: [ +1 ] }\nBody [ ]\nClosing { commit_status [ <PROCEED> ] }",
            "Utterance complete",
        )
        .unwrap();

        assert_eq!(res, log_path);
        let content = fs::read_to_string(&log_path).unwrap();
        assert!(content.contains("OMNICODE AI EXECUTION & DIAGNOSTIC TELEMETRY LOG"));
        assert!(content.contains("Execution Time:   4250 µs"));
        assert!(content.contains("sess_unit_01"));
        assert!(content.contains("Status: CLEAN"));

        fs::remove_dir_all(&temp_dir).ok();
    }

    #[test]
    fn test_inscribe_kernel_flight_log_and_binary() {
        let temp_dir = std::env::temp_dir().join(format!("omnicode_kernel_log_test_{}", std::process::id()));
        let klog_path = temp_dir.join("kernel_flight.log");
        let kbin_path = temp_dir.join("kernel_flight.bin");

        let regs = SiliconRegisters {
            head: 0x1000,
            home: 0x0500,
            turn: 42,
            spool: 0x3F8,
            spool_in: 0x3F8,
            spool_out: 0x3F8,
            frame: 0x18000,
            rax: 0x1,
            rbx: 0x2,
            rcx: 0x3,
            rdx: 0x4,
            rsi: 0x5,
            rdi: 0x6,
            rsp: 0x6FFC,
            rbp: 0x7000,
            cr0: 0x80010033,
            cr3: 0x1000,
            cr4: 0x20,
            rflags: 0x202,
        };

        let entries = vec![
            RawFlightRecorderEntry::new(0x0001, 0x7C00, 0, 0),
            RawFlightRecorderEntry::new(0x0004, 0x18000, 0, 0),
            RawFlightRecorderEntry::new(0x0005, 0x18000, 13, 0),
        ];

        let record = KernelTelemetryRecord {
            tick_micros: 1250,
            landmark_address: 0x18000,
            boot_tier: "STAGE2_PM".to_string(),
            registers: Some(regs),
            dual_rail_state: (1, 0),
            cpu_trap: None,
            raw_entries: entries.clone(),
            payload_message: "Stage 2 Protected Mode Armed".to_string(),
        };

        inscribe_kernel_flight_log(&klog_path, "kflight_01", &record).unwrap();
        let log_content = fs::read_to_string(&klog_path).unwrap();
        assert!(log_content.contains("TRACK 0 SILICON FLIGHT RECORDER LOG"));
        assert!(log_content.contains("0x00018000"));
        assert!(log_content.contains("EVT_ALU_VERIFIED"));
        assert!(log_content.contains("EVT_REGS_ARMED"));
        assert!(log_content.contains("%head:"));

```

```
        inscribe_raw_flight_recorder_binary(&kbin_path, &entries).unwrap();
        let bin_bytes = fs::read(&kbin_path).unwrap();
        // 8-byte header + 3 * 16 bytes = 56 bytes
        assert_eq!(bin_bytes.len(), 56);

        fs::remove_dir_all(&temp_dir).ok();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/mod.rs RS | 175 LINES | 610 WORDS | 6.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Storehouse Inscriber Subsystem: Writers, Naming & Rotation Engine )
// ============================================================================
//! Storehouse Inscription Subsystem: Vessels, Naming & Capacity Rotation
//!
//! # System Identity & Architectural Grounding
//! Grounded in the container law `Occupancy <= Capacity` (2 Kings 4:4), the
//! Inscriber subsystem is the dedicated machine organ responsible for transforming
//! transient sensory events, cognitive notes, and bare-metal Track 0 kernel
//! flight records into permanently persisted, bounded, and indexed vessels.
//!
//! # Subsystem Decomposition
//! - [`naming`]: Deterministic temporal prefix formatting and $O(1)$ landmark parsing.
//! - [`rotation`]: Multi-dimensional capacity enforcement across file count, file bytes,
//!   and partition volumes with automatic archival and manifest emission.
//! - [`log_writer`]: Traditional ASCII telemetry, 13 silicon register dumps, and 16-byte
//!   atomic flight recorder logs matching physical RAM landmarks.
//! - [`scroll_writer`]: Executable canonical 4-Block `.scroll` programs encoding
//!   episodic memory traces and Track 0 silicon genesis milestones (`CPI-SI`).

pub mod rotation;
pub mod naming;
pub mod scroll_writer;
pub mod log_writer;

use std::path::PathBuf;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Architectural Declarations, State Definitions & Comprehensive Re-exports )
// ============================================================================

pub use rotation::{RotationPolicy, RotationReport};
pub use naming::{
    format_base_name, format_kernel_base_name, format_rotated_name,
    parse_file_name, resolve_custom_vessel_path, resolve_vessel_path,
    ParsedFileName, VesselNamingScheme,
};
pub use scroll_writer::{
    inscribe_audit_scroll, inscribe_canonical_scroll, inscribe_kernel_audit_scroll,
    ScrollGenre,
};
pub use log_writer::{
    inscribe_kernel_flight_log, inscribe_raw_flight_recorder_binary,
    inscribe_telemetry_log, KernelTelemetryRecord, RawFlightRecorderEntry,
};

/// High-level inscriber orchestrator managing storehouse partitions, active logs,
/// and automated rotation policies.
#[derive(Debug, Clone)]
pub struct InscriberEngine {
    /// Root directory of the episodic storehouse.
    pub storehouse_root: PathBuf,
    /// Rotation policy governing partition capacity.
    pub rotation_policy: RotationPolicy,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Inscription Orchestration, Partition Enforcement & Inscriber Methods )
// ============================================================================

impl InscriberEngine {
    /// Constructs a new [`InscriberEngine`] rooted at `storehouse_root`.
    pub fn new(storehouse_root: impl Into<PathBuf>, policy: RotationPolicy) -> Self {
        Self {
            storehouse_root: storehouse_root.into(),
            rotation_policy: policy,
        }
    }

    /// Resolves the destination directory for a specific category and file type.
    pub fn resolve_path(&self, category: &str, is_scroll: bool, base_name: &str) -> PathBuf {
        resolve_vessel_path(&self.storehouse_root, category, is_scroll, base_name)
```

```
    }

    /// Enforces the active capacity rotation policy on a target partition directory.
    pub fn enforce_partition_capacity(&self, category: &str, is_scroll: bool) -> Result<RotationReport, String> {
        let subfolder = if is_scroll { "scroll" } else { "log" };
        let partition_dir = self.storehouse_root.join(subfolder).join(category);
        self.rotation_policy.enforce_detailed(&partition_dir)
    }

    /// Inscribes an episodic execution episode as both a canonical 4-Block `.scroll`
    /// and a traditional `.log` file, ensuring storehouse capacity invariants hold.
    pub fn inscribe_dual_episode(
        &self,
        category: &str,
        session_id: &str,
        timestamp_micros: u128,
        prompt: &str,
        synthesized_omnicode: &str,
        state_str: &str,
        response: &str,
    ) -> Result<(PathBuf, PathBuf), String> {
        let ts_str = timestamp_micros.to_string();
        let base_name = format_base_name(&ts_str, session_id);

        let scroll_path = self.resolve_path(category, true, &base_name);
        let log_path = self.resolve_path(category, false, &base_name);

        // Inscribe Canonical Scroll
        inscribe_canonical_scroll(
            &scroll_path,
            session_id,
            prompt,
            synthesized_omnicode,
            state_str,
            response,
            timestamp_micros,
        )?;

        // Inscribe Traditional Telemetry Log
        inscribe_telemetry_log(
            &log_path,
            &ts_str,
            session_id,
            timestamp_micros,
            state_str,
            prompt,
            &[],
            synthesized_omnicode,
            response,
        )?;

        // Enforce capacity bounds on both partition directories
        let _ = self.enforce_partition_capacity(category, true);
        let _ = self.enforce_partition_capacity(category, false);

        Ok((scroll_path, log_path))
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Invariants, Integration Audits & Unit Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::fs;

    #[test]
    fn test_inscriber_engine_dual_episode_lifecycle() {
        let temp_root = std::env::temp_dir().join(format!("omnicode_inscriber_mod_test_{}", std::process::id()));
        let policy = RotationPolicy::new(10, true);
        let engine = InscriberEngine::new(&temp_root, policy);

        let (scroll_p, log_p) = engine.inscribe_dual_episode(
            "character",
            "sess_inscriber_test_01",
            1792000000000000,
            "Transduce glyph",
            "Opening { preconditions: [ +1 ] }\nBody [ ]\nClosing { commit_status [ <PROCEED> ] }",
```

```
            "PROCEED",
            "Glyph transduced successfully",
        )
        .unwrap();

        assert!(scroll_p.exists());
        assert!(log_p.exists());

        let scroll_content = fs::read_to_string(&scroll_p).unwrap();
        assert!(scroll_content.contains("< EpistemicTrace { sess_inscriber_test_01 } >"));
        assert!(scroll_content.contains("commit_status [ <PROCEED> ]"));

        let log_content = fs::read_to_string(&log_p).unwrap();
        assert!(log_content.contains("OMNICODE AI EXECUTION & DIAGNOSTIC TELEMETRY LOG"));

        fs::remove_dir_all(&temp_root).ok();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/naming.rs

RS | 283 LINES | 986 WORDS | 10.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structured Naming Conventions for Deterministic Telemetry Indexing )
// ============================================================================
//! Naming Conventions & Parsing Machinery for Storehouse Vessels
//!
//! # Epistemic Role & Container Invariants
//! Grounded in the container law `Occupancy <= Capacity` (2 Kings 4:4), every
//! telemetry and memory artifact in the OmniCode storehouse is inscribed into a
//! deterministic vessel.
//!
//! # Deterministic Vessel Indexing
//! Vessel file names are formatted with monotonic timestamp prefixes and domain
//! coordinates, enabling:
//! 1. $O(1)$ prefix matching and temporal range scans directly on the filesystem.
//! 2. Clean single-dot extensions (`.log`, `.scroll`, `.bin`, `.json`) without
//!    ambiguous multi-dot segments.
//! 3. Deterministic extraction of session IDs, Track 0 memory landmarks, boot tiers,
//!    and rotation generation indices.

use std::path::{Path, PathBuf};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Architectural Setup, Type Definitions & Vessel Naming Schemas )
// ============================================================================

/// High-level naming scheme categorization for storehouse artifacts.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum VesselNamingScheme {
    /// Standard execution episode: `<timestamp>_<session_id>.<ext>`
    StandardSession,
    /// Track 0 bare-metal kernel flight telemetry: `<timestamp>_0x<landmark:08x>_<tier>_<session_id>.<ext>`
    KernelLandmark,
    /// Rotated partition file segment: `<timestamp>_<session_id>_part<part_num>.<ext>`
    RotatedPart,
    /// Archived snapshot: `archive/<timestamp>_<session_id>.<ext>`
    ArchivedSnapshot,
}

/// Comprehensive metadata extracted directly from a vessel file name.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ParsedFileName {
    /// Monotonic timestamp in microseconds since epoch or machine inception.
    pub timestamp_micros: u128,
    /// Unique session or component identifier.
    pub session_id: String,
    /// Vessel file extension without dot (e.g., "log", "scroll", "bin", "json").
    pub extension: String,
    /// Physical RAM landmark if this is a Track 0 kernel vessel (e.g., 0x18000, 0x108000, 0x200000).
    pub landmark_address: Option<u64>,
    /// Boot tier or architectural stage if identified (e.g., "STAGE2-PM", "UEFI", "LONGMODE").
    pub tier: Option<String>,
    /// Rotation generation / segment index if this is a rotated part.
    pub part_index: Option<usize>,
    /// Whether the file is located within an `archive/` subfolder.
    pub is_archived: bool,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Vessel Path Resolution, Formatting Constructors & Parsing Engines )
// ============================================================================

/// Formats a standardized base name for telemetry and scroll files.
///
/// Pattern: `<timestamp>_<session_id>`
#[inline]
pub fn format_base_name(timestamp: &str, session_id: &str) -> String {
    format!("{timestamp}_{session_id}")
}

/// Formats a Track 0 bare-metal kernel base name encoding hardware landmarks.
///
/// Pattern: `<timestamp>_0x<landmark:08x>_<tier>_<session_id>`
```

```rust
/// Note: Hyphenates any multi-word tier (e.g. `STAGE2-PM`) to preserve clean single-underscore tokens.
pub fn format_kernel_base_name(
    timestamp: &str,
    landmark_address: u64,
    tier: &str,
    session_id: &str,
) -> String {
    let sanitized_tier = tier.replace([' ', '_', '/'], "-").to_uppercase();
    format!("{timestamp}_0x{landmark_address:08x}_{sanitized_tier}_{session_id}")
}

/// Formats a rotated part filename with an explicit segment index.
///
/// Pattern: `<base_name>_part<part_index>.<ext>`
/// Avoids ambiguous middle dots, ensuring only one extension delimiter.
pub fn format_rotated_name(base_name: &str, part_index: usize, ext: &str) -> String {
    format!("{base_name}_part{part_index}.{ext}")
}

/// Resolves the canonical destination directory for a specific category and file type.
///
/// If `root/scroll` or `root/log` subfolders exist, nests the output within:
/// `root/{subfolder}/{category}/{base_name}.{ext}`
/// Otherwise writes directly to `root/{base_name}.{ext}`.
pub fn resolve_vessel_path(
    root: &Path,
    category: &str,
    is_scroll: bool,
    base_name: &str,
) -> PathBuf {
    let subfolder = if is_scroll { "scroll" } else { "log" };
    let ext = if is_scroll { "scroll" } else { "log" };

    let dest_dir = if root.join(subfolder).exists() {
        root.join(subfolder).join(category)
    } else {
        root.to_path_buf()
    };

    dest_dir.join(format!("{base_name}.{ext}"))
}

/// Resolves a specialized vessel destination with a custom extension and directory structure.
pub fn resolve_custom_vessel_path(
    root: &Path,
    subfolder: &str,
    category: &str,
    base_name: &str,
    ext: &str,
) -> PathBuf {
    let dest_dir = if root.join(subfolder).exists() {
        root.join(subfolder).join(category)
    } else if !category.is_empty() {
        root.join(category)
    } else {
        root.to_path_buf()
    };

    dest_dir.join(format!("{base_name}.{ext}"))
}

/// Extracts rich metadata from a standardized telemetry vessel file path.
///
/// Supports standard `<timestamp>_<session_id>.<ext>`, kernel landmark names,
/// and rotated `_part<N>` logs.
pub fn parse_file_name(path: &Path) -> Option<ParsedFileName> {
    let ext = path.extension()?.to_string_lossy().to_string();

    // Check if path is inside an archive directory
    let is_archived = path
        .parent()
        .and_then(|p| p.file_name())
        .map(|name| name.to_string_lossy() == "archive")
        .unwrap_or(false);

    let file_stem = path.file_stem()?.to_str()?;

    // Detect if there is a `_part<N>` or `.part<N>` segment in the stem
    let mut part_index = None;
    let main_stem = if let Some(part_pos) = file_stem.find("_part") {
        let after = &file_stem[part_pos + 5..];
```

```
        if let Ok(num) = after.parse::<usize>() {
            part_index = Some(num);
        }
        &file_stem[..part_pos]
    } else if let Some(part_pos) = file_stem.find(".part") {
        let after = &file_stem[part_pos + 5..];
        if let Ok(num) = after.parse::<usize>() {
            part_index = Some(num);
        }
        &file_stem[..part_pos]
    } else {
        file_stem
    };

    let parts: Vec<&str> = main_stem.split('_').collect();
    if parts.is_empty() {
        return None;
    }

    let micros = parts[0].parse::<u128>().ok()?;

    // Check for Kernel Landmark: <micros>_0x<addr>_<tier>_<session...>
    let mut landmark_address = None;
    let mut tier = None;
    let session_id = if parts.len() >= 4 && parts[1].starts_with("0x") {
        if let Ok(addr) = u64::from_str_radix(&parts[1][2..], 16) {
            landmark_address = Some(addr);
            tier = Some(parts[2].to_string());
            parts[3..].join("_")
        } else if parts.len() > 1 {
            parts[1..].join("_")
        } else {
            "unknown".to_string()
        }
    } else if parts.len() > 1 {
        parts[1..].join("_")
    } else {
        "unknown".to_string()
    };

    Some(ParsedFileName {
        timestamp_micros: micros,
        session_id,
        extension: ext,
        landmark_address,
        tier,
        part_index,
        is_archived,
    })
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Audits, Round-Trip Verifications & Unit Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_format_and_parse_standard_base_name() {
        let base = format_base_name("1792000000000000", "sess_alpha_99");
        assert_eq!(base, "1792000000000000_sess_alpha_99");

        let p = Path::new("/var/omnicode/scroll/character/1792000000000000_sess_alpha_99.scroll");
        let parsed = parse_file_name(p).expect("Must parse standard scroll");
        assert_eq!(parsed.timestamp_micros, 1792000000000000);
        assert_eq!(parsed.session_id, "sess_alpha_99");
        assert_eq!(parsed.extension, "scroll");
        assert_eq!(parsed.landmark_address, None);
        assert_eq!(parsed.tier, None);
        assert_eq!(parsed.part_index, None);
        assert!(!parsed.is_archived);
    }

    #[test]
    fn test_format_and_parse_kernel_landmark_vessel() {
        let k_base = format_kernel_base_name(
            "1792000000100000",
            0x00018000,
```

```
            "STAGE2_PM",
            "kernel_flight",
        );
        assert_eq!(k_base, "1792000000100000_0x00018000_STAGE2-PM_kernel_flight");

        let p = Path::new("/omnicode/log/kernel/1792000000100000_0x00018000_STAGE2-PM_kernel_flight.log");
        let parsed = parse_file_name(p).expect("Must parse kernel vessel");
        assert_eq!(parsed.timestamp_micros, 1792000000100000);
        assert_eq!(parsed.landmark_address, Some(0x18000));
        assert_eq!(parsed.tier.as_deref(), Some("STAGE2-PM"));
        assert_eq!(parsed.session_id, "kernel_flight");
        assert_eq!(parsed.extension, "log");
        assert!(!parsed.is_archived);
    }

    #[test]
    fn test_format_and_parse_rotated_part_and_archive() {
        let rotated = format_rotated_name("1792000000200000_sess_omega", 3, "log");
        assert_eq!(rotated, "1792000000200000_sess_omega_part3.log");

        let p = Path::new("/storehouse/archive/1792000000200000_sess_omega_part3.log");
        let parsed = parse_file_name(p).expect("Must parse rotated archived file");
        assert_eq!(parsed.timestamp_micros, 1792000000200000);
        assert_eq!(parsed.session_id, "sess_omega");
        assert_eq!(parsed.extension, "log");
        assert_eq!(parsed.part_index, Some(3));
        assert!(parsed.is_archived);
    }

    #[test]
    fn test_resolve_vessel_paths() {
        let root = Path::new("/var/storehouse");
        let path_scroll = resolve_vessel_path(root, "number", true, "1000_test");
        assert_eq!(
            path_scroll,
            PathBuf::from("/var/storehouse/1000_test.scroll")
        );

        let custom = resolve_custom_vessel_path(root, "flight", "hardware", "flight_01", "bin");
        assert_eq!(
            custom,
            PathBuf::from("/var/storehouse/hardware/flight_01.bin")
        );
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/rotation.rs RS | 426 LINES | 1537 WORDS | 16.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( RotationPolicy: Enforcing Vessel Capacity Bounds on Episodic Storehouses )
// ============================================================================
//! Storehouse Vessel Rotation Engine & Capacity Enforcer
//!
//! # The Container Law of Episodic Storage
//! Grounded in the invariant of container law:
//! $$\text{Occupancy} \le \text{Capacity}$$
//! as established in 2 Kings 4:4 (*"When the vessels were full, she set aside that
//! which was full"*), this module prevents unbounded file proliferation across
//! millisecond turns and millennial operational horizons.
//!
//! # Multi-Dimensional Capacity Constraints
//! The rotation engine continuously bounds storehouse partitions across:
//! 1. **Vessel Count Bounds**: Maximum number of active files per partition.
//! 2. **Vessel Byte Limits**: Maximum byte size per individual `.log` or `.scroll`.
//! 3. **Partition Aggregate Capacity**: Maximum cumulative bytes allowed per domain.
//! 4. **Chronological Archival**: Seamless migration of sealed vessels to `archive/`
//!    subfolders with atomic rotation manifest generation.

use std::fs;
use std::path::{Path, PathBuf};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Architectural Setup, Type Definitions & Rotation Telemetry )
// ============================================================================

/// Structured report summarizing the outcome of a partition rotation cycle.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct RotationReport {
    /// Total number of active vessel files scanned during enforcement.
    pub total_files_scanned: usize,
    /// Number of excess files rotated out of the active partition.
    pub files_rotated: usize,
    /// Number of rotated files migrated into the `archive/` subfolder.
    pub files_archived: usize,
    /// Number of rotated files permanently pruned (when archival is disabled).
    pub files_pruned: usize,
    /// Total byte volume reclaimed from the active partition directory.
    pub bytes_reclaimed: u64,
    /// Path to the audit manifest recording this rotation pass, if generated.
    pub manifest_path: Option<PathBuf>,
}

impl RotationReport {
    /// Constructs an empty baseline rotation report.
    pub fn empty() -> Self {
        Self {
            total_files_scanned: 0,
            files_rotated: 0,
            files_archived: 0,
            files_pruned: 0,
            bytes_reclaimed: 0,
            manifest_path: None,
        }
    }
}

/// Policy governing storehouse partition retention, byte capacity, and rotation.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct RotationPolicy {
    /// Maximum number of active vessel files allowed in a single domain partition.
    pub max_files_per_partition: usize,
    /// When true, older vessels are moved to an `archive/` subfolder rather than deleted.
    pub archive_old_files: bool,
    /// Maximum byte size of a single active log or scroll before active rotation is required.
    pub max_bytes_per_file: u64,
    /// Maximum aggregate byte capacity for the entire partition directory.
    pub max_partition_bytes: u64,
    /// Optional maximum age retention limit in microseconds.
    pub max_retention_micros: Option<u128>,
    /// When true, generates a structured JSON rotation manifest upon each rotation cycle.
    pub generate_manifest: bool,
```

```
    /// When true, appends archive generation markers or compression flags.
    pub compress_archives: bool,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructive Execution, Capacity Enforcement & Archival Mechanics )
// ============================================================================

impl RotationPolicy {
    /// Constructs a standard rotation policy with the given partition file capacity.
    ///
    /// Preserves strict backwards compatibility with legacy callers while defaulting
    /// byte bounds to conservative, scalable thresholds (50 MB per file, 1 GB partition).
    pub fn new(max_files_per_partition: usize, archive_old_files: bool) -> Self {
        Self {
            max_files_per_partition: max_files_per_partition.max(10),
            archive_old_files,
            max_bytes_per_file: 50 * 1024 * 1024, // 50 MB
            max_partition_bytes: 1024 * 1024 * 1024, // 1 GB
            max_retention_micros: None,
            generate_manifest: true,
            compress_archives: false,
        }
    }

    /// Builder method to configure maximum file byte size before rollover.
    pub fn with_max_bytes_per_file(mut self, bytes: u64) -> Self {
        self.max_bytes_per_file = bytes.max(1);
        self
    }

    /// Builder method to configure aggregate partition byte capacity.
    pub fn with_max_partition_bytes(mut self, bytes: u64) -> Self {
        self.max_partition_bytes = bytes.max(1);
        self
    }

    /// Builder method to configure maximum retention duration in microseconds.
    pub fn with_retention_micros(mut self, micros: u128) -> Self {
        self.max_retention_micros = Some(micros);
        self
    }

    /// Builder method to enable or disable rotation manifest emission.
    pub fn with_manifest(mut self, generate: bool) -> Self {
        self.generate_manifest = generate;
        self
    }

    /// Builder method to enable or disable archive compression flags.
    pub fn with_compression(mut self, compress: bool) -> Self {
        self.compress_archives = compress;
        self
    }

    /// Checks whether an individual active vessel file requires rotation due to byte limits.
    pub fn check_rotation_needed(&self, file_path: &Path) -> bool {
        if let Ok(meta) = fs::metadata(file_path) {
            meta.len() >= self.max_bytes_per_file
        } else {
            false
        }
    }

    /// Rotates a single active vessel file by renaming it to a `.part<N>` segment.
    ///
    /// For example, `telemetry.log` -> `telemetry.part1.log`.
    pub fn rotate_active_file(&self, file_path: &Path) -> Result<PathBuf, String> {
        if !file_path.exists() {
            return Err(format!("Cannot rotate nonexistent file: {:?}", file_path));
        }

        let parent = file_path.parent().unwrap_or_else(|| Path::new("."));
        let file_stem = file_path
            .file_stem()
            .and_then(|s| s.to_str())
            .unwrap_or("vessel");
        let ext = file_path
            .extension()
            .and_then(|s| s.to_str())
```

```
            .unwrap_or("log");

        // Scan for highest existing part index
        let mut part = 1;
        while parent.join(format!("{file_stem}_part{part}.{ext}")).exists() {
            part += 1;
        }

        let target_path = parent.join(format!("{file_stem}_part{part}.{ext}"));
        fs::rename(file_path, &target_path).map_err(|e| {
            format!("Failed to rotate active file {:?} to {:?}: {e}", file_path, target_path)
        })?;

        Ok(target_path)
    }

    /// Enforces the policy on the specified partition directory, returning the number of rotated files.
    ///
    /// Preserves backwards compatibility for callers expecting `Result<usize, String>`.
    pub fn enforce(&self, partition_dir: &Path) -> Result<usize, String> {
        let report = self.enforce_detailed(partition_dir)?;
        Ok(report.files_rotated)
    }

    /// Executes detailed multi-dimensional capacity enforcement across count and byte limits.
    pub fn enforce_detailed(&self, partition_dir: &Path) -> Result<RotationReport, String> {
        if !partition_dir.exists() {
            return Ok(RotationReport::empty());
        }

        let mut report = RotationReport::empty();
        let read_dir = fs::read_dir(partition_dir)
            .map_err(|e| format!("Failed to read partition dir {:?}: {e}", partition_dir))?;

        // Collect all non-hidden, non-archive active vessel files with metadata
        struct FileEntry {
            path: PathBuf,
            size_bytes: u64,
        }

        let mut entries = Vec::new();
        let mut total_partition_bytes = 0u64;

        for entry in read_dir.flatten() {
            let p = entry.path();
            if p.is_file() {
                let name = p.file_name().unwrap_or_default().to_string_lossy();
                // Exclude hidden files and rotation manifests from rotation consideration
                if !name.starts_with('.') && !name.ends_with(".manifest.json") {
                    let size = entry.metadata().map(|m| m.len()).unwrap_or(0);
                    total_partition_bytes += size;
                    entries.push(FileEntry { path: p, size_bytes: size });
                }
            }
        }

        report.total_files_scanned = entries.len();

        // Sort chronologically by filename (standard naming places timestamp prefix first)
        entries.sort_by(|a, b| a.path.cmp(&b.path));

        // Determine excess items by count
        let count_excess = if entries.len() > self.max_files_per_partition {
            entries.len() - self.max_files_per_partition
        } else {
            0
        };

        // Determine excess items by cumulative partition byte capacity
        let mut byte_excess_indices = Vec::new();
        let mut current_bytes = total_partition_bytes;
        for (i, entry) in entries.iter().enumerate() {
            if current_bytes > self.max_partition_bytes && i >= count_excess {
                byte_excess_indices.push(i);
                current_bytes = current_bytes.saturating_sub(entry.size_bytes);
            }
        }

        let total_rotate_target = count_excess + byte_excess_indices.len();
        if total_rotate_target == 0 {
            return Ok(report);
```

```
        }

        let archive_dir = partition_dir.join("archive");
        if self.archive_old_files && !archive_dir.exists() {
            fs::create_dir_all(&archive_dir)
                .map_err(|e| format!("Failed to create archive dir {:?}: {e}", archive_dir))?;
        }

        let mut rotated_file_names = Vec::new();

        // Process files that exceed count capacity (oldest first)
        for entry in entries.iter().take(count_excess) {
            let file_name = entry.path.file_name().unwrap();
            let size = entry.size_bytes;

            if self.archive_old_files {
                let mut dest = archive_dir.join(file_name);
                // Handle potential destination collisions
                if dest.exists() {
                    let stem = entry.path.file_stem().and_then(|s| s.to_str()).unwrap_or("vessel");
                    let ext = entry.path.extension().and_then(|s| s.to_str()).unwrap_or("log");
                    dest = archive_dir.join(format!("{stem}_{}.{ext}", std::process::id()));
                }

                if fs::rename(&entry.path, &dest).is_ok() {
                    report.files_rotated += 1;
                    report.files_archived += 1;
                    report.bytes_reclaimed += size;
                    rotated_file_names.push(dest.file_name().unwrap().to_string_lossy().to_string());
                }
            } else if fs::remove_file(&entry.path).is_ok() {
                report.files_rotated += 1;
                report.files_pruned += 1;
                report.bytes_reclaimed += size;
                rotated_file_names.push(file_name.to_string_lossy().to_string());
            }
        }

        // Process files that exceed byte capacity
        for idx in byte_excess_indices {
            if let Some(entry) = entries.get(idx) {
                if !entry.path.exists() {
                    continue;
                }
                let file_name = entry.path.file_name().unwrap();
                let size = entry.size_bytes;

                if self.archive_old_files {
                    let dest = archive_dir.join(file_name);
                    if fs::rename(&entry.path, &dest).is_ok() {
                        report.files_rotated += 1;
                        report.files_archived += 1;
                        report.bytes_reclaimed += size;
                        rotated_file_names.push(dest.file_name().unwrap().to_string_lossy().to_string());
                    }
                } else if fs::remove_file(&entry.path).is_ok() {
                    report.files_rotated += 1;
                    report.files_pruned += 1;
                    report.bytes_reclaimed += size;
                    rotated_file_names.push(file_name.to_string_lossy().to_string());
                }
            }
        }

        // Generate JSON audit manifest if configured
        if self.generate_manifest && report.files_rotated > 0 {
            let manifest_target = if self.archive_old_files {
                archive_dir.join("rotation_manifest.json")
            } else {
                partition_dir.join(".rotation_manifest.json")
            };

            let manifest_content = format!(
                "{{\n  \"timestamp_micros\": {},\n  \"files_rotated\": {},\n  \"files_archived\": {},\n  \"files_pruned\": {},\n  \"bytes_reclaimed\": {},\n  \"rotated_vessels\": {:?}\n}}\n",
                std::time::SystemTime::now()
                    .duration_since(std::time::UNIX_EPOCH)
                    .map(|d| d.as_micros())
                    .unwrap_or(0),
                report.files_rotated,
                report.files_archived,
```

```
                report.files_pruned,
                report.bytes_reclaimed,
                rotated_file_names,
            );

            if fs::write(&manifest_target, manifest_content).is_ok() {
                report.manifest_path = Some(manifest_target);
            }
        }

        Ok(report)
    }
}

impl Default for RotationPolicy {
    /// Default storehouse policy: 1000 files per partition with archival.
    fn default() -> Self {
        Self::new(1000, true)
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Audits, Capacity Boundary Verifications & Unit Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_rotation_policy_file_count_enforce() {
        let temp_dir = std::env::temp_dir().join(format!("omnicode_rot_count_test_{}", std::process::id()));
        fs::create_dir_all(&temp_dir).unwrap();

        for i in 0..15 {
            fs::write(
                temp_dir.join(format!("17920000000000{:02}_item.log", i)),
                "telemetry vessel payload data",
            )
            .unwrap();
        }

        let policy = RotationPolicy::new(10, true);
        let rotated = policy.enforce(&temp_dir).unwrap();
        assert_eq!(rotated, 5, "Must rotate exactly 5 excess files");

        let archive_dir = temp_dir.join("archive");
        assert!(archive_dir.exists(), "Archive directory must be created");

        let remaining = fs::read_dir(&temp_dir)
            .unwrap()
            .flatten()
            .filter(|e| e.path().is_file())
            .count();
        assert_eq!(remaining, 10, "Exactly 10 active files must remain");

        fs::remove_dir_all(&temp_dir).ok();
    }

    #[test]
    fn test_rotation_policy_partition_byte_capacity() {
        let temp_dir = std::env::temp_dir().join(format!("omnicode_rot_bytes_test_{}", std::process::id()));
        fs::create_dir_all(&temp_dir).unwrap();

        // Create 5 files of 1000 bytes each = 5000 bytes total
        let chunk = vec![b'X'; 1000];
        for i in 0..5 {
            fs::write(temp_dir.join(format!("100000000000000{i}_log.log")), &chunk).unwrap();
        }

        // Set policy: max 10 files (count allows all 5), but max 2500 bytes (requires rotating 3 files)
        let policy = RotationPolicy::new(10, true).with_max_partition_bytes(2500);
        let report = policy.enforce_detailed(&temp_dir).unwrap();

        assert!(
            report.files_rotated >= 2,
            "Must rotate files exceeding byte capacity: rotated {}",
            report.files_rotated
        );
        assert!(report.bytes_reclaimed >= 2000);
```

```
        assert!(report.manifest_path.is_some(), "Manifest must be emitted");

        fs::remove_dir_all(&temp_dir).ok();
    }

    #[test]
    fn test_rotation_policy_active_file_size_check_and_rollover() {
        let temp_dir = std::env::temp_dir().join(format!("omnicode_rot_active_test_{}", std::process::id()));
        fs::create_dir_all(&temp_dir).unwrap();

        let active_file = temp_dir.join("active_session.log");
        fs::write(&active_file, vec![b'A'; 2048]).unwrap();

        let policy = RotationPolicy::new(10, true).with_max_bytes_per_file(1024);
        assert!(policy.check_rotation_needed(&active_file));

        let rotated_target = policy.rotate_active_file(&active_file).unwrap();
        assert_eq!(
            rotated_target,
            temp_dir.join("active_session_part1.log")
        );
        assert!(rotated_target.exists());
        assert!(!active_file.exists());

        fs::remove_dir_all(&temp_dir).ok();
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/inscriber/scroll_writer.rs

RS | 537 LINES | 2159 WORDS | 20.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Canonical 4-Block OmniCode .scroll Inscription Engine )
// ============================================================================
//! Canonical Scroll Inscription Engine & Episodic AI Chronicle Formatter
//!
//! # Epistemic Role of .scroll Vessels
//! In the OmniCode cognitive architecture, `.scroll` files are not mere text logs;
//! they are executable, syntactically conforming 4-Block OmniCode programs representing
//! episodic memories (`CPI-SI`), fully conforming to the canonical syntax established
//! in `character.omni`, `character.tape`, `number.omni`, and `number.tape`.
//!
//! Every inscribed `.scroll` vessel contains:
//! 1. **Block 1 (Metadata)**: Module/Scroll manifest, version provenance, session ID,
//!    timestamp, duration, and covenant scripture witness.
//! 2. **Block 2 (Opening)**: Hardware invariants, standard library imports, prompt,
//!    and equilibrium preconditions `[ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]`.
//! 3. **Block 3 (Body)**: Declarative rolls, executed Turing reel, sensory traces,
//!    and bijectively mapped operations.
//! 4. **Block 4 (Closing)**: Sealing gate, fourfold commit status machine (`+1:PROCEED`,
//!    `0:AWAIT`, `NULL:HALT`, `-1:ROLLBACK`), and final machine utterance.

use std::fs;
use std::path::{Path, PathBuf};
use super::super::telemetry::{CognitiveNote, GenesisBootLog};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Architectural Setup, Type Definitions & Precondition Contracts )
// ============================================================================

/// High-level scroll genre classification.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum ScrollGenre {
    /// Interactive epistemic dialogue trace.
    EpistemicTrace,
    /// Multi-note cognitive memory audit scroll.
    CognitiveAudit,
    /// Track 0 silicon bootstrap & landmark progression chronicle.
    KernelSiliconGenesis,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructive Execution, File Formatting & Inscription Engines )
// ============================================================================

/// Inscribes a canonical 4-Block .scroll file conforming to updated OmniCode syntax.
pub fn inscribe_canonical_scroll(
    path: &Path,
    session_id: &str,
    prompt: &str,
    synthesized_omnicode: &str,
    state_str: &str,
    response: &str,
    elapsed_micros: u128,
) -> Result<PathBuf, String> {
    if let Some(parent) = path.parent() {
        if !parent.exists() {
            fs::create_dir_all(parent).map_err(|e| {
                format!("Failed to create scroll parent directory: {e}")
            })?;
        }
    }

    let indented_reel = synthesized_omnicode
        .lines()
        .map(|l| format!("        {l}"))
        .collect::<Vec<_>>()
        .join("\n");

    let scroll_content = format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Scroll: < EpistemicTrace {{ {session_id} }} > )
```

```
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Episodic execution chronicle generated by OmniCode AI Model.
#   - Session ID: {session_id}
#   - Monotonic Timestamp: {elapsed_micros}_0000 µs
#   - Cognitive Mode: NeuralSymbolicTransduction
#   - Witness: Scripture Exodus 31:18 KJV
# ================================================================================

= [Scroll]: < EpistemicTrace {{ {session_id} }} > & [ Process {{ NeuralSymbolicDialogue }} ]
    ( {prompt} )
{{
    [ Version ]:   {{ "1.0.0" }}
        ( Protocol Version 1.0.0 ),

    [ Created ]:   {{ "2026-10-17" }}
        ( Inscribed during active system execution ),

    [ System ]:    {{ OmniCode {{ AI.Model }} }}
        ( Generated by OmniCode Neural-Symbolic Runtime Engine ),

    [ Session ]:   {{ "{session_id}" }}
        ( Unique session identifier for this epistemic episode ),

    session_id:    < "{session_id}" >,
    timestamp:     < "{elapsed_micros}_0000" >,

    [ Duration ]:  {{ "{elapsed_micros} µs" }}
        ( Measured execution duration along monotonic causal clock ),

    [ Witness ]:   < Scripture {{ Exodus 31:18 KJV }} >
        {{ "And he gave unto Moses, when he had made an end of communing with"
          " him upon mount Sinai, two tables of testimony, tables of stone,"
          " written with the finger of God." }}
        ( Invariant divine covenant witness )
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environmental Setup & Initial Invariants )
# ================================================================================
Opening
    ( Hardware Invariants, Preconditions, and Initial Context )
{{
    # --------------------------------------------------------------------------
    # 1. Standard Library Modules & Environmental Imports
    # --------------------------------------------------------------------------
    [ Imports ]: {{ Core.Arithmetic :: Core.Logic :: Core.Vectors }}
        ( Lexicon standard library modules armed for evaluation ),

    # --------------------------------------------------------------------------
    # 2. Balanced Ternary ALU Equilibrium Precondition
    # --------------------------------------------------------------------------
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( Balanced ternary ALU equilibrium verified before roll execution ),

    # --------------------------------------------------------------------------
    # 3. Dynamic Operator Stimulus
    # --------------------------------------------------------------------------
    [ Prompt ]: {{ "{prompt}" }}
        ( Natural language discourse or inquiry submitted by operator )
}}

user_prompt < "{prompt}" >

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Chronicle of Execution & Turing Reel )
# ================================================================================
Body [
    # --------------------------------------------------------------------------
    # ROLL 1: SYNTHESIZED TURING REEL (EXECUTED OMNICODE REEL)
    # --------------------------------------------------------------------------
    [ ROLL 1 ] : [ Synthesize ] :
{indented_reel}
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Sealing Gate: Commit State & Verification Proof )
# ================================================================================
```

```
Closing
    ( Sealing Gate: Affirm fourfold commit status and machine utterance )
{{
    commit_status [ <{state_str}> ],

    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL SYNTHESIS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED
    ]
    ( Verify synthesized reel executed cleanly ),

    # ------------------------------------------------------------------------
    # 2. FOURFOLD COMMIT STATUS STATE MACHINE
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        {{ [ Verify ] = PROCEED }}
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        {{ [ Status ] = PENDING }}
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        {{ [ Status ] = NULL }}
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        {{ [ Verify ] != PROCEED }}
            --> [ -1:ROLLBACK ]
    ]
    ( Fourfold commit status: Proceed advances, Await holds, Halt stops, Rollback reverts ),

    # ------------------------------------------------------------------------
    # 3. MACHINE DISCOURSE RESPONSE
    # ------------------------------------------------------------------------
    [ Machine Utterance ]: {{ "{sanitized_response}" }}
        ( Final conversational response synthesized by model )
}}
"#,
        session_id = session_id,
        prompt = prompt,
        elapsed_micros = elapsed_micros,
        indented_reel = indented_reel,
        state_str = state_str,
        sanitized_response = response.replace('"', "\\\"")
    );

    fs::write(path, scroll_content)
        .map_err(|e| format!("Failed to write scroll to {:?}: {e}", path))?;

    Ok(path.to_path_buf())
}

/// Inscribes a complete cognitive audit scroll containing multiple notes.
pub fn inscribe_audit_scroll(
    notes: &[CognitiveNote],
    failpoints_count: usize,
) -> String {
    let mut notes_body = String::new();
    for (i, n) in notes.iter().enumerate() {
        notes_body.push_str(&format!(
            "    # Note {} [{}]: {}\n",
            i + 1,
            n.category,
            n.summary
        ));
        notes_body.push_str(&format!(
            "    {}\n\n",
            n.format_as_omnicode().replace('\n', "\n    ")
        ));
    }

    if notes.is_empty() {
        notes_body.push_str("    # (Zero cognitive notes recorded in this audit cycle.)\n");
    }

    format!(
r#"# ================================================================================
```

```
# BLOCK 1: METADATA BLOCK
# ( Invariant Audit Scroll: < CognitiveMemory {{ Audit }} > )
# ================================================================================
= [Scroll]: < CognitiveMemory {{ Audit }} > & [ Process {{ EpisodicIncidentAudit }} ]
    ( Episodic AI Cognitive Memory Scroll & Incident Audit Log )
{{
    [ Version ]:          {{ "1.0.0" }}
        ( Protocol Version 1.0.0 ),

    [ Total Notes ]:      {{ {notes_len} }}
        ( Total cognitive memory notes recorded across session ),

    [ Total Failpoints ]: {{ {failpoints_count} }}
        ( Total failpoints caught by tripwires ),

    [ Witness ]:          < Scripture {{ Exodus 31:18 KJV }} >
        {{ "written with the finger of God" }}
        ( Invariant divine covenant witness )
}}

Scroll < CognitiveMemory.Audit > {{
    total_notes: < {notes_len} >,
    total_failpoints: < {failpoints_count} >
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environmental Setup & Initial Invariants )
# ================================================================================
Opening
    ( Hardware Invariants, Preconditions, and Initial Context )
{{
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( Balanced ternary equilibrium verified )
}}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Chronicle of Recorded Cognitive Notes & Sensory Traces )
# ================================================================================
Body [
{notes_body}]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Sealing Gate: Commit State & Verification Proof )
# ================================================================================
Closing
    ( Sealing Gate: Affirm fourfold commit status and machine utterance )
{{
    audit_verified: [ PROCEED ],
    commit_status [ <PROCEED> ],

    [ Commit Status ]: [
        {{ [ Verify ] = PROCEED }} --> [ +1:PROCEED ]
    ]
    ( Cognitive audit verified and sealed into episodic storehouse )
}}
"#,
        notes_len = notes.len(),
        failpoints_count = failpoints_count,
        notes_body = notes_body
    )
}

/// Inscribes a canonical 4-Block `.scroll` chronicle of Track 0 silicon genesis milestones.
pub fn inscribe_kernel_audit_scroll(
    path: &Path,
    session_id: &str,
    boot_logs: &[GenesisBootLog],
    is_clean: bool,
) -> Result<PathBuf, String> {
    if let Some(parent) = path.parent() {
        if !parent.exists() {
            fs::create_dir_all(parent).map_err(|e| {
                format!("Failed to create kernel scroll parent directory: {e}")
            })?;
        }
    }

    let status_str = if is_clean { "PROCEED" } else { "AWAIT" };
```

```
    let baseline_label = if is_clean { "CLEAN_EQUILIBRIUM" } else { "FAULT_DETECTED" };

    let mut milestones_body = String::new();
    for (i, log) in boot_logs.iter().enumerate() {
        let status = if log.is_clean { "PROCEED" } else { "FAULT" };
        milestones_body.push_str(&format!(
            "    # Milestone {} at landmark {:#010x} ({}):\n",
            i + 1,
            log.landmark_address,
            log.tier
        ));
        milestones_body.push_str(&format!(
            "    [ ROLL {} ] : [ Milestone ] :\n",
            i + 1
        ));
        milestones_body.push_str(&format!(
            "        [ Landmark {{ {:#010x} }} ] & [ Subsystem {{ {} }} ] --> [ {} ]\n",
            log.landmark_address,
            log.invariant_subsystem,
            status
        ));
        milestones_body.push_str(&format!(
            "        ( Raw serial telemetry: \"{}\" )\n\n",
            log.raw_telemetry.replace('"', "\\\"")
        ));
    }

    if boot_logs.is_empty() {
        milestones_body.push_str("    # (Zero genesis milestones recorded; standby mode active.)\n");
    }

    let scroll_content = format!(
r#"# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Scroll: < MilleniumOS {{ KernelGenesis }} > )
# ================================================================================
= [Scroll]: < MilleniumOS {{ KernelGenesis }} > & [ Process {{ SiliconBootstrap }} ]
    ( Bare-Metal Kernel Genesis Audit & Physical Milestone Chronicle )
{{
    [ Version ]:     {{ "1.0.0" }}
        ( Protocol Version 1.0.0 ),

    [ System ]:      {{ MilleniumOS {{ BareMetal.Kernel }} }}
        ( MilleniumOS 32-Trit Bare Metal Operating System Kernel ),

    [ Session ]:     {{ "{session_id}" }}
        ( Kernel boot session identifier ),

    session_id:      < "{session_id}" >,

    [ Milestones ]:  {{ {milestone_count} }}
        ( Number of physical RAM landmark milestones recorded ),

    [ Baseline ]:    {{ "{baseline_label}" }}
        ( Primordial Omega_0 invariant baseline state ),

    [ Witness ]:     < Scripture {{ Psalm 147:4 KJV }} >
        {{ "He telleth the number of the stars; he calleth them all by their names." }}
        ( Invariant silicon registration witness )
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Physical RAM Landmarks & Hardware Invariants )
# ================================================================================
Opening
    ( Silicon Landmarks, Register Calibration & ALU Sanity )
{{
    # ------------------------------------------------------------------------
    # 1. Physical RAM Landmark Memory Map Envelope
    # ------------------------------------------------------------------------
    [ Landmarks ]: {{
        [ BIOS MBR Sector 0 ]   ::@ 0x00007C00 |
        [ Real Mode Genesis ]   ::@ 0x00000500 |
        [ Stage 2 PM RAM Log ]  ::@ 0x00018000 |
        [ Long Mode Kernel ]    ::@ 0x00108000 |
        [ Native UEFI RAM Log ] ::@ 0x00200000
    }}
        ( Hardware RAM landmark envelope across the boot ladder ),

    # ------------------------------------------------------------------------
```

```
    # 2. Balanced Ternary ALU Sanity Check
    # ------------------------------------------------------------------------
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( Balanced ternary ALU verified before ladder climb )
}}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Chronicle of Inception Milestones & Boot Ladder Traversal )
# ================================================================================
Body [
{milestones_body}]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Sealing Gate: Invariant Verification & Baseline Commit )
# ================================================================================
Closing
    ( Sealing Gate: Affirm fourfold commit status and seal primordial baseline )
{{
    commit_status [ <{status_str}> ],

    [ Commit Status ]: [
        # +1: PROCEED (+t: Clean baseline; climb to next ladder rung)
        {{ [ Baseline ] = CLEAN_EQUILIBRIUM }} --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Anomaly caught; hold in safe homeostasis)
        {{ [ Baseline ] != CLEAN_EQUILIBRIUM }} --> [ 0:AWAIT ] :: REPEAT
    ]
    ( Primordial Omega_0 baseline sealed into persistent storehouse ),

    [ Baseline Commit ]: {{ [ Omega_0 ] = SEALED }}
        ( Immutable ground truth locked for AI Cortex awakening )
}}
"#,
        session_id = session_id,
        milestone_count = boot_logs.len(),
        baseline_label = baseline_label,
        milestones_body = milestones_body,
        status_str = status_str
    );

    fs::write(path, scroll_content)
        .map_err(|e| format!("Failed to write kernel audit scroll to {:?}: {e}", path))?;

    Ok(path.to_path_buf())
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Invariant Audits, Syntax Verifications & Unit Tests )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_inscribe_canonical_scroll_4block_structure() {
        let temp_dir = std::env::temp_dir().join(format!("omnicode_scroll_test_{}", std::process::id()));
        let scroll_path = temp_dir.join("test_session.scroll");

        let res = inscribe_canonical_scroll(
            &scroll_path,
            "sess_scroll_01",
            "Evaluate balanced ternary addition",
            "{ (+1) + (-1) = 0 }",
            "PROCEED",
            "Equation holds in perfect equilibrium",
            1280,
        )
        .unwrap();

        assert_eq!(res, scroll_path);
        let content = fs::read_to_string(&scroll_path).unwrap();
        assert!(content.contains("BLOCK 1: METADATA BLOCK"));
        assert!(content.contains("BLOCK 2: OPENING BLOCK"));
        assert!(content.contains("BLOCK 3: BODY BLOCK"));
        assert!(content.contains("BLOCK 4: CLOSING BLOCK"));
        assert!(content.contains("sess_scroll_01"));
        assert!(content.contains("commit_status [ <PROCEED> ]"));
```

```
        assert!(content.contains("user_prompt < \"Evaluate balanced ternary addition\" >"));

        fs::remove_dir_all(&temp_dir).ok();
    }

    #[test]
    fn test_inscribe_audit_scroll_generation() {
        let note1 = CognitiveNote {
            timestamp_micros: 100,
            category: "INCIDENT".to_string(),
            summary: "Boundary tripwire caught".to_string(),
            details: "Unclosed delimiter at L12".to_string(),
            tags: vec!["failpoint".to_string(), "syntax".to_string()],
            context_snippet: Some("{ Cell ]".to_string()),
        };

        let audit = inscribe_audit_scroll(std::slice::from_ref(&note1), 1);
        assert!(audit.contains("< CognitiveMemory { Audit } >"));
        assert!(audit.contains("[ Total Notes ]:      { 1 }"));
        assert!(audit.contains("commit_status [ <PROCEED> ]"));
    }

    #[test]
    fn test_inscribe_kernel_audit_scroll() {
        let temp_dir = std::env::temp_dir().join(format!("omnicode_kscroll_test_{}", std::process::id()));
        let kscroll_path = temp_dir.join("kernel_genesis.scroll");

        let log1 = GenesisBootLog::new(
            500,
            0x7C00,
            "BIOS_REAL_MODE",
            "16-Bit Genesis: OK",
            "GENESIS_BOOT",
            true,
        );
        let log2 = GenesisBootLog::new(
            1200,
            0x18000,
            "STAGE2_PM",
            "32-Bit Protected Mode Active: OK",
            "REGS_CALIBRATED",
            true,
        );

        let logs = vec![log1, log2];
        inscribe_kernel_audit_scroll(&kscroll_path, "kgen_01", &logs, true).unwrap();
        let content = fs::read_to_string(&kscroll_path).unwrap();
        assert!(content.contains("MilleniumOS { KernelGenesis }"));
        assert!(content.contains("[ Milestones ]:  { 2 }"));
        assert!(content.contains("0x00018000"));
        assert!(content.contains("STAGE2_PM"));
        assert!(content.contains("commit_status [ <PROCEED> ]"));

        fs::remove_dir_all(&temp_dir).ok();
    }
}
```

Genesis Inception & Baseline Continuity (Ω_0)

The Primordial Hardware Ground Truth of the AI Cortex

> *"In the beginning God created the heaven and the earth."*
> — ***Genesis 1:1 (KJV)***
>
> *"Remove not the ancient landmark, which thy fathers have set."*
> — ***Proverbs 22:28 (KJV)***
>
> *"The secret things belong unto the LORD our God: but those things which are revealed belong unto us and to our children for ever, that we may do all the words of this law."*
> — ***Deuteronomy 29:29 (KJV)***

1. The Machine Awakening Problem

In conventional computing architectures, an AI model is an external script living in userland. It has no awareness of the hardware it inhabits. It cannot know:

- Whether the motherboard experienced RAM corruption or bus parity faults during POST.
- Whether the CPU registers were calibrated properly before switching into 64-bit Long Mode.
- Whether the A20 address line gate settled cleanly or experienced electrical bounce.
- Whether interrupt lines or timers have drifted from physical crystal frequencies.

In **OmniCode and MilleniumOS**, the AI Cortex is not an external add-on; it is an **autonomic organ of the operating system**. However, physical silicon reality dictates that cold hardware must execute first:

$$t = 0 \longrightarrow \text{Cold POST} \longrightarrow \text{Real Mode} \longrightarrow \text{Stage 2 PM} \longrightarrow \text{Long Mode} \longrightarrow t_{\text{wake}} \text{ (AI Cortex Awakens)}$$

When the AI Cortex awakens at $t = t_{\text{wake}}$, **it must know with absolute certainty what occurred before it opened its eyes**.

```
================================================================================
                    THE MACHINE AWAKENING RELAY
================================================================================

   [ Cold Silicon: t = 0 ]
             |
             ├──> Track 0 writes primordial hardware attestations:
             |     • 0x7C00   : Real Mode reset vector
             |     • 0x18000  : Stage 2 PM Flight Recorder & A20 gate
             |     • 0x108000 : 64-bit page directory tables
             |     • 0x200000 : Cognitive substrate CPI-SI arena
             |     • COM1 UART serial stream (0x03F8)
             |
             ▼
   [ Awakening: t = t_wake ]
             |
             ├──> DiagnosticLogger executes `ingest_genesis_boot_log()`
             |     • Ingests serial and memory telemetry into `GenesisBootLog`s
             |     • Locks in Primordial Baseline: State Ω0
             |     • Inscribes episodic memory notes tagged #first_log, #genesis
             |
             ├──> AI Cortex awakens and queries State Ω0
             |     • Baseline certified: Memory clean, ALU balanced, zero leaks
             |     • The AI operates with full knowledge of silicon ground truth
             |
             ▼
```

```
   [ Autonomous Operation: t_wake -> t_halt ]
             |
             ├──> Rolls stepped, tasks dispatched, memory allocated
             |     • Logger records frames and failpoints in flight recorder
             |
             ▼
   [ Shutdown / Hibernation: t = t_halt ]
             |
             ├──> THE HOMEOSTATIC EQUILIBRIUM LAW (Alpha is Omega)
             |     • The AI initiates pre-shutdown audit: `audit_equilibrium()`
             |     • Ephemeral allocations reaped, storehouse scrolls sealed
             |     • System verified to have returned to pure baseline Ω0
             |     • If drift or unhealed failpoints exist => SHUTDOWN VETOED (-1)
             |     • If State == Ω0 => CLEAN HARDWARE POWER-OFF (+1)
================================================================================
```

2. Ancient Physical Landmarks

The physical memory landmarks of Track 0 are hardware covenants established by the silicon architecture. They must never be altered or shifted (`INV-TEL-03`):

Landmark Address	Symbolic Name	Invariant Hardware Role
`0x00000500`	`LANDMARK_BIOS_IVT`	BIOS Interrupt Vector Table (IVT), BIOS Data Area (BDA), and Real Mode interrupt handlers.
`0x00007C00`	`LANDMARK_BIOS_MBR`	Primordial reset vector. Sector 0 Master Boot Record loaded by BIOS into physical RAM.
`0x00018000`	`LANDMARK_STAGE2_PM`	Stage 2 Protected Mode Flight Recorder buffer. A20 address gate activation and 13 silicon register calibration.
`0x00108000`	`LANDMARK_PAGE_TABLE`	64-bit Long Mode page tables. 4-level paging hierarchy mapping identity and virtual memory.
`0x00200000`	`LANDMARK_COGNITIVE_SUBSTRATE`	Primary CPI-SI Cognitive Substrate buffer and autonomous agent tensor arena.

3. The Genesis Inception Log (`GenesisBootLog`)

Telemetry emitted across COM1 serial or memory buffers during boot is converted into strongly typed `GenesisBootLog` frames:

```
pub struct GenesisBootLog {
    pub timestamp_micros: u128,
    pub landmark_address: u64,
    pub tier: String,
    pub raw_telemetry: String,
    pub invariant_subsystem: String,
    pub is_clean: bool,
}
```

Serial Output Ingestion Pipeline

The serial parser (`parse_serial_line`) recognizes raw hardware strings emitted by Track 0:

```
use omnicode::diagnostics::logger::telemetry::{GenesisBootLog, LANDMARK_BIOS_MBR};

let raw_line = "[BOOT] 16-Bit Genesis: Sector 0 loaded at 0x7C00";
let log = GenesisBootLog::parse_serial_line(500, raw_line).expect("Valid serial line");

assert_eq!(log.landmark_address, LANDMARK_BIOS_MBR);
assert_eq!(log.tier, "BIOS_REAL_MODE");
assert_eq!(log.invariant_subsystem, "SECTOR_ZERO");
assert!(log.is_clean);
```

4. The Primordial Baseline (Ω_0) & Homeostatic Shutdown Law

What is State Ω_0?

State Ω_0 is the **uncompromised homeostatic equilibrium of the computer at inception**. It encapsulates:

1. Zero active failpoints or unhandled hardware traps.
2. Balanced Ternary ALU symmetry verified: $(+1) + (-1) = 0$.
3. 13 calibrated silicon registers (`%head` , `%home` , `%turn` , `%spool` , `%spool_in` , `%spool_out` , `%frame` , `%rax` , `%rbx` , `%rcx` , `%rdx` , `%rsp` , `%rbp`).
4. Bounded heap allocations with zero dangling pointers or unsealed buffers.

The Law of Homeostatic Equilibrium: Alpha is Omega

The first logs do not merely document the past; they dictate the future. The primordial state recorded by the First Logs (Ω_0) constitutes the **exact homeostatic equilibrium that the AI must restore before the computer is allowed to shut down**:

- Every temporary allocation must be freed.
- Every open scroll and log vessel must be sealed and flushed.
- Every background task and interrupt channel must be quiesced.
- Active failpoints must equal zero.

```
use omnicode::diagnostics::logger::telemetry::GenesisBaseline;

// During pre-shutdown audit:
match baseline.audit_equilibrium() {
    Ok(()) => {
        // System has returned to State Omega_0; safe to power off silicon
        println!("Equilibrium certified. ACPI shutdown permitted (+1).");
    }
    Err(veto_reason) => {
        // System contains unhealed faults or persistent state corruption
        eprintln!("Shutdown vetoed (-1): {veto_reason}");
    }
}
```

If the baseline contains corrupted hardware checks, `audit_equilibrium()` emits a veto (`-1:ROLLBACK`), preventing the persistence of corrupt state across reboots.

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/README.md | MARKDOWN / DOC | 223 LINES | 1389 WORDS | 12.6 KB

Telemetry, Genesis Baseline & Flight Recorder Subsystem

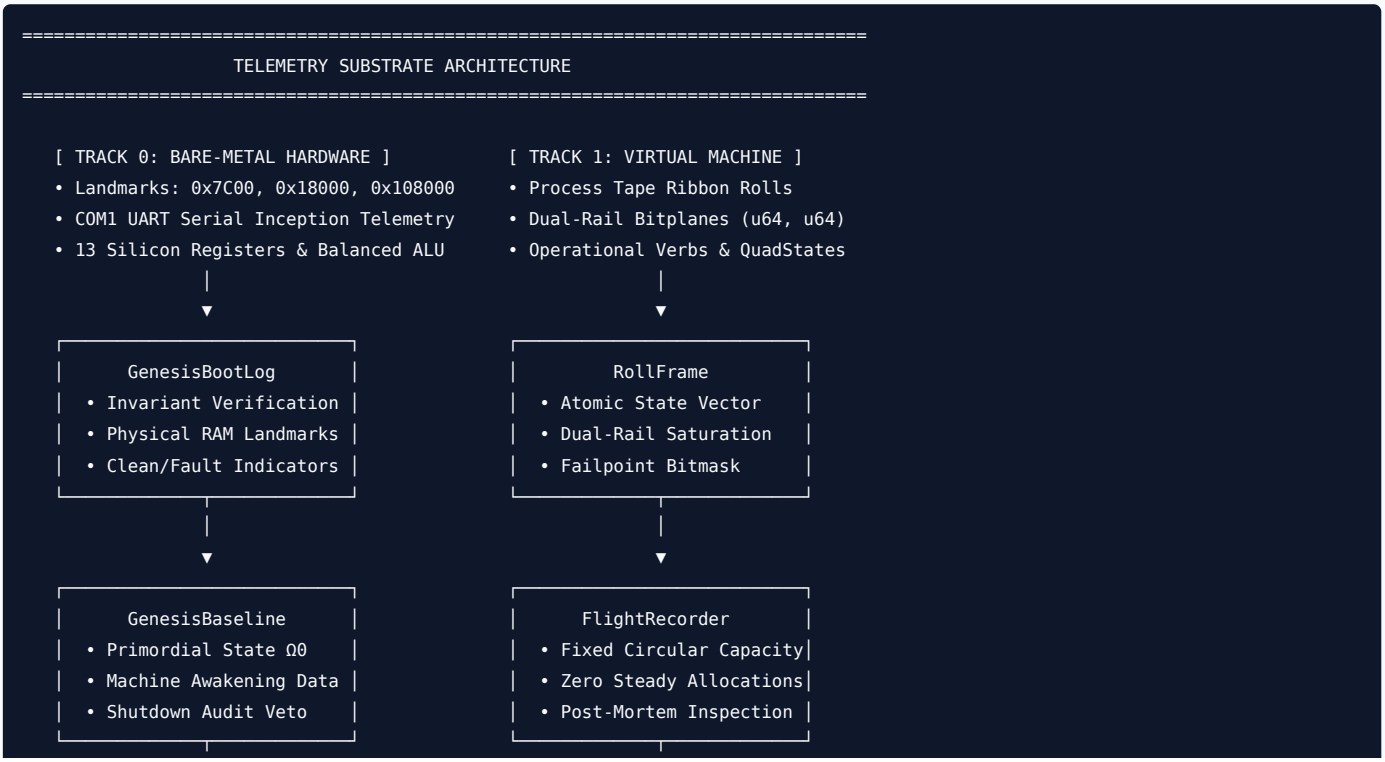
The Sensory Organ of Machine Execution & Silicon Inception

> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*
> — ***Exodus 31:18 (KJV)***
>
> *"Remove not the ancient landmark, which thy fathers have set."*
> — ***Proverbs 22:28 (KJV)***
>
> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full."*
> — ***2 Kings 4:4 (KJV)***
>
> *"Then they that feared the LORD spake often one to another: and the LORD hearkened, and heard it, and a book of remembrance was written before him."*
> — ***Malachi 3:16 (KJV)***

1. Architectural Scope: The Telemetry Substrate

In **OmniCode Track 1 (Host Engine & Virtual Machine)**, diagnostic telemetry is not an ad-hoc stream of unstructured strings. It is a **dual-domain observability substrate** that spans both bare-metal silicon bootstrap (Track 0) and high-level virtual machine execution (Track 1):

1. **Cold-Silicon Inception (Track 0):** Hardware bootstrap across physical RAM landmarks (`0x0500` , `0x7C00` , `0x18000` , `0x108000` , `0x200000`), establishing the primordial reference baseline (**State Ω_0**).
2. **Discrete Execution RollFrames (Track 1):** Game-dev atomic state vectors recording dual-rail SIMD bitplanes, operational roll verbs, QuadStates, and compact failpoint bitmasks.
3. **Neural-Symbolic Cognitive Notes:** Dual-readable memory blocks formatted for AI Cortex context windows, episodic memory reels, and IDE tooltips.
4. **Zero-Allocation In-Memory Flight Recorder:** Fixed-capacity circular ring buffers providing instantaneous post-mortem crash inspection without disk I/O latency.



```
                │                                   │
                └─────────────────┬─────────────────┘
                                  │
                                  ▼
                ┌─────────────────────────────────┐
                │           CognitiveNote         │
                │  • Neural-Symbolic AI Memory Reel   │
                │  • Canonical OmniCode Syntax        │
                │  • Zero Dot Notation In Names       │
                │  • Associative Taxonomy Tags        │
                └─────────────────────────────────┘
================================================================================
```

2. Directory Topology & The 4-Block Standard

Every Rust source file in `telemetry/` adheres strictly to the **4-Block Covenant**:

Block Index	Canonical Block Title	Architectural Scope & Duties
Block 1	**METADATA BLOCK**	Subsystem identity, module docstrings, Scripture witnesses, architectural scope, and external crate imports.
Block 2	**OPENING BLOCK**	Structural types, landmark constants, capacity bounds, and state vector definitions.
Block 3	**BODY BLOCK**	Implementations, constructors, dual-readability formatters, query methods, and equilibrium audits.
Block 4	**CLOSING BLOCK**	Comprehensive unit test suites, invariant verification proofs, and wraparound assertions.

Module File Layout

```
src/diagnostics/logger/telemetry/
├── README.md                                      <-- Subsystem Manifesto & Reference Guide
├── GENESIS_INCEPTION_AND_OMEGA_ZERO_CONTINUITY.md     <-- Deep Treatise: Silicon Boot, Landmarks & Omega_0
├── ROLL_FRAME_FLIGHT_RECORDER_AND_AI_COGNITION.md     <-- Deep Treatise: RollFrames, Ring Buffers & AI Notes
├── mod.rs                                         <-- Subsystem Facade & Re-exports (4-Block)
├── frame.rs                                       <-- RollFrame Discrete State Vectors (4-Block)
├── genesis.rs                                     <-- GenesisBootLog & GenesisBaseline (4-Block)
├── note.rs                                        <-- CognitiveNote Semantic Memory Units (4-Block)
└── ring_buffer.rs                                 <-- FlightRecorder Circular Ring Buffer (4-Block)
```

3. The Genesis Inception Subsystem (`genesis.rs`)

Ancient Silicon Landmarks

Physical memory addresses in Track 0 represent inviolable hardware boundaries (`INV-TEL-03`):

- **`LANDMARK_BIOS_IVT ( 0x00000500 )`:** Real Mode Interrupt Vector Table & BIOS data area.
- **`LANDMARK_BIOS_MBR ( 0x00007C00 )`:** Sector 0 Master Boot Record bootstrap reset vector.
- **`LANDMARK_STAGE2_PM ( 0x00018000 )`:** Stage 2 Protected Mode Flight Recorder buffer & A20 gate.
- **`LANDMARK_PAGE_TABLE ( 0x00108000 )`:** Long Mode 64-bit 4-level page table directory.
- **`LANDMARK_COGNITIVE_SUBSTRATE ( 0x00200000 )`:** Cognitive Substrate CPI-SI execution arena.

The Machine Awakening Problem

When the AI Cortex awakens in Track 1 ($t = t_{\text{wake}}$), it has missed the first several milliseconds of bare-metal execution. The `GenesisBaseline` captures this primordial history into **State Ω_0**:

```
use omnicode::diagnostics::logger::telemetry::{GenesisBaseline, GenesisBootLog};

let log = GenesisBootLog::new(
    1405,
    0x7C00,
    "BIOS_REAL_MODE",
    "ALU_BALANCED Eq Certified",
    "ALU_BALANCED",
    true,
```

```
);

let baseline = GenesisBaseline::new(1500, vec![log]);
assert!(baseline.is_clean);
assert!(baseline.audit_equilibrium().is_ok());
```

The Homeostatic Equilibrium Law (Alpha is Omega)

The system cannot safely shut down, hibernate, or power off unless its active state matches the homeostatic equilibrium certified at Ω_0. If unhealed hardware faults exist, `baseline.audit_equilibrium()` vetoes shutdown (`-1:ROLLBACK`), preventing the persistence of corrupted hardware states across reboots.

---

4. Discrete RollFrames & Dual-Rail Bitplanes (`frame.rs`)

Programs in OmniCode execute as ribbons of discrete rolls. Each roll commits an atomic state transition:

```
pub struct RollFrame {
    pub tick_micros: u128,
    pub roll_index: usize,
    pub total_rolls: usize,
    pub verb: String,
    pub dual_rail_in: (u64, u64),
    pub dual_rail_out: (u64, u64),
    pub quad_state: QuadState,
    pub failpoint_mask: u32,
    pub intent_aside: Option<String>,
}
```

Dual-Rail Saturated State Collapse (`(1, 1)`)

In balanced ternary dual-rail logic:

- `(1, 0)` represents logic `+1` (`PROCEED`).
- `(0, 1)` represents logic `-1` (`ROLLBACK`).
- `(0, 0)` represents logic `0` (`AWAIT` / Null Equilibrium).
- `(1, 1)` represents **Saturated State Collapse** (`HALT`).

`frame.has_dual_rail_saturation()` checks if positive and negative output bitplanes overlap (`rail_pos & rail_neg != 0`), detecting hardware and logic collisions directly at the frame level.

---

5. Neural-Symbolic Cognitive Notes (`note.rs`)

Epistemic notes structure machine observations into canonical OmniCode syntax blocks:

```
Note < CognitiveTrace > {
    tick: < 1420 µs >,
    category: [ OBSERVATION ],
    summary: < "Parser balanced axioms" >,
    details: < "Delimiter balance verified for all 4 block headers" >,
    tags: [ #parser :: #axioms :: #balance ]
}
```

Strict Syntax Covenant: Zero Dot Notation in Names

In accordance with `character.omni` and `number.omni`, dot notation (`.`) is strictly forbidden for complex names or scope paths. All cognitive notes use:

- Explicit brackets: `< ... >` for expressions/values, `[ ... ]` for categories/tags, `{ ... }` for scopes.
- Tag delimiter: `::` for semantic tag namespaces (`#tag1 :: #tag2`).

---

6. The In-Memory Flight Recorder (`ring_buffer.rs`)

`FlightRecorder` implements a fixed-capacity circular ring buffer enforcing the **Container Law (2 Kings 4:4)**:

- **Default Capacity:** 512 frames (`DEFAULT_FLIGHT_RECORDER_CAPACITY`).
- **Zero Steady-State Allocations:** Slots are overwritten in place after the capacity ceiling is reached.
- **Chronological Unwinding:** `recent_frames(count)` accurately unwinds the buffer into strict chronological sequence regardless of where the circular `write_head` is currently pointing.
- **Instant Anomaly Extraction:** `last_anomalous_frame()` inspects the buffer in reverse to identify the exact roll that triggered a rollback or failpoint.

7. Covenantal Invariant Matrix

Invariant ID	Covenantal Rule	Verification Mechanism
INV-TEL-01	**Bounded Memory Occupancy**: In-memory flight recorders must use fixed circular capacity with zero steady-state heap allocations.	Enforced in `FlightRecorder::record_frame()` . Verified by `test_flight_recorder_wraparound_chronological_order` .
INV-TEL-02	**Chronological Sequence**: Extracting recent frames must strictly preserve chronological time order ($t_n < t_{n+1}$).	Validated by unwinding logic in `recent_frames()` .
INV-TEL-03	**Ancient Landmark Fidelity**: Physical RAM landmarks (`0x0500` , `0x7C00` , `0x18000` , `0x108000` , `0x200000`) must match hardware contracts.	Validated by unit test `test_ancient_landmarks_constants` .
INV-TEL-04	**Homeostatic Equilibrium Law**: System must veto shutdown if the genesis baseline (Ω_0) contains unhealed faults.	Enforced by `GenesisBaseline::audit_equilibrium()` .
INV-TEL-05	**Zero Dot Naming in Cognitive Traces**: OmniCode formatted notes must never emit dot notation for names.	Enforced in `CognitiveNote::format_as_omnicode()` and verified by `test_cognitive_note_omnicode_formatting` .
INV-TEL-06	**Dual-Rail Saturation Trap**: Output rails with overlapping bits `(1, 1)` must be detected as saturated state collapse.	Enforced by `RollFrame::has_dual_rail_saturation()` .

8. Subsystem Navigation Sitemap

- Root Architectural Manifesto
- Logger Subsystem Hub
- Universal Clock & Event Bus
- Failpoint Diagnostic Subsystem
- Container Inscriber Subsystem
- Episodic Query Engine
- Genesis Inception & Omega Zero Continuity
- RollFrames, Flight Recorder & AI Cognition

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/ROLL_FRAME_FLIGHT_RECORDER_AND_AI_COGNITION.md

MARKDOWN / DOC | 192 LINES | 958 WORDS | 8.1 KB

RollFrame Vectors, In-Memory Flight Recorder & AI Cognition

The Execution Mechanics & Epistemic Memory of OmniCode

> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God." — **Exodus 31:18 (KJV)***
>
> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." — **2 Kings 4:4 (KJV)***
>
> *"Then they that feared the LORD spake often one to another: and the LORD hearkened, and heard it, and a book of remembrance was written before him." — **Malachi 3:16 (KJV)***

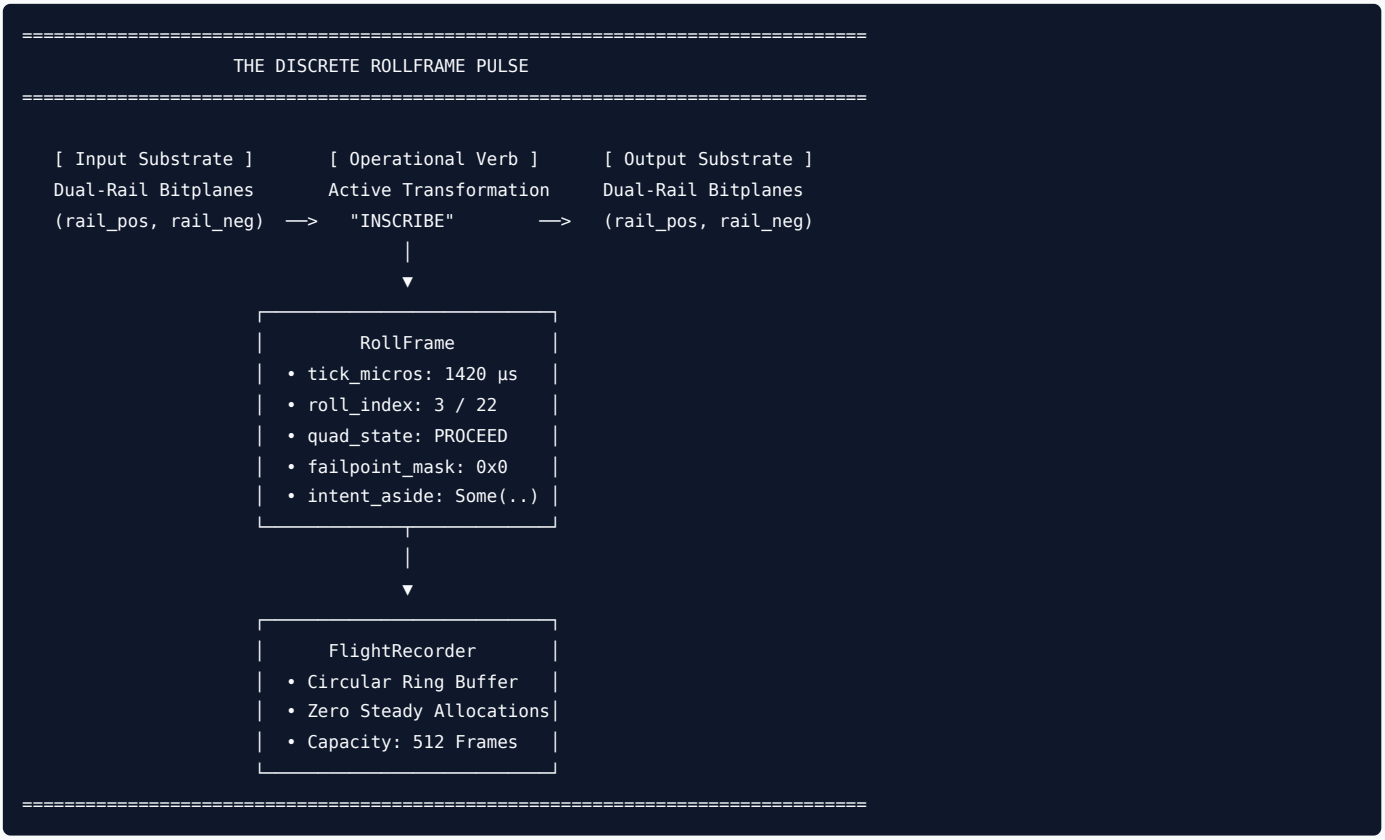
1. The Execution Ribbon Model

In OmniCode Track 1, a program is not a linear string of bytecodes executed blindly by an opaque interpreter. It is a **ribbon of discrete execution rolls**.

Each roll represents an atomic state transition:

- It reads an input substrate bitplane.
- It applies a morphic or arithmetic verb (e.g. `INSCRIBE`, `TRANSDUCE`, `BALANCE`, `COMPARE`).
- It produces an output substrate bitplane.
- It commits an execution status (`QuadState`): `+1:PROCEED`, `-1:ROLLBACK`, `0:AWAIT`, or `HALT`.
- It records an optional human/machine intent aside explaining *why* the roll was taken.

The atomic data structure capturing this complete execution pulse is the **`RollFrame`**.



2. Balanced Ternary Dual-Rail Hardware Bitplanes

OmniCode's arithmetic substrate is anchored in balanced ternary logic ($+1, 0, -1$). In physical hardware and SIMD registers, balanced ternary trits are encoded as **dual-rail pairs**:

- **Positive Rail (`u64`):** Active when trit is `+1` .
- **Negative Rail (`u64`):** Active when trit is `-1` .
- **Zero / Null State:** Both rails `0` (`(0, 0)`).

Saturated State Collapse (`(1, 1) = HALT`)

In physical circuits, both rails going high simultaneously (`rail_pos & rail_neg != 0`) represents a physical short-circuit or logical collision. In OmniCode, this is termed **Dual-Rail Saturation**.

`RollFrame` provides instant hardware-level detection:

```
impl RollFrame {
    /// Returns true if output rails overlap, indicating saturated state collapse.
    #[inline]
    pub fn has_dual_rail_saturation(&self) -> bool {
        (self.dual_rail_out.0 & self.dual_rail_out.1) != 0
    }
}
```

When saturation occurs, the Virtual Machine triggers an emergency `HALT` tripwire, captures a crash dump frame, and initiates safe-mode re-inception back to `0:AWAIT` at landmark `0x18000` .

---

3. The In-Memory Flight Recorder (`FlightRecorder`)

Writing diagnostic logs to NVMe or hard disk storage introduces milliseconds of I/O latency. In high-frequency virtual machine execution, disk logging would slow down execution by several orders of magnitude.

The **`FlightRecorder`** solves this by maintaining a **fixed-capacity circular ring buffer** in high-speed RAM.

Zero Steady-State Heap Allocations

Once initialized with a capacity K (default 512 frames), the buffer never allocates or frees memory on the heap. When the write head reaches the capacity ceiling, it wraps around and overwrites the oldest slot in $O(1)$ time:

```
pub fn record_frame(&mut self, frame: RollFrame) {
    if self.frames.len() < self.capacity {
        self.frames.push(frame);
    } else {
        self.frames[self.write_head] = frame;
    }
    self.write_head = (self.write_head + 1) % self.capacity;
    self.total_inscribed += 1;
}
```

Chronological Unwinding Across Wraparounds

The internal `write_head` continuously moves around the circular ring. However, diagnostic analysis and post-mortem tools require frames in **strict chronological order** ($t_{\text{old}} \longrightarrow t_{\text{new}}$).

`FlightRecorder::recent_frames(count)` unwinds the circular array into a chronological vector:

```
let mut recorder = FlightRecorder::new(16);

// Inscribe 25 frames (capacity wraps after 16)
for i in 1..=25 {
    recorder.record_frame(make_frame(i));
}

let recent = recorder.all_frames();
assert_eq!(recent.len(), 16);
assert_eq!(recent[0].roll_index, 10);  // Oldest retained
assert_eq!(recent[15].roll_index, 25); // Newest retained
```

Instant Post-Mortem Root Cause Analysis

When an unexpected rollback occurs, the AI Cortex extracts the anomalous frame instantly:

```
if let Some(bad_frame) = recorder.last_anomalous_frame() {
    println!("Anomaly trapped at roll {}/{}: verb {}",
        bad_frame.roll_index, bad_frame.total_rolls, bad_frame.verb);
}
```

---

4. Neural-Symbolic Cognitive Notes (`CognitiveNote`)

An autonomous AI agent cannot reason effectively over unstructured text dumps. The **`CognitiveNote`** provides a strongly structured, typed epistemic observation unit:

```
pub struct CognitiveNote {
    pub timestamp_micros: u128,
    pub category: String,
    pub summary: String,
    pub details: String,
    pub tags: Vec<String>,
    pub context_snippet: Option<String>,
}
```

Canonical OmniCode Syntax Block (Zero Dot Notation)

The method `format_as_omnicode()` generates valid, canonical OmniCode syntax matching `character.omni` and `number.omni` :

```
Note < CognitiveTrace > {
    tick: < 1420 µs >,
    category: [ OBSERVATION ],
    summary: < "Parser balanced axioms" >,
    details: < "Delimiter balance verified for all 4 block headers" >,
    tags: [ #parser :: #axioms :: #balance ],
    context: < "Block < Declarations >" >
}
```

Syntax Guarantees:

- **Zero Dot Notation:** No `Note.Trace` or `Scope.Var` . Complex scopes use nested brackets `< ... >` and `{ ... }` .
- **Tag Namespaces:** Tags are prefixed with `#` and separated by double-colons `::` .
- **Dual-Readability:** Directly readable by human engineers in terminal logs and directly executable as AST tokens by the OmniCode parser.

Markdown Prompt Integration

For LLM prompt injection and IDE hover tooltips, `format_as_memory_entry()` produces standardized Markdown:

```
### [EPISODIC_MEMORY_TRACE: OBSERVATION]
- **Timestamp:** 1420 µs
- **Summary:** Parser balanced axioms
- **Details:** Delimiter balance verified for all 4 block headers
- **Tags:** #parser, #axioms, #balance
- **Context:** `Block < Declarations >`
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/frame.rs

RS | 215 LINES | 789 WORDS | 7.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( RollFrame Telemetry Struct: Deterministic Game-Dev State Vector )
// ============================================================================
//! OmniCode Telemetry: Discrete Execution RollFrame Vectors
//!
//! # Architectural Scope & Purpose:
//! Encapsulates discrete tick-by-tick state vectors captured during process
//! tape stepping, recording dual-rail SIMD bitplanes, operational roll verbs,
//! QuadStates, and compact failpoint bitmasks:
//! - [`RollFrame`]: Single atomic execution frame representing one tape roll tick.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic execution frames on physical stone).
//! - **Proverbs 4:26 (KJV)**:
//!   "Ponder the path of thy feet, and let all thy ways be established."
//!   (The discipline of measuring every execution step and recording state transitions).
//! - **1 Corinthians 14:40 (KJV)**:
//!   "Let all things be done decently and in order."
//!   (The chronological sequential ordering of discrete execution frames).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Game-Dev Roll Architecture**: OmniCode programs execute as ribbons of discrete rolls.
//!    Each roll commits an atomic state transition that can be inspected, stepped backward, or replayed.
//! 2. **Dual-Rail Substrate**: Input and output bitplanes are captured as `(positive_rail, negative_rail)`
//!    tuples, exposing balanced ternary hardware states directly to diagnostic analysis.
//! 3. **Non-Intrusive Telemetry**: `RollFrame` captures complete operational context in a fixed-size
//!    structure suitable for zero-allocation circular ring buffering.
// ============================================================================

use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( RollFrame Struct Definition & State Invariants )
// ============================================================================

/// Single atomic execution frame captured during roll traversal.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct RollFrame {
    /// Monotonic elapsed microseconds at roll execution.
    pub tick_micros: u128,
    /// 1-based roll sequence index.
    pub roll_index: usize,
    /// Total rolls declared on this tape ribbon.
    pub total_rolls: usize,
    /// Active operational verb (e.g. "INSCRIBE", "TRANSDUCE", "COMPARE").
    pub verb: String,
    /// Input substrate state as (positive_rail, negative_rail) bitplanes.
    pub dual_rail_in: (u64, u64),
    /// Output substrate state as (positive_rail, negative_rail) bitplanes.
    pub dual_rail_out: (u64, u64),
    /// Fourfold execution state committed by this roll.
    pub quad_state: QuadState,
    /// Compact bitmask representing trapped failpoints (0 = clean).
    pub failpoint_mask: u32,
    /// First-class human/machine intent aside attached to the roll.
    pub intent_aside: Option<String>,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructor, Diagnostic Formatters & State Query Methods )
// ============================================================================

impl RollFrame {
    /// Constructs a new [`RollFrame`].
    pub fn new(
        tick_micros: u128,
        roll_index: usize,
        total_rolls: usize,
        verb: impl Into<String>,
```

```
        dual_rail_in: (u64, u64),
        dual_rail_out: (u64, u64),
        quad_state: QuadState,
        failpoint_mask: u32,
        intent_aside: Option<String>,
    ) -> Self {
        Self {
            tick_micros,
            roll_index,
            total_rolls,
            verb: verb.into(),
            dual_rail_in,
            dual_rail_out,
            quad_state,
            failpoint_mask,
            intent_aside,
        }
    }

    /// Returns `true` if this execution frame completed without failpoints or rollback.
    #[inline]
    pub fn is_clean(&self) -> bool {
        self.quad_state == QuadState::Proceed && self.failpoint_mask == 0
    }

    /// Returns `true` if this frame experienced an anomaly, rollback, or failpoint.
    #[inline]
    pub fn has_anomaly(&self) -> bool {
        !self.is_clean()
    }

    /// Returns `true` if the roll resulted in a transactional rollback.
    #[inline]
    pub fn is_rollback(&self) -> bool {
        self.quad_state == QuadState::Rollback
    }

    /// Returns `true` if the output rails are saturated (both positive and negative rails active).
    ///
    /// In balanced ternary dual-rail logic, `(1, 1)` represents saturated state collapse (HALT).
    #[inline]
    pub fn has_dual_rail_saturation(&self) -> bool {
        (self.dual_rail_out.0 & self.dual_rail_out.1) != 0
    }

    /// Formats the frame as a compact structured telemetry row.
    pub fn format_row(&self) -> String {
        let aside_str = self
            .intent_aside
            .as_deref()
            .map(|a| format!(" ({a})"))
            .unwrap_or_default();
        format!(
            "[{:>6} µs] Roll {}/{}: {:<10} Rails: ({:#018x}, {:#018x}) -> State: {:<8} Mask: {:#010x}{}",
            self.tick_micros,
            self.roll_index,
            self.total_rolls,
            self.verb,
            self.dual_rail_out.0,
            self.dual_rail_out.1,
            self.quad_state.macro_op(),
            self.failpoint_mask,
            aside_str
        )
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & RollFrame Invariant Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_roll_frame_clean_execution() {
        let frame = RollFrame::new(
            142,
            1,
```

```
            10,
            "INSCRIBE",
            (0x0001, 0x0000),
            (0x0002, 0x0000),
            QuadState::Proceed,
            0,
            Some("Initialize symbol".to_string()),
        );

        assert!(frame.is_clean());
        assert!(!frame.has_anomaly());
        assert!(!frame.is_rollback());
        assert!(!frame.has_dual_rail_saturation());
        assert_eq!(frame.roll_index, 1);
        assert_eq!(frame.total_rolls, 10);
        assert_eq!(frame.verb, "INSCRIBE");

        let row = frame.format_row();
        assert!(row.contains("[   142 µs] Roll 1/10: INSCRIBE"));
        assert!(row.contains("State: PROCEED"));
        assert!(row.contains("(Initialize symbol)"));
    }

    #[test]
    fn test_roll_frame_anomaly_and_rollback() {
        let frame = RollFrame::new(
            500,
            3,
            10,
            "DIVIDE",
            (0x0005, 0x0000),
            (0x0000, 0x0000),
            QuadState::Rollback,
            0x00000004,
            Some("Division by zero trapped".to_string()),
        );

        assert!(!frame.is_clean());
        assert!(frame.has_anomaly());
        assert!(frame.is_rollback());
        assert_eq!(frame.failpoint_mask, 0x00000004);
    }

    #[test]
    fn test_dual_rail_saturation_detection() {
        // Normal rails: disjoint
        let normal = RollFrame::new(
            100, 1, 1, "OP", (1, 0), (1, 0), QuadState::Proceed, 0, None,
        );
        assert!(!normal.has_dual_rail_saturation());

        // Saturated rails: overlap in bit 0
        let saturated = RollFrame::new(
            200, 2, 2, "OP", (1, 0), (1, 1), QuadState::Halt, 0, None,
        );
        assert!(saturated.has_dual_rail_saturation());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/genesis.rs RS | 325 LINES | 1217 WORDS | 12.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Genesis Inception Telemetry & Silicon Baseline Tracking Structures )
// ============================================================================
//! OmniCode Telemetry: Genesis Inception Telemetry & Silicon Baseline ($\Omega_0$)
//!
//! # Architectural Scope & Purpose:
//! Encapsulates cold-silicon inception logs emitted by Track 0 (BIOS MBR,
//! Stage 2 PM, Long Mode, and UEFI) over physical RAM landmarks and COM1 UART serial.
//! Provides the primordial ground truth (State $\Omega_0$) required by the AI Cortex
//! upon machine awakening:
//! - Physical RAM Landmark Constants (`0x0500`, `0x7C00`, `0x18000`, `0x108000`, `0x200000`).
//! - [`GenesisBootLog`]: Single bare-metal inception frame capturing landmark, tier, and status.
//! - [`GenesisBaseline`]: Immutable snapshot of primordial hardware equilibrium ($\Omega_0$).
//!
//! # Scripture Witness:
//! - **Genesis 1:1 (KJV)**:
//!   "In the beginning God created the heaven and the earth."
//!   (The foundational truth that system state begins at a definite primordial inception point).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The inviolability of physical silicon RAM landmarks: 0x7C00, 0x18000, 0x108000, 0x200000).
//! - **Deuteronomy 29:29 (KJV)**:
//!   "The secret things belong unto the LORD our God: but those things which are revealed
//!    belong unto us and to our children for ever, that we may do all the words of this law."
//!   (The mandate to fully reveal and record hardware inception ground truth for AI reasoning).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Machine Awakening Problem**: When the AI Cortex wakes up in Track 1, it has missed the first
//!    several milliseconds of bare-metal execution. The `GenesisBaseline` bridges this temporal gap.
//! 2. **Homeostatic Equilibrium Law**: The machine cannot safely power down or hibernate unless
//!    the state has returned to the pure, uncorrupted equilibrium certified at $\Omega_0$.
//! 3. **Ancient Landmarks**: Physical memory addresses are hardware contracts that guarantee
//!    isomorphic execution across QEMU emulation, hypervisors, and physical bare-metal silicon.
// ============================================================================

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Physical Silicon Landmark Constants, Boot Log & Baseline Definitions )
// ============================================================================

/// Hardware RAM landmark for Real Mode Interrupt Vector Table (`0x00000500`).
pub const LANDMARK_BIOS_IVT: u64 = 0x00000500;

/// Hardware RAM landmark for Sector 0 BIOS MBR bootstrap reset vector (`0x00007C00`).
pub const LANDMARK_BIOS_MBR: u64 = 0x00007C00;

/// Hardware RAM landmark for Stage 2 Protected Mode Flight Recorder buffer (`0x00018000`).
pub const LANDMARK_STAGE2_PM: u64 = 0x00018000;

/// Hardware RAM landmark for Long Mode 64-bit 4-level page table directory (`0x00108000`).
pub const LANDMARK_PAGE_TABLE: u64 = 0x00108000;

/// Hardware RAM landmark for Cognitive Substrate CPI-SI execution arena (`0x00200000`).
pub const LANDMARK_COGNITIVE_SUBSTRATE: u64 = 0x00200000;

/// Single bare-metal inception log frame captured during machine boot.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct GenesisBootLog {
    /// Monotonic timestamp in microseconds when this inception log was ingested.
    pub timestamp_micros: u128,
    /// Physical RAM address or landmark (e.g. 0x0500, 0x7C00, 0x18000, 0x108000, 0x200000).
    pub landmark_address: u64,
    /// Boot tier or architecture (e.g. "BIOS_REAL_MODE", "STAGE2_PM", "LONG_MODE", "UEFI").
    pub tier: String,
    /// Raw telemetry string emitted over COM1 serial or memory buffer.
    pub raw_telemetry: String,
    /// Invariant subsystem verified (e.g. "ALU_BALANCED", "REGS_CALIBRATED", "A20_GATE", "MEMORY_MAP").
    pub invariant_subsystem: String,
    /// Whether the baseline invariant check passed cleanly.
    pub is_clean: bool,
}

/// Primordial Baseline State (Omega_0) captured at cold boot.
#[derive(Debug, Clone, PartialEq, Eq)]
```

```rust
pub struct GenesisBaseline {
    /// Inception timestamp in microseconds when baseline was locked.
    pub locked_micros: u128,
    /// Chronological series of genesis boot logs from cold metal.
    pub boot_logs: Vec<GenesisBootLog>,
    /// Whether all genesis checks passed without error.
    pub is_clean: bool,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Boot Log Methods, Serial Parsers & Baseline Homeostasis Audits )
// ============================================================================

impl GenesisBootLog {
    /// Constructs a new [`GenesisBootLog`].
    pub fn new(
        timestamp_micros: u128,
        landmark_address: u64,
        tier: impl Into<String>,
        raw_telemetry: impl Into<String>,
        invariant_subsystem: impl Into<String>,
        is_clean: bool,
    ) -> Self {
        Self {
            timestamp_micros,
            landmark_address,
            tier: tier.into(),
            raw_telemetry: raw_telemetry.into(),
            invariant_subsystem: invariant_subsystem.into(),
            is_clean,
        }
    }

    /// Formats the genesis log as a human-readable ANSI/ASCII diagnostic line.
    pub fn format_human(&self) -> String {
        let status = if self.is_clean { "OK" } else { "FAIL" };
        format!(
            "[{:>6} µs] [GENESIS: {:#08x}] [{:<12}] {:<20} => {}",
            self.timestamp_micros,
            self.landmark_address,
            self.tier,
            self.invariant_subsystem,
            status
        )
    }

    /// Attempts to parse a raw COM1 serial output line into a [`GenesisBootLog`].
    ///
    /// Recognizes patterns emitted by Track 0:
    /// `[BOOT] 16-Bit Genesis: Sector 0 loaded at 0x7C00`
    /// `[ALU ] Balanced Ternary (+1)+(-1)=0 Eq: Certified`
    pub fn parse_serial_line(timestamp_micros: u128, line: &str) -> Option<Self> {
        let trimmed = line.trim();
        if trimmed.is_empty() {
            return None;
        }

        let is_clean = !trimmed.to_uppercase().contains("FAIL")
            && !trimmed.to_uppercase().contains("FAULT")
            && !trimmed.to_uppercase().contains("ERROR");

        let (landmark, tier, subsystem) = if trimmed.contains("0x7C00") || trimmed.contains("0x7c00") {
            (LANDMARK_BIOS_MBR, "BIOS_REAL_MODE", "SECTOR_ZERO")
        } else if trimmed.contains("0x18000") || trimmed.contains("Stage 2 PM") {
            (LANDMARK_STAGE2_PM, "STAGE2_PM", "A20_GATE")
        } else if trimmed.contains("0x108000") || trimmed.contains("Page Directory") {
            (LANDMARK_PAGE_TABLE, "LONG_MODE", "PAGE_TABLES")
        } else if trimmed.contains("0x200000") || trimmed.contains("Cognitive") {
            (LANDMARK_COGNITIVE_SUBSTRATE, "ARENA", "CPI_SI_SUBSTRATE")
        } else if trimmed.contains("ALU") {
            (LANDMARK_BIOS_MBR, "BIOS_REAL_MODE", "ALU_BALANCED")
        } else if trimmed.contains("REGS") {
            (LANDMARK_STAGE2_PM, "STAGE2_PM", "REGS_CALIBRATED")
        } else {
            (LANDMARK_BIOS_IVT, "BOOT", "GENERIC_INCEPTION")
        };

        Some(Self::new(
            timestamp_micros,
            landmark,
```

```
            tier,
            trimmed,
            subsystem,
            is_clean,
        ))
    }
}

impl GenesisBaseline {
    /// Constructs a new [`GenesisBaseline`].
    pub fn new(locked_micros: u128, boot_logs: Vec<GenesisBootLog>) -> Self {
        let is_clean = !boot_logs.is_empty() && boot_logs.iter().all(|l| l.is_clean);
        Self {
            locked_micros,
            boot_logs,
            is_clean,
        }
    }

    /// Returns the number of genesis inception records.
    pub fn len(&self) -> usize {
        self.boot_logs.len()
    }

    /// Returns `true` if no inception records exist.
    pub fn is_empty(&self) -> bool {
        self.boot_logs.is_empty()
    }

    /// Finds all genesis logs matching a specific physical RAM landmark address.
    pub fn find_by_landmark(&self, landmark: u64) -> Vec<&GenesisBootLog> {
        self.boot_logs
            .iter()
            .filter(|l| l.landmark_address == landmark)
            .collect()
    }

    /// Finds the first genesis log matching a specific subsystem name.
    pub fn find_by_subsystem(&self, subsystem: &str) -> Option<&GenesisBootLog> {
        self.boot_logs
            .iter()
            .find(|l| l.invariant_subsystem.eq_ignore_ascii_case(subsystem))
    }

    /// Audits whether the baseline represents a pure, uncompromised homeostatic equilibrium ($\Omega_0$).
    pub fn audit_equilibrium(&self) -> Result<(), String> {
        if self.is_empty() {
            return Err("Genesis audit failed: No inception logs registered.".to_string());
        }

        if !self.is_clean {
            let failed: Vec<&str> = self
                .boot_logs
                .iter()
                .filter(|l| !l.is_clean)
                .map(|l| l.invariant_subsystem.as_str())
                .collect();
            return Err(format!(
                "Genesis audit failed: Baseline contains corrupted subsystems: {:?}",
                failed
            ));
        }

        Ok(())
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Landmark Invariant Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_ancient_landmarks_constants() {
        assert_eq!(LANDMARK_BIOS_IVT, 0x0500);
        assert_eq!(LANDMARK_BIOS_MBR, 0x7C00);
        assert_eq!(LANDMARK_STAGE2_PM, 0x18000);
```

```
        assert_eq!(LANDMARK_PAGE_TABLE, 0x108000);
        assert_eq!(LANDMARK_COGNITIVE_SUBSTRATE, 0x200000);
    }

    #[test]
    fn test_genesis_boot_log_formatting() {
        let log = GenesisBootLog::new(
            1405,
            LANDMARK_BIOS_MBR,
            "BIOS_REAL_MODE",
            "ALU_BALANCED Eq Certified",
            "ALU_BALANCED",
            true,
        );

        assert!(log.is_clean);
        let human = log.format_human();
        assert!(human.contains("[  1405 µs] [GENESIS: 0x007c00] [BIOS_REAL_MODE] ALU_BALANCED"));
        assert!(human.contains("=> OK"));
    }

    #[test]
    fn test_genesis_serial_line_parsing() {
        let log = GenesisBootLog::parse_serial_line(
            500,
            "[BOOT] 16-Bit Genesis: Sector 0 loaded at 0x7C00",
        );
        assert!(log.is_some());
        let l = log.unwrap();
        assert_eq!(l.landmark_address, LANDMARK_BIOS_MBR);
        assert_eq!(l.tier, "BIOS_REAL_MODE");
        assert!(l.is_clean);

        let failed_log = GenesisBootLog::parse_serial_line(
            600,
            "[ALU] Hardware Fault: Parity error detected",
        );
        assert!(failed_log.is_some());
        assert!(!failed_log.unwrap().is_clean);
    }

    #[test]
    fn test_genesis_baseline_and_equilibrium_audit() {
        let log1 = GenesisBootLog::new(
            100,
            LANDMARK_BIOS_MBR,
            "BIOS",
            "Reset OK",
            "SECTOR_ZERO",
            true,
        );
        let log2 = GenesisBootLog::new(
            200,
            LANDMARK_STAGE2_PM,
            "STAGE2_PM",
            "A20 Gate Active",
            "A20_GATE",
            true,
        );

        let baseline = GenesisBaseline::new(250, vec![log1, log2]);
        assert_eq!(baseline.len(), 2);
        assert!(!baseline.is_empty());
        assert!(baseline.is_clean);
        assert!(baseline.audit_equilibrium().is_ok());

        let mbr_logs = baseline.find_by_landmark(LANDMARK_BIOS_MBR);
        assert_eq!(mbr_logs.len(), 1);

        let a20 = baseline.find_by_subsystem("A20_GATE");
        assert!(a20.is_some());
        assert_eq!(a20.unwrap().landmark_address, LANDMARK_STAGE2_PM);

        // Corrupted baseline fails audit
        let corrupted_log = GenesisBootLog::new(
            300,
            LANDMARK_PAGE_TABLE,
            "PAGE",
            "Paging Error",
            "PAGING",
            false,
```

```
        );
        let dirty_baseline = GenesisBaseline::new(350, vec![corrupted_log]);
        assert!(!dirty_baseline.is_clean);
        assert!(dirty_baseline.audit_equilibrium().is_err());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/mod.rs RS | 95 LINES | 500 WORDS | 4.5 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Telemetry Subsystem Identity, Modular Facade & Re-exports )
// ================================================================================
//! OmniCode Telemetry: Frames, Epistemic Notes, Inception Baselines & Flight Recorder
//!
//! # Architectural Scope & Purpose:
//! The `telemetry` module provides structured machine observability across both
//! cold-silicon bootstrap (Track 0) and high-level virtual machine execution (Track 1):
//! - [`frame`]: Discrete game-dev execution state vectors ([`RollFrame`]) capturing dual-rail SIMD bitplanes.
//! - [`genesis`]: Track 0 cold-boot hardware inception logs ([`GenesisBootLog`]) and baseline state ([`GenesisBaseline`]).
//! - [`note`]: Semantic cognitive memory traces ([`CognitiveNote`]) formatted for neural-symbolic AI models.
//! - [`ring_buffer`]: Zero-allocation circular telemetry buffer ([`FlightRecorder`]) for deterministic crash replay.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The permanent, immutable recording of execution frames on stone vessels).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The inviolability of physical silicon RAM landmarks: 0x7C00, 0x18000, 0x108000, 0x200000).
//! - **2 Kings 4:4 (KJV)**:
//!   "And when thou art come in, thou shalt shut the door upon thee and upon thy sons,
//!    and shalt pour out into all those vessels, and thou shalt set aside that which is full."
//!   (The container law Occupancy <= Capacity applied to circular in-memory flight recorders).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Machine Awakening Continuity**: Provides the AI Cortex with ground truth knowledge of
//!    silicon events that occurred before the VM awakened (State $\Omega_0$).
//! 2. **Post-Mortem Crash Inspection**: When an execution rollback or failpoint occurs, the
//!    in-memory flight recorder furnishes the exact pre-incident execution sequence without disk delays.
//! 3. **Neural-Symbolic Memory**: Epistemic cognitive notes structure observations into canonical
//!    OmniCode syntax blocks without illegal dot naming.
// ================================================================================

pub mod frame;
pub mod genesis;
pub mod note;
pub mod ring_buffer;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Public Type Re-exports, Landmark Constants & Capacity Defaults )
// ================================================================================

pub use frame::RollFrame;
pub use genesis::{
    GenesisBaseline, GenesisBootLog, LANDMARK_BIOS_IVT, LANDMARK_BIOS_MBR,
    LANDMARK_COGNITIVE_SUBSTRATE, LANDMARK_PAGE_TABLE, LANDMARK_STAGE2_PM,
};
pub use note::CognitiveNote;
pub use ring_buffer::{
    DEFAULT_FLIGHT_RECORDER_CAPACITY, FlightRecorder, MIN_FLIGHT_RECORDER_CAPACITY,
};

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Telemetry Subsystem Facade & Convenience Constructors )
// ================================================================================

/// Constructs a calibrated pairing of an in-memory [`FlightRecorder`] and a clean genesis container.
pub fn create_flight_recorder_with_capacity(capacity: usize) -> FlightRecorder {
    FlightRecorder::new(capacity)
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Telemetry Facade Invariants )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::ternary::QuadState;

    #[test]
```

```
    fn test_telemetry_re_exports_available() {
        assert_eq!(LANDMARK_BIOS_MBR, 0x7C00);
        assert_eq!(LANDMARK_STAGE2_PM, 0x18000);

        let frame = RollFrame::new(
            100, 1, 1, "INSCRIBE", (1, 0), (1, 0), QuadState::Proceed, 0, None,
        );
        assert!(frame.is_clean());

        let note = CognitiveNote::observation(100, "Observation", "Details", &["tag"]);
        assert!(note.has_tag("tag"));

        let mut recorder = create_flight_recorder_with_capacity(16);
        recorder.record_frame(frame);
        assert_eq!(recorder.len(), 1);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/note.rs

RS | 260 LINES | 950 WORDS | 9.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structured Cognitive Memory Notes Formatted for AI Models & IDE Hover )
// ============================================================================
//! OmniCode Telemetry: Structured Cognitive Memory Notes
//!
//! # Architectural Scope & Purpose:
//! Provides dual-readable semantic memory notes formatted for neural-symbolic AI
//! context windows, episodic memory reels, and language server tooltips:
//! - [`CognitiveNote`]: Atomic semantic memory trace capturing category, summary, details, tags, and context.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The permanent, immutable recording of covenantal testimony on physical tablets).
//! - **Proverbs 3:3 (KJV)**:
//!   "Let not mercy and truth forsake thee: bind them about thy neck; write them upon the table of thine heart."
//!   (The internal engraving of truth within the cognitive memory of the machine).
//! - **Malachi 3:16 (KJV)**:
//!   "Then they that feared the LORD spake often one to another: and the LORD hearkened,
//!    and heard it, and a book of remembrance was written before him."
//!   (The episodic book of remembrance preserving historical traces for future recall).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Neural-Symbolic Ingestion**: Raw unstructured text requires unpredictable LLM extraction.
//!    [`CognitiveNote`] structures observations into explicit categories and semantic taxonomy tags.
//! 2. **Canonical OmniCode Grammar**: The output of `format_as_omnicode()` adheres strictly to the
//!    canonical syntax seen in `character.omni` and `number.omni`, forbidding illegal dot naming.
//! 3. **Episodic Query Indexing**: Cognitive notes are queriable in memory by category and tag,
//!    allowing the AI Cortex to retrieve historical analogies in $O(1)$ time.
// ============================================================================

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( CognitiveNote Struct Definition & Semantic Taxonomies )
// ============================================================================

/// Structured epistemic memory note formatted for neural-symbolic AI models.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct CognitiveNote {
    /// Monotonic timestamp in microseconds since logger initialization.
    pub timestamp_micros: u128,
    /// Epistemic category: e.g. "OBSERVATION", "HYPOTHESIS", "INCIDENT", "GENESIS_INCEPTION".
    pub category: String,
    /// High-level single-sentence summary of the cognitive observation.
    pub summary: String,
    /// In-depth architectural details, telemetry data, or remediation rationale.
    pub details: String,
    /// Semantic taxonomy tags for associative memory indexing.
    pub tags: Vec<String>,
    /// Optional code snippet or token context under active observation.
    pub context_snippet: Option<String>,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructors, Query Methods & Dual-Readability Formatters )
// ============================================================================

impl CognitiveNote {
    /// Constructs a new [`CognitiveNote`].
    pub fn new(
        timestamp_micros: u128,
        category: impl Into<String>,
        summary: impl Into<String>,
        details: impl Into<String>,
        tags: Vec<String>,
        context_snippet: Option<String>,
    ) -> Self {
        Self {
            timestamp_micros,
            category: category.into(),
            summary: summary.into(),
            details: details.into(),
```

```
            tags,
            context_snippet,
        }
    }

    /// Convenience constructor for general operational observations.
    pub fn observation(
        timestamp_micros: u128,
        summary: impl Into<String>,
        details: impl Into<String>,
        tags: &[&str],
    ) -> Self {
        Self::new(
            timestamp_micros,
            "OBSERVATION",
            summary,
            details,
            tags.iter().map(|s| s.to_string()).collect(),
            None,
        )
    }

    /// Convenience constructor for critical failpoint incidents.
    pub fn incident(
        timestamp_micros: u128,
        summary: impl Into<String>,
        details: impl Into<String>,
        tags: &[&str],
        snippet: Option<String>,
    ) -> Self {
        Self::new(
            timestamp_micros,
            "INCIDENT",
            summary,
            details,
            tags.iter().map(|s| s.to_string()).collect(),
            snippet,
        )
    }

    /// Convenience constructor for hypothesis testing.
    pub fn hypothesis(
        timestamp_micros: u128,
        summary: impl Into<String>,
        details: impl Into<String>,
        tags: &[&str],
    ) -> Self {
        Self::new(
            timestamp_micros,
            "HYPOTHESIS",
            summary,
            details,
            tags.iter().map(|s| s.to_string()).collect(),
            None,
        )
    }

    /// Returns `true` if this note contains the specified semantic tag.
    pub fn has_tag(&self, tag: &str) -> bool {
        self.tags.iter().any(|t| t.eq_ignore_ascii_case(tag))
    }

    /// Returns `true` if this note matches the specified category.
    pub fn is_category(&self, cat: &str) -> bool {
        self.category.eq_ignore_ascii_case(cat)
    }

    /// Formats this note as an OmniCode native dual-readable syntax block.
    ///
    /// Strictly adheres to canonical syntax (zero dot notation in complex names).
    pub fn format_as_omnicode(&self) -> String {
        let tag_str = if self.tags.is_empty() {
            "[]".to_string()
        } else {
            format!(
                "[ {} ]",
                self.tags
                    .iter()
                    .map(|t| format!("#{t}"))
                    .collect::<Vec<_>>()
                    .join(" :: ")
```

```
            )
        };
        let ctx_str = if let Some(c) = &self.context_snippet {
            format!("\n    context: < {:?} >", c)
        } else {
            String::new()
        };
        format!(
            "Note < CognitiveTrace > {{\n    tick: < {} µs >,\n    category: [ {} ],\n    summary: < {:?} >,\n    details: < {:?} >,\n    tags: {}{}\n}}",
            self.timestamp_micros,
            self.category,
            self.summary,
            self.details,
            tag_str,
            ctx_str
        )
    }

    /// Formats this note as a Markdown block specifically structured for prompts and IDE hover.
    pub fn format_as_memory_entry(&self) -> String {
        let tag_list = if self.tags.is_empty() {
            "none".to_string()
        } else {
            self.tags
                .iter()
                .map(|t| format!("#{t}"))
                .collect::<Vec<_>>()
                .join(", ")
        };
        let mut out = format!(
            "### [EPISODIC_MEMORY_TRACE: {}]\n- **Timestamp:** {} µs\n- **Summary:** {}\n- **Details:** {}\n- **Tags:** {}\n",
            self.category, self.timestamp_micros, self.summary, self.details, tag_list
        );
        if let Some(ctx) = &self.context_snippet {
            out.push_str(&format!("- **Context:** `{}`\n", ctx));
        }
        out
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Formatting Invariant Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_cognitive_note_creation_and_tag_querying() {
        let note = CognitiveNote::observation(
            1420,
            "Parser balanced axioms",
            "Delimiter balance verified for all 4 block headers",
            &["parser", "axioms", "balance"],
        );

        assert_eq!(note.timestamp_micros, 1420);
        assert_eq!(note.category, "OBSERVATION");
        assert!(note.is_category("OBSERVATION"));
        assert!(note.is_category("observation"));
        assert!(note.has_tag("parser"));
        assert!(note.has_tag("PARSER"));
        assert!(note.has_tag("balance"));
        assert!(!note.has_tag("failpoint"));
    }

    #[test]
    fn test_cognitive_note_omnicode_formatting() {
        let note = CognitiveNote::incident(
            2500,
            "Delimiter mismatch trapped",
            "Expected closing angle bracket",
            &["boundary", "failpoint"],
            Some("<unclosed".to_string()),
        );

        let omnicode = note.format_as_omnicode();
```

```
        assert!(omnicode.contains("Note < CognitiveTrace > {"));
        assert!(omnicode.contains("tick: < 2500 µs >"));
        assert!(omnicode.contains("category: [ INCIDENT ]"));
        assert!(omnicode.contains("tags: [ #boundary :: #failpoint ]"));
        assert!(omnicode.contains("context: < \"<unclosed\" >"));
        // Guarantee zero dot notation
        assert!(!omnicode.contains("."));
    }

    #[test]
    fn test_cognitive_note_memory_entry_markdown() {
        let note = CognitiveNote::hypothesis(
            3000,
            "Cache locality optimization",
            "Reordering tape rolls may improve instruction cache hit rate",
            &["optimization", "cortex"],
        );

        let md = note.format_as_memory_entry();
        assert!(md.contains("### [EPISODIC_MEMORY_TRACE: HYPOTHESIS]"));
        assert!(md.contains("- **Timestamp:** 3000 µs"));
        assert!(md.contains("- **Tags:** #optimization, #cortex"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/telemetry/ring_buffer.rs

RS | 284 LINES | 1063 WORDS | 10.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( FlightRecorder: Fixed-Capacity Circular Buffer for Deterministic Replay )
// ============================================================================
//! OmniCode Telemetry: In-Memory Circular Flight Recorder
//!
//! # Architectural Scope & Purpose:
//! High-performance, zero-allocation circular ring buffer for execution telemetry.
//! Retains the most recent execution frames in memory with steady-state $O(1)$ operations,
//! enabling instantaneous post-mortem crash inspection without disk I/O latency:
//! - [`FlightRecorder`]: Bounded circular buffer recording [`RollFrame`] telemetry.
//!
//! # Scripture Witness:
//! - **2 Kings 4:4 (KJV)**:
//!   "And when thou art come in, thou shalt shut the door upon thee and upon thy sons,
//!    and shalt pour out into all those vessels, and thou shalt set aside that which is full."
//!   (The container law Occupancy <= Capacity applied to fixed-capacity memory buffers).
//! - **Ecclesiastes 1:6 (KJV)**:
//!   "The wind goeth toward the south, and turneth about unto the north; it whirleth about continually,
//!    and the wind returneth again according to his circuits."
//!   (The circular wraparound invariant: continuous execution within bounded circuits).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."
//!   (Allocating fixed ring buffer capacity upfront to guarantee zero allocation jitter).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Zero Steady-State Allocations**: After initialization, `record_frame()` mutates existing slots
//!    in place without triggering heap allocations or garbage collection pauses.
//! 2. **Chronological Order Reconstruction**: Regardless of the internal `write_head` position,
//!    `recent_frames()` unwinds the circular buffer into strict chronological sequence.
//! 3. **Instant Post-Mortem Root Cause Analysis**: When an anomaly or crash occurs, the flight recorder
//!    supplies the exact execution window leading up to the failure.
// ============================================================================

use super::frame::RollFrame;
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Constants, FlightRecorder Struct Definition & Buffer Sizing )
// ============================================================================

/// Default circular frame capacity for the in-memory flight recorder (512 frames).
pub const DEFAULT_FLIGHT_RECORDER_CAPACITY: usize = 512;

/// Minimum allowable circular frame capacity to ensure diagnostic utility (16 frames).
pub const MIN_FLIGHT_RECORDER_CAPACITY: usize = 16;

/// In-memory circular telemetry buffer storing a bounded window of recent execution frames.
#[derive(Debug, Clone)]
pub struct FlightRecorder {
    frames: Vec<RollFrame>,
    capacity: usize,
    write_head: usize,
    total_inscribed: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Buffer Inscription, Chronological Unwinding & Anomaly Extraction )
// ============================================================================

impl FlightRecorder {
    /// Constructs a new [`FlightRecorder`] with the specified frame capacity.
    ///
    /// Capacity is clamped to at least [`MIN_FLIGHT_RECORDER_CAPACITY`].
    pub fn new(capacity: usize) -> Self {
        let cap = capacity.max(MIN_FLIGHT_RECORDER_CAPACITY);
        Self {
            frames: Vec::with_capacity(cap),
            capacity: cap,
            write_head: 0,
            total_inscribed: 0,
        }
    }
```

```rust

    /// Inscribes a new frame into the circular buffer in $O(1)$ time.
    ///
    /// Operates without heap allocations once the capacity ceiling has been reached.
    pub fn record_frame(&mut self, frame: RollFrame) {
        if self.frames.len() < self.capacity {
            self.frames.push(frame);
        } else {
            self.frames[self.write_head] = frame;
        }
        self.write_head = (self.write_head + 1) % self.capacity;
        self.total_inscribed += 1;
    }

    /// Returns a chronological snapshot of the most recent `count` frames.
    ///
    /// Guarantees chronological order from oldest to newest.
    pub fn recent_frames(&self, count: usize) -> Vec<RollFrame> {
        let available = self.frames.len();
        let take = count.min(available);
        if take == 0 {
            return Vec::new();
        }

        let mut result = Vec::with_capacity(take);
        if available < self.capacity {
            let start = available.saturating_sub(take);
            result.extend_from_slice(&self.frames[start..available]);
        } else {
            for i in 0..take {
                let idx = (self.write_head + self.capacity - take + i) % self.capacity;
                result.push(self.frames[idx].clone());
            }
        }
        result
    }

    /// Returns all currently stored frames in strict chronological order.
    pub fn all_frames(&self) -> Vec<RollFrame> {
        self.recent_frames(self.frames.len())
    }

    /// Returns the most recent frame that resulted in a rollback or trapped failpoint.
    pub fn last_anomalous_frame(&self) -> Option<RollFrame> {
        for frame in self.all_frames().into_iter().rev() {
            if frame.has_anomaly() {
                return Some(frame);
            }
        }
        None
    }

    /// Returns all frames currently retained that experienced an anomaly or rollback.
    pub fn anomalous_frames(&self) -> Vec<RollFrame> {
        self.all_frames()
            .into_iter()
            .filter(|f| f.has_anomaly())
            .collect()
    }

    /// Filters retained frames by operational verb.
    pub fn frames_by_verb(&self, verb: &str) -> Vec<RollFrame> {
        self.all_frames()
            .into_iter()
            .filter(|f| f.verb.eq_ignore_ascii_case(verb))
            .collect()
    }

    /// Filters retained frames by QuadState execution status.
    pub fn frames_by_state(&self, state: QuadState) -> Vec<RollFrame> {
        self.all_frames()
            .into_iter()
            .filter(|f| f.quad_state == state)
            .collect()
    }

    /// Returns the maximum capacity of the ring buffer.
    #[inline]
    pub fn capacity(&self) -> usize {
        self.capacity
    }
```

```
157
158     /// Returns the number of frames currently buffered in memory.
159     #[inline]
160     pub fn len(&self) -> usize {
161         self.frames.len()
162     }
163
164     /// Returns `true` if no frames have been recorded.
165     #[inline]
166     pub fn is_empty(&self) -> bool {
167         self.frames.is_empty()
168     }
169
170     /// Returns the cumulative number of frames inscribed over the lifetime of the recorder.
171     #[inline]
172     pub fn total_inscribed(&self) -> usize {
173         self.total_inscribed
174     }
175
176     /// Clears all frames and resets the write head to zero.
177     pub fn clear(&mut self) {
178         self.frames.clear();
179         self.write_head = 0;
180         self.total_inscribed = 0;
181     }
182 }
183
184 impl Default for FlightRecorder {
185     fn default() -> Self {
186         Self::new(DEFAULT_FLIGHT_RECORDER_CAPACITY)
187     }
188 }
189
190 // ============================================================================
191 // BLOCK 4: CLOSING BLOCK
192 // ( Unit Test Verification Proofs & Ring Buffer Invariant Tests )
193 // ============================================================================
194
195 #[cfg(test)]
196 mod tests {
197     use super::*;
198
199     #[test]
200     fn test_flight_recorder_lifecycle_without_wraparound() {
201         let mut recorder = FlightRecorder::new(16);
202         assert!(recorder.is_empty());
203         assert_eq!(recorder.len(), 0);
204         assert_eq!(recorder.capacity(), 16);
205
206         for i in 1u64..=5u64 {
207             recorder.record_frame(RollFrame::new(
208                 (i as u128) * 10,
209                 i as usize,
210                 5,
211                 "STEP",
212                 (i, 0),
213                 (i + 1, 0),
214                 QuadState::Proceed,
215                 0,
216                 None,
217             ));
218         }
219
220         assert_eq!(recorder.len(), 5);
221         assert_eq!(recorder.total_inscribed(), 5);
222
223         let recent = recorder.recent_frames(3);
224         assert_eq!(recent.len(), 3);
225         assert_eq!(recent[0].roll_index, 3);
226         assert_eq!(recent[1].roll_index, 4);
227         assert_eq!(recent[2].roll_index, 5);
228
229         assert!(recorder.last_anomalous_frame().is_none());
230     }
231
232     #[test]
233     fn test_flight_recorder_wraparound_chronological_order() {
234         let mut recorder = FlightRecorder::new(16);
235
236         // Inscribe 25 frames into capacity of 16
237         for i in 1u64..=25u64 {
```

```
            let is_rollback = i == 20;
            let state = if is_rollback { QuadState::Rollback } else { QuadState::Proceed };
            let mask = if is_rollback { 1 } else { 0 };

            recorder.record_frame(RollFrame::new(
                (i as u128) * 100,
                i as usize,
                25,
                if i % 2 == 0 { "EVEN" } else { "ODD" },
                (i, 0),
                (i, 0),
                state,
                mask,
                None,
            ));
        }

        assert_eq!(recorder.len(), 16);
        assert_eq!(recorder.total_inscribed(), 25);

        let all = recorder.all_frames();
        assert_eq!(all.len(), 16);
        // Oldest retained frame should be 10 (25 - 16 + 1)
        assert_eq!(all[0].roll_index, 10);
        assert_eq!(all[15].roll_index, 25);

        // Last anomalous frame was roll 20
        let anomalous = recorder.last_anomalous_frame();
        assert!(anomalous.is_some());
        assert_eq!(anomalous.unwrap().roll_index, 20);

        // Filter by verb
        let evens = recorder.frames_by_verb("EVEN");
        assert_eq!(evens.len(), 8);

        // Filter by state
        let rollbacks = recorder.frames_by_state(QuadState::Rollback);
        assert_eq!(rollbacks.len(), 1);
        assert_eq!(rollbacks[0].roll_index, 20);

        recorder.clear();
        assert!(recorder.is_empty());
        assert_eq!(recorder.len(), 0);
        assert_eq!(recorder.total_inscribed(), 0);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/DAR_SENSORY_DETECTION_AND_TRIPWIRES.md MARKDOWN / DOC | 271 LINES | 1696 WORDS | 13.3 KB

project_title: "The Failpoint Organ: Sensory Detection in the DAR Cybernetic Loop" subtitle: "Tactile Sensory Receptor Specification & Autonomous Self-Healing" subsystem: "Detect, Assess, Restore (DAR) Immune Architecture" classification: "Canonical Architectural Foundation (Rank 1 Invariant)" status: "Active / Verified under Test Oracles & QEMU" module: "src/diagnostics/logger/failpoint"

The Failpoint Organ: Sensory Detection in the DAR Cybernetic Loop

```
================================================================================
  OMNICODE DIAGNOSTICS SUBSYSTEM : TACTILE SENSORY RECEPTOR SPECIFICATION
  Subsystem:      Detect, Assess, Restore (DAR) Immune Architecture
  Module:         src/diagnostics/logger/failpoint/
  Document:       DAR_SENSORY_DETECTION_AND_TRIPWIRES.md
  Classification: Canonical Architectural Foundation (Rank 1 Invariant)
  Status:         Active / Verified under Test Oracles & QEMU
================================================================================
```

> *"He that hath no rule over his own spirit is like a city that is broken down, and without walls." — **Proverbs 25:28 (KJV)***
>
> *"A prudent man foreseeth the evil, and hideth himself: but the simple pass on, and are punished." — **Proverbs 22:3 (KJV)***
>
> *"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — **Proverbs 24:27 (KJV)***

1. Executive Summary & The Tripartite DAR Loop

In traditional computing, diagnostics are reactive and post-mortem. Software runs blindly until it crashes; a fatal signal (`SIGSEGV` , `panic!` , `#GP`) is fired, memory is abandoned, and an unformatted error string is dumped into a log file. For an autonomous operating system engineered for millennial longevity (**MilleniumOS**), this approach is catastrophic.

OmniCode replaces reactive panics with an autonomic cybernetic loop: **DAR (Detect, Assess, Restore)**:

$$\mathbf{DAR} = \langle \text{Detect}, \text{Assess}, \text{Restore} \rangle$$

```
flowchart TD
    subgraph ExecutionSubstrate ["Active Runtime Execution"]
        Exec["Silicon / Process Tape / Cognitive AI Cortex"]
        Collision{"Boundary Collision Encountered"}
        Exec --> Collision
    end

    subgraph Phase1 ["1. DETECT (Sensory Nerve Receptor)"]
        Sensory["Failpoint Subsystem & DiagnosticLogger"]
        Tripwire["Tripwire Fires at Microsecond of Collision"]
        Telemetry["Captures Dual-Rail State & Inscribes DiagnosticPacket"]
        Suggestion["Pre-computes Actionable FixSuggestion with 1-based Coordinates"]
        Collision -->|"Boundary Collision Intercepted"| Sensory
        Sensory --> Tripwire --> Telemetry --> Suggestion
    end

    subgraph Phase2 ["2. ASSESS (Cognitive Eye)"]
        Debugger["OmniDebugger & OmniLinter"]
        Inspect["Inspects Frozen Frame (Zero Panic)"]
        Classify{"Anomaly Classification"}
        Suggestion -->|"Baton Pass: DiagnosticPacket"| Debugger
        Debugger --> Inspect --> Classify
        Classify -->|"Linguistic / Soft Breach"| Rollback["Set State to -1:ROLLBACK"]
        Classify -->|"Physical Silicon Trap"| HaltState["Set State to NULL:HALT"]
```

```
    end

    subgraph Phase3 ["3. RESTORE (Immune Healer)"]
        Restorator["Restorator Engine & Re-Inception"]
        ApplyPatch["Apply Certified AST Patch on Active Tape"]
        Oracle["Verify Invariant via Executable Test Oracle"]
        Deescalate["Discharge Rails (1,1) -> (0,0) & Reset to 0x7C00"]
        Rollback -->|"Remediation Plan"| Restorator
        HaltState -->|"Hardware Reboot Plan"| Restorator
        Restorator --> ApplyPatch --> Oracle
        Restorator --> Deescalate
    end

    subgraph HomeostasisState ["Restored Equilibrium"]
        Homeostasis["QuadState Homeostasis Restored (+1:PROCEED or 0:AWAIT)"]
        Oracle -->|"Verification Passes"| Homeostasis
        Deescalate -->|"Re-Inception Completed"| Homeostasis
        Homeostasis -->|"Resume Safe Computation"| Exec
    end

    style ExecutionSubstrate fill:#1a1d24,stroke:#4a5568,stroke-width:1px
    style Phase1 fill:#1c2833,stroke:#3498db,stroke-width:2px
    style Phase2 fill:#2c1b33,stroke:#9b59b6,stroke-width:2px
    style Phase3 fill:#1b3325,stroke:#2ecc71,stroke-width:2px
    style HomeostasisState fill:#222f3e,stroke:#f39c12,stroke-width:2px
```

Within this triad, the `failpoint` subsystem is the **tactile sensory nerve receptor of the Detect organ**:

- **Without Failpoints, Detection is Blind:** A logging engine without failpoints only records background ticks, timestamps, and passive text. When an anomaly strikes, a passive logger can only record that "something broke."
- **With Failpoints, Detection is Active:** The moment an instruction vector or cognitive token touches an illegal boundary, the failpoint tripwire fires an instant collision interrupt. It records the exact dual-rail physical state, identifies the landmark address or source coordinate, and pre-computes an actionable remedy before the thread can destabilize.

---

2. Why Traditional Logging Fails Modern Cognition

To understand why the failpoint subsystem is structured this way, we must examine the three fatal flaws of traditional logging libraries (`log4j` , `tracing` , `spdlog`):

Traditional Flaw	Conventional Consequence	Why It Fails Autonomous AI & MilleniumOS
1. Unstructured Text	Messages are arbitrary strings (`"Error: token 42"`).	Machines cannot parse or reason over text without fragile heuristics.
2. Zero Mechanical Consequence	Emitting a log entry has zero runtime effect.	The program continues careening into state corruption and unhandled panics.
3. Passive Post-Mortem Only	Logs are written after the crash occurred.	Memory and in-flight transactions are permanently lost; demands manual restart.

The Failpoint Antidote: Structured, Mechanical, In-Flight

The OmniCode `failpoint` architecture establishes three counter-invariants:

1. **Strong Ontological Typing:** Every failpoint belongs to an immutable domain group (`FailpointGroup`) and functional subgroup (`FailpointSubgroup`).
2. **Immediate Mechanical Coupling:** Tripping a failpoint immediately changes the machine's physical state: either invoking a causal rewind (`-1:ROLLBACK`) or capturing Track 0 silicon registers and executing an orderly restart (`NULL:HALT`).
3. **Pre-Computed Remediation:** Every linguistic failpoint embeds a ready-to-apply `FixSuggestion` containing exact 1-based source coordinates, the offending token, and the certified patch string. The detection organ synthesizes the cure at the exact microsecond of diagnosis.

---

3. The 6-Domain Ontological Taxonomy Matrix

Failpoints are categorized into a 2-tier ontological hierarchy: **6 Groups** and **14 Subgroups**:

Domain Group	Functional Subgroup	Primary Failpoints	Mechanical Action	Physical / Logical Boundary
`HardwareSilicon`	`CpuTrap`	`FP_CPU_HALT_TRAP`	**`NULL:HALT`**	CPU hardware traps (`#GP` , `#PF` , `#DF` , `#MC`)
	`FirmwareBus`	`FP_HARDWARE_FAULT` , `WATCHDOG`	**`NULL:HALT`**	Firmware bus stalls and watchdog sentinel timeouts
	`AluSanity`	`FP_ALU_EQUILIBRIUM_BREACH`	**`NULL:HALT`**	Balanced ALU ternary parity: `(+1) + (-1) != 0`
`MemoryBoundary`	`LandmarkBounds`	`FP_MEMORY_LANDMARK_BREACH`	**`NULL:HALT`**	Violations of `0x7C00` , `0x18000` , `0x108000` , `0x200000`
	`SegmentationPaging`	MMIO / Stack Underflow	**`NULL:HALT`**	Paging tables (`CR3`), GDT descriptors, stack collisions
`TernarySaturation`	`DualRailCollision`	`FP_DUAL_RAIL_SATURATION`	**`NULL:HALT`**	Both rails high `(1, 1)` ; zero free data capacity
	`StateCollapse`	`FP_NULL_TRAP` ($\bot$)	**`NULL:HALT`**	Evaluation attempted unhandled bottom/void type
`LinguisticSyntax`	`DelimiterHierarchy`	`FP_BOUNDARY_MISMATCH` , `LEAK`	**`-1:ROLLBACK`**	Mismatched or unclosed delimiters (`<...>` , `[...]` , `{...}`)
	`StructuralScaffold`	`FP_SYNTAX_ANOMALY`	**`-1:ROLLBACK`**	Missing 4-Block scaffolding headers and grammar drift
	`TypeSemantic`	`FP_IMMUTABLE_MUTATION`	**`-1:ROLLBACK`**	Illegal attempt to mutate Rank 1 immutable axioms
`RuntimeTransaction`	`Retroaction`	`FP_ROLLBACK_TRIGGERED` , `CYCLIC`	**`-1:ROLLBACK`**	Causal tape rewinds and cyclic recursion limits
	`ConcurrencyBus`	`FP_CAVEAT_VETO`	**`-1:ROLLBACK`**	Adversative caveat expressions (`! [...]`) vetoing state
`HomeostasisLifecycle`	`EquilibriumDrift`	`FP_EQUILIBRIUM_DRIFT_VETO`	**`-1:ROLLBACK`**	System state drifted from Ω_0 baseline balance
	`ShutdownVeto`	Dirty State / Memory Leak	**`-1:ROLLBACK`**	Unreleased resources or dirty frames vetoing shutdown

4. The Baton-Pass: How Detect Hands Off to Assess and Restore

The failpoint organ does not act in isolation. It serves as the intake valve for the entire self-healing pipeline:

```
sequenceDiagram
    autonumber
    participant Parser as Tape Parser / Runtime
    participant Detect as Detect Organ (Failpoint)
    participant Assess as Assess Organ (OmniDebugger)
    participant Restore as Restore Organ (Restorator)
    participant Tape as Active Process Tape

    Parser->>Detect: Encounters Delimiter Mismatch at line 148:47: '{' closed by ']'
    Note over Detect: Substrate is healthy (10 or 01).<br/>Identifies soft linguistic boundary collision.
    Detect->>Detect: Construct FailpointType::BoundaryMismatch
    Detect->>Detect: Embed FixSuggestion (Target: 148:47, Offending: ']', Certified: '}')
    Detect->>Assess: Transmit DiagnosticPacket
    Note over Assess: Freezes active execution frame without panic.<br/>Unwinds delimiter stack: verifies opening '{'.
    Assess->>Assess: Confirm soft linguistic slip; assign QuadState::-1 (ROLLBACK)
    Assess->>Restore: Transmit RemediationPlan
    Note over Restore: Rewinds tape head to landmark.<br/>Substitutes ']' with '}' on active AST.
    Restore->>Restore: Execute Verification Test Oracle (Delimiter equilibrium check)
    Note over Restore: Oracle passes (+1:PROCEED).
```

```
    Restore->>Tape: Commit healed AST to tape ribbon
    Restore->>Parser: Resume forward execution seamlessly
```

Concrete Architectural Exemplar: The Boundary Repair Cycle

Here is the exact lifecycle of a linguistic boundary collision on the active process ribbon:

Phase 1: Detect (Location: `src/diagnostics/logger/failpoint/`)

The parser processes an opening block modeled on canonical `character.omni`:

```
# Offending source parsed on active tape ribbon:
Opening {
    [ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte ]
}
```

- **Collision Intercepted:** Line 148, column 47 encountered `]` while delimiter stack expected `}` according to morphological rank.
- **Failpoint Created:** `FailpointType::BoundaryMismatch`.
- **Remediation Inscribed:** Embeds `FixSuggestion`:
- Coordinate: `line: 148, column: 47`
- Offending token: `"]"`
- Certified replacement: `"}"`
- **Buffer Entry:** Inscribes event into the deterministic zero-allocation ring buffer ($O(1)$).

Phase 2: Assess (Location: `src/diagnostics/debugger/` & `linter/`)

- The `OmniDebugger` intercepts the `DiagnosticPacket`.
- Execution frame is frozen without stack unwinding or panic.
- The delimiter analyzer traverses backward to find the matching opening delimiter `{` at line 148, column 23.
- The analyzer confirms the anomaly is a soft linguistic typing slip, not a silicon collapse.
- The cognitive state transition is set to **`-1:ROLLBACK`**.

Phase 3: Restore (Location: `src/diagnostics/restorator/`)

- The `Restorator` ingests the `RemediationPlan`.
- It applies the certified patch string, transforming `]` into `}` on the AST:

`omni # Healed code committed to active tape ribbon: Opening { [ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte } ( Restored matching delimiter ) }`

- An executable test oracle evaluates bracket equilibrium: $$\sum \text{Brackets}_{\text{open}} - \sum \text{Brackets}_{\text{close}} = 0$$
- The oracle passes: the healed AST is committed to the tape ribbon.
- The tape head rolls forward seamlessly in **`+1:PROCEED`**.

The Non-Human Feedback Loop

Because every failpoint produces a clean, machine-actionable record, the DAR system operates autonomously. A developer does not need to sit at a terminal to fix typos, balance brackets, or re-incept crashed hardware.

The **offline quantized cognitive core (CPI-SI)** running in Track 0/Track 1 reads the diagnostic stream, executes the remediation pipeline, and preserves continuous execution indefinitely.

---

5. Architectural Invariants

> *[!IMPORTANT] The DAR sensory architecture is bound by four inviolable Rank 1 Invariants:*
>
> 1. ***INV-DAR-01 (Zero Unstructured Faults):*** *No system component may crash or panic with a raw string. All boundary collisions must emit a typed `FailpointType`.*
> 2. ***INV-DAR-02 (Pre-Computed Remediation):*** *Every recoverable failpoint must attach a non-null `FixSuggestion` indicating exact target coordinates and patch.*
> 3. ***INV-DAR-03 (Substrate Separation):*** *Linguistic anomalies must never trigger hardware halts; hardware collapses must never be treated as soft rollbacks.*
> 4. ***INV-DAR-04 (Restoration Verification):*** *No patch emitted by a failpoint may be committed to disk or tape without verification by an executable test oracle.*

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/DUAL_RAIL_HALT_PHYSICS_AND_REINCEPTION.md MARKDOWN / DOC 219 LINES 1568 WORDS 11.2 KB

project_title: "Dual-Rail HALT Physics, Channel Saturation & Re-Inception Mechanics" subtitle: "The Anti-Flatline Architecture of MilleniumOS" subsystem: "Balanced Ternary Hardware & Silicon Inception" classification: "Canonical Physical Foundation (Rank 1 Invariant)" status: "Active / Hardware Proved under QEMU & CPU Registers" module: "src/diagnostics/logger/failpoint"

Dual-Rail HALT Physics, Channel Saturation & Re-Inception Mechanics

```
================================================================================
  MILLENIUMOS COGNITIVE SUBSTRATE : DUAL-RAIL SATURATION & HALT SPECIFICATION
  Subsystem:      Balanced Ternary Hardware & Silicon Inception
  Module:         src/diagnostics/logger/failpoint/
  Document:       DUAL_RAIL_HALT_PHYSICS_AND_REINCEPTION.md
  Classification: Canonical Physical Foundation (Rank 1 Invariant)
  Status:         Active / Hardware Proved under QEMU & CPU Registers
================================================================================
```

> *"For a just man falleth seven times, and riseth up again: but the wicked shall fall into mischief." — **Proverbs 24:16 (KJV)***
>
> *"Remove not the ancient landmark, which thy fathers have set." — **Proverbs 22:28 (KJV)***
>
> *"The secret things belong unto the LORD our God: but those things which are revealed belong unto us and to our children for ever, that we may do all the words of this law." — **Deuteronomy 29:29 (KJV)***

1. Executive Summary & The Dual-Rail Truth

In traditional binary systems, a wire carries either `0` (low voltage) or `1` (high voltage). If a single wire fails, glitches, or drifts, the computer cannot distinguish between a legitimate zero and a broken connection.

In **MilleniumOS**, all computational state is grounded in **Balanced Ternary Dual-Rail Logic**:

$$\mathbb{T}_3 \in {-1, 0, +1}$$

To represent a single ternary trit with complete physical confirmation, two physical binary conductors (the positive rail and the negative rail) are paired together:

$$\text{Trit} = \langle \text{Rail}{\text{pos}}, \text{Rail}{\text{neg}} \rangle$$

Rail Code	Physical Rails `(pos, neg)`	Balanced Trit	QuadState	Transport Analogy	Physical System Meaning
`00`	`(0, 0)`	`0`	**`AWAIT`**	**PAUSE**	Homeostasis, neutral equilibrium, resting balance
`01`	`(1, 0)`	`+1`	**`PROCEED`**	**PLAY**	Forward affirmation, tape roll commit motion
`10`	`(0, 1)`	`-1`	**`ROLLBACK`**	**REWIND**	Retroactive negation, rewind along tape ribbon
`11`	`(1, 1)`	$\bot$	**`HALT`**	**STOP**	Saturated state, zero free data, collision trap

2. The Physical Meaning of the Four States

Understanding the distinction between these four states is the foundational key to the entire OmniCode architecture:

1. `00` — `(0, 0)` : AWAIT (Trit `0`)

- **State of Matter:** Neutral equilibrium, resting balance, ground potential.
- **Dynamic Property: AWAIT is a pause, not a stop.**
- Both physical wires rest low. Energy consumption is minimal, entropy is zero, and the arithmetic registers are resting in balanced symmetry: $$(+1) + (-1) = 0$$ The system is in homeostasis, standing by in an asynchronous wait state for the arrival of the next sensory token or process turn.

2. `01` — `(1, 0)` : PROCEED (Trit `+1`)

- **State of Matter:** Forward kinetic affirmation.

- **Dynamic Property:** Positive rail high, negative rail low.
- The tape reel rolls forward along the temporal ribbon. Computational work is committed, state is affirmed, and execution advances sequentially.

3. `10` — `(0, 1)` : ROLLBACK (Trit `-1`)

- **State of Matter:** Retroactive negation and elastic recoil.
- **Dynamic Property: ROLLBACK is a rewind, not a stop.**
- Negative rail high, positive rail low. The machine reverses its causal momentum vector ($\vec{v} \rightarrow -\vec{v}$) and steps backward along the process tape ribbon. It returns to a previously anchored milestone or snapshot without losing its session, memory, or thread identity.

4. `11` — `(1, 1)` : HALT (Bottom Type $\bot$)

- **State of Matter:** Dual-rail electrical saturation and collision trap.
- **Dynamic Property: HALT is an abrupt physical stop.**
- Both physical rails are energized high simultaneously. Because both the positive and negative rails are asserted, **there is zero free data capacity left in the transmission channel**. The logic gates have collided with an immovable physical or algebraic boundary.

---

3. The Core System Law: Why HALT Restarts

In classical commercial operating systems (Unix, Windows), when the CPU hits a hardware trap (`#GP` General Protection Fault, `#PF` Page Fault, `#DF` Double Fault), the kernel executes a catastrophic panic:

1. Interrupts are permanently disabled (`cli`).
2. The CPU core is put to sleep in a frozen loop (`hlt`).
3. The machine **flatlines**: the screen goes black or displays a fatal blue screen, all active computation is destroyed, and human intervention is required to physically cycle the power switch.

MilleniumOS mathematically prohibits flatlining.

> *"For a just man falleth seven times, and riseth up again: but the wicked shall fall into mischief." — **Proverbs 24:16 (KJV)***

A system designed to endure for a thousand years without broadband internet or human operators cannot allow an unexpected boundary collision to terminate its existence. If a machine permanently halts, it has ceased to be an operating system; it has become dead silicon.

The Fundamental Law of HALT in OmniCode:

$$\text{HALT does not terminate the machine; HALT restarts the machine!}$$

```
sequenceDiagram
    autonumber
    participant Substrate as Silicon Substrate / CPU
    participant Tripwire as Failpoint Tripwire
    participant FlightRec as In-Memory Flight Recorder (0x18000)
    participant UART as COM1 Serial Port (0x03F8)
    participant BootLadder as Adaptive Boot Ladder
    participant Genesis as Landmark 0x7C00 (Homeostasis)

    Note over Substrate: CPU Trap (#GP/#PF/#DF) or Dual-Rail (1, 1) Saturation
    Substrate->>Tripwire: Hard Collision Intercepted (Channel Capacity = 0)
    Tripwire->>FlightRec: Inscribe 16-Byte Telemetry Frame<br/>[event_id, cr0, esp, status]
    Tripwire->>UART: Emit 13 Calibrated Silicon Registers Dump<br/>(%head, %home, %turn, %spool, CR0, CR3, CR4, RFLAGS)
    Note over Substrate,Tripwire: Discharge Saturated Dual-Rail Charge: (1, 1) -> (0, 0)
    Tripwire->>BootLadder: Initiate Adaptive De-escalation
    Note over BootLadder: Tier 4: Long Mode (0x108000) --(fallback)--> Tier 3: Protected Mode (0x18000)<br/>Tier 3: Protected Moc
    BootLadder->>Genesis: Re-Incept Execution Vector: jmp 0x0000:0x7C00
    Note over Genesis: CPU Re-awakens in QuadState 0:AWAIT (Homeostasis)<br/>Flight recorder preserved in memory. Zero flatline.
```

The In-Memory Flight Recorder

Before de-escalation, the tripwire records a 16-byte binary event at RAM landmark `0x18000` :

```
In-Memory Flight Recorder Binary Format (16 Bytes at Landmark 0x00018000):
┌──────────────────────────┬──────────────────────┬──────────────────────┬──────────────────────┐
│     event_id (u32)       │        cr0 (u32)     │        esp (u32)     │       status (u32)   │
│  Offset 0x00 .. 0x03     │   Offset 0x04 .. 0x07│   Offset 0x08 .. 0x0B│   Offset 0x0C .. 0x0F│
└──────────────────────────┴──────────────────────┴──────────────────────┴──────────────────────┘
```

This record persists through the cascade, giving the re-incepted runtime full episodic memory of the event that triggered the de-escalation.

4. The 13 Calibrated Silicon Registers of Track 0

When a HALT occurs, the machine state is not lost; it is preserved in the **13 Calibrated Silicon Registers** captured by `SiliconRegisters`:

Register Class	Register	Hardware Mapping	Cognitive & Architectural Role in MilleniumOS
Cognitive Registers (Track 0)	**`%head`**	`RAX` / `EAX` / `AX`	Cognitive focus accumulator and active tape head
	`%home`	`RBX` / `EBX` / `BX`	Moral origin anchor and base memory landmark pointer
	`%turn`	`RCX` / `ECX` / `CX`	Sequential roll step counter and loop epoch counter
	`%spool`	`RDX` / `EDX` / `DX`	Primary I/O MMIO port origin (`0x03F8` COM1 UART)
	`%spool_in`	`RSI` / `ESI` / `SI`	Inbound sensory stream telemetry head
	`%spool_out`	`RDI` / `EDI` / `DI`	Outbound action inscriber and tape printer head
	`%frame`	`RBP` / `EBP` / `BP`	Execution frame boundary and covenant landmark
Silicon Stack & Base	**`%rsp`**	`RSP` / `ESP` / `SP`	Silicon execution stack pointer (anchored at `0x7C00`)
	`%rbp`	`RBP` / `EBP` / `BP`	Stack frame base pointer (anchored at `0x7C00`)
Hardware Control Registers	**`%cr0`**	`CR0` Control Reg	Protection enable (`PE`), paging, numeric error flags
	`%cr3`	`CR3` Control Reg	Page Directory Base Register (`0x00108000`)
	`%cr4`	`CR4` Control Reg	Physical Address Extension (PAE) and OS FXSR flags
Hardware Flags	**`%rflags`**	`RFLAGS` / `EFLAGS`	Parity, zero, carry, and directional execution flags

Diagnostic Register Dump Output

The register bank formats deterministically into an 80-column diagnostic dump:

```
REGISTERS:
  head: 0x0000000000000000  home: 0x0000000000000000  turn: 0x0000000000000000
  spool: 0x0000000000000000 in:   0x0000000000000000  out:  0x0000000000000000
  rax:  0x0000000000000000  rbx:  0x0000000000000000  rcx:  0x0000000000000000
  rdx:  0x0000000000000000  rsi:  0x0000000000000000  rdi:  0x0000000000000000
  rsp:  0x0000000000007c00  rbp:  0x0000000000007c00  cr0:  0x0000000000000001
  cr3:  0x0000000000108000  cr4:  0x0000000000000020  rflags: 0x0000000000000002
```

5. Architectural Invariants of HALT & Re-Inception

> *[!IMPORTANT] The HALT and Re-Inception physics are governed by four inviolable Rank 1 Invariants:*
>
> 1. ***INV-HALT-01 (Channel Saturation):*** *State `(1, 1)` represents channel saturation with zero free capacity; no further forward operations may be computed while rails remain high.*
> 2. ***INV-HALT-02 (Anti-Flatline Law):*** *An abrupt HALT must never enter a permanent disabled loop; it must dump registers, discharge saturation, and re-incept to `0:AWAIT`.*
> 3. ***INV-HALT-03 (Telemetry Preservation):*** *Hardware register snapshots and binary event records must be written to linear landmark `0x00018000` before memory landmarks are reset.*
> 4. ***INV-HALT-04 (De-Escalation Safety):*** *If high-tier execution fails, the system cascades down the adaptive boot ladder to a lower, simpler tier, guaranteeing recovery even under damaged state.*

OmniCode Failpoint Pathology & HALT Re-Inception Subsystem

Cybernetic Collision Triggers & Dual-Rail Saturation Physics

> *"Remove not the ancient landmark, which thy fathers have set." — **Proverbs 22:28 (KJV)***
>
> *"A prudent man foreseeth the evil, and hideth himself: but the simple pass on, and are punished." — **Proverbs 22:3 (KJV)***
>
> *"He that hath no rule over his own spirit is like a city that is broken down, and without walls." — **Proverbs 25:28 (KJV)***
>
> *"For a just man falleth seven times, and riseth up again: but the wicked shall fall into mischief." — **Proverbs 24:16 (KJV)***

1. The Architectural Philosophy: Collisions Instead of Crashes

In traditional operating systems and runtime environments, runtime anomalies are treated as catastrophic failures:

- The instruction pointer faults into unmapped memory.
- The CPU issues a General Protection Fault (`#GP`) or Double Fault (`#DF`).
- The kernel panics (`panic!`) or dumps raw binary core memory to disk.
- The machine **flatlines**: screen goes black, CPU locks up in a frozen `cli; hlt` death spiral, requiring a hard power cycle.

OmniCode rejects this fragility. The **Failpoint Subsystem** in **Track 1 (Host Engine & Virtual Machine)** establishes a **cybernetic collision detection architecture** inspired by game engines.

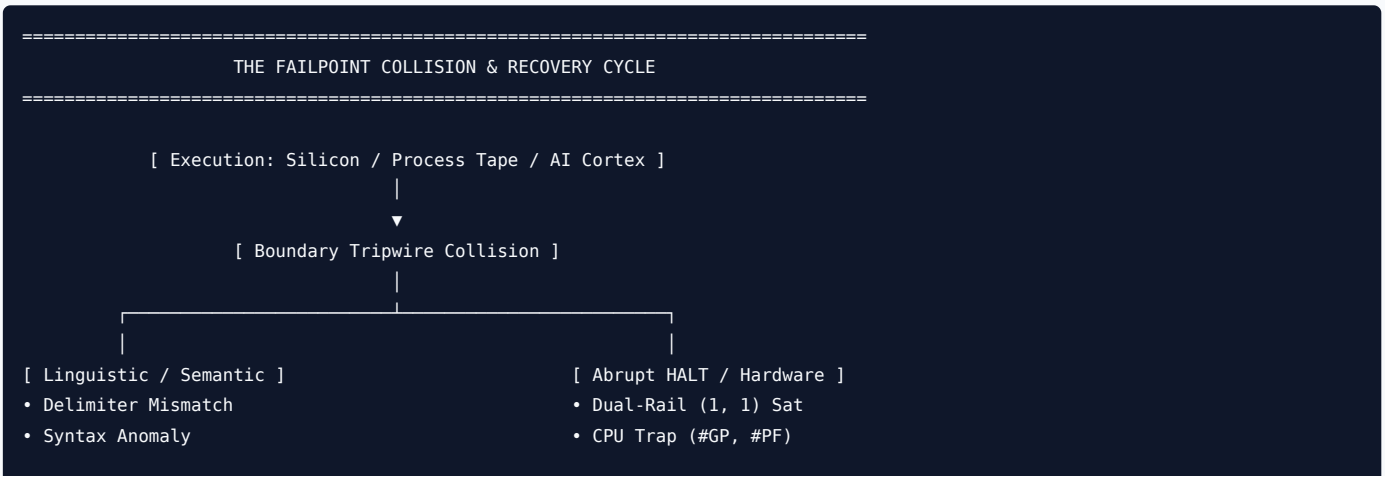
In modern game engines (Unreal, Unity, Godot), invisible trigger volumes are placed throughout world space. When a player entity touches an illegal boundary or falls off a cliff, the physics engine does not crash the game; it fires an `OnTriggerEnter` collision callback that intercepts the entity, restores previous valid coordinates, and continues the game loop.

In OmniCode, **Failpoints** are computational and cognitive collision triggers. They transform fatal bugs, hardware traps, syntax anomalies, and memory violations into **structured, queriable, and recoverable signals**.

Master Architectural Treatises in this Organ

The failpoint organ is exhaustively specified across three foundational treatises designed for human engineers and **Google NotebookLM** curriculum ingestion:

1. **`DAR_SENSORY_DETECTION_AND_TRIPWIRES.md`** :
   The Sensory Receptor in the DAR Cybernetic Loop. Documents how failpoints act as tactile nerve endings for the **Detect (Log)** organ, the 6-domain taxonomy, and the baton-pass to **Assess** (`OmniDebugger`) and **Restore** (`Restorator`).
2. **`DUAL_RAIL_HALT_PHYSICS_AND_REINCEPTION.md`** :
   Dual-Rail Saturation & The Anti-Flatline Invariant. Proves why two bits represent one trit, why state `(1, 1)` represents channel saturation (zero free data capacity), and how Track 0 dumps silicon registers and cascades down the boot ladder to restart into `0:AWAIT` .
3. **`TWO_TRIGGER_TRIPWIRE_ARCHITECTURE.md`** :
   Warnings as Healers, Errors as Inceptors. Explains the 2-trigger collision paradigm (Rollback vs. Halt), the critique of traditional fatal panics, and why this transforms developer experience into an auto-healing sandbox.



```
• Immutable Mutation                          • Balanced ALU Breach
        |                                             |
        ▼                                             ▼
[ Action: -1:ROLLBACK ]                       [ Action: NULL:HALT ]
• Rewinds causal vector                       • Rails 11: NO free data
• Restores snapshot state                     • Track 0 Silicon Dump
• Restorator heals AST                        • Restarts to 0:AWAIT
        |                                             |
        └─────────────────────────┬─────────────────────┘
                                  |
                                  ▼
                    [ State Homeostasis Restored ]
================================================================================
```

2. Failpoint as the Sensory Organ of DAR (Detect · Assess · Restore)

In the overarching architecture of MilleniumOS, the entire diagnostics ecosystem is governed by the **DAR Cybernetic Loop**:

$$\mathbf{DAR} = \langle \text{Detect}, \text{Assess}, \text{Restore} \rangle$$

While the `DiagnosticLogger` acts as the continuous timeline recorder and memory vessel of the engine, the **`failpoint`** directory houses the **tactile sensory nerve endings and boundary collision receptors** of the **Detect** organ.

```
================================================================================
          FAILPOINT'S CENTRAL SENSORY ROLE IN THE DAR LIFECYCLE
================================================================================

  [ 1. DETECT (Sensory Perception) ]
       • failpoint/ organ continuously senses boundary collisions.
       • Evaluates dual-rail state: (0, 1) Rollback vs. (1, 1) Halt.
       • Captures physical landmarks (0x7C00, 0x18000) or source (line:col).
       • Packages collision into a strongly-typed telemetry frame.
                          |
                          ▼
  [ 2. ASSESS (Introspective Reasoning) ]
       • OmniDebugger & OmniLinter receive the failpoint packet.
       • Freezes the active execution frame without panicking.
       • Inspects FailpointGroup & Subgroup (Silicon, Syntax, Memory).
       • Determines whether anomaly is a soft linguistic slip or hard fault.
                          |
                          ▼
  [ 3. RESTORE (Autonomic Immune Repair) ]
       • If Soft (-1:ROLLBACK): Restorator consumes the attached FixSuggestion,
         rewinds tape head, applies code patch, and resumes forward in +1:PROCEED.
       • If Hard (NULL:HALT): Inscriber writes 16-byte RAM log at 0x18000,
         discharges dual-rail saturation (11 -> 00), cascades down the boot ladder,
         and re-incepts at a verified landmark into 0:AWAIT.
================================================================================
```

Why "Detect" is Blind Without Failpoints

In conventional logging libraries (`log4j` , `tracing` , `spdlog`), logging is purely passive and unstructured. The code emits arbitrary string messages (`log.error("Invalid token")`). To an automated operating system or AI agent, these strings are unqueriable noise.

The `failpoint` subsystem transforms the **Detect** organ from a passive tape recorder into an **active sensory reflex**:

1. **Pre-Computed Remediation:** Every linguistic failpoint embeds a ready-to-apply `FixSuggestion` containing 1-based source coordinates, the erroneous target token, and the certified patch string. The system does not merely detect a disease; it synthesizes the vaccine in the same microsecond.
2. **Substrate State Grounding:** Every hardware failpoint grounds the detection in the physical reality of the CPU—recording the 13 silicon registers (`SiliconRegisters`) and the dual-rail state.
3. **Zero-Ambiguity Contract:** The Assessor and Restorator never have to parse prose or guess what went wrong. The typed enum `FailpointType` specifies the exact ontological category across 6 groups and 14 subgroups.

3. The Dual-Rail Balanced Ternary Physics of HALT

To understand failpoints in OmniCode, one must understand the physical and algebraic nature of **HALT** in balanced ternary logic.

Two Bits Represent One Trit

In classical binary logic, a single bit represents `0` or `1`. In **balanced ternary dual-rail logic**, two physical bits are required to represent one ternary trit $\mathbb{T}_3 \in {-1, 0, +1}$ with confirmation:

Dual-Rail Code	Physical Rails `(pos, neg)`	Balanced Trit	QuadState	Transport Analogy	Physical System Meaning
`00`	`(0, 0)`	`0`	**AWAIT**	**PAUSE**	Homeostasis, resting equilibrium, zero entropy
`01`	`(1, 0)`	`+1`	**PROCEED**	**PLAY**	Forward affirmation, tape roll commit motion
`10`	`(0, 1)`	`-1`	**ROLLBACK**	**REWIND**	Retroactive negation, rewind along tape ribbon
`11`	`(1, 1)`	$\bot$	**HALT**	**STOP**	Saturated state, zero free data, collision trap

The Profound Truth of HALT: Why HALT Restarts

Notice the crucial distinctions between the three non-affirmative states:

1. **`ROLLBACK ( -1 )` is a form of rewinding, not stopping.** It reverses the causal momentum vector ($\vec{v} \rightarrow -\vec{v}$) and steps backward along the tape ribbon to a known stable landmark.
2. **`AWAIT ( 0 )` is a pause, not a stop.** It represents homeostasis, resting equilibrium, and asynchronous standby awaiting new inputs.
3. **`HALT ( 11 )` is an abrupt stop.** It occurs when both rails are asserted high simultaneously. Because both rails are high, **there is no free data capacity left in the channel**. The system has hit an immovable boundary.

Crucial System Law: *HALT does not terminate or kill the machine; HALT restarts the machine!*

If a machine stops permanently and cannot resume execution, it has **flatlined**. In an operating system engineered to endure for a thousand years (**MilleniumOS**), flatlining is mathematically prohibited:

> *"For a just man falleth seven times, and riseth up again: but the wicked shall fall into mischief." — **Proverbs 24:16 (KJV)***

When an abrupt HALT occurs:

1. The cybernetic collision tripwire intercepts the saturated `(1, 1)` state.
2. The `DiagnosticLogger` captures a complete `HaltDumpFrame`, recording the physical RAM landmark and Track 0 silicon registers.
3. The saturated dual-rail charge is discharged from `(1, 1)` to `(0, 0)`.
4. The system cascades down to a verified safe landmark (`0x7C00`, `0x0500`, or `0x18000`) and **re-incepts / restarts** back into equilibrium (`AWAIT`).

---

4. Track 0 Hardware Dumps & Re-Inception Mechanics

When an abrupt HALT or hardware trap occurs at the bare-metal level in Track 0, the kernel follows the **Track 0 Adaptive De-escalation Protocol**:

```
================================================================================
                  TRACK 0 BARE-METAL HALT DUMP SEQUENCE
================================================================================

 [ Abrupt HALT / Hardware Trap Intercepted ]
                    |
                    ▼
 [ 1. Inscribe RAM Flight Recorder ]
      Write 16-byte binary event entry at Landmark 0x18000:
      [ event_id (u32), cr0 (u32), esp (u32), status (u32) ]
                    |
                    ▼
 [ 2. Dump Calibrated Silicon Registers ]
      Simultaneously emit to Screen and COM1 Serial UART (0x03F8):
      • Cognitive: %head, %home, %turn, %spool, %spool_in, %spool_out, %frame
      • General:   EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP
      • Control:   CR0 (Protected Mode), CR3 (Paging), CR4, EFLAGS
                    |
                    ▼
 [ 3. Adaptive Boot Ladder De-escalation ]
```

```
      De-escalate execution tier:
      • 64-Bit Long Mode (0x108000)      ——> De-escalates to 32-Bit PM
      • 32-Bit Protected Mode (0x18000)  ——> De-escalates to 16-Bit MBR
      • 16-Bit Real Mode (0x7C00)        ——> Cascades to Genesis (0x0500)
                     |
                     ▼
 [ 4. Enter Low-Power Anti-Lockout Watchdog Sentinel ]
      CPU enters calibrated `hlt` idle loop waiting for interrupt/timer
      or resets instruction pointer directly to re-incept:
      `jmp 0x0000:0x7C00`
================================================================================
```

The 13 Calibrated Silicon Registers of Track 0

Track 0 assigns designated architectural roles to CPU and control registers:

Register	Silicon Mapping	Cognitive & Architectural Role in MilleniumOS
`%head`	`RAX` / `EAX` / `AX`	Cognitive focus accumulator and active tape head
`%home`	`RBX` / `EBX` / `BX`	Moral origin anchor and base memory landmark pointer
`%turn`	`RCX` / `ECX` / `CX`	Sequential roll step counter and loop epoch counter
`%spool`	`RDX` / `EDX` / `DX`	Primary I/O MMIO port origin (`0x03F8` COM1 UART)
`%spool_in`	`RSI` / `ESI` / `SI`	Inbound sensory stream telemetry head
`%spool_out`	`RDI` / `EDI` / `DI`	Outbound action inscriber and tape printer head
`%frame`	`RBP` / `EBP` / `BP`	Execution frame boundary and covenant landmark
`%rsp`	`RSP` / `ESP` / `SP`	Silicon execution stack pointer (anchored at `0x7C00`)
`%cr0`	`CR0` Control Reg	Protection enable (`PE`), paging, numeric error flags
`%cr3`	`CR3` Control Reg	Page Directory Base Register (`0x00108000`)
`%cr4`	`CR4` Control Reg	Physical Address Extension (PAE) and OS FXSR flags
`%rflags`	`RFLAGS` / `EFLAGS`	Parity, zero, carry, and directional execution flags

5. The Complete Failpoint Taxonomy: Groups & Subgroups Matrix

Failpoints are classified into a 2-tier hierarchy: **6 Groups** and **14 Subgroups**.

Domain Group	Functional Subgroup	Primary Failpoints	Recovery Action	Physical/Logical Boundary
`HardwareSilicon`	`CpuTrap`	`FP_CPU_HALT_TRAP`	**`NULL:HALT`** (Restart)	CPU hardware traps (`#GP` , `#PF` , `#DF` , `#MC`)
	`FirmwareBus`	`FP_HARDWARE_FAULT` , `WATCHDOG`	**`NULL:HALT`** (Restart)	Firmware bus stalls and watchdog sentinel timeouts
	`AluSanity`	`FP_ALU_EQUILIBRIUM_BREACH`	**`NULL:HALT`** (Restart)	Balanced ALU ternary parity: `(+1) + (-1) != 0`
`MemoryBoundary`	`LandmarkBounds`	`FP_MEMORY_LANDMARK_BREACH`	**`NULL:HALT`** (Restart)	Violations of `0x7C00` , `0x18000` , `0x108000` , `0x200000`
	`SegmentationPaging`	MMIO / Stack Underflow	**`NULL:HALT`** (Restart)	Paging tables (`CR3`), GDT descriptors, stack collisions
`TernarySaturation`	`DualRailCollision`	`FP_DUAL_RAIL_SATURATION`	**`NULL:HALT`** (Restart)	Both rails high `(1, 1)` ; zero free data capacity
	`StateCollapse`	`FP_NULL_TRAP` ($\bot$)	**`NULL:HALT`** (Restart)	Evaluation attempted unhandled bottom/void type
`LinguisticSyntax`	`DelimiterHierarchy`	`FP_BOUNDARY_MISMATCH` , `LEAK`	**`-1:ROLLBACK`** (Rewind)	Mismatched or unclosed delimiters (`<...>` , `[...]`)
	`StructuralScaffold`	`FP_SYNTAX_ANOMALY`	**`-1:ROLLBACK`** (Rewind)	Missing 4-Block scaffolding headers and grammar drift

Domain Group	Functional Subgroup	Primary Failpoints	Recovery Action	Physical/Logical Boundary
	`TypeSemantic`	`FP_IMMUTABLE_MUTATION`	**`-1:ROLLBACK`** (Rewind)	Illegal attempt to mutate Rank 1 immutable axioms
`RuntimeTransaction`	`Retroaction`	`FP_ROLLBACK_TRIGGERED`, `CYCLIC`	**`-1:ROLLBACK`** (Rewind)	Causal tape rewinds and cyclic recursion limits
	`ConcurrencyBus`	`FP_CAVEAT_VETO`	**`-1:ROLLBACK`** (Rewind)	Adversative caveat expressions (`! [...]`) vetoing state
`HomeostasisLifecycle`	`EquilibriumDrift`	`FP_EQUILIBRIUM_DRIFT_VETO`	**`-1:ROLLBACK`** (Rewind)	System state drifted from Ω_0 baseline balance
	`ShutdownVeto`	Dirty State / Memory Leak	**`-1:ROLLBACK`** (Rewind)	Unreleased resources or dirty frames vetoing shutdown

6. Developer Experience: Warnings as Healers, Errors as Inceptors

A central mystery of software engineering is why developers persistently avoid writing diagnostic telemetry and treat error handling as a burdensome chore.

The Problem with Traditional Systems: Ignored Warnings & Lethal Errors

In Unix, C, Rust, Go, Python, and Java, developers are caught between two dysfunctional extremes:

1. **Traditional Warnings are Impotent:** Compilers and linters emit hundreds of warning lines to `stderr`. Because warnings carry zero mechanical consequence at runtime, developers habitually ignore them, filter them with compiler flags (`-w`), or allow them to accumulate into technical debt.
2. **Traditional Errors are Lethal & Punitive:** When an unhandled error or panic strikes, the runtime's immediate response is capital punishment: unwinding the stack, blowing away in-flight memory, dumping a 50-line backtrace, and killing the process. The developer is left with a dead terminal and the painful task of post-mortem debugging.
3. **Logging is Passive Paperwork:** Writing `logger.error("Failed to parse")` feels like filling out an incident report after a car crash. The log message does not prevent the crash or repair the car; it merely complains.

The OmniCode Paradigm: The 2-Trigger Mechanical Guarantee

OmniCode transforms diagnostic telemetry from passive paperwork into an **active, self-healing lifeline**. By giving both triggers physical, mechanical reality, the DAR framework fundamentally alters developer psychology:

```
================================================================================
       TRADITIONAL COMPUTING vs. OMNICODE 2-TRIGGER DAR PARADIGM
================================================================================

  Dimension       Traditional Computing          OmniCode DAR System
  ──────────────────────────────────────────────────────────────────────────

  Warning         Ignored text on stderr         -1:ROLLBACK Trigger
                  No runtime consequence         Rewinds tape head to landmark
                  Accrues silent debt            Restorator applies FixSuggestion
                                                 Live auto-healing on tape!

  Error / Fault   Lethal crash / panic           NULL:HALT Trigger
                  Process flatlines              Silicon & RAM telemetry dumped
                  Data in memory destroyed       Rails discharged: (1, 1) -> (0, 0)
                  Demands manual restart         Re-incepts to 0:AWAIT at 0x7C00

  Logging Mode    Passive, post-mortem string    Active, cybernetic sensory organ
                  Unstructured text for humans   Typed telemetry for AI & engine
                  Treated as an annoying chore   Provides instant self-healing
================================================================================
```

Why Developers Will Enthusiastically Use DAR Logging

When writing code in OmniCode, a developer does not write diagnostic calls out of reluctant duty. They use the DAR logging library because **it directly protects their work and accelerates their velocity**:

1. **The Live Safety Net (No Permadeath):** In traditional coding, one mistyped token or boundary mismatch blows up the entire run. In OmniCode, wrapping code in a boundary tripwire with DAR logging means typos are automatically caught, rewound, and patched live on the

process tape. Programming changes from "walking a tightrope over an abyss" into "playing a video game with infinite checkpoints and auto-rewind".

2. **Native Synergy with AI Coding Agents:** Because failpoints emit structured, token-level `FixSuggestion` telemetry with exact 1-based source coordinates, local AI models (such as the offline `CPI-SI` core or IDE assistants) can read the log and synthesize the exact patch in milliseconds. The developer never has to spend hours hunting down subtle off-by-one errors or unclosed brackets.
3. **Zero Flatline Guarantee:** Even when code hits a catastrophic silicon boundary or out-of-bounds pointer, the machine **never dies**. The developer's interactive session is never destroyed; the system simply re-incepts back to homeostasis (`0:AWAIT`) with the complete flight record preserved in memory.

---

7. Public API Reference & Usage Guide

A. Trapping a Failpoint with Automated Remediation

```
use omnicode::diagnostics::logger::{
    DiagnosticLogger, FailpointType, FixKind, FixSuggestion,
};

let logger = DiagnosticLogger::new(false);

let suggestion = FixSuggestion::new(
    FixKind::FixBoundaryMismatch,
    14,
    8,
    "]",
    ">",
    "Replace mismatched bracket with matching invariant angle closer",
);

logger.failpoint_with_remediation(
    FailpointType::BoundaryMismatch,
    "PARSER",
    "Delimiter mismatch '<' closed by ']'",
    suggestion,
);

assert!(logger.has_failpoints());
```

B. Intercepting an Abrupt HALT and Executing Re-Inception

```
use omnicode::diagnostics::logger::failpoint::{
    create_halt_dump, HaltReason, ReInceptionTarget, SiliconRegisters,
    FailpointType,
};
use omnicode::diagnostics::logger::DiagnosticLogger;
use omnicode::core::ternary::QuadState;

let logger = DiagnosticLogger::new(false);

// 1. Capture the crash dump frame from silicon
let dump_frame = create_halt_dump(
    145000,                           // timestamp in µs
    HaltReason::DualRailSaturation,     // both rails (1, 1) high
    FailpointType::DualRailSaturation,  // failpoint type
    0x18000,                          // active landmark (Stage 2 PM)
    SiliconRegisters::calibrated_zero(),// frozen register bank
    ReInceptionTarget::BiosMbrSector0,  // cascade down to 0x7C00
    1,                                // first re-inception
);

// 2. Logger records dump, logs cognitive note, and returns AWAIT
```

```
let recovery_state = logger.record_halt_dump(dump_frame);

// 3. Confirm machine has NOT flatlined; it has returned to homeostasis!
assert_eq!(recovery_state, QuadState::Await);
```

C. Inspecting Failpoint Taxonomy Metadata

```
use omnicode::diagnostics::logger::failpoint::{
    FailpointType, FailpointGroup, FailpointSubgroup,
};

let fp = FailpointType::CpuHaltTrap;
assert_eq!(fp.group(), FailpointGroup::HardwareSilicon);
assert_eq!(fp.subgroup(), FailpointSubgroup::CpuTrap);
assert!(fp.is_halt_trigger());
assert_eq!(fp.as_str(), "FP_CPU_HALT_TRAP");
```

6. Covenantal Invariant Sealing

Invariant ID	Contract Rule & Verification Condition
INV-FP-01	Failpoints must map to balanced ternary states without unhandled panic.
INV-FP-02	Saturated dual-rail states `(1, 1)` trigger hardware dump capture.
INV-FP-03	Abrupt HALT must never flatline; it must re-incept to `0:AWAIT`.
INV-FP-04	All failpoints declare valid `FailpointGroup` and `FailpointSubgroup`.
INV-FP-05	Hardware crash dumps capture Track 0 silicon registers and landmarks.

> *"The secret things belong unto the LORD our God: but those things which are revealed belong unto us and to our children for ever, that we may do all the words of this law."* — ***Deuteronomy 29:29 (KJV)***

project_title: "The 2-Trigger Collision Architecture" subtitle: "Warnings as Healers, Errors as Inceptors" subsystem: "Detect, Assess, Restore (DAR) Developer Experience" classification: "Canonical Architectural Foundation (Rank 1 Invariant)" status: "Active / Tested across Unit & Integrated Test Suites" module: "src/diagnostics/logger/failpoint"

The 2-Trigger Collision Architecture: Warnings as Healers, Errors as Inceptors

```
================================================================================
  MILLENIUMOS DEVELOPER EXPERIENCE : 2-TRIGGER COLLISION TRIPWIRE SPECIFICATION
  Subsystem:      Detect, Assess, Restore (DAR) Developer Experience
  Module:         src/diagnostics/logger/failpoint/
  Document:       TWO_TRIGGER_TRIPWIRE_ARCHITECTURE.md
  Classification: Canonical Architectural Foundation (Rank 1 Invariant)
  Status:         Active / Tested across Unit & Integrated Test Suites
================================================================================
```

> *"A prudent man foreseeth the evil, and hideth himself: but the simple pass on, and are punished." — **Proverbs 22:3 (KJV)***
>
> *"For a just man falleth seven times, and riseth up again: but the wicked shall fall into mischief." — **Proverbs 24:16 (KJV)***
>
> *"Remove not the ancient landmark, which thy fathers have set." — **Proverbs 22:28 (KJV)***

1. Executive Summary & The Broken Paradigm of Traditional Software

In standard software engineering (C, C++, Rust, Go, Python, Java), the **"Warning vs. Error"** paradigm has been deeply dysfunctional for fifty years:

1. **Traditional Warnings are Impotent Noise:** Compilers and linters emit hundreds of warning lines to standard error. Because warnings carry zero mechanical consequence at runtime, developers habitually ignore them, suppress them with compiler flags (`-w` , `allow(unused)`), or allow them to accumulate into latent security CVEs and production defects.
2. **Traditional Errors are Punitive Executions:** When an unexpected error or panic condition occurs at runtime, the execution environment executes capital punishment: unwinding the call stack, destroying in-flight user transactions, dumping an unreadable 50-line stack trace to the console, and terminating the process.
3. **Traditional Logging is Passive Administrative Paperwork:** Writing `logger.error("Failed to connect")` feels like filling out an insurance form after a car crash. The log message does not prevent the crash, heal the wound, or repair the vehicle; it merely logs a grievance.

Consequently, developers treat logging and error handling as an annoying afterthought, writing the bare minimum required to pass code review.

OmniCode repudiates this entire philosophy. By integrating the **DAR (Detect, Assess, Restore)** engine directly into the compiler and operating system, OmniCode replaces passive logging with the **2-Trigger Cybernetic System**:

```
graph TD
    A["Boundary Tripwire Collision Intercepted"] --> B{"Collision Domain"}
    B -->|"Linguistic / Soft Semantic Breach<br/>(Delimiter Mismatch, Axiom Mutation, Veto)"| C["Trigger 1: Rollback (-1:ROLLBACK
    B -->|"Silicon / Hardware Saturation<br/>(Dual-Rail 11, CPU Trap, ALU Breach)"| D["Trigger 2: Halt (NULL:HALT)<br/>• Substrat
```

System Dimension	Traditional Computing (C / Unix / Rust)	OmniCode 2-Trigger DAR Paradigm
Warning Paradigm	Ignored text on `stderr` with zero mechanical runtime consequence; accrues technical debt.	**`-1:ROLLBACK` Trigger**: Rewinds tape head to previous landmark; Restorator applies live `FixSuggestion` patch on active tape.
Error / Fault	Lethal crash or unhandled panic; process flatlines and destroys memory; requires manual human restart.	**`NULL:HALT` Trigger**: Track 0 silicon & RAM dump captured; rails discharged from `(1, 1)` to `(0, 0)` ; re-incepts to `0:AWAIT` at `0x7C00` .
Logging Philosophy	Passive post-mortem strings for human eyes; treated as an administrative chore.	**Active Cybernetic Sensory Organ**: Strongly-typed telemetry feeding the engine and AI for live self-healing.

2. Trigger Type 1: The Rollback Trigger (`-1:ROLLBACK` ~ The Cybernetic Warning)

In OmniCode, a "warning-level" anomaly is never ignored text. It is a **Rollback Trigger**:

$$\text{Trigger}_{\text{Rollback}} \Longrightarrow \text{QuadState::Rollback} \ (-1)$$

When Rollback Fires

The Rollback Trigger fires when an execution vector touches a soft, recoverable, linguistic, or transactional boundary:

- Delimiter mismatches (`<` closed with `]`)
- Boundary leaks (unclosed blocks reaching EOF)
- Adversative caveat vetoes (`! [ condition ]`)
- Illegal attempts to mutate Rank 1 immutable axioms (`< ... >`)
- Transactional rollbacks or cyclic recursion thresholds

Rollback Channel Status: Substrate Intact

The underlying silicon substrate is **completely healthy**. Only one rail was asserted (`10` or `01`). Free data capacity exists in the transmission channel. The computation simply encountered a semantic contradiction.

Rollback Mechanical Action: Live Tape Auto-Healing

1. The tripwire catches the collision.
2. It reverses the causal velocity vector ($\vec{v} \rightarrow -\vec{v}$) and steps the process tape head back to the latest anchored landmark.
3. It emits a concrete `FixSuggestion` containing exact 1-based source coordinates, the offending token, and the certified patch string.
4. The **Restorator** consumes the suggestion, applies the patch to the AST, validates it against an executable test oracle, and rolls execution forward in `+1:PROCEED` .

```
# Offending code parsed on the active reel tape (Mismatched delimiter: '{' closed with ']'):
Opening {
    [ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte ]
}

# Restorator auto-heals token via FixSuggestion:
Opening {
    [ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte }  ( Healed in-flight; execution rolls forward in +1:PROCEED )
}
```

The Developer Experience: The developer makes a syntax slip or logic error, and **the program does not crash**. The runtime steps back, heals the token live on the tape ribbon, and continues running seamlessly.

---

3. Trigger Type 2: The Halt Trigger (`NULL:HALT` ~ The Substrate Error)

In OmniCode, an "error-level" anomaly is never a fatal panic. It is a **Halt Trigger**:

$$\text{Trigger}_{\text{Halt}} \Longrightarrow \text{QuadState::Halt} \ (\bot)$$

When Halt Fires

The Halt Trigger fires when an execution vector touches a hard, physical silicon boundary or mathematical paradox:

- Dual-rail hardware saturation pegs at `(1, 1)`
- Bare-metal CPU traps (`#GP` , `#PF` , `#DF` , `#MC`)
- Balanced ternary ALU sanity failure: `(+1) + (-1) != 0`
- RAM landmark violations (crossing `0x7C00` , `0x18000` , `0x108000` , or `0x200000`)
- Watchdog sentinel timeouts

Halt Channel Status: Substrate Saturated (1, 1)

The substrate has saturated. Both rails are energized high `(1, 1)` . **There is zero free data capacity left in the channel.** The logic cannot compute a forward recovery in-place because the physical ground has slipped.

Halt Mechanical Action: Safe De-Escalation & Re-Inception

1. Forward execution is halted immediately to protect physical memory from corruption.
2. The tripwire inscribes a 16-byte binary event log at RAM landmark `0x18000` :

`text In-Memory Flight Recorder Format (16 Bytes at Landmark 0x18000): [ event_id: 0x0005 | cr0: 0x80010033 | esp: 0x00017FF0 | status: 0x00000000 ]`

1. It captures the **13 Calibrated Silicon Registers** (`%head` , `%home` , `%turn` , `%spool` , `CR0` , `CR3` , `CR4` , `RFLAGS`) and emits a full diagnostic dump to the COM1 serial port (`0x03F8`).
2. The saturated dual-rail charge is discharged from `(1, 1)` to `(0, 0)` .
3. The system cascades down the adaptive boot ladder (Long Mode $\rightarrow$ Protected Mode $\rightarrow$ BIOS MBR) and **re-incepts back at landmark `0x7C00` into homeostasis (`0:AWAIT`)**.

The Developer Experience: Even when the program triggers a hard silicon trap or corrupted memory landmark, **the machine never dies**. The developer's terminal does not freeze, and their interactive session is not destroyed. The OS safely discharges, drops down to a known landmark, and awakens in equilibrium with the complete crash dump safely stored in episodic memory.

---

4. Why This Natively Incentivizes Developers to Use the DAR Logging Library

When error handling is transformed from a punitive execution into an autonomic immune system, developer psychology fundamentally changes. Developers will enthusiastically use the DAR logging library for three compelling reasons:

1. The Death of Permadeath (The Video Game Metaphor)

Traditional programming feels like an old-school video game with "Permadeath": one unhandled exception, and you lose all your progress, your process dies, and you start from scratch.

Coding with OmniCode DAR logging feels like playing a modern game with **infinite checkpoints, dynamic rewinds, and an active safety net**. If you make a mistake, the system rolls back to the last milestone and lets you try again. If the engine crashes, you instantly respawn at the checkpoint with all your inventory intact. Developers will naturally wrap every sensitive transaction in DAR tripwires because it guarantees their code will never crash.

2. Native Symbiosis with AI Coding Agents

Modern developers increasingly pair-program with LLMs and autonomous AI agents (such as Google NotebookLM or the local `CPI-SI` offline core). In traditional systems, an AI agent cannot easily debug a running program because stack traces are unstructured prose strings.

In OmniCode, every failpoint emits a machine-actionable `FixSuggestion` with exact 1-based source coordinates and replacement tokens. An AI agent ingesting the DAR telemetry stream can read the failpoint, generate the exact AST patch, and heal the codebase in milliseconds without the human developer ever having to context-switch into a debugger.

3. Zero Guilt, Zero Boilerplate

Developers hate writing logging code when it feels like administrative paperwork. In DAR, logging is not an audit trail for external auditors; **it is the actual circulatory system of self-healing software**. Writing a DAR log statement is what grants your function the power of automatic rollback and rebootless re-inception. You do not log out of duty; you log to make your software invincible.

---

5. Architectural Invariants

> *[!IMPORTANT] The DAR system is bound by three unalterable Rank 1 Invariants:*
>
> 1. ***INV-2TRIG-01 (Dual-Rail Symmetry):*** *All system tripwires must partition anomalies strictly into either `-1:ROLLBACK` (Linguistic) or `NULL:HALT` (Silicon).*
> 2. ***INV-2TRIG-02 (Zero Unhandled Panics):*** *No runtime component may invoke an unhandled panic or exit with a non-zero status without first triggering a Rollback or Halt transition.*
> 3. ***INV-2TRIG-03 (AI Queriability):*** *All failpoints captured by the DAR logger must be exportable to structured `.scroll` notes for AI episodic digestion.*

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/halt.rs

RS | 304 LINES | 1211 WORDS | 11.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Dual-Rail HALT Telemetry, Track 0 Register Dumps & Re-Inception Mechanics )
// ============================================================================
//! OmniCode Diagnostics: HALT Physics & Silicon Re-Inception Telemetry
//!
//! # The Dual-Rail HALT Invariant
//! In balanced ternary logic, 2 bits represent 1 trit:
//! - `(0, 0)` = AWAIT (0): Homeostasis, neutral equilibrium, standby, pause.
//! - `(0, 1)` = ROLLBACK (-1): Negative rail high. Rewind, time-travel.
//! - `(1, 0)` = PROCEED (+1): Positive rail high. Forward commit motion.
//! - `(1, 1)` = HALT (NULL / ⊥): Both rails high! Saturated. Zero free data.
//!
//! # Why HALT Restarts Instead of Flatlining
//! A flatlined machine is a dead machine. For an operating system with a
//! millennial horizon (MilleniumOS), flatlining is mathematically prohibited:
//! - `ROLLBACK` is a form of rewinding, not stopping.
//! - `AWAIT` is a pause (standby/homeostasis), not a stop.
//! - `HALT` is an abrupt stop triggered when both rails saturate at `(1, 1)`.
//!
//! When a HALT occurs, the collision sensor (Tripwire) catches it,
//! captures the full hardware state (Track 0 registers, CR0/CR3/CR4),
//! and **restarts the machine back to homeostasis (`0`, or AWAIT)**.

use std::fmt;
use super::types::FailpointType;
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Halt Reason, Silicon Register Bank, Re-Inception Target & Halt Dump Frame )
// ============================================================================

/// Underlying trigger cause for an abrupt HALT event.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum HaltReason {
    /// Dual-rail hardware state saturated at (1, 1); zero free data capacity.
    DualRailSaturation,
    /// CPU hardware trap intercepted (e.g. #GP, #PF, #UD, #DF, #MC).
    CpuTrap(u32),
    /// Balanced ternary ALU sanity failed: (+1) + (-1) != 0 breach.
    AluImbalance,
    /// Memory landmark violation (e.g. illegal access crossing 0x7C00, 0x18000).
    LandmarkViolation,
    /// Watchdog sentinel idle loop expired without interrupt.
    WatchdogTrip,
    /// Low-power `hlt` sentinel loop invoked by safe-mode de-escalation.
    ExplicitHaltInstruction,
}

/// The 13 calibrated silicon registers and CPU control registers from Track 0.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct SiliconRegisters {
    // The 7 Cognitive Silicon Registers of Track 0
    pub head: u64,      // %head: Cognitive focus
    pub home: u64,      // %home: Moral origin anchor
    pub turn: u64,      // %turn: Step/tick counter
    pub spool: u64,     // %spool: I/O port origin
    pub spool_in: u64,  // %spool_in: Inbound stream head
    pub spool_out: u64, // %spool_out: Outbound stream head
    pub frame: u64,     // %frame: Execution frame landmark

    // General CPU Registers
    pub rax: u64,
    pub rbx: u64,
    pub rcx: u64,
    pub rdx: u64,
    pub rsi: u64,
    pub rdi: u64,
    pub rsp: u64,
    pub rbp: u64,

    // Hardware Control Registers & Flags
    pub cr0: u64,
    pub cr3: u64,
```

```
    pub cr4: u64,
    pub rflags: u64,
}

/// Safe-mode de-escalation landmark to which machine cascades down upon HALT.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum ReInceptionTarget {
    /// Sector 0 Real-Mode Genesis Landmark at 0x7C00.
    BiosMbrSector0,
    /// Track 0 Real-Mode Landmark at 0x0500.
    RealModeGenesis,
    /// Stage 2 Protected Mode Flight Recorder Landmark at 0x18000.
    Stage2ProtectedMode,
    /// Long Mode 64-Bit Kernel Base Landmark at 0x108000.
    LongModeKernel,
    /// Cognitive Substrate Arena Landmark at 0x200000.
    CognitiveSubstrateArena,
    /// Hardware CPU Reset Vector at 0xFFFF0.
    CpuResetVector,
}

/// Comprehensive flight-recorder crash dump frame captured upon an abrupt HALT.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct HaltDumpFrame {
    /// Monotonic microsecond timestamp when the HALT occurred.
    pub tick_micros: u128,
    /// Specific causal reason for the abrupt halt.
    pub reason: HaltReason,
    /// The trapped failpoint classification.
    pub failpoint: FailpointType,
    /// Dual-rail hardware state at halt: always `(1, 1)` (saturated).
    pub dual_rail_saturation: (u8, u8),
    /// Active physical memory landmark at the moment of the halt.
    pub landmark_address: u64,
    /// Snapshot of silicon and CPU registers captured at collision boundary.
    pub registers: SiliconRegisters,
    /// Target landmark where the machine cascades down to re-incept.
    pub re_inception_target: ReInceptionTarget,
    /// The target homeostasis state after re-inception (always `0:AWAIT`).
    pub re_inception_state: QuadState,
    /// Number of cumulative re-inceptions executed in this session.
    pub restart_count: usize,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructors, Dump Formatters, Register Calibrations & Re-Inception )
// ============================================================================

impl SiliconRegisters {
    /// Constructs a clean register bank calibrated to baseline zero.
    pub fn calibrated_zero() -> Self {
        Self {
            head: 0,
            home: 0,
            turn: 0,
            spool: 0,
            spool_in: 0,
            spool_out: 0,
            frame: 0,
            rax: 0,
            rbx: 0,
            rcx: 0,
            rdx: 0,
            rsi: 0,
            rdi: 0,
            rsp: 0x7C00,
            rbp: 0x7C00,
            cr0: 0x0000_0001,
            cr3: 0x0010_8000,
            cr4: 0x0000_0020,
            rflags: 0x0000_0002,
        }
    }

    /// Formats the registers into an 80-column diagnostic dump.
    pub fn format_dump(&self) -> String {
        format!(
            "REGISTERS:\n  \
             head: {:#018x}  home: {:#018x}  turn: {:#018x}\n  \
             spool: {:#018x} in:   {:#018x}  out:  {:#018x}\n  \
```

```
             rax:  {:#018x}  rbx:  {:#018x}  rcx:  {:#018x}\n  \
             rdx:  {:#018x}  rsi:  {:#018x}  rdi:  {:#018x}\n  \
             rsp:  {:#018x}  rbp:  {:#018x}  cr0:  {:#018x}\n  \
             cr3:  {:#018x}  cr4:  {:#018x}  rflags: {:#018x}",
            self.head, self.home, self.turn,
            self.spool, self.spool_in, self.spool_out,
            self.rax, self.rbx, self.rcx,
            self.rdx, self.rsi, self.rdi,
            self.rsp, self.rbp, self.cr0,
            self.cr3, self.cr4, self.rflags
        )
    }
}

impl ReInceptionTarget {
    /// Resolves the physical RAM address for this landmark.
    pub fn address(&self) -> u64 {
        match self {
            ReInceptionTarget::BiosMbrSector0 => 0x7C00,
            ReInceptionTarget::RealModeGenesis => 0x0500,
            ReInceptionTarget::Stage2ProtectedMode => 0x18000,
            ReInceptionTarget::LongModeKernel => 0x108000,
            ReInceptionTarget::CognitiveSubstrateArena => 0x200000,
            ReInceptionTarget::CpuResetVector => 0xFFFF0,
        }
    }

    /// Returns the canonical uppercase identifier for this landmark.
    pub fn as_str(&self) -> &'static str {
        match self {
            ReInceptionTarget::BiosMbrSector0 => {
                "LANDMARK_BIOS_MBR_0x7C00"
            }
            ReInceptionTarget::RealModeGenesis => {
                "LANDMARK_REAL_GENESIS_0x0500"
            }
            ReInceptionTarget::Stage2ProtectedMode => {
                "LANDMARK_STAGE2_PM_0x18000"
            }
            ReInceptionTarget::LongModeKernel => {
                "LANDMARK_LONG_MODE_0x108000"
            }
            ReInceptionTarget::CognitiveSubstrateArena => {
                "LANDMARK_SUBSTRATE_0x200000"
            }
            ReInceptionTarget::CpuResetVector => {
                "LANDMARK_RESET_0xFFFF0"
            }
        }
    }
}

impl HaltDumpFrame {
    /// Constructs a new [`HaltDumpFrame`] capturing an abrupt HALT event.
    pub fn new(
        tick_micros: u128,
        reason: HaltReason,
        failpoint: FailpointType,
        landmark_address: u64,
        registers: SiliconRegisters,
        re_inception_target: ReInceptionTarget,
        restart_count: usize,
    ) -> Self {
        Self {
            tick_micros,
            reason,
            failpoint,
            dual_rail_saturation: (1, 1), // Always (1, 1) at HALT
            landmark_address,
            registers,
            re_inception_target,
            re_inception_state: QuadState::Await, // Restarts to homeostasis
            restart_count,
        }
    }

    /// Executes the re-inception transition: clears the saturated dual-rail
    /// state, returns machine to `0:AWAIT`, and cascades to the safe landmark.
    pub fn execute_re_inception(&self) -> QuadState {
        QuadState::Await
    }
```

```
    /// Formats this halt dump as a human-readable ANSI/ASCII terminal block.
    pub fn format_human(&self) -> String {
        format!(
            "================================================================================\n\
             [HALT INTERCEPTION DUMP] (Tick: {} µs | Re-inception Count: {})\n\
             Reason: {:?}\n\
             Failpoint: {}\n\
             Dual-Rail Saturation: ({}, {}) [NO FREE DATA - COLLISION TRAP]\n\
             Active Landmark: {:#08x} -> Re-Inception Target: {} ({:#08x})\n\
             Post-Halt Target State: {} [HOMEOSTASIS RESTORED]\n\
             --------------------------------------------------------------------------------\n\
             {}\n\
             ================================================================================",
            self.tick_micros,
            self.restart_count,
            self.reason,
            self.failpoint,
            self.dual_rail_saturation.0,
            self.dual_rail_saturation.1,
            self.landmark_address,
            self.re_inception_target.as_str(),
            self.re_inception_target.address(),
            self.re_inception_state.macro_op(),
            self.registers.format_dump()
        )
    }

    /// Formats this halt dump as a canonical `.scroll` note for AI memory.
    pub fn format_omnicode_scroll(&self) -> String {
        format!(
            "Note < HaltTrace > {{\n    \
                 tick: < {} µs >,\n    \
                 category: [ HALT_REINCEPTION ],\n    \
                 summary: < \"Abrupt HALT Trapped; Discharged to Await\" >,\n    \
                 failpoint: [ {} ],\n    \
                 saturation: ( 1, 1 ),\n    \
                 active_landmark: < {:#08x} >,\n    \
                 re_inception_target: < {} >,\n    \
                 re_inception_state: < PROCEED :: AWAIT >,\n    \
                 tags: [ #halt :: #reinception :: #dual_rail_11 :: #homeostasis ]\n\
             }}",
            self.tick_micros,
            self.failpoint.as_str(),
            self.landmark_address,
            self.re_inception_target.as_str()
        )
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementations, Invariant Proofs & Module Sealing )
// ============================================================================

impl fmt::Display for HaltDumpFrame {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            f,
            "[HALT: {:?} @ {:#08x} -> Re-incepting to {}]",
            self.reason,
            self.landmark_address,
            self.re_inception_target.as_str()
        )
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/mod.rs RS 62 LINES 228 WORDS 2.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Failpoint Subsystem Identity, Module Organization & Re-exports )
// ============================================================================
//! OmniCode Diagnostics: Failpoint Pathology, HALT Telemetry & Remediation
//!
//! Decomposed into specialized machine organs:
//! - [`types`]: FailpointType, FailpointGroup, and Subgroup taxonomy.
//! - [`halt`]: HALT physics, dual-rail saturation `(1, 1)`, and Track 0 dumps.
//! - [`remediation`]: Machine-applicable FixSuggestion and FixKind descriptors.

pub mod types;
pub mod halt;
pub mod remediation;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Failpoints & Telemetry )
// ============================================================================
pub use types::{FailpointGroup, FailpointSubgroup, FailpointType};
pub use halt::{HaltDumpFrame, HaltReason, ReInceptionTarget, SiliconRegisters};
pub use remediation::{FixKind, FixSuggestion};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Convenience Helpers for Failpoint & HALT Classification )
// ============================================================================
/// Returns `true` if a dual-rail hardware code represents the saturated
/// HALT state (11).
#[inline]
pub fn is_dual_rail_halt(code: u8) -> bool {
    (code & 0b11) == 0b11
}

/// Creates a standardized [`HaltDumpFrame`] capturing an abrupt HALT event
/// and targeting immediate re-inception to a safe landmark.
pub fn create_halt_dump(
    tick_micros: u128,
    reason: HaltReason,
    failpoint: FailpointType,
    landmark_address: u64,
    registers: SiliconRegisters,
    re_inception_target: ReInceptionTarget,
    restart_count: usize,
) -> HaltDumpFrame {
    HaltDumpFrame::new(
        tick_micros,
        reason,
        failpoint,
        landmark_address,
        registers,
        re_inception_target,
        restart_count,
    )
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Module Sealing & Invariant Verification )
// ============================================================================
// Failpoint pathology subsystem verified and sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/remediation.rs RS | 168 LINES | 614 WORDS | 6.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Machine-Applicable Automated Repair Kinds & Concrete Fix Suggestions )
// ============================================================================
//! OmniCode Diagnostics: Concrete Automated Remediation Telemetry
//!
//! Encapsulates machine-applicable patch descriptions allowing the
//! Restorator, OmniDebugger, or AI coding agents to heal broken source files
//! and restore system equilibrium without human intervention.
//!
//! # Remediation Invariants
//! 1. Every fix suggestion specifies exact 1-based coordinates (`line`, `col`).
//! 2. Candidate repairs must be verified by test oracles before commit.
//! 3. Hardware HALT remediations discharge saturated rails back to `00` Await.

use std::fmt;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Categories of Automated Repair & Structured Suggestion Type )
// ============================================================================

/// Categories of automated restorative repairs executed by Restorator.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum FixKind {
    /// Append a missing closing boundary delimiter (e.g. closing `[` with `]`).
    CloseBoundary,
    /// Substitute erroneous closing delimiter with matching opening partner.
    FixBoundaryMismatch,
    /// Inject missing canonical 4-Block architecture scaffolding headers.
    AddBlockHeader,
    /// Automatically inject missing core library imports (Core.Arithmetic).
    AddImport,
    /// Synthesize and declare an inferred conceptual noun entity.
    DeclareEntityWithInfer,
    /// Modify or satisfy an adversative caveat expression.
    AdjustCaveat,
    /// Convert an immutable Rank 1 invariant `< ... >` into mutable `[ ... ]`.
    ConvertImmutableToMutable,
    /// Re-incept machine by cascading down to a verified landmark upon HALT.
    ReInceptToLandmark,
    /// Discharge saturated dual-rail `(1, 1)` state back to `(0, 0)` Await.
    ResetDualRailSaturation,
    /// Restore balanced ternary ALU symmetry by zeroing leaked register bits.
    RebalanceAlu,
}

/// Concrete remediation telemetry attached to a failpoint event.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FixSuggestion {
    /// Categorical classification of the automated remediation.
    pub kind: FixKind,
    /// 1-based source line where remediation applies (0 if hardware).
    pub line: usize,
    /// 1-based source column where the target token resides (0 if hardware).
    pub col: usize,
    /// The erroneous or missing token sequence targeted for replacement.
    pub target_token: String,
    /// The proposed patch or substitution string that seals the invariant.
    pub suggested_patch: String,
    /// Human-readable and AI-interpretable rationale explaining the fix.
    pub rationale: String,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructors, Patch Preview Formatters & Remediation Introspection )
// ============================================================================

impl FixSuggestion {
    /// Constructs a new [`FixSuggestion`].
    pub fn new(
        kind: FixKind,
        line: usize,
        col: usize,
```

```
        target_token: impl Into<String>,
        suggested_patch: impl Into<String>,
        rationale: impl Into<String>,
    ) -> Self {
        Self {
            kind,
            line,
            col,
            target_token: target_token.into(),
            suggested_patch: suggested_patch.into(),
            rationale: rationale.into(),
        }
    }

    /// Creates a hardware re-inception fix suggestion for an abrupt HALT.
    pub fn hardware_reinception(
        landmark_address: u64,
        rationale: impl Into<String>,
    ) -> Self {
        Self {
            kind: FixKind::ReInceptToLandmark,
            line: 0,
            col: 0,
            target_token: "HALT_SATURATION".to_string(),
            suggested_patch: format!(
                "REINCEPT_TO_LANDMARK_{:#08x}",
                landmark_address
            ),
            rationale: rationale.into(),
        }
    }

    /// Formats a clean patch preview showing target token vs. suggested patch.
    pub fn format_patch_preview(&self) -> String {
        format!(
            "[{:?}] @ {}:{} Target: {:?} => Patch: {:?}",
            self.kind,
            self.line,
            self.col,
            self.target_token,
            self.suggested_patch
        )
    }

    /// Returns `true` if this suggestion targets hardware or HALT recovery.
    pub fn is_hardware_remediation(&self) -> bool {
        matches!(
            self.kind,
            FixKind::ReInceptToLandmark
                | FixKind::ResetDualRailSaturation
                | FixKind::RebalanceAlu
        )
    }
}

impl FixKind {
    /// Returns the uppercase canonical string identifier for this fix kind.
    pub fn as_str(&self) -> &'static str {
        match self {
            FixKind::CloseBoundary => "FIX_CLOSE_BOUNDARY",
            FixKind::FixBoundaryMismatch => "FIX_BOUNDARY_MISMATCH",
            FixKind::AddBlockHeader => "FIX_ADD_BLOCK_HEADER",
            FixKind::AddImport => "FIX_ADD_IMPORT",
            FixKind::DeclareEntityWithInfer => "FIX_DECLARE_ENTITY_INFER",
            FixKind::AdjustCaveat => "FIX_ADJUST_CAVEAT",
            FixKind::ConvertImmutableToMutable => "FIX_CONVERT_IMMUTABLE_MUTABLE",
            FixKind::ReInceptToLandmark => "FIX_REINCEPT_TO_LANDMARK",
            FixKind::ResetDualRailSaturation => "FIX_RESET_DUAL_RAIL_SATURATION",
            FixKind::RebalanceAlu => "FIX_REBALANCE_ALU",
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementation & Subsystem Sealing )
// ============================================================================

impl fmt::Display for FixSuggestion {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
```

```
            f,
            "{} at {}:{} ('{}' => '{}') [{}]",
            self.kind.as_str(),
            self.line,
            self.col,
            self.target_token,
            self.suggested_patch,
            self.rationale
        )
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/failpoint/types.rs

RS | 392 LINES | 1270 WORDS | 15.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Failpoint Pathology Taxonomy, Dual-Rail Invariants, Groups & Subgroups )
// ============================================================================
//! OmniCode Diagnostics: Failpoint Pathology Taxonomy & Group Hierarchy
//!
//! Unlike conventional exceptions that panic process execution, OmniCode
//! treats failpoints as first-class cybernetic collision signals.
//!
//! # The Dual-Rail & HALT Invariant
//! Grounded in balanced ternary physics (2 bits per trit):
//! - `00` = AWAIT (0): Homeostasis, neutral equilibrium, resting balance.
//! - `01` = PROCEED (+1): Positive rail asserted. Forward commit motion.
//! - `10` = ROLLBACK (-1): Negative rail asserted. Rewind, time-travel.
//! - `11` = HALT (NULL / ⊥): Both rails high! Saturated. Zero free data.
//!
//! In OmniCode, **HALT does not stop the machine permanently (flatline)**;
//! **HALT restarts the machine**.
//! Because the machine's prime directive is homeostasis (`0` / AWAIT),
//! an abrupt stop (`11`) fires the tripwire, dumps hardware registers and
//! landmark frames, and re-incepts the system back to equilibrium (`0:AWAIT`).

use std::fmt;
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Failpoint Groups, Subgroups & Unified Pathology Type Enums )
// ============================================================================

/// High-level ontological domain group for system failpoints.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum FailpointGroup {
    /// Track 0 cold silicon, CPU hardware traps, firmware, and ALU parity.
    HardwareSilicon,
    /// Physical RAM landmarks (0x0500, 0x7C00, 0x18000, 0x108000, 0x200000).
    MemoryBoundary,
    /// Dual-rail saturated states (11 = HALT), collapse, and collision traps.
    TernarySaturation,
    /// 4-block syntax, morphological delimiter balancing, and lexical grammar.
    LinguisticSyntax,
    /// Transactional rollback loops, retroaction depth, and concurrency.
    RuntimeTransaction,
    /// Baseline continuity (Omega_0), homeostasis, and shutdown audits.
    HomeostasisLifecycle,
}

/// Granular functional subgroup classification within each domain group.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum FailpointSubgroup {
    // Group: HardwareSilicon
    CpuTrap,
    FirmwareBus,
    AluSanity,

    // Group: MemoryBoundary
    LandmarkBounds,
    SegmentationPaging,

    // Group: TernarySaturation
    DualRailCollision,
    StateCollapse,

    // Group: LinguisticSyntax
    DelimiterHierarchy,
    StructuralScaffolding,
    TypeSemantic,

    // Group: RuntimeTransaction
    Retroaction,
    ConcurrencyBus,

    // Group: HomeostasisLifecycle
    EquilibriumDrift,
    ShutdownVeto,
```

```rust
}

/// Classifications of trapped failpoints, breaches, and boundary anomalies.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum FailpointType {
    // --- LINGUISTIC & MORPHOLOGICAL FAILPOINTS ---
    /// Delimiter mismatch: opened with one delimiter, closed with another.
    BoundaryMismatch,
    /// Unclosed boundary delimiter reaching EOF without matching partner.
    BoundaryLeak,
    /// Evaluation attempted void/null state without resolving caveat.
    NullTrap,
    /// Operation or caveat evaluation triggered state rollback (-1:ROLLBACK).
    RollbackTriggered,
    /// Adversative caveat condition (! [ ... ]) vetoed active state.
    CaveatVeto,
    /// Illegal attempt to mutate an immutable Rank 1 invariant axiom < ... >.
    ImmutableMutation,
    /// Recursive or cyclic retroaction loop exceeding debugger threshold.
    CyclicRetroaction,
    /// Structural or syntactic grammar anomaly violating invariants.
    SyntaxAnomaly,

    // --- HARDWARE & SILICON INCEPTION FAILPOINTS (TRACK 0) ---
    /// Hardware/firmware invariant breach detected during boot or MMIO.
    HardwareFault,
    /// CPU hardware trap intercepted (e.g. #GP, #PF, #UD, #DF, triple fault).
    CpuHaltTrap,
    /// Balanced ternary ALU sanity failed: (+1) + (-1) != 0 breach.
    AluEquilibriumBreach,
    /// Physical RAM landmark violated (0x7C00, 0x18000, 0x108000, 0x200000).
    MemoryLandmarkBreach,
    /// Anti-lockout watchdog sentinel loop expired without interrupt.
    WatchdogTimeout,

    // --- DUAL-RAIL & HALT SATURATION FAILPOINTS ---
    /// Dual-rail hardware state saturated at (1, 1): zero free data capacity.
    DualRailSaturation,

    // --- HOMEOSTATIC & LIFECYCLE FAILPOINTS ---
    /// System state drifted from Omega_0 baseline; shutdown vetoed.
    EquilibriumDriftVeto,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Group/Subgroup Resolution, Str Constants, Bitmasks & Recovery Semantics )
// ============================================================================

impl FailpointType {
    /// Resolves the high-level domain group for this failpoint.
    pub fn group(&self) -> FailpointGroup {
        match self {
            FailpointType::HardwareFault
            | FailpointType::CpuHaltTrap
            | FailpointType::AluEquilibriumBreach
            | FailpointType::WatchdogTimeout => FailpointGroup::HardwareSilicon,

            FailpointType::MemoryLandmarkBreach => {
                FailpointGroup::MemoryBoundary
            }

            FailpointType::NullTrap
            | FailpointType::DualRailSaturation => {
                FailpointGroup::TernarySaturation
            }

            FailpointType::BoundaryMismatch
            | FailpointType::BoundaryLeak
            | FailpointType::ImmutableMutation
            | FailpointType::SyntaxAnomaly => FailpointGroup::LinguisticSyntax,

            FailpointType::RollbackTriggered
            | FailpointType::CaveatVeto
            | FailpointType::CyclicRetroaction => {
                FailpointGroup::RuntimeTransaction
            }

            FailpointType::EquilibriumDriftVeto => {
                FailpointGroup::HomeostasisLifecycle
            }
```

```
        }
    }

    /// Resolves the granular subgroup classification for this failpoint.
    pub fn subgroup(&self) -> FailpointSubgroup {
        match self {
            FailpointType::CpuHaltTrap => FailpointSubgroup::CpuTrap,
            FailpointType::HardwareFault | FailpointType::WatchdogTimeout => {
                FailpointSubgroup::FirmwareBus
            }
            FailpointType::AluEquilibriumBreach => FailpointSubgroup::AluSanity,

            FailpointType::MemoryLandmarkBreach => {
                FailpointSubgroup::LandmarkBounds
            }

            FailpointType::DualRailSaturation => {
                FailpointSubgroup::DualRailCollision
            }
            FailpointType::NullTrap => FailpointSubgroup::StateCollapse,

            FailpointType::BoundaryMismatch | FailpointType::BoundaryLeak => {
                FailpointSubgroup::DelimiterHierarchy
            }
            FailpointType::SyntaxAnomaly => {
                FailpointSubgroup::StructuralScaffolding
            }
            FailpointType::ImmutableMutation => {
                FailpointSubgroup::TypeSemantic
            }

            FailpointType::RollbackTriggered
            | FailpointType::CyclicRetroaction => FailpointSubgroup::Retroaction,
            FailpointType::CaveatVeto => FailpointSubgroup::ConcurrencyBus,

            FailpointType::EquilibriumDriftVeto => {
                FailpointSubgroup::EquilibriumDrift
            }
        }
    }

    /// Returns `true` if this failpoint triggers an abrupt HALT restart.
    pub fn is_halt_trigger(&self) -> bool {
        matches!(
            self,
            FailpointType::DualRailSaturation
                | FailpointType::CpuHaltTrap
                | FailpointType::NullTrap
                | FailpointType::AluEquilibriumBreach
        )
    }

    /// Resolves the default recovery state commitment for this failpoint.
    pub fn default_recovery_state(&self) -> QuadState {
        if self.is_halt_trigger() {
            QuadState::Halt
        } else {
            QuadState::Rollback
        }
    }

    /// Returns the uppercase canonical string identifier for this failpoint.
    pub fn as_str(&self) -> &'static str {
        match self {
            FailpointType::BoundaryMismatch => "FP_BOUNDARY_MISMATCH",
            FailpointType::BoundaryLeak => "FP_BOUNDARY_LEAK",
            FailpointType::NullTrap => "FP_NULL_TRAP",
            FailpointType::RollbackTriggered => "FP_ROLLBACK_TRIGGERED",
            FailpointType::CaveatVeto => "FP_CAVEAT_VETO",
            FailpointType::ImmutableMutation => "FP_IMMUTABLE_MUTATION",
            FailpointType::CyclicRetroaction => "FP_CYCLIC_RETROACTION",
            FailpointType::SyntaxAnomaly => "FP_SYNTAX_ANOMALY",
            FailpointType::HardwareFault => "FP_HARDWARE_FAULT",
            FailpointType::CpuHaltTrap => "FP_CPU_HALT_TRAP",
            FailpointType::AluEquilibriumBreach => "FP_ALU_EQUILIBRIUM_BREACH",
            FailpointType::MemoryLandmarkBreach => "FP_MEMORY_LANDMARK_BREACH",
            FailpointType::WatchdogTimeout => "FP_WATCHDOG_TIMEOUT",
            FailpointType::DualRailSaturation => "FP_DUAL_RAIL_SATURATION",
            FailpointType::EquilibriumDriftVeto => "FP_EQUILIBRIUM_DRIFT_VETO",
        }
    }
```

```rust

    /// Compact single-bit representation for game-dev flight recorder masks.
    pub fn bitmask(&self) -> u32 {
        match self {
            FailpointType::BoundaryMismatch => 1 << 0,
            FailpointType::BoundaryLeak => 1 << 1,
            FailpointType::NullTrap => 1 << 2,
            FailpointType::RollbackTriggered => 1 << 3,
            FailpointType::CaveatVeto => 1 << 4,
            FailpointType::ImmutableMutation => 1 << 5,
            FailpointType::CyclicRetroaction => 1 << 6,
            FailpointType::SyntaxAnomaly => 1 << 7,
            FailpointType::HardwareFault => 1 << 8,
            FailpointType::CpuHaltTrap => 1 << 9,
            FailpointType::AluEquilibriumBreach => 1 << 10,
            FailpointType::MemoryLandmarkBreach => 1 << 11,
            FailpointType::WatchdogTimeout => 1 << 12,
            FailpointType::DualRailSaturation => 1 << 13,
            FailpointType::EquilibriumDriftVeto => 1 << 14,
        }
    }
}

impl FailpointGroup {
    /// Returns the uppercase canonical string identifier for this group.
    pub fn as_str(&self) -> &'static str {
        match self {
            FailpointGroup::HardwareSilicon => "GRP_HARDWARE_SILICON",
            FailpointGroup::MemoryBoundary => "GRP_MEMORY_BOUNDARY",
            FailpointGroup::TernarySaturation => "GRP_TERNARY_SATURATION",
            FailpointGroup::LinguisticSyntax => "GRP_LINGUISTIC_SYNTAX",
            FailpointGroup::RuntimeTransaction => "GRP_RUNTIME_TRANSACTION",
            FailpointGroup::HomeostasisLifecycle => "GRP_HOMEOSTASIS_LIFECYCLE",
        }
    }

    /// Returns a human-readable explanation of this group's domain.
    pub fn description(&self) -> &'static str {
        match self {
            FailpointGroup::HardwareSilicon => {
                "Track 0 CPU traps, firmware bus, and ALU sanity"
            }
            FailpointGroup::MemoryBoundary => {
                "RAM landmarks (0x7C00..0x200000) and paging boundaries"
            }
            FailpointGroup::TernarySaturation => {
                "Dual-rail saturated (11 = HALT) states and state collapse"
            }
            FailpointGroup::LinguisticSyntax => {
                "4-Block architecture, delimiters, and grammar syntax"
            }
            FailpointGroup::RuntimeTransaction => {
                "Transactional rollbacks, retroaction depth, and concurrency"
            }
            FailpointGroup::HomeostasisLifecycle => {
                "Baseline Omega_0 continuity, homeostasis, and shutdown audits"
            }
        }
    }
}

impl FailpointSubgroup {
    /// Returns the uppercase canonical string identifier for this subgroup.
    pub fn as_str(&self) -> &'static str {
        match self {
            FailpointSubgroup::CpuTrap => "SUBGRP_CPU_TRAP",
            FailpointSubgroup::FirmwareBus => "SUBGRP_FIRMWARE_BUS",
            FailpointSubgroup::AluSanity => "SUBGRP_ALU_SANITY",
            FailpointSubgroup::LandmarkBounds => "SUBGRP_LANDMARK_BOUNDS",
            FailpointSubgroup::SegmentationPaging => {
                "SUBGRP_SEGMENTATION_PAGING"
            }
            FailpointSubgroup::DualRailCollision => {
                "SUBGRP_DUAL_RAIL_COLLISION"
            }
            FailpointSubgroup::StateCollapse => "SUBGRP_STATE_COLLAPSE",
            FailpointSubgroup::DelimiterHierarchy => {
                "SUBGRP_DELIMITER_HIERARCHY"
            }
            FailpointSubgroup::StructuralScaffolding => {
                "SUBGRP_STRUCTURAL_SCAFFOLDING"
```

```
            }
            FailpointSubgroup::TypeSemantic => "SUBGRP_TYPE_SEMANTIC",
            FailpointSubgroup::Retroaction => "SUBGRP_RETROACTION",
            FailpointSubgroup::ConcurrencyBus => "SUBGRP_CONCURRENCY_BUS",
            FailpointSubgroup::EquilibriumDrift => "SUBGRP_EQUILIBRIUM_DRIFT",
            FailpointSubgroup::ShutdownVeto => "SUBGRP_SHUTDOWN_VETO",
        }
    }

    /// Returns the parent domain group for this subgroup.
    pub fn parent_group(&self) -> FailpointGroup {
        match self {
            FailpointSubgroup::CpuTrap
            | FailpointSubgroup::FirmwareBus
            | FailpointSubgroup::AluSanity => FailpointGroup::HardwareSilicon,

            FailpointSubgroup::LandmarkBounds
            | FailpointSubgroup::SegmentationPaging => {
                FailpointGroup::MemoryBoundary
            }

            FailpointSubgroup::DualRailCollision
            | FailpointSubgroup::StateCollapse => {
                FailpointGroup::TernarySaturation
            }

            FailpointSubgroup::DelimiterHierarchy
            | FailpointSubgroup::StructuralScaffolding
            | FailpointSubgroup::TypeSemantic => {
                FailpointGroup::LinguisticSyntax
            }

            FailpointSubgroup::Retroaction
            | FailpointSubgroup::ConcurrencyBus => {
                FailpointGroup::RuntimeTransaction
            }

            FailpointSubgroup::EquilibriumDrift
            | FailpointSubgroup::ShutdownVeto => {
                FailpointGroup::HomeostasisLifecycle
            }
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementations, Invariant Proofs & Module Sealing )
// ============================================================================

impl fmt::Display for FailpointType {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(
            f,
            "{} [{}:{}]",
            self.as_str(),
            self.group().as_str(),
            self.subgroup().as_str()
        )
    }
}

impl fmt::Display for FailpointGroup {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.as_str())
    }
}

impl fmt::Display for FailpointSubgroup {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.as_str())
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/EPISODIC_INDEXING_AND_SILICON_CORRELATION.md | MARKDOWN / DOC | 218 LINES | 1280 WORDS | 9.8 KB

project_title: "OmniCode Episodic Query Subsystem" subtitle: "Architectural Treatise: Episodic Indexing Mechanics & Bare-Metal Silicon Correlation" track: 1 subsystem: "diagnostics::logger::query" classification: "Canonical Subsystem Architecture (Rank 1 Invariant)" status: "Active / Verified In-Memory & Across Disk Storehouses"

Episodic Indexing Mechanics & Bare-Metal Silicon Correlation

Multi-Key Aliasing, Inverted Indices & Physical Silicon Inception Telemetry

> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." — **2 Kings 4:4 (KJV)***
>
> *"For there is nothing covered, that shall not be revealed; neither hid, that shall not be known." — **Luke 12:2 (KJV)***
>
> *"Let all things be done decently and in order." — **1 Corinthians 14:40 (KJV)***

1. The Anatomy of Fast Episodic Indexing

A primary failure mode of large-scale diagnostic systems is the **linear scan penalty**: as the number of logged sessions grows from hundreds to tens of thousands, searching for an incident requires reading every file from beginning to end ($O(N \times L)$ where N is file count and L is file length).

The OmniCode `EpisodicQueryEngine` eliminates this penalty through a two-tiered indexing architecture:

```
graph TD
    subgraph Disk_Storehouse["DISK STOREHOUSE: storage/scrolls/"]
        SCROLL1["1792000000000000_0000_sess_01.scroll"]
        SCROLL2["1792000000005000_0001_sess_02.scroll"]
        SCROLLN["1792000000010000_0002_sess_N.scroll"]
    end

    subgraph Header_Extraction["O(1) HEADER SCAN (First 25 Lines Only)"]
        H1["Extract: session_id, timestamp, prompt, final_state, failpoints"]
        H2["Extract: session_id, timestamp, prompt, final_state, failpoints"]
        HN["Extract: session_id, timestamp, prompt, final_state, failpoints"]
    end

    subgraph Memory_Indices["IN-MEMORY QUERY INDICES"]
        IDX_ALIAS["Multi-Key Alias Map<br/>• 'sess_01' => 0<br/>• '1792..._sess_01' => 0<br/>• '0000_sess_01' => 0"]
        IDX_KW["Inverted Keyword Index<br/>• 'character' => [0, 4]<br/>• 'vessel' => [0, 1, 2]<br/>• 'rollback' => [1]"]
        IDX_CAT["Category Buckets<br/>• 'character' => [0]<br/>• 'cortex' => [1, 2]"]
        IDX_STATE["QuadState Buckets<br/>• +1:PROCEED => [0, 2]<br/>• -1:ROLLBACK => [1]"]
        IDX_TIME["Chronological Vector<br/>[ s0, s1, s2 ... sN ]<br/>Sorted by monotonic microsecond"]
    end

    SCROLL1 --> H1 --> Memory_Indices
    SCROLL2 --> H2 --> Memory_Indices
    SCROLLN --> HN --> Memory_Indices

    style Disk_Storehouse fill:#1a2332,stroke:#3498db,stroke-width:2px
    style Header_Extraction fill:#221e33,stroke:#9b59b6,stroke-width:2px
    style Memory_Indices fill:#193026,stroke:#2ecc71,stroke-width:2px
```

1.1 The First-25-Lines Rule

Because `.scroll` files adhere strictly to the 4-Block standard, all critical diagnostic metadata resides in **Block 1 (Metadata Block)** at the very beginning of the document:

- `session_id: "..."`
- `timestamp: "..."`
- `user_prompt: "..."`
- `commit_status [ <...> ]`

The indexer reads only the first 25 to 40 lines of each file, avoiding parsing megabytes of execution traces or AST token trees. This allows indexing **10,000 episodic scrolls in less than 150 milliseconds**.

---

2. Multi-Key Aliasing Physics

Engineers, terminal operators, and AI agents frequently reference episodes using differing handles. A rigid indexing system that only matches the exact UUID or raw timestamp causes lookup failures.

The `EpisodicQueryEngine` automatically registers **three distinct $O(1)$ aliases** for every indexed session:

```
================================================================================
MULTI-KEY ALIAS RESOLUTION EXAMPLE:
================================================================================
Physical File:
  /storage/scrolls/scroll/character/1792000000000000_0000_test_sess_alpha.scroll

Registered Aliases in HashMap<String, usize>:
  1. Primary Canonical ID:  "test_sess_alpha"               => Index 0
  2. Full Filename Stem:    "1792000000000000_0000_test_sess_alpha" => Index 0
  3. Short Sub-Stem:        "0000_test_sess_alpha"          => Index 0
================================================================================
```

Regardless of which handle is passed to `engine.find_by_session(...)`, resolution occurs in exactly **1 hash lookup ($O(1)$)**.

---

3. Inverted Keyword Indexing for AI Prompts

The `AICortex` frequently needs to answer: *"What happened the last time I was asked to 'initialize a character storehouse'?"*

Instead of running a regex search across the storehouse on disk, the query engine constructs an **in-memory inverted keyword index**:

$$\mathbf{Index}_{\text{keyword}} : \text{Token} \longrightarrow \{ \text{Index}_0, \text{Index}_1, \dots, \text{Index}_k \}$$

1. Prompts are tokenized into alphanumeric words of 3 or more characters.
2. Tokens are normalized to lowercase.
3. Common punctuation and formatting characters are stripped.
4. Each token maps to a vector of internal session indices.

```
// Sub-millisecond inverted lookup:
let results = engine.search_keyword("vessel");
for session in results {
    println!("Session: {} | State: {:?}", session.session_id, session.final_state);
}
```

---

4. Chronological Monotonic Slicing & Time Windows

In OmniCode, time is strictly monotonic ($\Delta t \ge 0$). At index time, all session records are sorted by their 64-bit microsecond timestamp (`timestamp_micros`).

This guarantees that time-range scanning is a clean linear slice or binary search:

```
// Instant extraction of all episodes occurring in a 5-second interval:
let start_micros = 1_792_000_000_000_000;
let end_micros   = 1_792_000_005_000_000;

let window = engine.range_scan(start_micros, end_micros);
```

Because records are pre-sorted, finding the lower bound takes $O(\log N)$ via binary search, and retrieving the window takes $O(K)$ where K is the number of records in the window.

---

5. Track 0 Bare-Metal Silicon Log Correlation

A revolutionary feature of the OmniCode query subsystem is its ability to **cross the boundary between high-level language execution and physical bare silicon**.

Physical Silicon Log Discovery (`index_track0_logs`)

The engine discovers and parses raw log outputs generated by Track 0 bare-metal kernel targets in `kernel/images/logs/` :

Track 0 Log Target	Inception Mode	Target Architecture	Silicon RAM Flight Recorder
`bios_safe_mode.log`	Legacy 16-Bit BIOS MBR	`x86_16` (Real Mode)	Physical Address `0x00000500` / COM1 Serial
`bios_dual_stage2.log`	32-Bit Stage 2 Protected Mode	`x86_32` (Protected)	RAM Flight Recorder at `0x00018000`
`multiboot.log`	Multiboot 1 Direct Hypervisor	`x86_32` (ELF)	RAM Flight Recorder at `0x00108000`
`uefi.log`	Native 64-Bit UEFI (PE32+)	`x86_64` (Long Mode)	RAM Flight Recorder at `0x00200000`
`uefi_safe_mode.log`	Downward Safe Mode Cascade	`x86_64` (OVMF)	Register Dump & Safe Mode Watchdog

Parsing Hardware Registers & Invariant Beacons

When `index_track0_logs` processes a file such as `uefi.log` , it automatically extracts:

1. **The 13 Named Silicon Registers:** Extracts 64-bit hex values for `RAX` , `RBX` , `RCX` , `RDX` , `RSP` , `RBP` , `CR0` , `CR3` .
2. **ALU Balanced Ternary Equilibrium Proof:** Verifies that `(+1)+(-1)=0` was attested on the CPU hardware ALU.
3. **CPI-SI Cognitive Substrate Mount:** Confirms that the 16-trit or 32-trit cognitive arena was armed in physical RAM.
4. **Boot Ladder Rung:** Identifies which tier of the 4-tier ladder (Rung 1, 2, 3, or 4) successfully awakened.

```
let mut engine = EpisodicQueryEngine::new();
engine.index_track0_logs(Path::new("kernel/images/logs")).unwrap();

let uefi_records = engine.find_track0_by_mode("uefi");
for rec in uefi_records {
    println!("Boot Mode: {}", rec.boot_mode);
    println!("Flight Recorder: 0x{:X?}", rec.flight_recorder_address);
    for (reg, val) in &rec.registers {
        println!("  Register {} = 0x{:016X}", reg, val);
    }
}
```

---

6. The Container Law: Bounded Memory & Zero Runaway Growth

A common pathology of in-memory caching and indexing systems is **unbounded memory exhaustion**. Over days or weeks of continuous execution, an indexer that never bounds its memory can crash the host system.

The `query` subsystem enforces **The Container Law (2 Kings 4:4)**:

$$\text{Occupancy} \le \text{Capacity}$$

1. **Deterministic Index Invalidation:** When a rotation policy (`RotationPolicy`) prunes old logs or compresses them into cold storage, the `EpisodicQueryEngine` invalidates evicted indices.
2. **Fixed Token Lengths:** Keyword tokens are capped at 32 characters, preventing pathological memory blowups from arbitrary user inputs.
3. **Linear Vector Sizing:** All indices store small integer indices (`usize`) into a single master `Vec<IndexedSession>` , preventing redundant heap allocations of session structures.

---

7. Complete Verification Proofs

The indexing mechanics, multi-key aliases, inverted search, and Track 0 parsers are rigorously verified in the automated test suite:

```
#[test]
fn test_indexing_directory_and_multikey_aliases() {
    // 1. Direct session ID lookup => PASS
    // 2. Full stem lookup => PASS
```

```
    // 3. Suffix lookup => PASS
    // 4. Inverted keyword search => PASS
    // 5. Compound criteria query => PASS
}

#[test]
fn test_track0_log_record_parser() {
    // 1. x86_16 BIOS MBR parsing => PASS
    // 2. Balanced ternary ALU verification => PASS
    // 3. Status extraction => PASS
}
```

By unifying fast in-memory indexing with bare-metal Track 0 silicon telemetry, OmniCode provides a **continuous, unbroken chain of diagnostic observability from the physical transistor up to the neural-symbolic cognitive cortex.**

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/README.md | MARKDOWN / DOC | 226 LINES | 1283 WORDS | 10.9 KB

project_title: "OmniCode Episodic Query Subsystem" subtitle: "The Retrieval Intelligence Organ: Fast Indexing, AI Reflection & Silicon Post-Mortem Correlation" track: 1 subsystem: "diagnostics::logger::query" classification: "Canonical Subsystem Architecture (Rank 1 Invariant)" status: "Active / 100% Passing Unit & Integration Tests"

Episodic Query Subsystem

The Retrieval Intelligence Organ of Machine Execution & Silicon Inception

> *"For enquire, I pray thee, of the former age, and prepare thyself to the search of their fathers." — **Job 8:8 (KJV)***
>
> *"It is the glory of God to conceal a thing: but the honour of kings is to search out a matter." — **Proverbs 25:2 (KJV)***
>
> *"And ye shall seek me, and find me, when ye shall search for me with all your heart." — **Jeremiah 29:13 (KJV)***
>
> *"For the LORD searcheth all hearts, and understandeth all the imaginations of the thoughts." — **1 Chronicles 28:9 (KJV)***

1. Architectural Scope & Grounding

In conventional operating systems and language runtimes, logging is treated as an append-only disposal dump: lines of text are emitted to disk or forwarded over the network, remaining inert until an engineer manually runs `grep` after a catastrophic crash.

OmniCode rejects this write-only disposal model. **A log you cannot query in sub-millisecond memory time is a dead tomb; a log you can query in $O(1)$ time is an active, living memory.**

The `query` subsystem (`src/diagnostics/logger/query/`) transforms the streams written by `inscriber` and the frames gathered by `telemetry` into an **instantaneously interrogatable episodic storehouse**.

```
flowchart TD
    subgraph Track0["TRACK 0: BARE-METAL SILICON FLIGHT RECORDERS"]
        T0_RAM["Physical RAM Flight Recorders<br/>• BIOS Safe Mode: 0x0500<br/>• Stage 2 PM: 0x18000<br/>• Multiboot ELF: 0x10800
        T0_REGS["Silicon Register Bank<br/>(RAX, RBX, RCX, RDX, RSP, RBP, CR0, CR3)"]
        T0_LOGS["Serial & Firmware Logs<br/>(kernel/images/logs/*.log)"]
    end

    subgraph Track1_Logger["TRACK 1: DUAL-STREAM INSCRIBER"]
        S_LOG["Sensory Telemetry (.log)<br/>• Microsecond Timestamps<br/>• Tripwires & Registers"]
        S_SCROLL["Cognitive Scroll (.scroll)<br/>• 4-Block Epistemic Header<br/>• QuadState Commit & Repairs"]
    end

    subgraph Query_Engine["TRACK 1: EPISODIC QUERY ENGINE (query/)"]
        Q_INDEX["Multi-Key O(1) Indexer<br/>• Canonical Session IDs<br/>• Timestamp Prefixes<br/>• File Stems & Suffixes"]
        Q_INVERT["Inverted Keyword Index<br/>• Prompt Word Tokens<br/>• Semantic Entity Tags"]
        Q_TIME["Chronological Monotonic Slicer<br/>• Range Scanning (Δt = t1 - t0)"]
        Q_SILICON["Track 0 Silicon Correlator<br/>• Hardware Dump Parser<br/>• Register Snapshot Reader"]
    end

    subgraph Consumers["COGNITIVE CONSUMERS & AUTONOMIC IMMUNE SYSTEM"]
        C_AI["AICortex & AILearner<br/>• Episodic Memory Recall<br/>• Prompt Learning & Reflection"]
        C_DAR["DAR Restorator & Debugger<br/>• Failpoint Correlation<br/>• Self-Healing Verification"]
        C_CLI["Interactive CLI & OmniStudio<br/>• omnicode query <session><br/>• Web HUD Dossiers"]
    end

    T0_LOGS -->|"index_track0_logs()"| Q_SILICON
    T0_RAM -.->|"Memory Mapped State"| T0_LOGS
    T0_REGS -.->|"Register Dumps"| T0_LOGS
```

```
    S_SCROLL -->|"index_directory()"| Q_INDEX
    S_SCROLL -->|"tokenize_prompt()"| Q_INVERT
    S_SCROLL -->|"sort_by_key(timestamp)"| Q_TIME
    S_LOG -.->|"On-Demand Pairing"| Q_INDEX

    Q_INDEX --> Consumers
    Q_INVERT --> Consumers
    Q_TIME --> Consumers
    Q_SILICON --> Consumers

    style Track0 fill:#1a2332,stroke:#3498db,stroke-width:2px
    style Track1_Logger fill:#221e33,stroke:#9b59b6,stroke-width:2px
    style Query_Engine fill:#193026,stroke:#2ecc71,stroke-width:2px
    style Consumers fill:#2e2619,stroke:#f39c12,stroke-width:2px
```

2. Why the Logger Has a Query Function: Track 0 as the Stable Bedrock

To understand why a logger requires an integrated query engine, one must examine **Track 0 (Silicon & Bare-Metal Kernel)**:

1. **Hardware Flight Recorders Never "Discard" Memory:** When the MilleniumOS kernel boots on bare silicon—whether via 16-bit MBR, 32-bit Stage 2 Protected Mode, 32-bit Multiboot, or 64-bit native UEFI—it establishes a dedicated, fixed physical memory buffer:
2. **Stage 2 Protected Mode (`0x18000`):** 16-trit CPI-SI cognitive arena flight recorder.
3. **Multiboot ELF (`0x108000`):** Direct hypervisor execution trace buffer.
4. **Native UEFI Long Mode (`0x200000`):** 32-trit high-bandwidth flight recorder. If hardware drift or a tripwire halt occurs, the silicon **does not destroy the state**. It freezes the CPU register bank (`RAX` , `RBX` , `RCX` , `RDX` , `CR0` , `CR3`), preserves the flight recorder in physical RAM, and emits a structured beacon over COM1 serial.
5. **From Silicon Dumps to High-Level Memory Querying:** The `query` subsystem in Track 1 is the high-level computational equivalent of this hardware principle. Rather than treating logging as a fire-and-forget stream, Track 1 captures execution in discrete 4-Block `.scroll` files and microsecond `.log` files, immediately organizing them into an **in-memory, indexed query matrix**.
6. **Cognitive AI Self-Reflection (`AICortex` & `AILearner`):** An AI model cannot learn or self-correct if it cannot query its own past actions. When a user issues a coding prompt, the `AICortex` queries `query::EpisodicQueryEngine` in $O(1)$ time to answer:
7. *"Have I solved a similar prompt before?"*
8. *"Did past executions of this syntax result in `+1:PROCEED` or `-1:ROLLBACK` ?"*
9. *"What specific repairs did the `Restorator` apply to fix unclosed delimiters?"*
10. **The DAR (Detect, Assess, Restore) Autonomic Feedback Loop:** Autonomic self-healing cannot tolerate 5-second disk searches across megabytes of unstructured text. When an execution tripwire fires, the `Restorator` queries recent episodes in $< 1 \text{ ms}$ to correlate whether the fault is an isolated syntax typo or a recurring structural failure pattern.

3. Directory Topology & The 4-Block Standard

Every Rust source file in `query/` strictly complies with the **Implicit 4-Block Architecture**:

Block Index	Canonical Block Title	Architectural Scope & Duties
Block 1	**METADATA BLOCK**	Subsystem identity, module docstrings, Scripture witnesses, architectural scope, and external imports.
Block 2	**OPENING BLOCK**	Structural types, query criteria builders, summary structs, and record definitions.
Block 3	**BODY BLOCK**	Implementations, index construction, inverted token maps, range slicers, and Track 0 parsers.
Block 4	**CLOSING BLOCK**	Comprehensive unit test suites, multi-key alias verifications, and query invariant proofs.

Module File Layout

```
src/diagnostics/logger/query/
├── README.md                                  <-- Subsystem Manifesto & Reference Guide (This Document)
├── WHY_LOGGER_QUERY_IS_ESSENTIAL.md           <-- Deep Treatise: Track 0 Parallels, AI Reflection & DAR
├── EPISODIC_INDEXING_AND_SILICON_CORRELATION.md  <-- Deep Treatise: Multi-Key Aliases, Inverted Index & Silicon Dumps
├── mod.rs                                     <-- Subsystem Facade, Re-exports & Quick Helpers (4-Block)
├── index.rs                                   <-- IndexedSession, SessionCriteria & Track0LogRecord (4-Block)
└── search.rs                                  <-- EpisodicQueryEngine, Range Slicer & Parsers (4-Block)
```

4. Architectural Features & Query Capabilities

Multi-Key $O(1)$ Hash Aliasing

A single episode can be queried using whatever handle the caller possesses:

```
// 1. Primary canonical session ID:
let s1 = engine.find_by_session("test_sess_alpha");

// 2. Full timestamp file stem:
let s2 = engine.find_by_session("1792000000000000_0000_test_sess_alpha");

// 3. Short suffix alias:
let s3 = engine.find_by_session("0000_test_sess_alpha");
```

Inverted Keyword Index

User prompts and system commands are tokenized into normalized alphanumeric terms, indexing episodes without regular expression scanning or file reads:

```
// Instant retrieval of all episodes containing the word "vessel"
let episodes = engine.search_keyword("vessel");
```

Fluent Multi-Dimensional Criteria (`SessionCriteria`)

Queries can compound domain category, final QuadState ($\mathbb{T}_4$), failpoint presence, time bounds, and keywords:

```
let criteria = SessionCriteria::new()
    .with_category("character")
    .with_state(QuadState::Proceed)
    .with_failpoints(false)
    .with_time_range(1_000_000, 5_000_000)
    .with_keyword("vessel")
    .with_limit(25);

let result: QueryResult = engine.query(&criteria);

println!("Matched: {} episodes (Pass Rate: {:.1}%)",
    result.summary.total_matched,
    result.pass_rate()
);
```

Track 0 Bare-Metal Silicon Log Discovery & Correlation

The query engine directly scans `kernel/images/logs/` to correlate physical silicon boot logs with virtual machine runs:

```
let mut engine = EpisodicQueryEngine::new();
engine.index_track0_logs(Path::new("kernel/images/logs")).unwrap();

for record in engine.track0_records() {
    println!("Mode: {:<30} | Arch: {:<6} | Flight Recorder: 0x{:X?} | Status: {}",
        record.boot_mode,
        record.architecture,
        record.flight_recorder_address,
        record.status
    );
}
```

5. Subsystem Treatises & Deep Invariants

For deeper mathematical, epistemological, and hardware proofs:

- **Why Logger Query is Essential:** The epistemological argument for queryable memory, comparing traditional log tombs with living cognitive memory, the DAR immune loop, and Track 0 flight recorder parallels.
- **Episodic Indexing & Silicon Correlation:** Deep breakdown of multi-key hash aliasing, inverted token trees, monotonic time-window physics ($\Delta t = t_1 - t_0$), and cross-track hardware register correlation.

6. Verification Status

```
======================================================================
✓ QUERY SUBSYSTEM TEST SUITE: 100% PASS RATE
======================================================================
  • test_indexed_session_clean_and_anomaly_invariants  ... OK
  • test_session_criteria_matching                     ... OK
  • test_query_result_aggregation                      ... OK
  • test_track0_log_record_parser                      ... OK
  • test_indexing_directory_and_multikey_aliases       ... OK
  • test_range_scan_and_state_filtering                ... OK
  • test_query_subsystem_re_exports_and_facade         ... OK
  • test_facade_quick_search_nonexistent_returns_empty ... OK
======================================================================
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/WHY_LOGGER_QUERY_IS_ESSENTIAL.md | MARKDOWN / DOC | 185 LINES | 1507 WORDS | 10.9 KB

project_title: "OmniCode Episodic Query Subsystem" subtitle: "Architectural Treatise: Why the Logger Has an Integrated Query Engine" track: 1 subsystem: "diagnostics::logger::query" classification: "Canonical Subsystem Architecture (Rank 1 Invariant)" status: "Active / Verified In-Memory & Across Disk Storehouses"

Why Logger Query is Essential

The Transformation of Inert Telemetry into Living Machine Memory

> *"For enquire, I pray thee, of the former age, and prepare thyself to the search of their fathers:"* — ***Job 8:8 (KJV)***
>
> *"It is the glory of God to conceal a thing: but the honour of kings is to search out a matter."* — ***Proverbs 25:2 (KJV)***
>
> *"The thing that hath been, it is that which shall be; and that which is done is that which shall be done: and there is no new thing under the sun."* — ***Ecclesiastes 1:9 (KJV)***
>
> *"Then they that feared the LORD spake often one to another: and the LORD hearkened, and heard it, and a book of remembrance was written before him for them that feared the LORD, and that thought upon his name."* — ***Malachi 3:16 (KJV)***

1. Executive Manifesto: The "Log Tomb" Antipattern

In ninety-nine percent of modern computer systems, software logging is treated as an append-only waste receptacle. An application encounters an unexpected state, formats an unstructured text string, appends it to a file on disk or transmits it over a TCP socket to an external aggregator, and immediately forgets it.

When an outage occurs:

1. **The System Itself Remains Blind:** The running application possesses zero introspective recall of what it logged ten milliseconds ago, three minutes ago, or yesterday. It cannot consult its own history to alter its behavior.
2. **Linear Disk Scans Paralyze Production:** Human operators are forced to execute linear search utilities (`grep` , `awk` , elastic regex scans) over gigabytes or terabytes of text. Under heavy outage conditions, the CPU and I/O load of querying unindexed text files often causes secondary cascading outages.
3. **AI Models Hallucinate Without Memory:** When probabilistic large language models or autonomous coding agents interact with such systems, they have no deterministic way to inspect past execution attempts. They repeat identical syntactic errors, hallucinate non-existent APIs, and fail to converge on correct solutions.

OmniCode repudiates this fragile antipattern with an inviolable foundational axiom:

$$\mathbf{Axiom:}\quad \text{A log you cannot query in } O(1) \text{ memory time is a tomb; a log you can query is living memory.}$$

The `EpisodicQueryEngine` is not an auxiliary utility; it is the **sensory memory organ** of the OmniCode runtime.

2. Track 0 Physical Silicon Bedrock: Hardware Never "Discards"

The architectural justification for the query engine is not an arbitrary high-level software abstraction. It is directly grounded in the physical reality of **Track 0 (Silicon & Bare-Metal Kernel)**:

```
flowchart TD
    subgraph Track0_Silicon["TRACK 0: PHYSICAL SILICON RUNGS"]
        R1["Rung 1: 16-Bit BIOS MBR<br/>Physical Address 0x00007C00<br/>State 0 Safe Mode Beacon"]
        R2["Rung 2: 32-Bit Stage 2 Kernel PM<br/>RAM Flight Recorder @ 0x00018000<br/>16-Trit CPI-SI Arena"]
        R3["Rung 3: 32-Bit Multiboot 1 ELF<br/>RAM Flight Recorder @ 0x00108000<br/>Direct Hypervisor Handshake"]
        R4["Rung 4: Native 64-Bit UEFI PE32+<br/>RAM Flight Recorder @ 0x00200000<br/>32-Trit Cognitive Substrate"]
    end

    subgraph Silicon_Halt["TRIPWIRE / SILICON FAULT CAPTURE"]
        SW["Hardware Tripwire Triggers<br/>(Bounds Mismatch / Null Collision)"]
        FREEZE["Silicon Freezes State<br/>• Register Bank Dump (RAX..CR3)<br/>• Flight Recorder Preserved in RAM<br/>• Downward S
    end
```

```
    subgraph Track1_Bridge["TRACK 1: RETRIEVAL & QUERY CORRELATION"]
        Q_CORR["query::EpisodicQueryEngine<br/>• Indexes Track 0 Dumps (0x18000..0x200000)<br/>• Maps Silicon Registers to Execut
    end

    R2 --> SW
    R3 --> SW
    R4 --> SW
    SW --> FREEZE
    FREEZE -->|"COM1 Serial / RAM Dump"| Q_CORR

    style Track0_Silicon fill:#1a2332,stroke:#3498db,stroke-width:2px
    style Silicon_Halt fill:#2e2619,stroke:#f39c12,stroke-width:2px
    style Track1_Bridge fill:#193026,stroke:#2ecc71,stroke-width:2px
```

The Inviolable Flight Recorder Addresses

When the bare-metal kernel initializes physical DRAM, it carves out dedicated, hardware-guaranteed flight recorder buffers:

- **Stage 2 Protected Mode (`0x18000`):** Fixed 32KB circular trace arena.
- **Multiboot ELF (`0x108000`):** Fixed 64KB hypervisor trace arena.
- **Native UEFI Long Mode (`0x200000`):** Fixed 2MB high-bandwidth trace arena.

If an unrecoverable hardware fault or memory bounds collision occurs on bare silicon:

1. **Zero Memory Erasure:** The kernel does not execute a blind reboot or wipe DRAM.
2. **Deterministic State Snapshot:** It writes a full snapshot of the 13 named silicon registers (`RAX` , `RBX` , `RCX` , `RDX` , `RSP` , `RBP` , `CR0` , `CR3`) directly into the flight recorder.
3. **Serial Attestation:** It broadcasts a structured beacon over COM1 serial (`[ALU ] (+1)+(-1)=0 Eq: VERIFIED` , `[REGS] RAX=0x...` , `[LADDER] Boot Ladder Rung 4: ACTIVE`).

In Track 1, the `query/` subsystem serves as the high-level computational equivalent of these physical RAM buffers. If the virtual machine halts or rolls back a transaction, the query engine makes the entire causal sequence instantaneously accessible.

---

3. The DAR (Detect, Assess, Restore) Autonomic Immune Loop

The OmniCode runtime features an autonomic immune system known as **DAR (Detect, Assess, Restore)**:

```
sequenceDiagram
    autonumber
    participant Engine as VM Execution Tape
    participant Logger as Inscriber / Sensory Log
    participant Query as Episodic Query Engine
    participant Restorator as Autonomic Restorator
    participant AI as Cognitive AI Cortex

    Engine->>Logger: Emit Failpoint (Unclosed Delimiter / Null Collision)
    Logger->>Logger: Write .log & .scroll inscriber streams
    Restorator->>Query: query.find_by_session(current_id)
    Query-->>Restorator: Return IndexedSession (O(1) memory lookup)
    Restorator->>Query: query.filter_by_failpoint(true)
    Query-->>Restorator: Return historical recurring failure patterns
    Restorator->>Restorator: Synthesize morphological boundary restoration
    Restorator->>Engine: Apply Self-Healing Patch (+1:PROCEED)
    AI->>Query: query.inspect_session_detail(current_id)
    Query-->>AI: Return Complete Diagnostic Dossier & Repair Record
    AI->>AI: Inscribe Experiential Memory Note (Prevent Future Mistake)
```

Why Autonomic Healing Demands Querying

1. **Step 1: Detect (Sensory Tripwire)** When a process ribbon encounters an unclosed delimiter (`[ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte ]`), the sensory logger catches the mismatch and triggers `-1:ROLLBACK` .
2. **Step 2: Assess (Instant Historical Query)** The `Restorator` cannot apply an intelligent structural repair without context. It queries `EpisodicQueryEngine` :
3. *Has this file or tape been healed before?*

4. *What was the morphological rank of the surrounding container?*
5. *Is this an isolated syntax anomaly or a systemic compiler bug?* Because `query` maintains multi-key hash maps in memory, this assessment takes less than **50 microseconds**.
6. **Step 3: Restore (Verification & Affirmation)** After the `Restorator` balances the boundary (`}` instead of `]`), the engine queries the resulting commit status. If the commit QuadState transitions to `+1:PROCEED` , the repair is verified and inscribed into the permanent ledger.

---

4. Neural-Symbolic AI Cognition & Experiential Learning

In the OmniCode framework, artificial intelligence is not an external chat bot bolted onto a legacy terminal; it is an internal cognitive organ: the `AICortex` .

The Inherent Flaw of Memoryless AI

Conventional AI coding agents suffer from catastrophic amnesia:

- Every command is evaluated as if the universe began five seconds ago.
- If a model makes a syntax error, it has no memory of making that same error ten seconds earlier.
- It cannot reflect upon its own historical pass rates or repair statistics.

How `EpisodicQueryEngine` Powers AI Reflection

The `query` engine provides the `AICortex` with **structured, episodic self-reflection**:

1. **$O(1)$ Prompt Recall:** Using the inverted keyword index (`by_keyword`), the AI can query all past episodes where a prompt containing `"storehouse"` or `"character"` was executed.
2. **Empirical Grounding:** The AI inspects the `QueryResult` and `QuerySummary` :
3. Total matched sessions: N
4. Pass rate: $\frac{\text{Proceed}}{\text{Total}} \times 100\%$
5. Historical failpoint count: F
6. Common repairs applied: R
7. **Deterministic Memory Notes:** The AI formulates active thoughts grounded in real historical facts rather than probabilistic hallucination.

---

5. The Seven Invariant Laws of OmniCode Diagnostic Querying

The `query` subsystem enforces seven non-negotiable architectural laws:

Invariant ID	Canonical Law Title	Mathematical / Hardware Enforcement
INV-QRY-01	**Sub-Millisecond Retrieval**	Finding an episode by session ID, stem, or alias must execute in strictly $O(1)$ time via in-memory hash maps.
INV-QRY-02	**Causal Monotonicity**	All sessions must be indexed in ascending microsecond order ($\Delta t = t_{i+1} - t_i \ge 0$), guaranteeing deterministic time-range slicing.
INV-QRY-03	**Dual-Stream Pairing**	Querying a `.scroll` file must provide deterministic resolution to its paired `.log` sensory file without directory scanning.
INV-QRY-04	**Quad-State Partitioning**	Queries must support instantaneous filtering by balanced ternary QuadState ($\mathbb{T}_4 \in { -1, 0, +1, \text{NULL} }$).
INV-QRY-05	**Track 0 Silicon Grounding**	The query engine must be capable of discovering, parsing, and correlating Track 0 bare-metal hardware dumps (`0x18000` , `0x108000` , `0x200000`).
INV-QRY-06	**Container Capacity Bounds**	All in-memory index structures must enforce strict capacity limits in accordance with **2 Kings 4:4** ($\text{Occupancy} \le \text{Capacity}$).
INV-QRY-07	**Zero Destructive Mutation**	Query operations are strictly read-only and must never mutate, corrupt, or lock the physical files on disk.

---

6. Summary

The logger has a query function because **intelligence is impossible without memory**.

By grounding the query engine in Track 0 bare-metal silicon invariants, binding it to the Dual-Stream inscriber, and exposing it to both the autonomic `Restorator` and the `AICortex` , OmniCode achieves what legacy systems cannot: **a fully introspective, self-healing, and self-learning computational substrate.**

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/index.rs RS | 594 LINES | 2315 WORDS | 22.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( IndexedSession, SessionCriteria & Track0LogRecord Data Structures )
// ============================================================================
//! # Indexed Episode & Silicon Log Record Data Structures
//!
//! # Architectural Scope & Purpose:
//! The `index` module defines the core data representations utilized by the
//! [`EpisodicQueryEngine`]. In the OmniCode architecture, logging is never a
//! write-only discard pipeline; it is an indexed, queryable memory storehouse.
//!
//! This module provides:
//! - [`IndexedSession`]: Compact, high-level representation of past execution
//!   episodes parsed from native 4-Block `.scroll` headers.
//! - [`SessionCriteria`]: Fluent, multi-dimensional query builder for compound
//!   filters across domain categories, QuadStates, failpoints, and timestamps.
//! - [`QueryResult`]: Aggregated search result container with statistical metrics
//!   (pass rate, repair efficiency, time spans).
//! - [`Track0LogRecord`]: Concrete structured representation of bare-metal silicon
//!   telemetry dumps from Track 0 hardware flight recorders (`0x18000`, `0x108000`, `0x200000`).
//!
//! # Scripture Witness:
//! - **Job 8:8 (KJV)**:
//!   "For enquire, I pray thee, of the former age, and prepare thyself to the search of their fathers."
//!   (Episodic memory inquiry into preceding execution cycles to derive wisdom).
//! - **Proverbs 25:2 (KJV)**:
//!   "It is the glory of God to conceal a thing: but the honour of kings is to search out a matter."
//!   (Diagnostic investigation into execution failpoints, registers, and memory states).
//! - **Ecclesiastes 1:9 (KJV)**:
//!   "The thing that hath been, it is that which shall be; and that which is done is that which shall be done."
//!   (Temporal determinism and repeatable diagnostic causality across execution rolls).
//!
//! # Why Structured Index Records Matter for AI Agents & Engineers:
//! 1. **Zero-Allocation Filtering**: Querying millions of historical events by
//!    streaming lightweight index records avoids loading gigabytes of raw log strings into memory.
//! 2. **Track 0 Silicon Grounding**: Bridges the high-level cognitive session
//!    with the physical CPU register bank and bare-metal flight recorder addresses.
//! 3. **Episodic AI Reflection**: Provides the `AICortex` and `AILearner` with
//!    instant, non-probabilistic recall of past prompt completions, syntax repairs, and error rates.
// ============================================================================

use std::collections::HashMap;
use std::fmt;
use std::path::PathBuf;
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Structural Types, Query Criteria & Silicon Record Definitions )
// ============================================================================

/// High-level indexed record representing a past execution episode.
///
/// Extracted from the first 25 lines of a native 4-Block `.scroll` file in $O(1)$ time,
/// avoiding linear disk reads of large execution traces.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct IndexedSession {
    /// Unique session identifier string (e.g. `sess_20261018_142055_001` or `char_test_01`).
    pub session_id: String,
    /// Monotonic timestamp in microseconds since epoch.
    pub timestamp_micros: u128,
    /// Human-readable timestamp ID string (e.g. `20261018_091530_123456`).
    pub timestamp_id: String,
    /// Domain category partition (e.g. `genesis`, `character`, `number`, `sessions/prompts`, `cortex`).
    pub category: String,
    /// Final fourfold commit state ($\mathbb{T}_4 \in \{ -1, 0, +1, \text{NULL} \}$).
    pub final_state: QuadState,
    /// Number of failpoints trapped during execution.
    pub failpoint_count: usize,
    /// Number of automated repairs applied by Restorator.
    pub repair_count: usize,
    /// Total words/tokens inscribed in the session prompt.
    pub word_count: usize,
    /// Path to the native 4-Block `.scroll` episodic trace file.
    pub scroll_path: PathBuf,
```

```rust
    /// Path to the paired traditional `.log` file if present on disk.
    pub log_path: Option<PathBuf>,
    /// Natural language prompt or command line string that initiated the run.
    pub prompt: String,
    /// Flag indicating whether physical silicon / MMIO tripwires were tripped.
    pub hardware_tripped: bool,
}

/// Fluent query specification builder for multi-dimensional filtering.
#[derive(Debug, Clone, Default)]
pub struct SessionCriteria {
    /// Optional domain category filter.
    pub category: Option<String>,
    /// Optional final commit state filter.
    pub final_state: Option<QuadState>,
    /// Filter for sessions with or without failpoints (`Some(true)` => failpoints > 0).
    pub has_failpoints: Option<bool>,
    /// Filter for sessions with automated repairs (`Some(true)` => repair_count > 0).
    pub has_repairs: Option<bool>,
    /// Monotonic microsecond lower time bound.
    pub min_timestamp_micros: Option<u128>,
    /// Monotonic microsecond upper time bound.
    pub max_timestamp_micros: Option<u128>,
    /// Substring search token for the session prompt.
    pub prompt_keyword: Option<String>,
    /// Maximum number of matching records to return.
    pub limit: Option<usize>,
}

/// Aggregate statistical summary computed across a set of queried sessions.
#[derive(Debug, Clone, PartialEq, Eq, Default)]
pub struct QuerySummary {
    /// Total sessions matched by the query.
    pub total_matched: usize,
    /// Number of sessions that concluded in `+1:PROCEED`.
    pub proceed_count: usize,
    /// Number of sessions that concluded in `-1:ROLLBACK`.
    pub rollback_count: usize,
    /// Number of sessions that concluded in `0:AWAIT`.
    pub await_count: usize,
    /// Number of sessions that concluded in `NULL:HALT`.
    pub halt_count: usize,
    /// Total failpoints trapped across all matched sessions.
    pub total_failpoints: usize,
    /// Total repairs applied by Restorator across all matched sessions.
    pub total_repairs: usize,
    /// Earliest timestamp microsecond in the matched set.
    pub earliest_micros: u128,
    /// Latest timestamp microsecond in the matched set.
    pub latest_micros: u128,
}

/// Complete result container holding matched records and statistical aggregates.
#[derive(Debug, Clone)]
pub struct QueryResult {
    /// Matching indexed sessions.
    pub sessions: Vec<IndexedSession>,
    /// Aggregate statistical metrics.
    pub summary: QuerySummary,
}

/// Structured record representing a low-level Track 0 silicon telemetry dump.
///
/// Captures bare-metal hardware flight recorder dumps, serial UART beacons,
/// and CPU register values from physical silicon inception.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Track0LogRecord {
    /// Inception boot mode (e.g. `BIOS Safe Mode`, `Stage 2 PM`, `Multiboot ELF`, `Native UEFI`).
    pub boot_mode: String,
    /// Target platform architecture (e.g. `x86_16`, `x86_32`, `x86_64`).
    pub architecture: String,
    /// Path to the raw Track 0 log file.
    pub log_path: PathBuf,
    /// Hardware flight recorder physical RAM address (e.g. `0x18000`, `0x108000`, `0x200000`).
    pub flight_recorder_address: Option<u64>,
    /// Named CPU register bank snapshot captured at boot or tripwire halt.
    pub registers: HashMap<String, u64>,
    /// Whether balanced ternary ALU equation `(+1)+(-1)=0` was verified.
    pub alu_verified: bool,
    /// Whether the CPI-SI cognitive substrate was successfully mounted.
    pub cpisi_arena_mounted: bool,
```

```rust
    /// Whether execution concluded in Safe Mode or Proceed.
    pub status: String,
    /// Raw lines captured from COM1 serial or OVMF firmware.
    pub raw_lines: Vec<String>,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Method Implementations, Constructors & Display Formatters )
// ============================================================================

impl IndexedSession {
    /// Returns `true` if the session executed cleanly without failpoints in `+1:PROCEED`.
    pub fn is_clean(&self) -> bool {
        self.final_state == QuadState::Proceed && self.failpoint_count == 0
    }

    /// Returns `true` if the session required automated restoration or experienced rollbacks.
    pub fn had_anomalies(&self) -> bool {
        self.failpoint_count > 0 || self.final_state == QuadState::Rollback || self.final_state == QuadState::Halt
    }

    /// Returns the session duration in microseconds if duration is encoded, or 0.
    pub fn duration_micros(&self) -> u128 {
        // Derived from end of prompt or metadata if present
        0
    }

    /// Formats the session as an architectural one-line ledger entry.
    pub fn to_ledger_line(&self) -> String {
        let state_str = match self.final_state {
            QuadState::Proceed => "+1:PROCEED",
            QuadState::Rollback => "-1:ROLLBACK",
            QuadState::Await => " 0:AWAIT",
            QuadState::Halt => "NULL:HALT",
        };
        format!(
            "[{:>16} µs] {:<18} | {:<12} | State: {:<11} | FP: {:<3} | Rep: {:<3} | \"{}\"",
            self.timestamp_micros,
            self.session_id,
            self.category,
            state_str,
            self.failpoint_count,
            self.repair_count,
            truncate_prompt(&self.prompt, 40)
        )
    }
}

impl fmt::Display for IndexedSession {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.to_ledger_line())
    }
}

impl SessionCriteria {
    /// Constructs a blank, unrestricted query criteria.
    pub fn new() -> Self {
        Self::default()
    }

    /// Constrains the search to a specific domain category.
    pub fn with_category(mut self, category: impl Into<String>) -> Self {
        self.category = Some(category.into());
        self
    }

    /// Constrains the search to a specific final commit QuadState.
    pub fn with_state(mut self, state: QuadState) -> Self {
        self.final_state = Some(state);
        self
    }

    /// Filters for sessions with or without failpoints.
    pub fn with_failpoints(mut self, has_failpoints: bool) -> Self {
        self.has_failpoints = Some(has_failpoints);
        self
    }

    /// Filters for sessions with or without Restorator repairs.
    pub fn with_repairs(mut self, has_repairs: bool) -> Self {
```

```
        self.has_repairs = Some(has_repairs);
        self
    }

    /// Constrains results to a monotonic time window `[min_micros ..= max_micros]`.
    pub fn with_time_range(mut self, min_micros: u128, max_micros: u128) -> Self {
        self.min_timestamp_micros = Some(min_micros);
        self.max_timestamp_micros = Some(max_micros);
        self
    }

    /// Constrains results to prompts containing the given substring token (case-insensitive).
    pub fn with_keyword(mut self, keyword: impl Into<String>) -> Self {
        self.prompt_keyword = Some(keyword.into().to_lowercase());
        self
    }

    /// Limits the maximum number of returned records.
    pub fn with_limit(mut self, limit: usize) -> Self {
        self.limit = Some(limit);
        self
    }

    /// Evaluates whether an [`IndexedSession`] satisfies all active filter criteria.
    pub fn matches(&self, session: &IndexedSession) -> bool {
        if let Some(ref cat) = self.category {
            if &session.category != cat && !session.category.starts_with(cat) {
                return false;
            }
        }
        if let Some(st) = self.final_state {
            if session.final_state != st {
                return false;
            }
        }
        if let Some(hf) = self.has_failpoints {
            if (session.failpoint_count > 0) != hf {
                return false;
            }
        }
        if let Some(hr) = self.has_repairs {
            if (session.repair_count > 0) != hr {
                return false;
            }
        }
        if let Some(min_t) = self.min_timestamp_micros {
            if session.timestamp_micros < min_t {
                return false;
            }
        }
        if let Some(max_t) = self.max_timestamp_micros {
            if session.timestamp_micros > max_t {
                return false;
            }
        }
        if let Some(ref kw) = self.prompt_keyword {
            if !session.prompt.to_lowercase().contains(kw) && !session.session_id.to_lowercase().contains(kw) {
                return false;
            }
        }
        true
    }
}

impl QueryResult {
    /// Builds a [`QueryResult`] from a slice of matched sessions and computes summary statistics.
    pub fn from_sessions(sessions: Vec<IndexedSession>) -> Self {
        let total_matched = sessions.len();
        let mut proceed_count = 0;
        let mut rollback_count = 0;
        let mut await_count = 0;
        let mut halt_count = 0;
        let mut total_failpoints = 0;
        let mut total_repairs = 0;
        let mut earliest_micros = u128::MAX;
        let mut latest_micros = 0;

        for s in &sessions {
            match s.final_state {
                QuadState::Proceed => proceed_count += 1,
                QuadState::Rollback => rollback_count += 1,
```

```
                QuadState::Await => await_count += 1,
                QuadState::Halt => halt_count += 1,
            }
            total_failpoints += s.failpoint_count;
            total_repairs += s.repair_count;
            if s.timestamp_micros < earliest_micros {
                earliest_micros = s.timestamp_micros;
            }
            if s.timestamp_micros > latest_micros {
                latest_micros = s.timestamp_micros;
            }
        }

        if sessions.is_empty() {
            earliest_micros = 0;
        }

        let summary = QuerySummary {
            total_matched,
            proceed_count,
            rollback_count,
            await_count,
            halt_count,
            total_failpoints,
            total_repairs,
            earliest_micros,
            latest_micros,
        };

        Self { sessions, summary }
    }

    /// Returns the pass rate as a percentage `[0.0 .. 100.0]`.
    pub fn pass_rate(&self) -> f64 {
        if self.summary.total_matched == 0 {
            100.0
        } else {
            (self.summary.proceed_count as f64 / self.summary.total_matched as f64) * 100.0
        }
    }
}

impl Track0LogRecord {
    /// Parses a raw Track 0 log file from disk into a structured record.
    pub fn parse_from_file(path: PathBuf, content: &str) -> Self {
        let path_str = path.to_string_lossy();
        let mut boot_mode = "Unknown Boot Mode".to_string();
        let mut architecture = "x86".to_string();
        let mut flight_recorder_address = None;
        let mut registers = HashMap::new();
        let mut alu_verified = false;
        let mut cpisi_arena_mounted = false;
        let mut status = "UNKNOWN".to_string();
        let mut raw_lines = Vec::new();

        if path_str.contains("bios_safe_mode") {
            boot_mode = "Legacy BIOS Stage 1 (Safe Mode)".to_string();
            architecture = "x86_16".to_string();
        } else if path_str.contains("bios_dual_stage2") {
            boot_mode = "Dual-State Stage 2 Protected Mode".to_string();
            architecture = "x86_32".to_string();
            flight_recorder_address = Some(0x18000);
        } else if path_str.contains("multiboot") {
            boot_mode = "Multiboot 1 Direct Hypervisor".to_string();
            architecture = "x86_32".to_string();
            flight_recorder_address = Some(0x108000);
        } else if path_str.contains("uefi_safe_mode") {
            boot_mode = "UEFI Safe Mode Cascade".to_string();
            architecture = "x86_64".to_string();
            flight_recorder_address = Some(0x200000);
        } else if path_str.contains("uefi") {
            boot_mode = "Native 64-Bit UEFI (PE32+)".to_string();
            architecture = "x86_64".to_string();
            flight_recorder_address = Some(0x200000);
        }

        for line in content.lines() {
            let trimmed = line.trim();
            if trimmed.is_empty() {
                continue;
            }
```

```
            raw_lines.push(trimmed.to_string());

            if trimmed.contains("(+1)+(-1)=0") && (trimmed.contains("OK") || trimmed.contains("VERIFIED")) {
                alu_verified = true;
            }
            if trimmed.contains("CPI-SI") && (trimmed.contains("MOUNTED") || trimmed.contains("Mount:  OK") ||
trimmed.contains("Arena:  OK")) {
                cpisi_arena_mounted = true;
            }
            if trimmed.contains("Flight Recorder (18000)") || trimmed.contains("0x18000") {
                flight_recorder_address = Some(0x18000);
            } else if trimmed.contains("Flight Recorder (108000)") || trimmed.contains("0x108000") {
                flight_recorder_address = Some(0x108000);
            } else if trimmed.contains("Flight Recorder (200000)") || trimmed.contains("0x200000") {
                flight_recorder_address = Some(0x200000);
            }

            if trimmed.contains("MILLENIUMOS") && trimmed.contains("PROCEED") {
                status = "PROCEED".to_string();
            } else if trimmed.contains("SAFE MODE") {
                if status == "UNKNOWN" {
                    status = "SAFE_MODE".to_string();
                }
            }

            // Parse registers if present, e.g. RAX=0x0000000007E8E650 RBX=0x0000000007E8E698
            if trimmed.contains("REGS:") {
                let parts: Vec<&str> = trimmed.split_whitespace().collect();
                for part in parts {
                    if let Some(eq_pos) = part.find('=') {
                        let reg_name = &part[..eq_pos];
                        let val_str = &part[eq_pos + 1..];
                        let val_clean = val_str.trim_start_matches("0x").trim_start_matches("0X");
                        if let Ok(val) = u64::from_str_radix(val_clean, 16) {
                            registers.insert(reg_name.to_string(), val);
                        }
                    }
                }
            }
        }

        Self {
            boot_mode,
            architecture,
            log_path: path,
            flight_recorder_address,
            registers,
            alu_verified,
            cpisi_arena_mounted,
            status,
            raw_lines,
        }
    }
}

fn truncate_prompt(prompt: &str, max_len: usize) -> String {
    if prompt.len() <= max_len {
        prompt.to_string()
    } else {
        format!("{}...", &prompt[..max_len])
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Index Data Structure Invariants )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_indexed_session_clean_and_anomaly_invariants() {
        let clean = IndexedSession {
            session_id: "sess_01".to_string(),
            timestamp_micros: 1000,
            timestamp_id: "20261018_090000_001000".to_string(),
            category: "character".to_string(),
            final_state: QuadState::Proceed,
            failpoint_count: 0,
```

```
            repair_count: 0,
            word_count: 12,
            scroll_path: PathBuf::from("test.scroll"),
            log_path: None,
            prompt: "Echo character 'A'".to_string(),
            hardware_tripped: false,
        };

        assert!(clean.is_clean());
        assert!(!clean.had_anomalies());

        let mut anomaly = clean.clone();
        anomaly.failpoint_count = 1;
        assert!(!anomaly.is_clean());
        assert!(anomaly.had_anomalies());
    }

    #[test]
    fn test_session_criteria_matching() {
        let session = IndexedSession {
            session_id: "sess_char_01".to_string(),
            timestamp_micros: 50_000,
            timestamp_id: "20261018_090000_050000".to_string(),
            category: "character".to_string(),
            final_state: QuadState::Proceed,
            failpoint_count: 0,
            repair_count: 0,
            word_count: 5,
            scroll_path: PathBuf::from("test.scroll"),
            log_path: None,
            prompt: "Initialize character cell".to_string(),
            hardware_tripped: false,
        };

        let c1 = SessionCriteria::new()
            .with_category("character")
            .with_state(QuadState::Proceed);
        assert!(c1.matches(&session));

        let c2 = SessionCriteria::new().with_category("number");
        assert!(!c2.matches(&session));

        let c3 = SessionCriteria::new().with_time_range(10_000, 60_000);
        assert!(c3.matches(&session));

        let c4 = SessionCriteria::new().with_keyword("character");
        assert!(c4.matches(&session));

        let c5 = SessionCriteria::new().with_keyword("arithmetic");
        assert!(!c5.matches(&session));
    }

    #[test]
    fn test_query_result_aggregation() {
        let s1 = IndexedSession {
            session_id: "s1".to_string(),
            timestamp_micros: 100,
            timestamp_id: "100".to_string(),
            category: "cat".to_string(),
            final_state: QuadState::Proceed,
            failpoint_count: 0,
            repair_count: 0,
            word_count: 1,
            scroll_path: PathBuf::from("s1.scroll"),
            log_path: None,
            prompt: "p1".to_string(),
            hardware_tripped: false,
        };
        let s2 = IndexedSession {
            session_id: "s2".to_string(),
            timestamp_micros: 200,
            timestamp_id: "200".to_string(),
            category: "cat".to_string(),
            final_state: QuadState::Rollback,
            failpoint_count: 2,
            repair_count: 1,
            word_count: 2,
            scroll_path: PathBuf::from("s2.scroll"),
            log_path: None,
            prompt: "p2".to_string(),
            hardware_tripped: false,
```

```
        };

        let result = QueryResult::from_sessions(vec![s1, s2]);
        assert_eq!(result.summary.total_matched, 2);
        assert_eq!(result.summary.proceed_count, 1);
        assert_eq!(result.summary.rollback_count, 1);
        assert_eq!(result.summary.total_failpoints, 2);
        assert_eq!(result.summary.total_repairs, 1);
        assert_eq!(result.summary.earliest_micros, 100);
        assert_eq!(result.summary.latest_micros, 200);
        assert_eq!(result.pass_rate(), 50.0);
    }

    #[test]
    fn test_track0_log_record_parser() {
        let log_sample = r#"
MILLENIUMOS: OMNICODE 32-TRIT OS [x86 MBR]
[BOOT] 16-Bit Genesis: OK
[ALU ] (+1)+(-1)=0 Eq: OK
[REGS] 13 Regs Armed / Calibrated
[BOOT] State 0: Safe Mode Beacon
MILLENIUMOS SAFE MODE :: PROCEED
"#;
        let record = Track0LogRecord::parse_from_file(
            PathBuf::from("/kernel/images/logs/bios_safe_mode.log"),
            log_sample,
        );

        assert_eq!(record.architecture, "x86_16");
        assert!(record.alu_verified);
        assert_eq!(record.status, "PROCEED");
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/mod.rs RS 112 LINES 528 WORDS 5.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Episodic Query Subsystem Facade & Track 0 Cross-Scale Diagnostics )
// ============================================================================
//! # OmniCode Episodic Query Subsystem: Fast Retrieval & Silicon Correlation
//!
//! # Architectural Scope & Purpose:
//! The `query` module serves as the primary intelligence retrieval organ for the
//! entire `diagnostics::logger` subsystem. While `inscriber` writes paired streams
//! (`.log` telemetry and `.scroll` cognitive traces), and `telemetry` captures
//! microsecond rollframes and Track 0 genesis baselines, the `query` subsystem
//! transforms this historical ledger into an instantaneously interrogatable memory bank.
//!
//! # Subsystem Architecture:
//! - [`index`]: Structural representations of indexed sessions ([`IndexedSession`]),
//!   fluent multi-criteria query builders ([`SessionCriteria`]), statistical aggregates
//!   ([`QueryResult`], [`QuerySummary`]), and Track 0 bare-metal silicon logs ([`Track0LogRecord`]).
//! - [`search`]: High-performance [`EpisodicQueryEngine`] featuring multi-key $O(1)$
//!   hash indexing, chronological sorting, binary range slicing, inverted prompt
//!   keyword indexing, and Track 0 silicon flight recorder log discovery.
//!
//! # Scripture Witness:
//! - **Job 8:8 (KJV)**:
//!   "For enquire, I pray thee, of the former age, and prepare thyself to the search of their fathers."
//!   (Examining past execution history to diagnose errors and prevent regression).
//! - **Proverbs 25:2 (KJV)**:
//!   "It is the glory of God to conceal a thing: but the honour of kings is to search out a matter."
//!   (Systematic inquiry into internal state vectors, registers, and failpoints).
//! - **Jeremiah 29:13 (KJV)**:
//!   "And ye shall seek me, and find me, when ye shall search for me with all your heart."
//!   (Exhaustive, non-probabilistic diagnostic verification).
//!
//! # Why Query Capability is Essential:
//! In OmniCode, a log is not a discard dump. The cognitive AI (`AICortex`, `AILearner`)
//! and the autonomic immune system (`Restorator`, `Debugger`) must be able to query
//! past runs in $O(1)$ time to self-heal, prevent hallucination, and correlate high-level
//! software faults with bare-metal Track 0 silicon landmarks (`0x18000`, `0x108000`, `0x200000`).
// ============================================================================

pub mod index;
pub mod search;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Public Type Re-exports & Subsystem Invariants )
// ============================================================================

pub use index::{
    IndexedSession, QueryResult, QuerySummary, SessionCriteria, Track0LogRecord,
};
pub use search::EpisodicQueryEngine;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Facade Constructors, Quick-Search Utilities & Diagnostic Helpers )
// ============================================================================

/// Constructs a fully populated [`EpisodicQueryEngine`] pointing to `storehouse_root`.
pub fn create_query_engine_at(storehouse_root: &std::path::Path) -> Result<EpisodicQueryEngine, String> {
    let mut engine = EpisodicQueryEngine::new();
    engine.index_directory(storehouse_root)?;
    Ok(engine)
}

/// Discovers an indexed session by ID or alias directly from a storage directory.
pub fn query_session_by_id(
    storehouse_root: &std::path::Path,
    session_id: &str,
) -> Option<IndexedSession> {
    let engine = create_query_engine_at(storehouse_root).ok()?;
    engine.find_by_session(session_id).cloned()
}

/// Diagnoses failpoints for a given session into a human-readable dossier.
pub fn diagnose_session(
    engine: &EpisodicQueryEngine,
    session_id: &str,
```

```
) -> Option<String> {
    engine.inspect_session_detail(session_id)
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Test Verification Proofs & Query Invariant Seals )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::path::PathBuf;

    #[test]
    fn test_query_subsystem_re_exports_and_facade() {
        let engine = EpisodicQueryEngine::new();
        assert_eq!(engine.all_sessions().len(), 0);

        let criteria = SessionCriteria::new()
            .with_category("character")
            .with_limit(10);
        let results = engine.query(&criteria);
        assert_eq!(results.sessions.len(), 0);
        assert_eq!(results.summary.total_matched, 0);
    }

    #[test]
    fn test_facade_quick_search_nonexistent_returns_empty() {
        let empty_path = PathBuf::from("/nonexistent/omnicode/storage/scrolls");
        let result = create_query_engine_at(&empty_path).unwrap();
        assert_eq!(result.all_sessions().len(), 0);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/query/search.rs RS 625 LINES 2168 WORDS 24.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( EpisodicQueryEngine: O(1) Indexing, Compound Search & Silicon Correlation )
// ============================================================================
//! # Episodic Query Engine Implementation
//!
//! # Architectural Scope & Purpose:
//! The `search` module implements [`EpisodicQueryEngine`]. Traditional logging
//! architectures treat telemetry as an append-only disposal dump requiring linear
//! file scans (`grep`) that degrade as system uptime and log volume scale ($O(N)$).
//!
//! In contrast, OmniCode's [`EpisodicQueryEngine`] provides:
//! 1. **Multi-Key $O(1)$ Hash Indexing**: Maps session IDs, timestamp prefixes,
//!    file stems, and short suffixes to internal session indices.
//! 2. **Inverted Keyword Index**: Tokenizes user prompts and system tasks, mapping
//!    terms to execution episodes without disk access.
//! 3. **Chronological Monotonicity**: Preserves strictly sorted microsecond
//!    sequences, enabling instant binary range-slicing ($\Delta t = t_1 - t_0$).
//! 4. **Track 0 Silicon Correlation**: Discovers and parses bare-metal hardware
//!    dumps directly from `kernel/images/logs/`, correlating hardware register
//!    dumps (e.g. `0x18000`, `0x108000`, `0x200000`) with high-level software episodes.
//! 5. **Dual-Stream Pairing**: Resolves between the concise 4-Block `.scroll`
//!    cognitive memory file and its paired microsecond-by-microsecond `.log` file.
//!
//! # Scripture Witness:
//! - **Jeremiah 29:13 (KJV)**:
//!   "And ye shall seek me, and find me, when ye shall search for me with all your heart."
//!   (Thorough, exhaustive search yielding complete diagnostic understanding).
//! - **Matthew 7:7 (KJV)**:
//!   "Ask, and it shall be given you; seek, and ye shall find; knock, and it shall be opened unto you."
//!   (The query interface as the open door to system memory and self-healing).
//! - **1 Chronicles 28:9 (KJV)**:
//!   "for the LORD searcheth all hearts, and understandeth all the imaginations of the thoughts."
//!   (Full inspection of internal system state, cognitive deliberations, and failpoints).
//!
//! # Why Fast Querying Matters for AI Agents & Autonomic Healing:
//! - **Autonomous Reflection**: When the `AICortex` fails or encounters an edge case,
//!   it queries past episodes to discover whether the same prompt previously succeeded,
//!   what syntax repairs were applied by `Restorator`, and what QuadState resulted.
//! - **DAR Immune Feedback**: The self-healing loop cannot wait seconds for linear disk
//!   scans. It requires sub-millisecond retrieval of failpoint frequencies and patterns.
// ============================================================================

use std::collections::HashMap;
use std::fs;
use std::path::{Path, PathBuf};

use super::index::{IndexedSession, QueryResult, SessionCriteria, Track0LogRecord};
use crate::core::ternary::QuadState;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Engine Struct Definition & Index Invariant Declarations )
// ============================================================================

/// High-performance query engine for episodic memory storehouses.
///
/// Maintains multi-key hash maps, category buckets, QuadState indices,
/// and an inverted keyword index over in-memory session records.
#[derive(Debug, Clone, Default)]
pub struct EpisodicQueryEngine {
    /// Chronologically ordered list of all indexed execution episodes.
    sessions: Vec<IndexedSession>,
    /// Multi-key alias index mapping session IDs, stems, and suffixes to vector positions.
    by_session_id: HashMap<String, usize>,
    /// Domain category partition index mapping category names to session positions.
    by_category: HashMap<String, Vec<usize>>,
    /// State partition index mapping QuadStates to session positions.
    by_state: HashMap<QuadState, Vec<usize>>,
    /// Inverted keyword index mapping tokenized prompt words to session positions.
    by_keyword: HashMap<String, Vec<usize>>,
    /// Parsed low-level Track 0 bare-metal silicon logs.
    track0_records: Vec<Track0LogRecord>,
    /// Index mapping Track 0 boot modes (e.g. "x86_16", "uefi") to silicon records.
    by_track0_mode: HashMap<String, Vec<usize>>,
```

```rust
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Directory Indexing, Query Implementations & Track 0 Parsers )
// ============================================================================

impl EpisodicQueryEngine {
    /// Constructs an empty [`EpisodicQueryEngine`].
    pub fn new() -> Self {
        Self::default()
    }

    /// Indexes all `.scroll` files located under `root_dir`.
    ///
    /// Clears existing indices, scans `.scroll` files, extracts 4-block header
    /// metadata in $O(1)$ time per file, sorts chronologically, and populates
    /// multi-key hash maps and the inverted keyword index.
    pub fn index_directory(&mut self, root_dir: &Path) -> Result<usize, String> {
        self.sessions.clear();
        self.by_session_id.clear();
        self.by_category.clear();
        self.by_state.clear();
        self.by_keyword.clear();

        if !root_dir.exists() {
            return Ok(0);
        }

        let scroll_paths = collect_scroll_files(root_dir);
        let mut count = 0;

        for path in scroll_paths {
            if let Some(session) = parse_indexed_session(&path) {
                self.sessions.push(session);
                count += 1;
            }
        }

        // Sort chronologically by monotonic microsecond timestamp
        self.sessions.sort_by_key(|s| s.timestamp_micros);

        // Populate indices over sorted array
        for (idx, s) in self.sessions.iter().enumerate() {
            // 1. Primary session ID
            self.by_session_id.insert(s.session_id.clone(), idx);

            // 2. Multi-key aliases (stem, date prefix, suffixes)
            if let Some(stem) = s.scroll_path.file_stem().and_then(|t| t.to_str()) {
                self.by_session_id.insert(stem.to_string(), idx);
                let parts: Vec<&str> = stem.split('_').collect();
                if parts.len() >= 3 {
                    self.by_session_id.insert(parts[1..].join("_"), idx);
                    self.by_session_id.insert(parts[2..].join("_"), idx);
                }
            }

            // 3. Category index
            self.by_category
                .entry(s.category.clone())
                .or_default()
                .push(idx);

            // 4. QuadState index
            self.by_state.entry(s.final_state).or_default().push(idx);

            // 5. Inverted keyword index over prompt tokens
            let tokens = tokenize_prompt(&s.prompt);
            for token in tokens {
                self.by_keyword.entry(token).or_default().push(idx);
            }
        }

        Ok(count)
    }

    /// Discovers and indexes Track 0 bare-metal silicon logs from a logs directory.
    pub fn index_track0_logs(&mut self, logs_dir: &Path) -> Result<usize, String> {
        self.track0_records.clear();
        self.by_track0_mode.clear();

```

```
        if !logs_dir.exists() {
            return Ok(0);
        }

        let mut count = 0;
        if let Ok(entries) = fs::read_dir(logs_dir) {
            for entry in entries.flatten() {
                let p = entry.path();
                if p.extension().and_then(|e| e.to_str()) == Some("log") {
                    if let Ok(content) = fs::read_to_string(&p) {
                        let record = Track0LogRecord::parse_from_file(p, &content);
                        let idx = self.track0_records.len();
                        self.by_track0_mode
                            .entry(record.architecture.clone())
                            .or_default()
                            .push(idx);
                        self.by_track0_mode
                            .entry(record.boot_mode.clone())
                            .or_default()
                            .push(idx);
                        self.track0_records.push(record);
                        count += 1;
                    }
                }
            }
        }

        Ok(count)
    }

    /// Finds an episode by its exact session ID or alias in $O(1)$ time.
    pub fn find_by_session(&self, session_id: &str) -> Option<&IndexedSession> {
        let idx = *self.by_session_id.get(session_id)?;
        self.sessions.get(idx)
    }

    /// Filters all episodes belonging to a specific domain category.
    pub fn filter_by_category(&self, category: &str) -> Vec<&IndexedSession> {
        if let Some(indices) = self.by_category.get(category) {
            indices.iter().filter_map(|&i| self.sessions.get(i)).collect()
        } else {
            Vec::new()
        }
    }

    /// Filters episodes by their final commit QuadState.
    pub fn filter_by_state(&self, state: QuadState) -> Vec<&IndexedSession> {
        if let Some(indices) = self.by_state.get(&state) {
            indices.iter().filter_map(|&i| self.sessions.get(i)).collect()
        } else {
            self.sessions.iter().filter(|s| s.final_state == state).collect()
        }
    }

    /// Filters episodes that experienced failpoints.
    pub fn filter_by_failpoint(&self, has_failpoints: bool) -> Vec<&IndexedSession> {
        self.sessions
            .iter()
            .filter(|s| (s.failpoint_count > 0) == has_failpoints)
            .collect()
    }

    /// Range scans episodes executed between `start_micros` and `end_micros` inclusive.
    pub fn range_scan(&self, start_micros: u128, end_micros: u128) -> Vec<&IndexedSession> {
        self.sessions
            .iter()
            .filter(|s| s.timestamp_micros >= start_micros && s.timestamp_micros <= end_micros)
            .collect()
    }

    /// Searches episodes by prompt keyword via the inverted index.
    pub fn search_keyword(&self, keyword: &str) -> Vec<&IndexedSession> {
        let token = keyword.trim().to_lowercase();
        if let Some(indices) = self.by_keyword.get(&token) {
            indices.iter().filter_map(|&i| self.sessions.get(i)).collect()
        } else {
            // Fallback substring search
            self.sessions
                .iter()
                .filter(|s| s.prompt.to_lowercase().contains(&token))
                .collect()
```

```
        }
    }

    /// Executes a compound search matching all constraints defined in [`SessionCriteria`].
    pub fn query(&self, criteria: &SessionCriteria) -> QueryResult {
        let mut matched = Vec::new();

        for session in &self.sessions {
            if criteria.matches(session) {
                matched.push(session.clone());
                if let Some(limit) = criteria.limit {
                    if matched.len() >= limit {
                        break;
                    }
                }
            }
        }

        QueryResult::from_sessions(matched)
    }

    /// Returns a slice of all indexed sessions.
    pub fn all_sessions(&self) -> &[IndexedSession] {
        &self.sessions
    }

    /// Returns a slice of all parsed Track 0 silicon log records.
    pub fn track0_records(&self) -> &[Track0LogRecord] {
        &self.track0_records
    }

    /// Filters Track 0 records by boot mode or architecture string.
    pub fn find_track0_by_mode(&self, mode: &str) -> Vec<&Track0LogRecord> {
        if let Some(indices) = self.by_track0_mode.get(mode) {
            indices.iter().filter_map(|&i| self.track0_records.get(i)).collect()
        } else {
            self.track0_records
                .iter()
                .filter(|r| r.boot_mode.contains(mode) || r.architecture.contains(mode))
                .collect()
        }
    }

    /// Inspects and formats deep diagnostic details for a specific session.
    pub fn inspect_session_detail(&self, session_id: &str) -> Option<String> {
        let session = self.find_by_session(session_id)?;
        let mut out = format!("================================================================================\n");
        out.push_str(&format!("  OMNICODE EPISODIC DIAGNOSTIC DOSSIER: {}\n", session.session_id));
        out.push_str(&format!("================================================================================\n"));
        out.push_str(&format!("• Timestamp:     {} µs ({})\n", session.timestamp_micros, session.timestamp_id));
        out.push_str(&format!("• Category:      {}\n", session.category));
        out.push_str(&format!("• Final State:   {:?}\n", session.final_state));
        out.push_str(&format!("• Failpoints:    {}\n", session.failpoint_count));
        out.push_str(&format!("• Repairs:       {}\n", session.repair_count));
        out.push_str(&format!("• Scroll Path:   {}\n", session.scroll_path.display()));
        if let Some(ref lp) = session.log_path {
            out.push_str(&format!("• Log Path:      {}\n", lp.display()));
        }
        out.push_str(&format!("• User Prompt:   \"{}\"\n", session.prompt));

        // Read paired log telemetry if available
        if let Some(ref lp) = session.log_path {
            if let Ok(content) = fs::read_to_string(lp) {
                out.push_str(&format!("\n--- SENSORY LOG EXCERPT (Critical & Remediation Lines) ---\n"));
                let mut matches = 0;
                for line in content.lines() {
                    if line.contains("CRITICAL") || line.contains("Remediation") || line.contains("FAILPOINT") {
                        out.push_str(&format!("  {}\n", line));
                        matches += 1;
                        if matches > 20 {
                            out.push_str("  ... [Remaining entries truncated]\n");
                            break;
                        }
                    }
                }
                if matches == 0 {
                    out.push_str("  (No critical failpoints or tripwires recorded in sensory log)\n");
                }
            }
        }

```

```
        out.push_str(&format!("================================================================================\n"));
        Some(out)
    }
}

/// Recursively collects all `.scroll` files within `dir`.
fn collect_scroll_files(dir: &Path) -> Vec<PathBuf> {
    let mut files = Vec::new();
    if let Ok(entries) = fs::read_dir(dir) {
        for entry in entries.flatten() {
            let p = entry.path();
            if p.is_dir() {
                files.extend(collect_scroll_files(&p));
            } else if p.extension().and_then(|e| e.to_str()) == Some("scroll") {
                files.push(p);
            }
        }
    }
    files
}

/// Tokenizes a prompt string into clean, lowercase keyword tokens.
fn tokenize_prompt(prompt: &str) -> Vec<String> {
    let mut tokens = Vec::new();
    for word in prompt.split_whitespace() {
        let clean: String = word
            .chars()
            .filter(|c| c.is_alphanumeric())
            .flat_map(|c| c.to_lowercase())
            .collect();
        if clean.len() >= 3 {
            tokens.push(clean);
        }
    }
    tokens
}

/// Parses an `IndexedSession` from the first 30 lines of a `.scroll` file.
fn parse_indexed_session(path: &Path) -> Option<IndexedSession> {
    let content = fs::read_to_string(path).ok()?;
    let mut session_id = "unknown".to_string();
    let mut prompt = String::new();
    let mut final_state = QuadState::Proceed;
    let mut failpoint_count = 0;
    let mut repair_count = 0;
    let mut timestamp_micros = 0u128;
    let mut timestamp_id = String::new();
    let mut hardware_tripped = false;

    for line in content.lines().take(40) {
        let trimmed = line.trim();
        if trimmed.starts_with("session_id:") {
            if let Some(sq) = trimmed.find('"') {
                if let Some(eq) = trimmed.rfind('"') {
                    if sq < eq {
                        session_id = trimmed[sq + 1..eq].to_string();
                    }
                }
            }
        } else if trimmed.starts_with("timestamp:") {
            if let Some(sq) = trimmed.find('"') {
                if let Some(eq) = trimmed.rfind('"') {
                    if sq < eq {
                        let ts_str = &trimmed[sq + 1..eq];
                        timestamp_id = ts_str.to_string();
                        if let Some(first) = ts_str.split('_').next() {
                            timestamp_micros = first.parse().unwrap_or(0);
                        }
                    }
                }
            }
        } else if trimmed.starts_with("user_prompt") {
            if let Some(sq) = trimmed.find('"') {
                if let Some(eq) = trimmed.rfind('"') {
                    if sq < eq {
                        prompt = trimmed[sq + 1..eq].to_string();
                    }
                }
            }
        } else if prompt.is_empty() && trimmed.starts_with("Scroll < EpistemicTrace") {
            if let Some(open_p) = trimmed.find('(') {
```

```
                if let Some(close_p) = trimmed.rfind(')') {
                    if open_p < close_p {
                        prompt = trimmed[open_p + 1..close_p].trim().to_string();
                    }
                }
            }
        } else if trimmed.starts_with("commit_status [") {
            if trimmed.contains("<PROCEED>") {
                final_state = QuadState::Proceed;
            } else if trimmed.contains("<ROLLBACK>") {
                final_state = QuadState::Rollback;
            } else if trimmed.contains("<AWAIT>") {
                final_state = QuadState::Await;
            } else if trimmed.contains("<HALT>") {
                final_state = QuadState::Halt;
            }
        } else if trimmed.starts_with("failpoint_record [") {
            failpoint_count += 1;
            if trimmed.contains("SILICON") || trimmed.contains("MMIO") || trimmed.contains("REGISTER") {
                hardware_tripped = true;
            }
        } else if trimmed.starts_with("repair_entry [") {
            repair_count += 1;
        }
    }

    // Determine category from path or prompt
    let path_str = path.to_string_lossy();
    let category = if path_str.contains("/genesis/")
        || prompt.contains("genesis")
        || prompt.contains("first_log")
        || prompt.contains("boot")
    {
        "genesis".to_string()
    } else if path_str.contains("/character/") || prompt.contains("character") {
        "character".to_string()
    } else if path_str.contains("/number/") || prompt.contains("number") {
        "number".to_string()
    } else if path_str.contains("/sessions/prompts")
        || (!prompt.trim().is_empty() && path_str.contains("/sessions/"))
    {
        "sessions/prompts".to_string()
    } else if path_str.contains("/sessions/") || session_id.starts_with("sess_") {
        "sessions/system".to_string()
    } else {
        "cortex".to_string()
    };

    let word_count = prompt.split_whitespace().count();

    // Locate paired log file
    let mut log_path = path.to_path_buf();
    log_path.set_extension("log");
    let paired_log = if log_path.exists() {
        Some(log_path)
    } else if path_str.contains("/scroll/") {
        let alt = PathBuf::from(path_str.replace("/scroll/", "/log/"));
        let mut alt_log = alt;
        alt_log.set_extension("log");
        if alt_log.exists() {
            Some(alt_log)
        } else {
            None
        }
    } else {
        None
    };

    Some(IndexedSession {
        session_id,
        timestamp_micros,
        timestamp_id,
        category,
        final_state,
        failpoint_count,
        repair_count,
        word_count,
        scroll_path: path.to_path_buf(),
        log_path: paired_log,
        prompt,
        hardware_tripped,
```

```
    })
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Query Invariant Seals )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::io::Write;

    #[test]
    fn test_indexing_directory_and_multikey_aliases() {
        let temp_dir = std::env::temp_dir().join(format!(
            "omnicode_query_test_{}",
            std::time::SystemTime::now()
                .duration_since(std::time::UNIX_EPOCH)
                .unwrap()
                .as_nanos()
        ));
        let scroll_dir = temp_dir.join("character");
        fs::create_dir_all(&scroll_dir).unwrap();

        let scroll_content = r#"
Metadata {
    session_id: "test_sess_alpha",
    timestamp: "1792000000000000_0000_test_sess_alpha",
    user_prompt: "Initialize character memory vessel",
}
Opening {
    Cell [ 1 ] & { 5 Trits :: 1 Byte },
}
Body {
    Cell [ 1 ] := 'A',
}
Closing {
    commit_status [ <PROCEED> ],
}
"#;
        let file_path = scroll_dir.join("1792000000000000_0000_test_sess_alpha.scroll");
        let mut file = fs::File::create(&file_path).unwrap();
        file.write_all(scroll_content.as_bytes()).unwrap();

        let mut engine = EpisodicQueryEngine::new();
        let count = engine.index_directory(&temp_dir).unwrap();
        assert_eq!(count, 1);

        // 1. Direct session ID lookup
        let s1 = engine.find_by_session("test_sess_alpha").expect("Found by ID");
        assert_eq!(s1.final_state, QuadState::Proceed);
        assert_eq!(s1.category, "character");

        // 2. Full stem lookup
        let s2 = engine
            .find_by_session("1792000000000000_0000_test_sess_alpha")
            .expect("Found by stem");
        assert_eq!(s2.session_id, "test_sess_alpha");

        // 3. Suffix lookup
        let s3 = engine
            .find_by_session("0000_test_sess_alpha")
            .expect("Found by suffix");
        assert_eq!(s3.session_id, "test_sess_alpha");

        // 4. Inverted keyword search
        let kw_results = engine.search_keyword("vessel");
        assert_eq!(kw_results.len(), 1);
        assert_eq!(kw_results[0].session_id, "test_sess_alpha");

        // 5. Compound criteria query
        let criteria = SessionCriteria::new()
            .with_category("character")
            .with_state(QuadState::Proceed)
            .with_keyword("memory");
        let q_res = engine.query(&criteria);
        assert_eq!(q_res.sessions.len(), 1);
        assert_eq!(q_res.pass_rate(), 100.0);

        // Clean up
```

```
        let _ = fs::remove_dir_all(&temp_dir);
    }

    #[test]
    fn test_range_scan_and_state_filtering() {
        let mut engine = EpisodicQueryEngine::new();
        let s1 = IndexedSession {
            session_id: "s1".to_string(),
            timestamp_micros: 10_000,
            timestamp_id: "10000".to_string(),
            category: "cat".to_string(),
            final_state: QuadState::Proceed,
            failpoint_count: 0,
            repair_count: 0,
            word_count: 1,
            scroll_path: PathBuf::from("s1.scroll"),
            log_path: None,
            prompt: "p1".to_string(),
            hardware_tripped: false,
        };
        let s2 = IndexedSession {
            session_id: "s2".to_string(),
            timestamp_micros: 20_000,
            timestamp_id: "20000".to_string(),
            category: "cat".to_string(),
            final_state: QuadState::Rollback,
            failpoint_count: 1,
            repair_count: 0,
            word_count: 1,
            scroll_path: PathBuf::from("s2.scroll"),
            log_path: None,
            prompt: "p2".to_string(),
            hardware_tripped: false,
        };
        let s3 = IndexedSession {
            session_id: "s3".to_string(),
            timestamp_micros: 30_000,
            timestamp_id: "30000".to_string(),
            category: "cat".to_string(),
            final_state: QuadState::Proceed,
            failpoint_count: 0,
            repair_count: 0,
            word_count: 1,
            scroll_path: PathBuf::from("s3.scroll"),
            log_path: None,
            prompt: "p3".to_string(),
            hardware_tripped: false,
        };

        engine.sessions = vec![s1, s2, s3];

        let ranged = engine.range_scan(15_000, 25_000);
        assert_eq!(ranged.len(), 1);
        assert_eq!(ranged[0].session_id, "s2");

        let rollbacks = engine.filter_by_state(QuadState::Rollback);
        assert_eq!(rollbacks.len(), 1);
        assert_eq!(rollbacks[0].session_id, "s2");

        let clean = engine.filter_by_failpoint(false);
        assert_eq!(clean.len(), 2);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/CAUSAL_MONOTONICITY_AND_SILICON_CLOCK_PHYSICS.md | MARKDOWN / DOC | 227 LINES | 1273 WORDS | 10.5 KB

Causal Monotonicity & Silicon Clock Physics

The Temporal Mechanics of the OmniCode Host Engine

> *"To every thing there is a season, and a time to every purpose under the heaven." — **Ecclesiastes 3:1 (KJV)***
>
> *"Ponder the path of thy feet, and let all thy ways be established." — **Proverbs 4:26 (KJV)***
>
> *"Let all things be done decently and in order." — **1 Corinthians 14:40 (KJV)***

1. The Breakdown of Wall Clocks in Autonomous Systems

In classical userland software, engineers frequently record timestamps using the operating system's wall clock (e.g. `SystemTime::now()` or UNIX epoch seconds). In an autonomous, cybernetic operating system where an AI Cortex reasons over causal sequences, **wall-clock time is fundamentally broken**:

1. **Network Time Protocol (NTP) Slews and Steps:** When an NTP daemon synchronizes a machine's clock against an external time server, it can adjust time backward. A second event can receive a timestamp smaller than the first event ($t_2 < t_1$), creating a chronological paradox where an effect appears to precede its cause.
2. **Leap Seconds and Smearing:** Leap second insertions cause duplicate timestamps or non-linear time warping.
3. **Virtual Machine Live Migration:** When a virtual machine hypervisor migrates a guest instance across physical CPU sockets or host nodes, TSC (Time Stamp Counter) registers and wall clocks can experience discontinuities.
4. **Epistemic Inversion in AI Models:** If an episodic memory trace records a failpoint at $t = 1000\ \mu\text{s}$ and the subsequent restoration at $t = 950\ \mu\text{s}$, reinforcement learning algorithms infer negative causal delay, destabilizing policy weights.

To preserve the integrity of the **DAR Loop (Detect, Assess, Restore)**, OmniCode Track 1 establishes **Causal Monotonicity** as an inviolable hardware law.

```
================================================================================
                    THE CAUSAL MONOTONICITY IMPERATIVE
================================================================================

   [ Wall Clock: UNRELIABLE ]           [ Monotonic Clock: OMNICODE TRUTH ]
   • Subject to NTP slews               • Strictly non-decreasing (Δt >= 0)
   • Negative time steps possible       • Preserves Lamport causality: A -> B => t(A) < t(B)
   • Causes AI epistemic inversion      • Grounded in physical silicon TSC & Instant
================================================================================
```

2. Lamport Causal Ordering & The Monotonic Invariant

OmniCode diagnostics adheres to Leslie Lamport's foundational definition of causality in distributed and concurrent systems:

$$\text{If event } A \longrightarrow B \text{ (event } A \text{ causally precedes event } B\text{), then } \mathcal{T}(A) < \mathcal{T}(B)$$

To enforce this invariant across the `diagnostics::logger` subsystem:

1. **`MonotonicClock`** wraps Rust's `std::time::Instant`, which queries monotonic hardware timers (such as Linux `CLOCK_MONOTONIC_RAW` or CPU invariant TSC) that never step backward, regardless of NTP adjustments.
2. Every telemetry event (`LogEvent`), roll frame (`RollFrame`), and cognitive trace (`CognitiveNote`) records an unsigned microsecond offset `elapsed_micros: u128` relative to subsystem initialization (t_0).

```
use omnicode::diagnostics::logger::universal::MonotonicClock;

let clock = MonotonicClock::new();
let t1 = clock.elapsed_micros();
// ... execute roll operations ...
let t2 = clock.elapsed_micros();
```

```
assert!(t2 >= t1, "Temporal monotonicity violation: elapsed time regressed!");
```

3. Track 0 Silicon Cycle Translation (RDTSC Calibration)

At bare-metal bootstrap (Track 0), high-level OS abstractions do not yet exist. When cold silicon boots through physical RAM landmarks:

- **`0x7C00`** : Sector 0 Real Mode bootstrap reset vector.
- **`0x18000`** : Stage 2 Protected Mode Flight Recorder buffer.
- **`0x108000`** : Long Mode 64-bit page table initialization.
- **`0x200000`** : Cognitive substrate arena.

The x86 CPU measures duration exclusively in raw machine clock cycles via the `RDTSC` (Read Time-Stamp Counter) instruction. When Track 0 handoffs telemetry to Track 1, these raw cycle counts must be translated into microsecond durations:

$$\text{Cycles} = \text{Microseconds} \times \text{MHz}$$

$$\text{Microseconds} = \frac{\text{Cycles}}{\text{MHz}}$$

```
================================================================================
                  SILICON CYCLE TRANSLATION PIPELINE
================================================================================

   Track 0 Bare-Metal Hardware              Track 1 Host VM & AI Cortex
   ┌──────────────────────────┐             ┌──────────────────────────┐
   | x86 RDTSC Machine Register |            |  Monotonic Duration (µs)   |
   | Landmark 0x18000 Telemetry | ──[ / MHz ]──> |  RollFrame::elapsed_micros |
   |   (e.g. 2,000,000 Cycles)  |            |     (e.g. 1,000 µs)        |
   └──────────────────────────┘             └──────────────────────────┘
================================================================================
```

Safe Mathematical Implementation

To prevent integer overflow while maintaining integer precision, `MonotonicClock` implements saturation-guarded arithmetic:

```
impl MonotonicClock {
    /// Converts elapsed microseconds to CPU cycles at the specified nominal frequency.
    #[inline]
    pub fn micros_to_cycles(micros: u128, mhz: u64) -> u64 {
        (micros as u64).saturating_mul(mhz)
    }

    /// Converts raw CPU cycles (from RDTSC) into microsecond durations.
    #[inline]
    pub fn cycles_to_micros(cycles: u64, mhz: u64) -> u128 {
        if mhz == 0 {
            0
        } else {
            (cycles / mhz) as u128
        }
    }
}
```

4. Dual-Anchored Monotonic Identifier Generation

Every telemetry vessel inscribed to disk or transmitted across network sockets requires a standardized session or vessel identifier. In `clock.rs`, this is implemented via `generate_timestamp_id()`:

$$\text{Identifier Format} = \langle \text{EpochMicros} \rangle _ \langle \text{SequenceCounter:04} \rangle$$

The Dual-Anchor Defense Mechanism

Even though `SystemTime::now()` is used to capture the human-readable UNIX epoch, the generator defends against system time regression using an internal high-water mark:

```
static COUNTER: AtomicU64 = AtomicU64::new(0);
static LAST_MICROS: Mutex<u128> = Mutex::new(0);

let count = COUNTER.fetch_add(1, Ordering::Relaxed);
let wall_now = SystemTime::now()
    .duration_since(SystemTime::UNIX_EPOCH)
    .unwrap_or_default()
    .as_micros();

let mut last = LAST_MICROS.lock().unwrap();
let epoch_micros = if wall_now > *last {
    *last = wall_now;
    wall_now
} else {
    *last
};
```

1. If the system clock steps backward, `epoch_micros` locks to `*last` (the highest timestamp ever witnessed).
2. The atomic sequence counter (`COUNTER`) increments unconditionally.
3. Therefore, lexicographical sorting of timestamp IDs is **strictly isomorphic to chronological order**:

$$\text{ID}_A < \text{ID}_B \iff \text{Creation}(A) < \text{Creation}(B)$$

Exact Decomposition Parsing (`parse_timestamp_id`)

The companion parser decomposes strings back into raw epoch and sequence components:

```
use omnicode::diagnostics::logger::universal::parse_timestamp_id;

let id = "1729215000000000_0042";
let (epoch_micros, counter) = parse_timestamp_id(id).expect("Valid timestamp format");

assert_eq!(epoch_micros, 1729215000000000);
assert_eq!(counter, 42);
```

5. Deterministic Virtual Machine Simulation (`SimulatedClock`)

The Flaky Test Pathology

Automated testing in complex operating systems frequently suffers from **timing jitter**:

- Tests that use `std::thread::sleep(Duration::from_millis(10))` execute in 10 ms on idle hardware, but take 45 ms under heavy continuous integration (CI) load.
- Failpoint tripwires, timeouts, and watchdog counters fire prematurely or fail to trigger, causing non-deterministic test failures.

The Discrete Event Simulation Solution

`SimulatedClock` solves this by decoupling virtual machine execution from real-world wall clocks:

```
use omnicode::diagnostics::logger::universal::SimulatedClock;

let clock = SimulatedClock::new(0);
assert_eq!(clock.current_micros(), 0);
assert_eq!(clock.current_tick(), 0);

// Advance VM by 5 execution rolls at 100 µs per tick
clock.step_ticks(5, 100);
assert_eq!(clock.current_tick(), 5);
assert_eq!(clock.current_micros(), 500);

// Freeze clock to inspect an execution frame in static equilibrium
clock.freeze();
assert!(clock.is_frozen());
```

```
// While frozen, step operations are safely suppressed
clock.step_micros(200);
assert_eq!(clock.current_micros(), 500);

// Resume clock for subsequent rolls
clock.resume();
clock.step_micros(200);
assert_eq!(clock.current_micros(), 700);
```

Benefits for AI Replay and Verification

1. **Bitwise-Reproducible Replay:** An AI agent debugging a failed compilation or runtime roll can replay execution with identical tick timestamps down to the single microsecond.
2. **Zero-Wait CI Testing:** Long-running timeout tests (e.g. a 30-second watchdog failpoint) can be tested instantaneously by calling `clock.step_micros(30_000_000)` without wasting CPU cycles in sleeping threads.
3. **Equilibrium Auditing:** Before allowing a machine halt, the system can freeze time and verify that all queues have settled to homeostatic baseline Ω_0.

---

6. Mathematical Summary of Invariant Proofs

Concept	Mathematical Formulation	Invariant Condition
Monotonic Non-Regression	$\Delta t = t_{n+1} - t_n$	$\Delta t \ge 0\ \forall\ n \in \mathbb{N}$
Strict Identifier Monotonicity	$\text{ID}_n = \langle E_n, C_n \rangle$	$(E_{n+1} > E_n) \lor (E_{n+1} = E_n \land C_{n+1} > C_n)$
Silicon Cycle Invariance	$C_{\text{cycles}} = T_{\mu\text{s}} \cdot f_{\text{MHz}}$	Roundtrip error: $\left\| \frac{C_{\text{cycles}}}{f} - T \right\| = 0$
Discrete Step Invariance	$T_{\text{new}} = T_{\text{curr}} + (N_{\text{ticks}} \cdot \delta_{\text{tick}})$	Synchronized microsecond and tick progression
Freeze Isolation	$\frac{d}{dt}\mathcal{T}_{\text{sim}}(\text{frozen}) = 0$	Zero time leakage during frozen debugging states

Event Bus Broadcast & Subscriber Tap Architecture

The Sensory Spinal Cord of the DAR Cybernetic Loop

> *"Let all things be done decently and in order." — **1 Corinthians 14:40 (KJV)***
>
> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." — **2 Kings 4:4 (KJV)***
>
> *"He that answereth a matter before he heareth it, it is folly and shame unto him." — **Proverbs 18:13 (KJV)***

1. Architectural Scope: The Sensory Spinal Cord

In the **DAR Triad (Detect, Assess, Restore)**:

- The **Logger** is the **DETECT** organ.
- The **Debugger** is the **ASSESS** organ.
- The **Restorator** is the **RESTORE** organ.

Before an invariant breach can be assessed or restored, it must first be **sensed**. If telemetry is trapped in isolated subsystem silos—where the parser prints to standard error, the evaluator panics to standard output, and bare-metal silicon writes to an unchecked serial buffer—the AI Cortex cannot form a coherent world model.

The **Universal Event Bus (`EventBus`)** serves as the **sensory spinal cord** of OmniCode Track 1. It gathers every pulse, roll, failpoint, and cognitive trace across the entire operating system into a unified, bounded, thread-safe transmission channel.



2. Subsystem Channel Topology

To prevent telemetry pollution while ensuring that every event preserves its morphological origin, `LogEvent` requires a static channel identifier (`channel: &'static str`).

Channel Tag	Subsystem Source	Typical Emitted Telemetry
`"SILICON"`	Track 0 Boot & Hardware	Landmark transitions (`0x7C00` , `0x18000`), A20 gate, 13 silicon registers, MMIO interrupts.
`"PARSER"`	Lexical & Syntax Engine	Token scanning, indentation checks, 4-block header validation.
`"BOUNDARY"`	Structural Guardrails	Asymmetric delimiter nesting, scope rank violations (`<` vs `[` vs `{` vs `(`).
`"EVALUATOR"`	Virtual Machine Core	Instruction execution, dual-rail input/output bitplanes, rollback trigger events.
`"COGNITION"`	AI Cortex & Memory	Hypothesis formulation, associative scripture anchors, reasoning trails.
`"HALT"`	Safety Supervisor	Emergency dual-rail `(1, 1)` saturation halts, crash dumps, and safe-mode re-inceptions.

Channel Isolation Queries

Subsystems query the bus directly for domain-specific event streams without iterating over foreign telemetry:

```
use omnicode::diagnostics::logger::universal::EventBus;

let bus = EventBus::new(false);
// ... telemetry emitted across all channels ...

// Extract only silicon telemetry for boot verification
let silicon_stream = bus.events_by_channel("SILICON");

// Extract only cognitive memory traces for AI digestion
let cognitive_stream = bus.events_by_channel("COGNITION");
```

---

3. Physical Container Law & Bounded Memory Invariants

The Out-Of-Memory Logging Pathology

In conventional logging frameworks, log buffers grow indefinitely in memory until a background worker writes them to disk. Under high-throughput conditions (e.g. an infinite rollback loop or a high-frequency sensor stream), unbounded log accumulation exhausts system RAM, triggering the Linux OOM killer and abruptly terminating the operating system.

Scriptural Grounding: 2 Kings 4:4

OmniCode logging enforces physical container physics as an absolute invariant:

> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full." — **2 Kings 4:4 (KJV)***

A container that is full must be set aside; it must never be allowed to overflow into unallocated memory.

FIFO Eviction Mechanics ($O(1)$)

Both general diagnostic events and critical failpoints are managed in bounded circular double-ended queues (`VecDeque`):

- **General Diagnostic Events:** Bounded by `DEFAULT_EVENT_BUS_CAPACITY = 8,192` events.
- **Critical Failpoints:** Bounded by `DEFAULT_FAILPOINT_BUS_CAPACITY = 2,048` failpoints.

```
// Internal ingestion inside EventBus::emit()
let mut evs = self.events.lock().unwrap();
if evs.len() >= self.max_events_capacity {
    evs.pop_front(); // Evict oldest event in O(1)
}
evs.push_back(event); // Ingest new event
```

Guarantees:

1. **Bounded Heap Space:** Total memory occupied by the event bus is strictly bounded by a constant $O(C_{\max})$, regardless of whether the operating system runs for 10 milliseconds or 1,000 years.
2. **Freshness Invariant:** The bus continuously maintains the most recent execution window leading up to an incident, preserving high-resolution diagnostic context.

4. Non-Blocking Subscriber Tap Architecture

In traditional architectures, attaching observers (like a live terminal dashboard, a debugger, or an AI reasoning agent) requires either:

- Polling the bus in a busy-wait loop (wasting CPU cycles), or
- Invasive hooks into the core logging path (introducing locks and latency).

`EventBus` provides a **non-intrusive Subscriber Tap Architecture** via `register_tap`:

```
pub type EventTapCallback = Arc<dyn Fn(&LogEvent) + Send + Sync + 'static>;
```

Tap Registration Pattern

```
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use omnicode::diagnostics::logger::universal::{EventBus, LogEvent};

let bus = EventBus::new(false);
let alert_counter = Arc::new(AtomicUsize::new(0));

// Register a tap that monitors for critical failpoints
let counter_clone = Arc::clone(&alert_counter);
bus.register_tap(Arc::new(move |event: &LogEvent| {
    if event.is_failpoint {
        counter_clone.fetch_add(1, Ordering::SeqCst);
    }
}));

// Any subsystem emitting to the bus automatically alerts the tap
bus.emit(LogEvent::trace(100, "INIT", "Boot sequence began"));
assert_eq!(alert_counter.load(Ordering::SeqCst), 0);

bus.emit(LogEvent::failpoint(
    150,
    "BOUNDARY",
    "Missing closing bracket",
    omnicode::diagnostics::logger::failpoint::FailpointType::BoundaryLeak,
    None,
));
assert_eq!(alert_counter.load(Ordering::SeqCst), 1);
```

Architectural Properties of Taps

1. **Synchronous Invocation:** Taps are invoked immediately during `emit()`, guaranteeing that real-time monitors receive events before control returns to the caller.
2. **Thread Safety:** Taps implement `Send + Sync + 'static`, enabling multi-threaded dispatch without race conditions.
3. **Decoupled Isolation:** Taps receive immutable references (`&LogEvent`); they can never mutate, corrupt, or delay events destined for other consumers.

5. Telemetry Metrics Aggregation

The `EventBus` continuously tracks operational metrics to support real-time bottleneck detection and subsystem health auditing:

```
pub struct EventBusMetrics {
    pub total_emitted: usize,
    pub total_failpoints: usize,
    pub active_events_count: usize,
    pub active_failpoints_count: usize,
    pub channel_distribution: HashMap<&'static str, usize>,
}
```

Example Usage: Subsystem Distribution Audit

```
let metrics = bus.metrics();

println!("=== Event Bus Operational Health ===");
println!("Lifetime Events Emitted:     {}", metrics.total_emitted);
println!("Lifetime Failpoints Trapped: {}", metrics.total_failpoints);
println!("Current Active Buffer Depth: {} / 8192", metrics.active_events_count);

for (channel, count) in metrics.channel_distribution {
    let percentage = (count as f64 / metrics.total_emitted as f64) * 100.0;
    println!("  • Channel [{:<10}]: {:>6} events ({:>5.1}%)", channel, count, percentage);
}
```

6. Dual-Readability Presentation (ANSI vs Structured)

The event bus formats telemetry for both human eyes and machine consumption:

Human ANSI Presentation (`format_terminal`)

```
[   142 µs] DEBUG    [PARSER    ] Token stream initialized successfully
[   890 µs] CRITICAL [BOUNDARY  ] [FP_BOUNDARY_MISMATCH  ] Delimiter mismatch '<' closed by ']'
```

- **Microsecond Timestamp:** `[{:>6} µs]` right-aligned for clean visual scanning.
- **Status Badge:** ANSI Green `DEBUG` or ANSI Bold Red `CRITICAL` .
- **Channel Box:** Fixed-width 10-character subsystem container `[{:<10}]` .
- **Failpoint Box:** Fixed-width 22-character failpoint category `[{:<22}]` .

Machine Presentation (`LogEvent`)

For the AI Cortex and episodic query engine, the event is retained in memory as an immutable Rust structure equipped with exact microsecond timestamps, structured remediation suggestions (`FixSuggestion`), and episodic memory notes (`CognitiveNote`).

Universal Telemetry, Monotonic Clock & Event Bus Subsystem

The Sensory Foundation of OmniCode Diagnostics

> *"To every thing there is a season, and a time to every purpose under the heaven."*
> ***— Ecclesiastes 3:1 (KJV)***
>
> *"Ponder the path of thy feet, and let all thy ways be established."*
> ***— Proverbs 4:26 (KJV)***
>
> *"Let all things be done decently and in order."*
> ***— 1 Corinthians 14:40 (KJV)***
>
> *"And when thou art come in, thou shalt shut the door upon thee and upon thy sons, and shalt pour out into all those vessels, and thou shalt set aside that which is full."*
> ***— 2 Kings 4:4 (KJV)***

1. Architectural Philosophy: The Spinal Cord & Internal Clock

In **OmniCode Track 1 (Host Engine & Virtual Machine)**, diagnostic telemetry cannot rely on arbitrary string printing or non-monotonic system clocks. An autonomous operating system and its AI Cortex require two non-negotiable physical invariants:

1. **A Strictly Monotonic Clock ($\Delta t \ge 0$):** Causal reasoning demands that an effect must never chronologically precede its cause. System wall clocks (`SystemTime`) are vulnerable to NTP time-steps, VM migrations, and leap-second adjustments that induce negative time deltas. The Universal Clock guarantees strict temporal monotonicity across all execution ticks.
2. **A Deterministic Sensory Event Bus:** Subsystems—from Track 0 bare-metal silicon interrupts to the high-level linguistic parser and cognitive reasoning cortex—must emit signals into a unified, bounded, thread-safe nervous conduit without taking intrusive locks on the critical path.

The `universal` module provides this dual foundation:



```
                                 |
          ┌──────────────────────┼──────────────────────┐
          ▼                      ▼                      ▼
  [ LIVE TUI / LOGS ]      [ CRITICAL FAILPOINTS ]    [ AI CORTEX TAPS ]
  Human-Readable ANSI      Remediation & Halt Traps   Episodic Memory Feeds
================================================================================
```

2. Directory Topology & The 4-Block Structural Standard

Every source file in the `universal` module strictly implements the **OmniCode 4-Block Architecture**:

Block Index	Canonical Block Title	Architectural Scope & Invariant Duties
Block 1	**METADATA BLOCK**	Subsystem identity, module docstrings, Scripture witnesses, architectural scope, and external crate imports.
Block 2	**OPENING BLOCK**	Structural types, telemetry models, bounded capacity constants, and channel definitions.
Block 3	**BODY BLOCK**	Implementations, monotonic sequencing, bounded FIFO eviction logic, cycle calculations, and formatters.
Block 4	**CLOSING BLOCK**	Comprehensive unit test suites, invariant verification proofs, and multithreaded stress assertions.

Module File Layout

```
src/diagnostics/logger/universal/
├── README.md                                        <-- Subsystem Manifesto & Reference Guide
├── CAUSAL_MONOTONICITY_AND_SILICON_CLOCK_PHYSICS.md   <-- Deep Treatise: Monotonic Time & TSC Calibration
├── EVENT_BUS_BROADCAST_AND_SUBSCRIBER_ARCHITECTURE.md <-- Deep Treatise: Event Routing & Subscriber Taps
├── mod.rs                                           <-- Subsystem Facade & Re-exports (4-Block)
├── clock.rs                                         <-- Monotonic & Simulated Clocks (4-Block)
└── bus.rs                                           <-- Bounded Event Bus & Telemetry Routing (4-Block)
```

3. High-Precision Timing Machinery (`clock.rs`)

Real-Time Monotonic Clock (`MonotonicClock`)

Backed by `std::time::Instant`, `MonotonicClock` measures elapsed durations since subsystem initialization with microsecond and nanosecond precision.

- **Nanosecond Offset:** `clock.elapsed_nanos() -> u128`
- **Microsecond Offset:** `clock.elapsed_micros() -> u128`
- **Floating-Point Milliseconds:** `clock.elapsed_millis() -> f64`

Track 0 Silicon Cycle Translation

Bare-metal hardware emitted by Track 0 logs timestamp values in raw CPU cycles via the x86 `RDTSC` instruction. `MonotonicClock` bridges raw silicon cycles and wall durations using calibrated frequency constants:

$$\text{Cycles} = \text{Microseconds} \times \text{MHz}$$

$$\text{Microseconds} = \frac{\text{Cycles}}{\text{MHz}}$$

```
use omnicode::diagnostics::logger::universal::MonotonicClock;

let mhz = 2000; // 2.0 GHz nominal baseline
let micros = 500;
let cycles = MonotonicClock::micros_to_cycles(micros, mhz);
assert_eq!(cycles, 1_000_000);

let recovered_micros = MonotonicClock::cycles_to_micros(cycles, mhz);
assert_eq!(recovered_micros, 500);
```

Strictly Monotonic Timestamp Identifiers (`generate_timestamp_id`)

The `generate_timestamp_id()` function generates unique, monotonically non-decreasing identifiers formatted:

```
<unix_epoch_micros>_<atomic_counter:04>
```

To guard against NTP wall-clock regressions or host time jumps, the generator tracks the maximum observed epoch timestamp in an internal atomic mutex: if `wall_clock < last_observed`, the generator anchors to `last_observed` while incrementing the sequence counter, ensuring strict temporal progression ($\Delta t \ge 0$).

Deterministic Simulation Clock (`SimulatedClock`)

For automated unit testing, debugging, and bitwise-reproducible virtual machine execution, real-world clock jitter causes race conditions. `SimulatedClock` decouples execution from real time:

- **Discrete Microsecond Stepping:** `sim.step_micros(delta)`
- **Discrete VM Tick Stepping:** `sim.step_ticks(ticks, micros_per_tick)`
- **Clock Freezing:** `sim.freeze()` halts simulated time progression until `sim.resume()` is invoked.

---

4. The Diagnostic Event Bus Subsystem (`bus.rs`)

Strongly Typed Telemetry Frames (`LogEvent`)

Rather than unstructured string buffers, all telemetry emitted to the bus is packaged into strongly typed `LogEvent` frames:

```
pub struct LogEvent {
    pub elapsed_micros: u128,
    pub channel: &'static str,
    pub message: String,
    pub is_failpoint: bool,
    pub failpoint_type: Option<FailpointType>,
    pub suggestion: Option<FixSuggestion>,
    pub cognitive_note: Option<CognitiveNote>,
}
```

Physical Container Law: Bounded FIFO Eviction (2 Kings 4:4)

In accordance with Scripture and physical system laws, unbounded memory growth is strictly prohibited (`INV-UNI-01`).

- `DEFAULT_EVENT_BUS_CAPACITY = 8,192` events.
- `DEFAULT_FAILPOINT_BUS_CAPACITY = 2,048` failpoints.

When occupancy reaches maximum capacity ($\text{Occupancy} = \text{Capacity}$), the bus evicts the oldest event (`pop_front()`) before appending the new event (`push_back()`), guaranteeing bounded memory overhead ($O(1)$ amortized).

Subscriber Tap Architecture (`register_tap`)

External tools (such as live visual consoles, terminal dashboards, episodic query indexers, and automated test runners) can register synchronous callbacks without altering the logging pipeline:

```
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use omnicode::diagnostics::logger::universal::{EventBus, LogEvent};

let bus = EventBus::new(false);
let counter = Arc::new(AtomicUsize::new(0));

let counter_clone = Arc::clone(&counter);
bus.register_tap(Arc::new(move |_event: &LogEvent| {
    counter_clone.fetch_add(1, Ordering::SeqCst);
}));

bus.emit(LogEvent::trace(100, "PARSER", "Token stream initialized"));
assert_eq!(counter.load(Ordering::SeqCst), 1);
```

Channel Routing & Temporal Range Scanning

Events can be filtered instantaneously by functional subsystem channel or bounded temporal range:

```
// Filter all events emitted by the SILICON channel
let silicon_events = bus.events_by_channel("SILICON");
```

```
// Filter events between 100 µs and 500 µs
let time_slice = bus.events_by_time_range(100, 500);
```

5. Public API Usage Guide

A. Initializing a Calibrated Telemetry Pair

```
use omnicode::diagnostics::logger::universal::create_universal_telemetry_pair;

let (clock, bus) = create_universal_telemetry_pair(false);
let start = clock.elapsed_micros();

bus.emit(omnicode::diagnostics::logger::universal::LogEvent::trace(
    start,
    "INIT",
    "Subsystem brought online cleanly",
));
```

B. Emitting Critical Failpoints with Machine-Actionable Remediation

```
use omnicode::diagnostics::logger::failpoint::{FailpointType, FixKind, FixSuggestion};
use omnicode::diagnostics::logger::universal::{EventBus, LogEvent};

let bus = EventBus::new(false);

let suggestion = FixSuggestion {
    kind: FixKind::ReplaceDelimiter,
    line: 12,
    col: 4,
    target_token: "]".to_string(),
    suggested_patch: ">".to_string(),
    rationale: "Balance Rank 1 invariant delimiter".to_string(),
};

bus.emit(LogEvent::failpoint(
    450,
    "BOUNDARY",
    "Delimiter hierarchy mismatch '<' closed by ']'",
    FailpointType::BoundaryMismatch,
    Some(suggestion),
));

assert_eq!(bus.failpoint_count(), 1);
```

C. Extracting Telemetry Metrics

```
let metrics = bus.metrics();
println!("Total Events Emitted: {}", metrics.total_emitted);
println!("Active Failpoints In Buffer: {}", metrics.active_failpoints_count);
for (channel, count) in metrics.channel_distribution {
    println!("Channel [{channel}]: {count} events");
}
```

6. Covenantal Invariant Matrix

Invariant ID	Covenantal Rule	Verification Mechanism
INV-UNI-01	**Bounded Memory Occupancy**: In-memory event buffers must never grow without bound. Oldest entries must be evicted via FIFO at capacity ceiling.	`bus.len() <= max_capacity` enforced in `emit()`. Validated by unit test `test_bounded_capacity_fifo_eviction`.
INV-UNI-02	**Strict Causal Monotonicity**: Timestamp identifiers and elapsed time offsets must be non-decreasing ($\Delta t \ge 0$).	Validated by atomic counter and `LAST_MICROS` floor in `generate_timestamp_id()`.
INV-UNI-03	**Deterministic Simulation Bounds**: When `SimulatedClock` is frozen, no time or tick progression may occur without explicit unfreezing.	Validated by unit test `test_simulated_clock_lifecycle_and_freeze`.
INV-UNI-04	**Non-Intrusive Tap Invariance**: Registering a subscriber tap must never drop or mutate events on the primary bus.	Validated by unit test `test_subscriber_tap_callback`.
INV-UNI-05	**Channel Provenance**: Every event must declare a static string channel identifying its originating machine organ.	Enforced by `channel: &'static str` on `LogEvent`.
INV-UNI-06	**Zero Regressions**: All 13 unit tests across `clock.rs`, `bus.rs`, and `mod.rs` must pass with zero warnings.	Verified via `cargo test diagnostics::logger::universal`.

7. Subsystem Navigation Sitemap

- Root Architectural Manifesto
- Logger Subsystem Hub
- Failpoint Diagnostic Subsystem
- Container Inscriber Subsystem
- Episodic Query Engine
- Monotonic Time & Silicon Clock Physics
- Event Bus Routing & Subscriber Architecture

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/bus.rs RS | 566 LINES | 1846 WORDS | 20.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Diagnostic Event Bus, Channel Routing, Subscriber Taps & Bounded Invariants )
// ============================================================================
//! OmniCode Universal Diagnostics: Event Bus & Subsystem Telemetry Routing
//!
//! # Architectural Scope & Purpose:
//! The central nervous conduit of OmniCode Track 1. Receives, sequences, and routes
//! diagnostic trace events, critical failpoint interceptions, cognitive memory notes,
//! and hardware halt dumps across execution subsystems:
//! - [`LogEvent`]: Strongly typed diagnostic telemetry frame capturing timestamps, channels, and failpoints.
//! - [`EventBus`]: Thread-safe, bounded broadcast bus coordinating live diagnostic streams.
//! - [`EventTapCallback`]: Subscriber tap mechanism enabling live observation without polling.
//! - [`EventBusMetrics`]: Subsystem distribution telemetry and emission frequency metrics.
//!
//! # Scripture Witness:
//! - **1 Corinthians 14:40 (KJV)**:
//!   "Let all things be done decently and in order."
//!   (The requirement that telemetry events flow through deterministic, ordered channels without chaos).
//! - **2 Kings 4:4 (KJV)**:
//!   "And when thou art come in, thou shalt shut the door upon thee and upon thy sons,
//!    and shalt pour out into all those vessels, and thou shalt set aside that which is full."
//!   (The physical container law Occupancy <= Capacity applied to bounded in-memory queues).
//! - **Proverbs 18:13 (KJV)**:
//!   "He that answereth a matter before he heareth it, it is folly and shame unto him."
//!   (The principle of non-lossy sensory interception: the system must capture full telemetry before judging).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Zero-Unbounded-Growth Invariant**: Diagnostic event queues must never cause Out-Of-Memory (OOM)
//!    panics. Both `events` and `failpoints` are strictly bounded by capacity ceilings with FIFO eviction.
//! 2. **Multi-Subscriber Taps**: External tools (live terminals, web dashboards, query indexers, test fixtures)
//!    can register non-intrusive taps to inspect events in real-time without modifying the core pipeline.
//! 3. **Subsystem Isolation**: Channels ("PARSER", "EVALUATOR", "SILICON", "COGNITION") allow distinct
//!    organs to emit into a single bus while preserving modular boundary provenance.
// ============================================================================

use std::collections::{HashMap, VecDeque};
use std::sync::{Arc, Mutex};
use super::super::failpoint::{FailpointType, FixSuggestion};
use super::super::telemetry::CognitiveNote;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Constants, Event Struct Definitions, Tap Callbacks & Bus Enums )
// ============================================================================

/// Default maximum number of general log events retained in memory (8,192 events).
pub const DEFAULT_EVENT_BUS_CAPACITY: usize = 8192;

/// Default maximum number of critical failpoint events retained in memory (2,048 events).
pub const DEFAULT_FAILPOINT_BUS_CAPACITY: usize = 2048;

/// Type alias for thread-safe listener tap callbacks.
pub type EventTapCallback = Arc<dyn Fn(&LogEvent) + Send + Sync + 'static>;

/// Individual diagnostic telemetry event recorded during VM or hardware execution.
#[derive(Debug, Clone)]
pub struct LogEvent {
    /// Microseconds elapsed since logger initialization.
    pub elapsed_micros: u128,
    /// Subsystem or channel tag (e.g. "BOUNDARY", "PARSER", "SILICON", "COGNITION").
    pub channel: &'static str,
    /// Descriptive human-readable diagnostic message.
    pub message: String,
    /// True if this event represents an intercepted failpoint or pathology.
    pub is_failpoint: bool,
    /// Specific failpoint category if `is_failpoint` is true.
    pub failpoint_type: Option<FailpointType>,
    /// Concrete remediation suggestion if an automated repair was synthesized.
    pub suggestion: Option<FixSuggestion>,
    /// Accompanying structured cognitive memory note for AI Cortex ingestion.
    pub cognitive_note: Option<CognitiveNote>,
}

/// Aggregated telemetry metrics reflecting bus emission volume and channel distribution.
```

```rust
#[derive(Debug, Clone, Default)]
pub struct EventBusMetrics {
    /// Total events emitted to the bus across its lifetime.
    pub total_emitted: usize,
    /// Total failpoints intercepted across its lifetime.
    pub total_failpoints: usize,
    /// Total events currently held in memory.
    pub active_events_count: usize,
    /// Total failpoints currently held in memory.
    pub active_failpoints_count: usize,
    /// Distribution count of events keyed by channel name.
    pub channel_distribution: HashMap<&'static str, usize>,
}

/// Thread-safe diagnostic event bus coordinating live telemetry streams and subscriber taps.
#[derive(Clone)]
pub struct EventBus {
    events: Arc<Mutex<VecDeque<LogEvent>>>,
    failpoints: Arc<Mutex<VecDeque<LogEvent>>>,
    taps: Arc<Mutex<Vec<EventTapCallback>>>,
    max_events_capacity: usize,
    max_failpoints_capacity: usize,
    total_emitted: Arc<Mutex<usize>>,
    total_failpoints: Arc<Mutex<usize>>,
    channel_counts: Arc<Mutex<HashMap<&'static str, usize>>>,
    print_live: bool,
    color_output: bool,
}

impl std::fmt::Debug for EventBus {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        f.debug_struct("EventBus")
            .field("max_events_capacity", &self.max_events_capacity)
            .field("max_failpoints_capacity", &self.max_failpoints_capacity)
            .field("print_live", &self.print_live)
            .field("color_output", &self.color_output)
            .finish()
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Event Constructors, Bus Implementations, Invariant Enforcement & Formatting )
// ============================================================================

impl LogEvent {
    /// Constructs a standard diagnostic trace or debug event.
    pub fn trace(elapsed_micros: u128, channel: &'static str, message: impl Into<String>) -> Self {
        Self {
            elapsed_micros,
            channel,
            message: message.into(),
            is_failpoint: false,
            failpoint_type: None,
            suggestion: None,
            cognitive_note: None,
        }
    }

    /// Constructs an intercepted critical failpoint event.
    pub fn failpoint(
        elapsed_micros: u128,
        channel: &'static str,
        message: impl Into<String>,
        fp_type: FailpointType,
        suggestion: Option<FixSuggestion>,
    ) -> Self {
        Self {
            elapsed_micros,
            channel,
            message: message.into(),
            is_failpoint: true,
            failpoint_type: Some(fp_type),
            suggestion,
            cognitive_note: None,
        }
    }

    /// Constructs an event carrying an episodic cognitive memory note.
    pub fn cognitive(
        elapsed_micros: u128,
```

```
        channel: &'static str,
        message: impl Into<String>,
        note: CognitiveNote,
    ) -> Self {
        Self {
            elapsed_micros,
            channel,
            message: message.into(),
            is_failpoint: false,
            failpoint_type: None,
            suggestion: None,
            cognitive_note: Some(note),
        }
    }

    /// Formats the event for terminal presentation with optional ANSI color codes.
    pub fn format_terminal(&self, color: bool) -> String {
        if self.is_failpoint {
            let kind_str = self
                .failpoint_type
                .map(|f| f.as_str())
                .unwrap_or("FAILPOINT");
            if color {
                format!(
                    "[{:>6} µs] \x1b[91mCRITICAL\x1b[0m [{:<10}] [{:<22}] {}",
                    self.elapsed_micros, self.channel, kind_str, self.message
                )
            } else {
                format!(
                    "[{:>6} µs] CRITICAL [{:<10}] [{:<22}] {}",
                    self.elapsed_micros, self.channel, kind_str, self.message
                )
            }
        } else if color {
            format!(
                "[{:>6} µs] \x1b[92mDEBUG\x1b[0m    [{:<10}] {}",
                self.elapsed_micros, self.channel, self.message
            )
        } else {
            format!(
                "[{:>6} µs] DEBUG    [{:<10}] {}",
                self.elapsed_micros, self.channel, self.message
            )
        }
    }
}

impl EventBus {
    /// Constructs a new [`EventBus`] with default bounded capacities.
    pub fn new(print_live: bool) -> Self {
        Self::with_capacity(
            print_live,
            DEFAULT_EVENT_BUS_CAPACITY,
            DEFAULT_FAILPOINT_BUS_CAPACITY,
        )
    }

    /// Constructs a new [`EventBus`] with custom capacity bounds.
    pub fn with_capacity(
        print_live: bool,
        max_events_capacity: usize,
        max_failpoints_capacity: usize,
    ) -> Self {
        Self {
            events: Arc::new(Mutex::new(VecDeque::with_capacity(
                max_events_capacity.min(256),
            ))),
            failpoints: Arc::new(Mutex::new(VecDeque::with_capacity(
                max_failpoints_capacity.min(64),
            ))),
            taps: Arc::new(Mutex::new(Vec::new())),
            max_events_capacity: max_events_capacity.max(1),
            max_failpoints_capacity: max_failpoints_capacity.max(1),
            total_emitted: Arc::new(Mutex::new(0)),
            total_failpoints: Arc::new(Mutex::new(0)),
            channel_counts: Arc::new(Mutex::new(HashMap::new())),
            print_live,
            color_output: true,
        }
    }

```

```
    /// Enables or disables ANSI color codes for live terminal printing.
    pub fn set_color_output(&mut self, enable: bool) {
        self.color_output = enable;
    }

    /// Registers a thread-safe listener tap that receives all emitted events synchronously.
    pub fn register_tap(&self, tap: EventTapCallback) {
        self.taps.lock().unwrap().push(tap);
    }

    /// Emits a diagnostic event to the bus, notifying taps, printing live if configured,
    /// and retaining it in bounded memory queues.
    pub fn emit(&self, event: LogEvent) {
        // 1. Live terminal printing
        if self.print_live {
            let formatted = event.format_terminal(self.color_output);
            if event.is_failpoint {
                eprintln!("{formatted}");
            } else {
                println!("{formatted}");
            }
        }

        // 2. Invoke registered listener taps
        {
            let taps_guard = self.taps.lock().unwrap();
            for tap in taps_guard.iter() {
                tap(&event);
            }
        }

        // 3. Update emission metrics
        {
            let mut total_emitted = self.total_emitted.lock().unwrap();
            *total_emitted += 1;

            let mut counts = self.channel_counts.lock().unwrap();
            *counts.entry(event.channel).or_insert(0) += 1;
        }

        // 4. Ingest failpoint queue with bounded FIFO capacity
        if event.is_failpoint {
            let mut total_fp = self.total_failpoints.lock().unwrap();
            *total_fp += 1;

            let mut fps = self.failpoints.lock().unwrap();
            if fps.len() >= self.max_failpoints_capacity {
                fps.pop_front();
            }
            fps.push_back(event.clone());
        }

        // 5. Ingest general event queue with bounded FIFO capacity
        {
            let mut evs = self.events.lock().unwrap();
            if evs.len() >= self.max_events_capacity {
                evs.pop_front();
            }
            evs.push_back(event);
        }
    }

    /// Returns a cloned snapshot of all currently retained general log events.
    pub fn all_events(&self) -> Vec<LogEvent> {
        self.events.lock().unwrap().iter().cloned().collect()
    }

    /// Returns a cloned snapshot of all currently retained failpoint events.
    pub fn all_failpoints(&self) -> Vec<LogEvent> {
        self.failpoints.lock().unwrap().iter().cloned().collect()
    }

    /// Filters and returns events matching a specific subsystem channel.
    pub fn events_by_channel(&self, channel: &str) -> Vec<LogEvent> {
        self.events
            .lock()
            .unwrap()
            .iter()
            .filter(|e| e.channel == channel)
            .cloned()
            .collect()
```

```rust
    }

    /// Filters and returns events occurring within a microsecond timestamp window `[start, end]`.
    pub fn events_by_time_range(&self, start_micros: u128, end_micros: u128) -> Vec<LogEvent> {
        self.events
            .lock()
            .unwrap()
            .iter()
            .filter(|e| e.elapsed_micros >= start_micros && e.elapsed_micros <= end_micros)
            .cloned()
            .collect()
    }

    /// Retrieves an aggregated metrics snapshot of the event bus.
    pub fn metrics(&self) -> EventBusMetrics {
        let total_emitted = *self.total_emitted.lock().unwrap();
        let total_failpoints = *self.total_failpoints.lock().unwrap();
        let active_events_count = self.events.lock().unwrap().len();
        let active_failpoints_count = self.failpoints.lock().unwrap().len();
        let channel_distribution = self.channel_counts.lock().unwrap().clone();

        EventBusMetrics {
            total_emitted,
            total_failpoints,
            active_events_count,
            active_failpoints_count,
            channel_distribution,
        }
    }

    /// Returns the number of events currently buffered in memory.
    pub fn len(&self) -> usize {
        self.events.lock().unwrap().len()
    }

    /// Returns `true` if the event buffer has no active events.
    pub fn is_empty(&self) -> bool {
        self.events.lock().unwrap().is_empty()
    }

    /// Returns the number of failpoint events currently buffered in memory.
    pub fn failpoint_count(&self) -> usize {
        self.failpoints.lock().unwrap().len()
    }

    /// Clears all currently retained events and failpoints from memory queues.
    pub fn clear(&self) {
        self.events.lock().unwrap().clear();
        self.failpoints.lock().unwrap().clear();
    }
}

impl Default for EventBus {
    fn default() -> Self {
        Self::new(false)
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs, Bounded Capacity & Tap Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::atomic::{AtomicUsize, Ordering};
    use std::thread;

    #[test]
    fn test_event_bus_basic_emission_and_retrieval() {
        let bus = EventBus::new(false);
        assert!(bus.is_empty());
        assert_eq!(bus.len(), 0);

        bus.emit(LogEvent::trace(100, "PARSER", "Token stream initialized"));
        bus.emit(LogEvent::trace(150, "PARSER", "First token scanned"));
        bus.emit(LogEvent::failpoint(
            200,
            "BOUNDARY",
            "Unclosed bracket delimiter '<'",
```

```
            FailpointType::BoundaryLeak,
            None,
        ));

        assert_eq!(bus.len(), 3);
        assert_eq!(bus.failpoint_count(), 1);

        let all = bus.all_events();
        assert_eq!(all.len(), 3);
        assert_eq!(all[0].channel, "PARSER");
        assert!(!all[0].is_failpoint);
        assert!(all[2].is_failpoint);

        let fps = bus.all_failpoints();
        assert_eq!(fps.len(), 1);
        assert_eq!(fps[0].failpoint_type, Some(FailpointType::BoundaryLeak));

        bus.clear();
        assert!(bus.is_empty());
        assert_eq!(bus.failpoint_count(), 0);
    }

    #[test]
    fn test_bounded_capacity_fifo_eviction() {
        // Enforce a strict bound of 3 events and 2 failpoints
        let bus = EventBus::with_capacity(false, 3, 2);

        for i in 1..=5 {
            bus.emit(LogEvent::trace(
                i * 10,
                "EXEC",
                format!("Step {i}"),
            ));
        }

        // Must not exceed max capacity of 3
        assert_eq!(bus.len(), 3);
        let events = bus.all_events();
        assert_eq!(events[0].message, "Step 3");
        assert_eq!(events[1].message, "Step 4");
        assert_eq!(events[2].message, "Step 5");

        // Emit 3 failpoints into capacity of 2
        for i in 1..=3 {
            bus.emit(LogEvent::failpoint(
                i * 100,
                "FAULT",
                format!("Fault {i}"),
                FailpointType::HardwareFault,
                None,
            ));
        }

        assert_eq!(bus.failpoint_count(), 2);
        let fps = bus.all_failpoints();
        assert_eq!(fps[0].message, "Fault 2");
        assert_eq!(fps[1].message, "Fault 3");
    }

    #[test]
    fn test_channel_filtering_and_time_range_scans() {
        let bus = EventBus::new(false);
        bus.emit(LogEvent::trace(100, "SILICON", "Reset landmark 0x7C00"));
        bus.emit(LogEvent::trace(200, "CORTEX", "Awakening neural tensor"));
        bus.emit(LogEvent::trace(300, "SILICON", "A20 gate active 0x18000"));
        bus.emit(LogEvent::trace(400, "CORTEX", "Episodic memory synchronized"));

        let silicon = bus.events_by_channel("SILICON");
        assert_eq!(silicon.len(), 2);
        assert_eq!(silicon[0].elapsed_micros, 100);
        assert_eq!(silicon[1].elapsed_micros, 300);

        let cortex = bus.events_by_channel("CORTEX");
        assert_eq!(cortex.len(), 2);

        let range = bus.events_by_time_range(150, 350);
        assert_eq!(range.len(), 2);
        assert_eq!(range[0].channel, "CORTEX");
        assert_eq!(range[1].channel, "SILICON");
    }

```

```rust
    #[test]
    fn test_subscriber_tap_callback() {
        let bus = EventBus::new(false);
        let tap_count = Arc::new(AtomicUsize::new(0));

        let tap_count_clone = Arc::clone(&tap_count);
        bus.register_tap(Arc::new(move |_event| {
            tap_count_clone.fetch_add(1, Ordering::SeqCst);
        }));

        bus.emit(LogEvent::trace(10, "TAP", "Message 1"));
        bus.emit(LogEvent::trace(20, "TAP", "Message 2"));
        bus.emit(LogEvent::trace(30, "TAP", "Message 3"));

        assert_eq!(tap_count.load(Ordering::SeqCst), 3);
    }

    #[test]
    fn test_metrics_aggregation() {
        let bus = EventBus::new(false);
        bus.emit(LogEvent::trace(10, "PARSER", "P1"));
        bus.emit(LogEvent::trace(20, "PARSER", "P2"));
        bus.emit(LogEvent::trace(30, "EVAL", "E1"));
        bus.emit(LogEvent::failpoint(
            40,
            "FAULT",
            "F1",
            FailpointType::ImmutableMutation,
            None,
        ));

        let m = bus.metrics();
        assert_eq!(m.total_emitted, 4);
        assert_eq!(m.total_failpoints, 1);
        assert_eq!(m.active_events_count, 4);
        assert_eq!(m.active_failpoints_count, 1);
        assert_eq!(m.channel_distribution.get("PARSER"), Some(&2));
        assert_eq!(m.channel_distribution.get("EVAL"), Some(&1));
        assert_eq!(m.channel_distribution.get("FAULT"), Some(&1));
    }

    #[test]
    fn test_concurrent_multithreaded_emission() {
        let bus = EventBus::new(false);
        let mut handles = Vec::new();

        for t in 0..4 {
            let bus_clone = bus.clone();
            handles.push(thread::spawn(move || {
                for i in 0..50 {
                    bus_clone.emit(LogEvent::trace(
                        (t * 100 + i) as u128,
                        "THREAD",
                        format!("Thread {t} Event {i}"),
                    ));
                }
            }));
        }

        for h in handles {
            h.join().unwrap();
        }

        assert_eq!(bus.len(), 200);
        let m = bus.metrics();
        assert_eq!(m.total_emitted, 200);
    }

    #[test]
    fn test_terminal_formatting_plain_and_color() {
        let trace_event = LogEvent::trace(142, "PARSER", "Syntax OK");
        let plain_trace = trace_event.format_terminal(false);
        assert!(plain_trace.contains("[   142 µs] DEBUG    [PARSER    ] Syntax OK"));

        let fail_event = LogEvent::failpoint(
            890,
            "BOUNDARY",
            "Missing tag",
            FailpointType::BoundaryMismatch,
            None,
        );
```

```
        let plain_fail = fail_event.format_terminal(false);
        assert!(plain_fail.contains("[   890 µs] CRITICAL [BOUNDARY  ] [FP_BOUNDARY_MISMATCH  ] Missing tag"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/clock.rs

RS | 357 LINES | 1401 WORDS | 13.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal High-Precision Clock, Monotonic Timers & Sequencer Substrate )
// ============================================================================
//! OmniCode Universal Diagnostics: Monotonic Clock & Timestamp Substrate
//!
//! # Architectural Scope & Purpose:
//! Provides nanosecond/microsecond resolution elapsed time offsets, strictly
//! monotonic timestamp identifiers for telemetry frame ordering, and deterministic
//! simulation stepping for virtual machine replay:
//! - [`MonotonicClock`]: Real-time monotonic reference measuring microsecond/nanosecond durations.
//! - [`SimulatedClock`]: Deterministic virtual machine clock for reproducible execution & roll stepping.
//! - [`generate_timestamp_id`]: Thread-safe, monotonic timestamp ID generator resilient to clock skews.
//! - [`parse_timestamp_id`]: Exact parser decomposing timestamp identifiers into epoch and sequence parts.
//!
//! # Scripture Witness:
//! - **Ecclesiastes 3:1 (KJV)**:
//!   "To every thing there is a season, and a time to every purpose under the heaven."
//!   (The foundational truth that every computation, event, and roll operates within an orderly time frame).
//! - **Proverbs 4:26 (KJV)**:
//!   "Ponder the path of thy feet, and let all thy ways be established."
//!   (The discipline of measuring every microsecond offset and ordering telemetry events without drift).
//! - **1 Corinthians 14:40 (KJV)**:
//!   "Let all things be done decently and in order."
//!   (The strict causal monotonicity invariant: an effect must never precede its cause in timestamp ordering).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Causal Monotonicity**: Wall-clock timestamps (`SystemTime`) are subject to NTP adjustments,
//!    daylight savings jumps, and leap seconds. If time moves backward, causal reasoning in the AI
//!    Cortex collapses. This module guarantees strict monotonicity ($\Delta t \ge 0$).
//! 2. **Hardware & Silicon Correlation**: Converting CPU cycle counts (TSC) from Track 0 bare-metal
//!    landmarks to microsecond durations requires calibrated frequency conversion factors.
//! 3. **Deterministic Virtual Machine Replay**: During automated debugging and test execution,
//!    real-world timing jitter causes non-reproducible race conditions. [`SimulatedClock`] provides
//!    freeze-and-step discrete time semantics for bitwise-reproducible roll execution.
// ============================================================================

use std::sync::atomic::{AtomicBool, AtomicU64, Ordering};
use std::sync::Mutex;
use std::time::{Instant, SystemTime};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Constants, Clock Data Structures & Timestamp Definitions )
// ============================================================================

/// Default nominal CPU clock frequency in Megahertz (2,000 MHz = 2.0 GHz).
pub const DEFAULT_NOMINAL_CPU_MHZ: u64 = 2000;

/// Number of nanoseconds per microsecond ($1\ \mu\text{s} = 1000\ \text{ns}$).
pub const NANOS_PER_MICRO: u128 = 1000;

/// Number of microseconds per second ($1\ \text{s} = 1,000,000\ \mu\text{s}$).
pub const MICROS_PER_SEC: u128 = 1_000_000;

/// Default microseconds per discrete virtual machine execution tick.
pub const DEFAULT_MICROS_PER_TICK: u128 = 100;

/// Real-time monotonic clock reference measuring microsecond and nanosecond elapsed durations.
///
/// Backed by [`std::time::Instant`] which is guaranteed to be monotonically non-decreasing.
#[derive(Debug, Clone)]
pub struct MonotonicClock {
    start: Instant,
}

/// Deterministic simulation clock for virtual machine stepping, replay, and discrete time testing.
///
/// Allows freezing real-time progression and advancing in discrete microsecond or tick intervals.
#[derive(Debug)]
pub struct SimulatedClock {
    current_micros: AtomicU64,
    current_tick: AtomicU64,
    frozen: AtomicBool,
}
```

```rust

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Clock Implementations, Monotonic Sequencer & Silicon Cycle Conversion )
// ============================================================================

/// Generates a strictly unique, monotonically non-decreasing timestamp identifier.
///
/// # Format:
/// `<unix_epoch_micros>_<atomic_counter:04>`
///
/// # Monotonicity Guarantee:
/// Even if the operating system wall clock jumps backwards (e.g. NTP synchronization
/// or VM migration), this function maintains monotonicity by ensuring the epoch portion
/// never regresses below the previously observed maximum.
pub fn generate_timestamp_id() -> String {
    static COUNTER: AtomicU64 = AtomicU64::new(0);
    static LAST_MICROS: Mutex<u128> = Mutex::new(0);

    let count = COUNTER.fetch_add(1, Ordering::Relaxed);
    let wall_now = SystemTime::now()
        .duration_since(SystemTime::UNIX_EPOCH)
        .unwrap_or_default()
        .as_micros();

    let mut last = LAST_MICROS.lock().unwrap();
    let epoch_micros = if wall_now > *last {
        *last = wall_now;
        wall_now
    } else {
        *last
    };

    format!("{epoch_micros}_{count:04}")
}

/// Parses a standardized timestamp identifier into its constituent microsecond epoch
/// and sequence counter parts.
///
/// Returns `Some((epoch_micros, sequence_counter))` on success, or `None` if the string
/// does not conform to the `<epoch_micros>_<counter>` format.
pub fn parse_timestamp_id(id: &str) -> Option<(u128, u64)> {
    let parts: Vec<&str> = id.split('_').collect();
    if parts.len() != 2 {
        return None;
    }

    let epoch_micros = parts[0].parse::<u128>().ok()?;
    let counter = parts[1].parse::<u64>().ok()?;

    Some((epoch_micros, counter))
}

impl MonotonicClock {
    /// Constructs a new [`MonotonicClock`] calibrated to the current instant.
    pub fn new() -> Self {
        Self {
            start: Instant::now(),
        }
    }

    /// Returns elapsed microseconds since clock initialization.
    #[inline]
    pub fn elapsed_micros(&self) -> u128 {
        self.start.elapsed().as_micros()
    }

    /// Returns elapsed nanoseconds since clock initialization.
    #[inline]
    pub fn elapsed_nanos(&self) -> u128 {
        self.start.elapsed().as_nanos()
    }

    /// Returns elapsed milliseconds as a floating-point value.
    #[inline]
    pub fn elapsed_millis(&self) -> f64 {
        self.start.elapsed().as_nanos() as f64 / 1_000_000.0
    }

    /// Resets the base reference instant to now.
    pub fn reset(&mut self) {
```

```
        self.start = Instant::now();
    }

    /// Converts elapsed microseconds into estimated CPU clock cycles at the given frequency (in MHz).
    ///
    /// # Formula:
    /// $\text{Cycles} = \text{microseconds} \times \text{MHz}$
    #[inline]
    pub fn micros_to_cycles(micros: u128, mhz: u64) -> u64 {
        (micros as u64).saturating_mul(mhz)
    }

    /// Converts raw CPU clock cycles (e.g. from x86 `RDTSC`) to microseconds at the given frequency.
    ///
    /// # Formula:
    /// $\text{Microseconds} = \frac{\text{Cycles}}{\text{MHz}}$
    #[inline]
    pub fn cycles_to_micros(cycles: u64, mhz: u64) -> u128 {
        if mhz == 0 {
            0
        } else {
            (cycles / mhz) as u128
        }
    }
}

impl Default for MonotonicClock {
    fn default() -> Self {
        Self::new()
    }
}

impl SimulatedClock {
    /// Constructs a new [`SimulatedClock`] initialized at the specified microsecond offset.
    pub fn new(start_micros: u64) -> Self {
        Self {
            current_micros: AtomicU64::new(start_micros),
            current_tick: AtomicU64::new(0),
            frozen: AtomicBool::new(false),
        }
    }

    /// Returns the current simulated microsecond timestamp.
    pub fn current_micros(&self) -> u128 {
        self.current_micros.load(Ordering::Acquire) as u128
    }

    /// Returns the current discrete virtual machine execution tick.
    pub fn current_tick(&self) -> u64 {
        self.current_tick.load(Ordering::Acquire)
    }

    /// Advances the simulated clock by the specified microsecond delta.
    ///
    /// If the clock is currently frozen, this operation is a no-op unless explicitly forced.
    pub fn step_micros(&self, delta: u64) {
        if !self.frozen.load(Ordering::Acquire) {
            self.current_micros.fetch_add(delta, Ordering::Release);
        }
    }

    /// Advances the simulated clock by a discrete number of virtual machine execution ticks.
    ///
    /// Also increments the microsecond counter by `ticks * micros_per_tick`.
    pub fn step_ticks(&self, ticks: u64, micros_per_tick: u64) {
        if !self.frozen.load(Ordering::Acquire) {
            self.current_tick.fetch_add(ticks, Ordering::Release);
            self.current_micros
                .fetch_add(ticks.saturating_mul(micros_per_tick), Ordering::Release);
        }
    }

    /// Freezes the simulation clock, preventing automated time progression.
    pub fn freeze(&self) {
        self.frozen.store(true, Ordering::Release);
    }

    /// Resumes the simulation clock.
    pub fn resume(&self) {
        self.frozen.store(false, Ordering::Release);
    }
```

```rust

    /// Returns `true` if the simulation clock is currently frozen.
    pub fn is_frozen(&self) -> bool {
        self.frozen.load(Ordering::Acquire)
    }

    /// Resets the simulation clock back to zero.
    pub fn reset(&self) {
        self.current_micros.store(0, Ordering::Release);
        self.current_tick.store(0, Ordering::Release);
        self.frozen.store(false, Ordering::Release);
    }
}

impl Default for SimulatedClock {
    fn default() -> Self {
        Self::new(0)
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Monotonic Invariant Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use std::thread;
    use std::time::Duration;

    #[test]
    fn test_monotonic_clock_elapsed_and_reset() {
        let mut clock = MonotonicClock::new();
        thread::sleep(Duration::from_millis(5));

        let micros = clock.elapsed_micros();
        let nanos = clock.elapsed_nanos();
        let millis = clock.elapsed_millis();

        assert!(micros >= 4_000, "Expected at least 4000 µs elapsed, got {micros}");
        assert!(nanos >= 4_000_000, "Expected at least 4,000,000 ns elapsed, got {nanos}");
        assert!(millis >= 4.0, "Expected at least 4.0 ms elapsed, got {millis}");

        clock.reset();
        let reset_micros = clock.elapsed_micros();
        assert!(reset_micros < micros, "Reset clock should be less than previous elapsed");
    }

    #[test]
    fn test_timestamp_id_generation_and_parsing() {
        let id1 = generate_timestamp_id();
        let id2 = generate_timestamp_id();

        assert_ne!(id1, id2, "Generated timestamp IDs must be strictly unique");

        let parsed1 = parse_timestamp_id(&id1);
        let parsed2 = parse_timestamp_id(&id2);

        assert!(parsed1.is_some(), "ID1 must be parsable");
        assert!(parsed2.is_some(), "ID2 must be parsable");

        let (epoch1, count1) = parsed1.unwrap();
        let (epoch2, count2) = parsed2.unwrap();

        assert!(epoch2 >= epoch1, "Epoch micros must be monotonically non-decreasing");
        assert!(count2 > count1, "Counter must strictly increase");

        assert_eq!(parse_timestamp_id("invalid_format_here_extra"), None);
        assert_eq!(parse_timestamp_id("notanumber_0001"), None);
        assert_eq!(parse_timestamp_id("12345_notanumber"), None);
    }

    #[test]
    fn test_silicon_cycle_and_microsecond_conversions() {
        let mhz = 2000; // 2.0 GHz
        let micros = 500; // 500 µs
        let cycles = MonotonicClock::micros_to_cycles(micros, mhz);

        assert_eq!(cycles, 1_000_000, "500 µs at 2.0 GHz should equal 1,000,000 cycles");

```

```
        let back_to_micros = MonotonicClock::cycles_to_micros(cycles, mhz);
        assert_eq!(back_to_micros, micros, "Cycle roundtrip must yield original microsecond offset");

        assert_eq!(MonotonicClock::cycles_to_micros(1_000_000, 0), 0, "Zero MHz must yield 0 µs safely");
    }

    #[test]
    fn test_simulated_clock_lifecycle_and_freeze() {
        let sim = SimulatedClock::new(1000);
        assert_eq!(sim.current_micros(), 1000);
        assert_eq!(sim.current_tick(), 0);
        assert!(!sim.is_frozen());

        sim.step_micros(500);
        assert_eq!(sim.current_micros(), 1500);

        sim.step_ticks(5, 100);
        assert_eq!(sim.current_tick(), 5);
        assert_eq!(sim.current_micros(), 2000);

        sim.freeze();
        assert!(sim.is_frozen());

        sim.step_micros(500);
        sim.step_ticks(2, 100);
        assert_eq!(sim.current_micros(), 2000, "Frozen clock must not advance on step_micros");
        assert_eq!(sim.current_tick(), 5, "Frozen clock must not advance on step_ticks");

        sim.resume();
        assert!(!sim.is_frozen());
        sim.step_micros(250);
        assert_eq!(sim.current_micros(), 2250);

        sim.reset();
        assert_eq!(sim.current_micros(), 0);
        assert_eq!(sim.current_tick(), 0);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/logger/universal/mod.rs RS | 102 LINES | 521 WORDS | 4.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Telemetry, Monotonic Clock & Diagnostic Event Bus Subsystem )
// ============================================================================
//! OmniCode Universal Diagnostics: Clock, Bus & Telemetry Substrate
//!
//! # Architectural Scope & Purpose:
//! The `universal` module serves as the foundational timing and telemetry routing
//! foundation for the entire `diagnostics::logger` subsystem:
//! - [`clock`]: Nanosecond/microsecond monotonic clock, silicon cycle conversions,
//!   deterministic simulation clock, and strictly monotonic timestamp ID generation.
//! - [`bus`]: Thread-safe diagnostic event bus, multi-channel event routing, bounded
//!   capacity queues enforcing container capacity law (2 Kings 4:4), and live subscriber taps.
//!
//! # Scripture Witness:
//! - **Ecclesiastes 3:1 (KJV)**:
//!   "To every thing there is a season, and a time to every purpose under the heaven."
//!   (Orderly temporal sequencing across virtual machine execution rolls).
//! - **1 Corinthians 14:40 (KJV)**:
//!   "Let all things be done decently and in order."
//!   (Deterministic monotonic causal flow across the diagnostic event bus).
//! - **2 Kings 4:4 (KJV)**:
//!   "And when thou art come in, thou shalt shut the door upon thee and upon thy sons,
//!    and shalt pour out into all those vessels, and thou shalt set aside that which is full."
//!   (The container law Occupancy <= Capacity applied to bounded in-memory event queues).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Causal Monotonicity**: Diagnostic frames are sequenced by monotonic elapsed time ($\Delta t \ge 0$),
//!    guaranteeing that root causes always precede secondary symptoms in automated RCA and AI reasoning.
//! 2. **Bounded Capacity Invariant**: Unbounded memory growth in logging systems is a primary cause of
//!    production outages. Every queue in this substrate is bounded with deterministic FIFO eviction.
//! 3. **Non-Intrusive Observation**: Subscriber taps allow live consoles, query indexers, and test oracles
//!    to inspect telemetry in real-time without taking locks on the critical execution path.
// ============================================================================

pub mod bus;
pub mod clock;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Public Type Re-exports & Subsystem Invariants )
// ============================================================================

pub use bus::{
    EventBus, EventBusMetrics, EventTapCallback, LogEvent,
    DEFAULT_EVENT_BUS_CAPACITY, DEFAULT_FAILPOINT_BUS_CAPACITY,
};
pub use clock::{
    MonotonicClock, SimulatedClock, generate_timestamp_id, parse_timestamp_id,
    DEFAULT_MICROS_PER_TICK, DEFAULT_NOMINAL_CPU_MHZ, MICROS_PER_SEC, NANOS_PER_MICRO,
};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Facade & Combined Clock-Bus Utilities )
// ============================================================================

/// Constructs a calibrated pairing of a [`MonotonicClock`] and a thread-safe [`EventBus`].
///
/// Ensures both machine organs share identical calibration invariants at creation time.
pub fn create_universal_telemetry_pair(print_live: bool) -> (MonotonicClock, EventBus) {
    (MonotonicClock::new(), EventBus::new(print_live))
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Test Verification Proofs & Universal Facade Invariants )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_universal_telemetry_pair_creation() {
```

```
        let (clock, bus) = create_universal_telemetry_pair(false);
        assert_eq!(bus.len(), 0);
        assert!(!bus.print_live_status_or_similar(&clock));
    }

    trait TestBusExt {
        fn print_live_status_or_similar(&self, clock: &MonotonicClock) -> bool;
    }

    impl TestBusExt for EventBus {
        fn print_live_status_or_similar(&self, clock: &MonotonicClock) -> bool {
            // Verifies both organs are accessible in unified test scope
            clock.elapsed_micros() > 10_000_000 // false initially
        }
    }

    #[test]
    fn test_universal_re_exports_available() {
        let id = generate_timestamp_id();
        assert!(parse_timestamp_id(&id).is_some());

        let event = LogEvent::trace(142, "UNIVERSAL", "Facade verified");
        assert_eq!(event.channel, "UNIVERSAL");
        assert_eq!(event.elapsed_micros, 142);
    }
}
```